\ here are the text rules:
\ in a non-wrapped text, the rows end with a return byte.
\ in a wrapped text, the rows end with either a return byte or a space.
\ when text is converted to a string, linefeed bytes are added after return bytes.
\ when a string is converted to text, linefeed bytes are removed.
\ there is always at least one row.
\ there is always a return byte at the end of the last row.

The a-key is a key equal to 65.

An abc is a record with a number called abca, a number called abcb, a number called abcc.

An abc pointer is a pointer to an abc.

An abca is a number.

An abcc is a number.

An absolute position is a number.

The accent byte is a byte equal to 96.

The accent key is a key equal to 192.

The acknowledge byte is a byte equal to 6.

The acute-accent byte is a byte equal to 180.

To add a byte to another byte:
Intel $8B8508000000. \ mov eax,[ebp+8] \ the byte
Intel $0FB600. \ movzx eax,[eax]
Intel $8B9D0C000000. \ mov ebx,[ebp+12] \ the other byte
Intel $0003. \ add [ebx],al

To add a byte to a number:
Intel $8B8508000000. \ mov eax,[ebp+8] \ the byte
Intel $0FB600. \ movzx eax,[eax]
Intel $8B9D0C000000. \ mov ebx,[ebp+12] \ the number
Intel $0103. \ add [ebx],eax

To add a fraction to another fraction:
Privatize the fraction.
Normalize the fraction and the other fraction.
Add the fraction’s numerator to the other fraction’s numerator.
Reduce the other fraction.

To add some horizontal twips and some vertical twips to the current spot:
Add the horizontal twips to the context’s spot’s x.
Add the vertical twips to the context’s spot’s y.

To add a line to a figure:
If the figure is nil, create the figure; append the figure to the figures.
Add the line’s start to the figure.

Add the line's end to the figure.

To add a name to some choices:
Allocate memory for a choice.
Put the name into the choice's name.
Put the choice at the end of the choices.

To add a number and another number to a pair:
Add the number to the pair's x.
Add the other number to the pair's y.

To add a number to another number and a third number to a fourth number:
Add the number to the other number.
Add the third number to the fourth number.

To add a number to a byte:
Intel $8B8508000000. \ mov eax,[ebp+8] \ the number
Intel $8B00. \ mov eax,[eax]
Intel $8B9D0C000000. \ mov ebx,[ebp+12] \ the byte
Intel $0FB60B. \ movzx ecx,[ebx]
Intel $03C8. \ add ecx,eax
Intel $880B. \ mov [ebx],cl

To add a number to a fraction:
Add the number / 1 to the fraction.

To add a number to a pair:
Add the number to the pair's x.
Add the number to the pair's y.

To add a number to a pointer;
To add a number to another number:
Intel $8B8508000000. \ mov eax,[ebp+8] \ the number
Intel $8B00. \ mov eax,[eax]
Intel $8B9D0C000000. \ mov ebx,[ebp+12] \ the other number
Intel $0103. \ add [ebx],eax

To add a pair to another pair:
Add the pair's x to the other pair's x.
Add the pair's y to the other pair's y.

To add a pdf object given a kind:
Create the pdf object given the kind.
Append the pdf object to the pdf state's objects.
Add 1 to the pdf state's object number.
Put the pdf state's object number into the pdf object's number.

To add a quora to a terminal:
Create the quora.
Append the quora to the terminal's quoras.
If the terminal is not full, exit.
Put the terminal's quoras' first into a doomed quora.
Remove the doomed quora from the terminal's quoras.
Destroy the doomed quora.

To add a spot to a figure: append the spot to the figure.

To add a string to some string things:
Create a string thing given the string.
Append the string thing to the string things.

An addrinfo is a record with
A number called ai_flags,
A number called ai_family,
A number called ai_sockettype,
A number called ai_protocol,
A number called ai_addrlen,
A pchar called ai_canonname,
A sockaddrptr called ai_addr,
A addrinfoptr called ai_next.

Some addrinfo routines is a record with
A getaddrinfo pointer,
A freeaddrinfo pointer.

An addrinfoptr is a pointer to an addrinfo.

To adjust a box given a number and another number and a third number and a fourth number:
Add the number to the box's left.
Add the other number to the box's top.
Add the third number to the box's right.
Add the fourth number to the box's bottom.

To adjust an item:
Put the item's win32finddata's dwfileattributes into a number.
Bitwise and the number with 16 [file_attribute_directory].
If the number is 0, put "file" into the item's kind.
If the number is not 0, put "directory" into the item's kind.
Put the item's win32finddata's cfilename's whereabouts into a pchar.
Convert the pchar to the item's designator.
If the item's kind is "directory", append "\" to the item's designator.
Put the item's directory then the item's designator into the item's path.
Extract the item's extension from the item's designator as a path.
Put the item's win32finddata's nfilesizelow into the item's size.
If the item's designator's first's target is not the period byte, exit.
Get the item (not first time).

To adjust a line with a number and another number and a third number and a fourth number:
Add the number to the line's start's x.
Add the other number to the line's start's y.
Add the third number to the line's end's x.
Add the fourth number to the line's end's y.

To adjust a picture (extract boxes from gpbitmap):
If the picture is nil, exit.
Put 0 into the picture's box's left.
Put 0 into the picture's box's top.
Put the picture's gpbitmap's width minus 1 times the tpp into the picture's box's right.
Put the picture's gpbitmap's height minus 1 times the tpp into the picture's box's bottom.
Put the picture's box into the picture's uncropped box.

To adjust spacing given a string:
If the current canvas is not the printer canvas, exit.
Call "gdi32.dll" "SetTextCharacterExtra" with the printer canvas and 0.
Call "gdi32.dll" "GetCurrentObject" with the printer canvas and 6 [obj_font] returning a handle.
Call "gdi32.dll" "SelectObject" with the memory canvas and the handle.
Get a width given the string and the memory canvas.
Call "gdi32.dll" "SelectObject" with the memory canvas and the null hfont.
Get another width given the string and the printer canvas.
Put the width minus the other width divided by the string's length into a number.
Call "gdi32.dll" "SetTextCharacterExtra" with the printer canvas and the number.

To align a text given an alignment:
Put the alignment into the text's alignment.

An alignment is a string [center, left, or right].

The alt key is a key equal to 18.

An amount is a number.

The ampersand byte is a byte equal to 38.

An anchor is a position.

An and-mask is a mask.

An angle is some precise degrees [0 to 3599].

To append a buffer to a file:
Clear the i/o error.
Call "kernel32.dll" "SetFilePointer" with the file and 0 and 0 and 2 [file_end] returning a result number.
If the result number is -1, put "Error positioning file pointer." into the i/o error; exit.
Call "kernel32.dll" "WriteFile" with the file and the buffer's first and the buffer's length and a number's whereabouts and 0 returning the result number.
If the result number is 0, put "Error writing file." into the i/o error; exit.

To append a byte to a string:
Put the string's length into a saved length.
Reassign the string's first given the string's length plus 1.
Put the string's first plus the saved length into the string's last.
Put the byte into the string's last's target.

To append a byte to a string given a count:
Privatize the count.
Loop.
If the count is less than 1, exit.
Append the byte to the string.
Subtract 1 from the count.
Repeat.

To append a flag to a string:
Convert the flag to another string.
Append the other string to the string.

```
To append a fraction to a string:
Convert the fraction to another string.
Append the other string to the string.

To append a number to a string:
Convert the number to another string.
Append the other string to the string.

To append a pointer to a string:
Convert the pointer to another string.
Append the other string to the string.

To append a spot to a polygon:
If the polygon is nil, exit.
Create a vertex given the spot.
Append the vertex to the polygon's vertices.

To append a string to another string:
If the string is blank, exit.
Put the string's length into a combined length.
Put the other string's length into a saved length.
Add the saved length to the combined length.
Reassign the other string's first given the combined length.
Put the other string's first plus the saved length into a pointer.
Copy bytes from the string's first to the pointer for the string's length.
Put the other string's first plus the combined length minus 1 into the other string's last.

To append a string to another string (handling email transparency):
If the string is blank, exit.
Slap a rider on the string.
Loop.
Move the rider (text file rules).
If the rider's token is blank, exit.
If the rider's token starts with ".", append "." to the other string.
Append the rider's token to the other string.
Repeat.

To append a string to another string given a count:
Privatize the count.
Loop.
If the count is less than 1, exit.
Append the string to the other string.
Subtract 1 from the count.
Repeat.

To append a string to a pdf object: \ this guys adds CRLF
Append the string to the pdf object's data.
Append the crlf string to the pdf object's data.

To append a string to a pdf object without advancing:
Append the string to the pdf object's data.

To append some things to some other things:
Put the things' first into a thing.
If the thing is nil, exit.
```

Remove the thing from the things.
Append the thing to the other things.
Repeat.

To append a timer to a string:
Convert the timer to another string.
Append the other string to the string.

To append a vertex to a polygon:
If the polygon is nil, exit.
Append the vertex to the polygon's vertices.

To append an x coord and a y coord to a polygon:
If the polygon is nil, exit.
Create a vertex given the x and the y.
Append the vertex to the polygon's vertices.

To append zeros to a string until its length is a number:
If the string's length is greater than or equal to the number, exit.
Append "0" to the string.
Repeat.

The arrow cursor is a cursor.

To assign a pointer given a byte count:
If the byte count is 0, void the pointer; exit.
Privatize the byte count.
Round the byte count up to the nearest power of two.
Call "kernel32.dll" "HeapAlloc" with the heap pointer and 8 [heap_zero_memory] and the byte count returning the pointer.
If the pointer is not nil, add 1 to the heap count; exit.

The asterisk byte is a byte equal to 42.

The at-sign byte is a byte equal to 64.

To autoscroll a text given a spot and a flag:
If the text is nil, clear the flag; exit.
Put the text's font's height into a number.
Clear a difference.
Put the text's box into a box.
Indent the box given the tpp.
If the spot's y is less than the box's top, put the number into the difference's y.
If the spot's y is greater than the box's bottom, put the number into the difference's y; negate the difference's y.
If the spot's x is less than the box's left, put the number into the difference's x.
If the spot's x is greater than the box's right, put the number into the difference's x; negate the difference's x.
If the text's horizontal scroll flag is not set, put 0 into the difference's x.
If the text's vertical scroll flag is not set, put 0 into the difference's y.
If the difference is 0, clear the flag; exit.
Set the flag.
Scroll the text given the difference.
Wait for 50 milliseconds.

The b-key is a key equal to 66.

The backslash byte is a byte equal to 92.

The backspace key is a key equal to 8.

The bar byte is a byte equal to 124.

A baseline is a number.

To beep:
Call "user32.dll" "MessageBeep" with 0.

To begin a landscape sheet:
Make the landscape sheet 11 inches by 8-1/2 inches.
Begin a sheet with the landscape sheet.

To begin a landscape sheet given a title string:
If the pdf document flag is not set, clear the landscape sheet; exit.
Make the landscape sheet 11 inches by 8-1/2 inches.
Begin the sheet given the box and the title (pdf style).

To begin a portrait sheet:
Make the portrait sheet 8-1/2 inches by 11 inches.
Begin a sheet with the portrait sheet.

To begin a portrait sheet given a title string:
If the pdf document flag is not set, clear the portrait sheet; exit.
Make the portrait sheet 8-1/2 inches by 11 inches.
Begin the sheet given the box and the title (pdf style).

To begin printing:
Initialize the printer canvas.
Put a docinfo's magnitude into the docinfo's cbsize.
Put the module's name's first into the docinfo's lpszdocname.
Call "gdi32.dll" "StartDocA" with the printer canvas and the docinfo's whereabouts.

To begin printing a pdf:
Set the pdf state's document flag.
Put 0 into the pdf state's object number.
Create the pdf state's font index given 113.
Begin printing the pdf (start the root).
Begin printing the pdf (start the parent).

To begin printing a pdf (start the parent):
Add a parent pdf object given "parent".
Put the parent into the pdf state's parent.
Append the parent's number then " 0 obj" to the parent.
Append "<<" to the parent.
Append "/Type /Pages" to the parent.

To begin printing a pdf (start the root):
Add a root pdf object given "root".
Put the root into the pdf state's root.
Append the root's number then " 0 obj" to the root.

Append "<<" to the root.
Append "/Type /Catalog" to the root.

To begin a sheet:
Begin the sheet as a portrait sheet.

To begin a sheet given a box:
If the pdf state's document flag is set, begin the sheet given the box (pdf style); exit.
Call "kernel32.dll" "GlobalLock" with the printer device mode handle returning a pdevmode.
If the pdevmode is nil, exit.
Bitwise or the pdevmode's dmfields with 1 [dm_orientation].
Put 1 [dmorient_portrait] into the pdevmode's dmorientation.
If the box's width is greater than the box's height, put 2 [dmorient_landscape] into the pdevmode's dmorientation.
Call "gdi32.dll" "ResetDCA" with the printer canvas and the pdevmode.
Call "kernel32.dll" "GlobalUnlock" with the printer device mode handle.
Call "gdi32.dll" "SetGraphicsMode" with the printer canvas and 2 [gm_advanced].
Call "gdi32.dll" "SetBkMode" with the printer canvas and 1 [transparent].
Call "gdi32.dll" "SetMapMode" with the printer canvas and 8 [mm_anisotropic].
Call "gdi32.dll" "GetDeviceCaps" with the printer canvas and 112 [physicaloffsetx] returning a pair's x.
Call "gdi32.dll" "GetDeviceCaps" with the printer canvas and 113 [physicaloffsety] returning the pair's y.
Negate the pair.
Call "gdi32.dll" "SetViewportOrgEx" with the printer canvas and the pair's x and the pair's y and nil.
Call "gdi32.dll" "GetDeviceCaps" with the printer canvas and 88 [logpixelsx] returning the pair's x.
Call "gdi32.dll" "GetDeviceCaps" with the printer canvas and 90 [logpixelsy] returning the pair's y.
Call "gdi32.dll" "SetViewportExtEx" with the printer canvas and the pair's x and the pair's y and nil.
Call "gdi32.dll" "SetWindowOrgEx" with the printer canvas and 0 and 0 and nil.
Call "gdi32.dll" "SetWindowExtEx" with the printer canvas and the tpi and the tpi and nil.
Call "gdi32.dll" "StartPage" with the printer canvas.
Put the printer canvas into the current canvas.
Call "gdi32.dll" "GetDeviceCaps" with the printer canvas and 88 [logpixelsx] returning a number.
Put the tpp into the saved tpp.
Put the tpi divided by the number into the tpp.

To begin a sheet given a box (pdf style):
Begin the sheet given the box and "" (pdf style).

To begin a sheet given a box and a title string:
Begin the sheet given the box and the title (pdf style).

To begin a sheet given a box and a title string (pdf style - start the current page):
Add the pdf state's current page given "page".
Append the pdf state's current page's number then " 0 obj" to the pdf state's current page.
Append "<<" to the pdf state's current page.
Append "/Type /Page" to the pdf state's current page.
Append "/Parent " then the pdf state's parent's number then " 0 R" to the pdf state's current page.
Put the box's width minus the tpp times 72 / the tpi into a width.
Put the box's height minus the tpp times 72 / the tpi into a height.
Append "/MediaBox [0 0 " then the width then " " then the height then "]" to the pdf state's current page.
Put the box's height minus the tpp into the pdf state's current height.
Add the pdf state's current contents given "contents".
Append "/Contents " then the pdf state's current contents' number then " 0 R" to the pdf state's current page.
Append "0.05 0 0 0.05 1 1 cm" to the pdf state's current contents. \ set matrix to scale 72/1440
Append "13 w 0 J 0 j 0 i" to the pdf state's current contents. \ penwidth, linecap, linejoin, flatness \ 15 w on

penwidth comes out to wide

To begin a sheet given a box and a title string (pdf style):
Set the pdf state's page flag.
Put the clear color into the pdf state's current border.
Put the clear color into the pdf state's current fill.
Begin the sheet given the box and the title (pdf style - start the current page).
If the title is blank, exit.
Create a pdf outline entry given the title and the pdf state's current height and the pdf state's current page's number.
Append the pdf outline entry to the pdf state's outline entries.

To begin a sheet given a title string:
Begin a portrait sheet given the title.

The bell byte is a byte equal to 7.

The Bible is a thing with some verses.
\A verse is a thing with a book abbreviation, a chapter number, a verse number and a string.
A verse is a thing with a string.

The big-a byte is a byte equal to 65.

The big-a-acute byte is a byte equal to 193.

The big-a-circumflex byte is a byte equal to 194.

The big-a-diaeresis byte is a byte equal to 196.

The big-a-grave byte is a byte equal to 192.

The big-a-ring byte is a byte equal to 197.

The big-a-tilde byte is a byte equal to 195.

The big-ae byte is a byte equal to 198.

The big-b byte is a byte equal to 66.

The big-c byte is a byte equal to 67.

The big-c-cedilla byte is a byte equal to 199.

The big-d byte is a byte equal to 68.

The big-e byte is a byte equal to 69.

The big-e-acute byte is a byte equal to 201.

The big-e-circumflex byte is a byte equal to 202.

The big-e-diaeresis byte is a byte equal to 203.

The big-e-grave byte is a byte equal to 200.

A big-endian unsigned wyrd is a record with 2 bytes.

The big-eth byte is a byte equal to 208.

The big-f byte is a byte equal to 70.

The big-g byte is a byte equal to 71.

The big-h byte is a byte equal to 72.

The big-i byte is a byte equal to 73.

The big-i-acute byte is a byte equal to 205.

The big-i-circumflex byte is a byte equal to 206.

The big-i-diaeresis byte is a byte equal to 207.

The big-i-grave byte is a byte equal to 204.

The big-j byte is a byte equal to 74.

The big-k byte is a byte equal to 75.

The big-l byte is a byte equal to 76.

The big-m byte is a byte equal to 77.

The big-n byte is a byte equal to 78.

The big-n-tilde byte is a byte equal to 209.

The big-o byte is a byte equal to 79.

The big-o-acute byte is a byte equal to 211.

The big-o-circumflex byte is a byte equal to 212.

The big-o-diaeresis byte is a byte equal to 214.

The big-o-grave byte is a byte equal to 210.

The big-o-stroke byte is a byte equal to 216.

The big-o-tilde byte is a byte equal to 213.

The big-oe byte is a byte equal to 140.

The big-p byte is a byte equal to 80.

The big-q byte is a byte equal to 81.

The big-r byte is a byte equal to 82.

The big-s byte is a byte equal to 83.

The big-s-caron byte is a byte equal to 138.

The big-t byte is a byte equal to 84.

The big-thorn byte is a byte equal to 222.

The big-u byte is a byte equal to 85.

The big-u-acute byte is a byte equal to 218.

The big-u-circumflex byte is a byte equal to 219.

The big-u-diaeresis byte is a byte equal to 220.

The big-u-grave byte is a byte equal to 217.

The big-v byte is a byte equal to 86.

The big-w byte is a byte equal to 87.

The big-x byte is a byte equal to 88.

The big-y byte is a byte equal to 89.

The big-y-acute byte is a byte equal to 221.

The big-y-diaeresis byte is a byte equal to 159.

The big-z byte is a byte equal to 90.

The big-z-caron byte is a byte equal to 142.

A billion is 1000 millions.

A binary string is a string.

A bit is a unit.

A bitmapdata is a record with
A width,
A height,
A number called stride,
A number called pixelformat,
A pointer called scan0,
A number called reserved.

To bitwise and a byte with another byte:
Intel $8B850C000000. \ mov eax,[ebp+12] \ the other byte
Intel $8A00. \ mov al,[eax]
Intel $8B9D08000000. \ mov ebx,[ebp+8] \ the byte
Intel $2003. \ and [ebx],al

To bitwise and a byte with a number:
Intel $8B850C000000. \ mov eax,[ebp+12] \ the number

Intel $8B00. \ mov eax,[eax]
Intel $8B9D08000000. \ mov ebx,[ebp+8] \ the byte
Intel $2003. \ and [ebx],al

To bitwise and a number with another number:
Intel $8B850C000000. \ mov eax,[ebp+12] \ the other number
Intel $8B00. \ mov eax,[eax]
Intel $8B9D08000000. \ mov ebx,[ebp+8] \ the number
Intel $2103. \ and [ebx],eax

To bitwise or a byte with another byte:
Intel $8B850C000000. \ mov eax,[ebp+12] \ the other byte
Intel $8A00. \ mov al,[eax]
Intel $8B9D08000000. \ mov ebx,[ebp+8] \ the byte
Intel $0803. \ or [ebx],al

To bitwise or a byte with a number:
Intel $8B850C000000. \ mov eax,[ebp+12] \ the number
Intel $8B00. \ mov eax,[eax]
Intel $8B9D08000000. \ mov ebx,[ebp+8] \ the byte
Intel $0803. \ or [ebx],al

To bitwise or a number with another number:
Intel $8B850C000000. \ mov eax,[ebp+12] \ the other number
Intel $8B00. \ mov eax,[eax]
Intel $8B9D08000000. \ mov ebx,[ebp+8] \ the number
Intel $0903. \ or [ebx],eax

To bitwise xor a byte with another byte:
Intel $8B850C000000. \ mov eax,[ebp+12] \ the other byte
Intel $8A00. \ mov al,[eax]
Intel $8B9D08000000. \ mov ebx,[ebp+8] \ the byte
Intel $3003. \ xor [ebx],al

To bitwise xor a byte with a number:
Intel $8B850C000000. \ mov eax,[ebp+12] \ the number
Intel $8B00. \ mov eax,[eax]
Intel $8B9D08000000. \ mov ebx,[ebp+8] \ the byte
Intel $3003. \ or [ebx],al

To bitwise xor a number with another number:
Intel $8B850C000000. \ mov eax,[ebp+12] \ the other number
Intel $8B00. \ mov eax,[eax]
Intel $8B9D08000000. \ mov ebx,[ebp+8] \ the number
Intel $3103. \ xor [ebx],eax

The black color is a color.

The black pen is a pen.

The blue color is a color.

The blue pen is a pen.

A box has

A left coord, a top coord, a right coord, a bottom coord,
A left-top spot at the left, and a right-bottom spot at the right,
A top-left spot at the left, and a bottom-right spot at the right.

A brightness is a lightness.

The broken-bar byte is a byte equal to 166.

The brown color is a color.

The brown pen is a pen.

A brownish color is a color.

A bucket count is a count.

A bucket is a pointer to a bucket record.

A bucket record has some refers.

A bucket# is a number.

A buffer is a string.

The bullet byte is a byte equal to 149.

To bump a byte limiting it to another byte and a third byte:
Add 1 to the byte.
If the byte is greater than the third byte, put the other byte into the byte.

To bump a number: add 1 to the number.

To bump a number limiting it to another number and a third number:
Add 1 to the number.
If the number is greater than the third number, put the other number into the number.

To bump a rider:
Intel $8B9D08000000. \ mov ebx,[ebp+8] \ the rider
Intel $FF8314000000. \ inc [ebx+20] \ the rider’s token’s last
Intel $FF8308000000. \ inc [ebx+8] \ the rider’s source’s first

To bump a rider by a number:
Add the number to the rider’s token’s last.
Add the number to the rider’s source’s first.

To buzz:
Call "kernel32.dll" "Beep" with 220 and 200.

A byte count is a count.

A byte pointer is a pointer to a byte.

A byte# is a number.

The c-key is a key equal to 67.

The cancel byte is a byte equal to 24.

A canvas is an hdc.

To capitalize any selected rows in a text:
If the text is nil, exit.
Loop.
Get a row from the text's rows.
If the row is nil, exit.
If the row of the text is not selected, repeat.
If the row is blank, repeat.
Capitalize the row's string.
Repeat.

To capitalize a string:
Slap a substring on the string.
Loop.
If the substring is blank, exit.
If the substring's first's target is not noise, break.
Add 1 to the substring's first.
Repeat.
Uppercase the substring's first's target.

To capitalize a text:
If the text is nil, exit.
Loop.
Get a row from the text's rows.
If the row is nil, break.
Capitalize the row's string.
Repeat.
Wrap the text.

The caps key is a key equal to 20.

The caret byte is a byte equal to 94.

A caret is a position.

The cedilla byte is a byte equal to 184.

The cent-sign byte is a byte equal to 162.

To center a box at the bottom of another box:
Center the box in the other box (horizontally).
Put the box's height into a height.
Put the other box's bottom into the box's bottom.
Put the box's bottom minus the height into the box's top.

To center a box in another box (horizontally):
Put the other box's center's x minus the box's center's x into a number.
Round the number to the nearest multiple of the tpp.
Move the box given the number and 0.

To center a box in another box (vertically):

Put the other box's center's y minus the box's center's y into a number.
Round the number to the nearest multiple of the tpp.
Move the box given 0 and the number.

To center a dot on the current spot:
Center the dot on the context's spot.

To center an ellipse in a box (horizontally):
Center the ellipse's box in the box (horizontally).

To center an ellipse in a box (vertically):
Center the ellipse's box in the box (vertically).

To center a line in a box (horizontally):
Put the box's center's x minus the line's center's x into a number.
Round the number to the nearest multiple of the tpp.
Move the line given the number and 0.

To center a line in a box (vertically):
Put the box's center's y minus the line's center's y into a number.
Round the number to the nearest multiple of the tpp.
Move the line given 0 and the number.

To center a picture in a box (horizontally):
If the picture is nil, exit.
Put the box's center's x minus the picture's box's center's x into a number.
Round the number to the nearest multiple of the tpp.
Move the picture given the number and 0.

To center a picture in a box (vertically):
If the picture is nil, exit.
Put the box's center's y minus the picture's box's center's y into a number.
Round the number to the nearest multiple of the tpp.
Move the picture given 0 and the number.

To center a polygon in a box (horizontally):
If the polygon is nil, exit.
Put the box's center's x minus the polygon's box's center's x into a number.
Round the number to the nearest multiple of the tpp.
Move the polygon given the number and 0.

To center a polygon in a box (vertically):
If the polygon is nil, exit.
Put the box's center's y minus the polygon's box's center's y into a number.
Round the number to the nearest multiple of the tpp.
Move the polygon given 0 and the number.

To center a spot in a box (horizontally):
Put the box's center's x minus the spot's x into a number.
Round the number to the nearest multiple of the tpp.
Move the spot given the number and 0.

To center a spot in a box (vertically):
Put the box's center's y minus the spot's y into a number.
Round the number to the nearest multiple of the tpp.

Move the spot given 0 and the number.

To center a text in a box (horizontally):
If the text is nil, exit.
Put the box's center's x minus the text's box's center's x into a number.
Round the number to the nearest multiple of the tpp.
Move the text given the number and 0.

To center a text in a box (vertically):
If the text is nil, exit.
Put the box's center's y minus the text's box's center's y into a number.
Round the number to the nearest multiple of the tpp.
Move the text given 0 and the number.

To change the current hue by some points:
Change the context's hue by the points.

To change a hue by some points:
Add the points to the hue.

To change a roundy box given a radius:
Put the radius into the roundy box's radius.

To change a text given a box:
If the text is nil, exit.
Put the box into the text's box.
Wrap the text.

To change a text given a font height:
If the text is nil, exit.
Subtract the text's margin from the text's x.
Put the text's origin divided by the text's grid into a pair.
Put the font height into the text's font's height.
Scale the text's font's height given the text's scale.
Put the pair times the text's grid into the text's origin.
Add the text's margin to the text's x.
Limit the origin of the text.
Wrap the text.

To change a text given a font name:
If the text is nil, exit.
Put the font name into the text's font's name.
Wrap the text.

A character is a byte.

A choice is a thing with a name and a box.

The choices are some choices.

The circumflex byte is a byte equal to 136.

To clear a box:
\ Draw the box with the black color. \ like "Clear the screen"
Put 0 and 0 and 0 and 0 into the box. \ writer depends on this

```
To clear a byte:
Put the null byte into the byte.

To clear a color:
Put 0 and 0 and 0 into the color.

The clear color is a color.

To clear an ellipse:
Clear the ellipse's box.

To clear a flag:
Put no into the flag.

To clear a font:
Put "" and 0 into the font.

To clear a fraction:
Put 0 and 1 into the fraction.

To clear an ip address:
Clear the ip address' number.
Clear the ip address' string.

The clear key is a key equal to 12.

To clear a line:
Clear the line's start.
Clear the line's end.

To clear a number:
Put 0 into the number.

To clear a pair:
Put 0 and 0 into the pair.

The clear pen is a pen.

To clear a rider:
Clear the rider's original.
Clear the rider's source.
Clear the rider's token.

To erase the screen;
To blank out the screen;
To wipe off the screen;
To clear the screen:
Unmask everything.
Draw the screen's box with the black color and the black color.
Refresh the screen.
Put the screen's box into the context's box.

To clear the screen to a color:
Unmask everything.
```

Draw the screen’s box with the color and the color.
Refresh the screen.
Put the screen’s box into the context’s box.

To clear the screen to white:
Unmask everything.
Draw the screen’s box with the white color and the white color.
Refresh the screen.
Put the screen’s box into the context’s box.

To clear the screen without refreshing it:
Unmask everything.
Draw the screen’s box with the black color and the black color.
Put the screen’s box into the context’s box.

To clear a selection:
Clear the selection’s anchor.
Clear the selection’s caret.

To clear the stack:
Destroy the stack.

To clear a string:
Unassign the string’s first.
Void the string’s last.

To clear a substring:
Void the substring’s first.
Void the substring’s last.

To clear a terminal:
Destroy the terminal’s quoras.

To clear some things:
Void the things’ first.
Void the things’ last.

To clear a wyrd:
Put 0 into the wyrd.

To close a file:
Call "kernel32.dll" "CloseHandle" with the file.

A clsid is a uuid.

To cluck:
Play the cluck sound.

The cluck sound is a wave equal to
$524946463A02000057415645666D74201200000001000100401F0000401F00000100080000006661637404000000070200006461746107020000 7F7F807F7F807F7F808080807F807F7F80817F81817E7E827E7D847C79877D5F6D99B2A25D608269A5979869667F7D8D738C7D8C5E7E878F767A75868D84797278829A7D7E857A73929271657492907D7E7D787E8B887C758388827E817F7C7B89897C7A7E84838183827E7A8488877D7E8181808484817C7F84838181807E7F8283807E8081808182807F7F81818018007F7F8081818080807F7F808180807F80808080807F8080807F8080808080807F7F7F7F7F8080807F7

F7F7C7B8182817C7B7D7E8082807D7D7C7F8281807F7C7D808082807E7E7E7D7E807D7B7C7B7D7
D7B7A7979797875727269797A490F4571FFF4FF0C001297FBF492525BB0F5B26A001C69BEE5BA76
476F9EBD953C3757BAC9BB705C7F9AA883645D7897AA9B80697795958869697699 9B83766F828C9
07F7375828E91877B757D868B837A757C858D8880787B8187847B7A7D8788807B79808486817D7C7
F8384817D7C7F8484817B7B7D83847F7C7B7E8182827D7C7C8082817D7C7D7F81807F7D7D7F8281
7F7B7B7E8081807E7D7D7F80807E7D7D7E80807E7D7D7E7F807F7D7D7E7F807F7E7D7E7E80807F
7E7D7E80807F7E7E7E7F7F7E7E7E7F7F7F7F7E7E7F7F807F7F7E7E7E7F8080807E7E7E80807F7E7
E7F8081807F7F7E7F7F80807F7E7F8081807F7E7E7E80808080807F807F7F7F7F7F7F7F8080808080
7F7F7F808080807F80808180807F80808100.

The colon byte is a byte equal to 58.

A color has a hue, a saturation, a lightness, and a brightness at the lightness.

A colorref is a number [like $00BBGGRR].

A column# is a number.

The comma byte is a byte equal to 44.

To compare a string to another string given a length and another length (equal only):
Intel $8BB508000000. \ mov esi,[ebp+8] \ the string
Intel $8B36. \ mov esi,[esi] \ the string’s first
Intel $8BBD0C000000. \ mov edi,[ebp+12] \ the other string
Intel $8B3F. \ mov edi,[edi] \ the other string’s first
Intel $8B8510000000. \ mov eax,[ebp+16] \ the string’s length
Intel $8B00. \ mov eax,[eax]
Intel $8B9514000000. \ mov edx,[ebp+20] \ the other string’s length
Intel $8B12. \ mov edx,[edx]
Intel $3BD0. \ cmp eax,edx \ if the length’s differ, say no
Intel $0F8548000000. \ jne sayno
Intel $8BC8. \ mov ecx,eax \ put length into ecx
\L3: \ loop:
Intel $85C9. \ test ecx,ecx
Intel $0F8449000000. \ jz say yes
\ uppercase current byte in the string
Intel $8A1E. \ mov bl,[esi]
Intel $80FB61. \ cmp bl,’a’
Intel $0F820C000000. \ jb L4
Intel $80FB7A. \ cmp bl,’z’
Intel $0F8703000000. \ ja L4
Intel $80EB20. \ sub bl,$20
\L4: \ uppercase current byte in the other string
Intel $8A3F. \ mov bh,[edi]
Intel $80FF61. \ cmp bh,’a’
Intel $0F820C000000. \ jb L5
Intel $80FF7A. \ cmp bh,’z’
Intel $0F8703000000. \ ja L5
Intel $80EF20. \ sub bh,$20
\L5: \ compare the two uppercased bytes
Intel $3ADF. \ cmp bl,bh
Intel $0F8508000000. \ jne say no
Intel $46. \ inc esi
Intel $47. \ inc edi
Intel $49. \ dec ecx

Intel $E9BAFFFFFF. \ jmp L3
\SAY NO:
Intel $C7C000000000. \ mov eax,0
Intel $E906000000. \ jmp end
\SAY YES:
Intel $C7C001000000. \ mov eax,1

To compare a string to another string given a length and another length returning a number:
Intel $8BB508000000. \ mov esi,[ebp+8] \ the string
Intel $8B36. \ mov esi,[esi] \ the string's first
Intel $8BBD0C000000. \ mov edi,[ebp+12] \ the other string
Intel $8B3F. \ mov edi,[edi] \ the other string's first
Intel $8B8510000000. \ mov eax,[ebp+16] \ the string's length
Intel $8B00. \ mov eax,[eax]
Intel $8B9514000000. \ mov edx,[ebp+20] \ the other string's length
Intel $8B12. \ mov edx,[edx]
\ get the minimum length
Intel $8BC8. \ mov ecx,eax
Intel $3BCA. \ cmp ecx,edx
Intel $0F8602000000. \ jbe L2
Intel $8BCA. \ mov ecx,edx
\L3: \ loop:
Intel $85C9. \ test ecx,ecx
Intel $0F8444000000. \ jz L6
\ uppercase current byte in the string
Intel $8A1E. \ mov bl,[esi]
Intel $80FB61. \ cmp bl,'a'
Intel $0F820C000000. \ jb L4
Intel $80FB7A. \ cmp bl,'z'
Intel $0F8703000000. \ ja L4
Intel $80EB20. \ sub bl,$20
\L4: \ uppercase current byte in the other string
Intel $8A3F. \ mov bh,[edi]
Intel $80FF61. \ cmp bh,'a'
Intel $0F820C000000. \ jb L5
Intel $80FF7A. \ cmp bh,'z'
Intel $0F8703000000. \ ja L5
Intel $80EF20. \ sub bh,$20
\L5: \ compare the two uppercased bytes
Intel $3ADF. \ cmp bl,bh
Intel $0F8508000000. \ jne L5a
Intel $46. \ inc esi
Intel $47. \ inc edi
Intel $49. \ dec ecx
Intel $E9BAFFFFFF. \ jmp L3
\L5a: \ load bytes into eax and edx for final compare
Intel $0FB6C3. \ movzx eax,bl
Intel $0FB6D7. \ movzx edx,bh
\L6: \ subtract either the lengths or the last two bytes to set the eax to <0, =0, >0
Intel $2BC2. \ sub eax,edx
Intel $8B9D18000000. \ mov ebx,[ebp+24] \ the number
Intel $8903. \ mov [ebx],eax

To compatibly handle any message with a window a message number a w-param and an l-param:
If the message is 006, handle any wm-activate with the w-param; put 0 into eax; exit.

If the message is 258, handle any wm-char with the w-param and the l-param; put 0 into eax; exit.
If the message is 001, handle any wm-create with the window; put 0 into eax; exit.
If the message is 002, handle any wm-destroy; put 0 into eax; exit.
If the message is 256, handle any wm-keydown with the w-param and the l-param; put 0 into eax; exit.
If the message is 513, handle any wm-lbuttondown with the l-param; put 0 into eax; exit.
If the message is 515, handle any wm-lbuttondblclk with the l-param; put 0 into eax; exit.
If the message is 015, handle any wm-paint with the window; put 0 into eax; exit.
If the message is 516, handle any wm-rbuttondown with the l-param; put 0 into eax; exit.
If the message is 518, handle any wm-rbuttondblclk with the l-param; put 0 into eax; exit.
If the message is 032, handle any wm-setcursor; put 1 into eax; exit.
If the message is 260, handle any wm-syskeydown with the w-param and the l-param; put 0 into eax; exit.
Call "user32.dll" "DefWindowProcA" with the window and the message and the w-param and the l-param.

To compatibly wait for a process pointer:
Call "kernel32.dll" "WaitForSingleObject" with the process pointer's target and -1 [infinite].
Call "kernel32.dll" "CloseHandle" with the process pointer's target.
Put 0 into the process pointer's target.
Call "user32.dll" "GetForegroundWindow" returning a window.
If the window is the main window, put 0 into eax; exit.
Call "user32.dll" "ShowWindow" with the main window and 6 [sw_minimize].
Call "user32.dll" "ShowWindow" with the main window and 9 [sw_restore].
Put 0 into eax. \ set return value of thread

A console is a thing with
A box,
A border color,
A fill color,
A text,
A grid,
A reply string.

The context is a context.

A context is a thing with a spot, a box, a heading, a letter height, a color, a number, \ pen width? ***
And a letter size at the letter height, and a pen at the color.

The context stack is some contexts.

To convert an absolute position to a position given a text:
If the text is nil, clear the position; exit.
Privatize the absolute position.
Loop.
Get a row from the text's rows.
If the row is nil, clear the position; exit.
Put the row's row# into the position's row#.
Put the absolute position into the position's column#.
Subtract the row's string's length from the absolute position.
If the absolute position is less than 1, exit.
Repeat.

To convert a binary string into a number:
Put 0 into the number.
Put 1 into a value number.
Loop.

If the binary string is blank, exit.
Get a character from the binary string (backwards). \ was backwards
If the character is "1", add the value to the number.
Double the value.
Repeat.

To convert a box to a string:
Clear the string.
Append the box's left to the string.
Append " " to the string.
Append the box's top to the string.
Append " " to the string.
Append the box's right to the string.
Append " " to the string.
Append the box's bottom to the string.

To convert a byte to a nibble:
Put the byte into the nibble as a byte.
Uppercase the nibble.
If the nibble is greater than the nine byte, subtract 7 from the nibble.
Subtract 48 from the nibble.

To convert a byte to a nibble string:
Split the byte into a nibble and another nibble.
Convert the nibble to the nibble string.
Convert the other nibble to another nibble string.
Append the other nibble string to the nibble string.

To convert a byte to a query byte:
If the byte is between 48 and 57, put the byte into the query byte; exit. \ 0-9
If the byte is between 65 and 90, put the byte into the query byte; exit. \ A-Z
If the byte is between 97 and 122, put the byte into the query byte; exit. \ a-z
If the byte is 32, put "+" into the query byte; exit. \ space
Convert the byte to a nibble string.
Put "%" then the nibble string into the query byte.

To convert a color to a colorref:
If the color is clear, put 16777215 [$00FFFFFF] into the colorref; exit. \ clear pen becomes white
Privatize the color.
Scale the color's saturation given 240/1000.
Limit the color's saturation to 1 and 239.
Scale the color's lightness given 240/1000.
Limit the color's lightness to 1 and 239.
Scale the color's hue given 240/3600.
Limit the color's hue to 1 and 239.
Call "shlwapi.dll" "ColorHLSToRGB" with the color's hue and the color's lightness and the color's saturation returning the colorref.

To convert a color to a rgb:
Convert the color to a colorref.
Convert the colorref to the rgb.

To convert a colorref to a color:
Call "shlwapi.dll" "ColorRGBToHLS" with the colorref and a wyrd's whereabouts and another wyrd's whereabouts and a third wyrd's whereabouts.

Put the wyrd into the color's hue.
Put the other wyrd into the color's lightness.
Put the third wyrd into the color's saturation.
Scale the color's hue given 3600/240.
Limit the color's hue to 0 and 3600.
Scale the color's saturation given 1000/240.
Limit the color's saturation to 0 and 1000.
Scale the color's lightness given 1000/240.
Limit the color's lightness to 0 and 1000.

To convert a colorref to a rgb:
Privatize the colorref.
Shift the colorref right 0 bits.
Put the colorref into the rgb's red byte.
Shift the colorref right 8 bits.
Put the colorref into the rgb's green byte.
Shift the colorref right 8 bits.
Put the colorref into the rgb's blue byte.

To convert a flag to a hex string:
Reassign the hex string's first given the flag's magnitude.
Copy bytes from the flag's whereabouts to the hex string's first for the flag's magnitude.
Put the hex string's first plus the flag's magnitude minus 1 into the hex string's last.

To convert a font to an hfont:
Privatize the font.
Null terminate the font's name.
Call "gdi32.dll" "CreateFontA" with - the font's height times 3 divided by 4 and 0 and 0 and 0 and 0 and 0 and 0 and 0
And 1 [default_charset] and 0 and 0 and 5 [cleartype_quality] and 4 [truetype_fonttype] and the font's name's first returning the hfont.

To convert a font info to pdf em units:
If the font info is nil, exit.
Convert the font info's internal leading to pdf em units given the font info's emsquare and the font info's font.
Convert the font info's ascent to pdf em units given the font info's emsquare and the font info's font.
Convert the font info's descent to pdf em units given the font info's emsquare and the font info's font.
Convert the font info's capheight to pdf em units given the font info's emsquare and the font info's font.
\ convert the font info's italicangle to pdf em units given the font info's emsquare and the font info's font.
Convert the font info's stemv to pdf em units given the font info's emsquare and the font info's font.
Convert the font info's font box's left to pdf em units given the font info's emsquare and the font info's font.
Convert the font info's font box's top to pdf em units given the font info's emsquare and the font info's font.
Convert the font info's font box's right to pdf em units given the font info's emsquare and the font info's font.
Convert the font info's font box's bottom to pdf em units given the font info's emsquare and the font info's font.
Swap the font info's font box's top with the font info's font box's bottom.
Convert the font info's font widths to pdf em units.

To convert some font widths to pdf em units:
Get an outlinetextmetric given the font widths' font.
Put the font widths' data into a number pointer.

Loop.
If a counter is past the font widths' count, break.
Convert the number pointer's target to pdf em units given the outlinetextmetric's otmemsquare and the font widths' font.
Add a number's magnitude to the number pointer.
Repeat.

To convert a fraction to a hex string:
Reassign the hex string's first given the fraction's magnitude.
Copy bytes from the fraction's whereabouts to the hex string's first for the fraction's magnitude.
Put the hex string's first plus the fraction's magnitude minus 1 into the hex string's last.

To convert a fraction to a mixed:
If the fraction's denominator is 0, put 0 into the mixed's whole number; put 0 into the mixed's ratio; exit.
Divide the fraction's numerator by the fraction's denominator giving the mixed's whole number and a remainder.
Put the remainder and the fraction's denominator into the mixed's fraction.

To convert a fraction to a string given a number: \ converts to a decimal representation with "the number" of places
Clear the string.
If the number is less than 1, exit.
Put 10 into a value number.
Raise the value to the number.
Privatize the fraction.
If the fraction is negative, set a flag; de-sign the fraction.
Multiply the value by the fraction.
Zero fill the value given the number and append it to the string.
Put the string's length minus the number plus 1 into a byte#.
Insert "." into the string before the byte#.
If the string's first's target is the period byte, prepend "0" to the string.
If the flag is set, prepend "-" to the string.

To convert a gpbitmap to a buffer (pdf style):
Clear the buffer.
If the gpbitmap is nil, exit.
Lock the gpbitmap given a bitmapdata (24-bit rgb).
Put 1 into a row#.
Put 1 into a column#.
Loop.
If the column# is greater than the bitmapdata's width, put 1 into the column#; add 1 to the row#.
If the row# is greater than the bitmapdata's height, break.
Get a rgb pointer from the bitmapdata at the row# and the column#.
Append the rgb pointer's red byte to the buffer.
Append the rgb pointer's green byte to the buffer.
Append the rgb pointer's blue byte to the buffer.
Add 1 to the column#.
Repeat.
Unlock the gpbitmap given the bitmapdata.

To convert a hex string to a fraction:
If the hex string's length is not the fraction's magnitude, put 0 and 1 into the ratio; exit.
Copy bytes from the hex string's first to the fraction's whereabouts for the fraction's magnitude.

To convert a hex string to a number:

If the hex string's length is not the number's magnitude, clear the number; exit.
Copy bytes from the hex string's first to the number's whereabouts for the number's magnitude.

To convert an l-param to a key: \ assumes l-param from wm_char message
Put the l-param into the key.
Shift the key right 16 bits.
Bitwise and the key with 255.
Call "user32.dll" "MapVirtualKeyA" with the key and 1 returning the key.
If the numlock key was not toggled, exit.
If the key is the insert key, put the numpad-zero key into the key; exit.
If the key is the delete key, put the numpad-period key into the key; exit.
If the key is the home key, put the numpad-seven key into the key; exit.
If the key is the end key, put the numpad-one key into the key; exit.
If the key is the page-up key, put the numpad-nine key into the key; exit.
If the key is the page-down key, put the numpad-three key into the key; exit.
If the key is the left-arrow key, put the numpad-four key into the key; exit.
If the key is the up-arrow key, put the numpad-eight key into the key; exit.
If the key is the right-arrow key, put the numpad-six key into the key; exit.
If the key is the down-arrow key, put the numpad-two key into the key; exit.
If the key is the clear key, put the numpad-five key into the key; exit.

To convert an l-param to a spot:
Split the l-param into a wyrd and another wyrd.
Put the wyrd into the spot's y.
Put the other wyrd into the spot's x.
Multiply the spot by the tpp.

To convert a mixed to a fraction:
Put the mixed's fraction into the fraction.
Add the mixed's whole number times the fraction's denominator to the fraction's numerator.

To convert a nibble to a nibble string:
Privatize the nibble.
If the nibble is greater than 9, add 7 to the nibble.
Add 48 to the nibble.
Put the nibble into the nibble string.

To convert a nibble string to a hex string:
Privatize the nibble string.
Clear the hex string.
If the nibble string's length is odd, prepend the zero byte to the nibble string.
Slap a substring on the nibble string.
Loop.
If the substring is blank, exit.
Convert the substring's first's target to a nibble.
Shift the nibble left 4 bits.
Add 1 to the substring's first.
Convert the substring's first's target to another nibble.
Bitwise or the nibble with the other nibble.
Append the nibble to the hex string.
Add 1 to the substring's first.
Repeat.

To convert a number to a byte:
Put the number into the byte.

To convert a number to pdf em units given an emsquare number and a font:
Multiply the number by the emsquare / the font's adjusted height.
Multiply the number by 1000 / the emsquare.

To convert a pchar to a string:
Clear the string.
If the pchar is nil, exit.
Privatize the pchar.
Loop.
If the pchar's target is the null byte, exit.
Append the pchar's target to the string.
Add 1 to the pchar.
Repeat.

To convert a pointer and a length to a string:
Clear the string.
If the pointer is nil, exit.
If the length is 0, exit.
Reassign the string's first given the length.
Copy bytes from the pointer to the string's first for the length.
Put the string's first plus the length minus 1 into the string's last.

To convert a pointer to a hex string;
To convert a number to a hex string:
Reassign the hex string's first given the number's magnitude.
Copy bytes from the number's whereabouts to the hex string's first for the number's magnitude.
Put the hex string's first plus the number's magnitude minus 1 into the hex string's last.

To convert a pointer to a nibble string;
To convert a number to a nibble string:
Split the number into a wyrd and another wyrd.
Convert the wyrd to the nibble string.
Convert the other wyrd to another nibble string.
Append the other nibble string to the nibble string.

To convert a pointer to a string:
Convert the pointer to the string as a nibble string.

To convert some points to some precise degrees:
Put the points times 3840 divided by 3600 into the precise degrees.

To convert a position to an absolute position given a text:
If the text is nil, clear the absolute position; exit.
Put 0 into the absolute position.
Loop.
Get a row from the text's rows.
If the row is nil, exit.
If the row's row# is less than the position's row#, add the row's string's length to the absolute position; repeat.
Add the position's column# to the absolute position.

To convert some precise degrees to some points:
Put the precise degrees times 3600 divided by 3840 into the points.

To convert a query string to a string:
Clear the string.
If the query string is blank, exit.
Slap a substring on the query string.
Loop.
If the substring is blank, exit.
If the substring's first's target is the cross byte, append " " to the string; add 1 to the substring's first; repeat.
If the substring's first's target is not the percent-sign byte, append the substring's first's target to the string; add 1 to the substring's first; repeat.
If the substring's length is less than 3, exit.
Add 1 to the substring's first.
Convert the substring's first's target to a nibble.
Shift the nibble left 4 bits.
Add 1 to the substring's first.
Convert the substring's first's target to another nibble.
Bitwise or the nibble with the other nibble.
Add 1 to the substring's first.
Append the nibble to the string.
Repeat.

To convert some rows to a string:
Clear the string.
Loop.
Get a row from the rows.
If the row is nil, exit.
Append the row's string to the string.
If the string's last's target is the return byte, append the linefeed byte to the string; repeat.
Repeat.

To convert some rows to a string (no linefeed additions):
Clear the string.
Loop.
Get a row from the rows.
If the row is nil, exit.
Append the row's string to the string.
Repeat.

To convert a string and an actual font info and an original font info into a buffer for pdf output:
Clear the buffer.
Put 0 into a current byte#.
Slap a substring on the first byte of the string.
Loop.
If the substring's first is greater than the string's last, break.
If the substring's last is the string's last, break.
Get a current width given the current byte# and the actual font info's font widths.
Get an original width given the substring's last's target and the original font info's font widths.
Put the original width minus the current width into an offset.
If the offset is 0, add 1 to the substring's last; add 1 to the current byte#; repeat.
Convert the substring to a pdf string.
Append the pdf string then " " then the offset then " " to the buffer.
Put the substring's last plus 1 into the substring's last.
Put the substring's last into the substring's first.
Add 1 to the current byte#.
Repeat.

If the substring’s first is greater than the string’s last, exit.
Convert the substring to another pdf string.
Append the other pdf string to the buffer.

To convert a string to a flag:
If the string is "y", set the flag; exit.
If the string is "yes", set the flag; exit.
Clear the flag.

To convert a string to a fraction:
Clear the fraction.
If the string is blank, exit.
If the string is any integer, convert the string to the fraction’s numerator; exit.
Slap a substring on the string.
If the substring’s first’s target is any sign, add 1 to the substring’s first.
If the substring is blank, exit.
Split the substring into an integer substring and a fraction substring given the dash byte.
If the integer substring is blank, put the substring into the fraction substring.
Split the fraction substring into a numerator substring and a denominator substring given the slash byte.
Convert the integer substring to a whole number.
Convert the numerator substring to a numerator number.
Convert the denominator substring to a denominator number.
If the whole number is negative, exit.
If the numerator number is negative, exit.
If the denominator number is negative, exit.
If the denominator number is 0, exit.
If the whole number is not 0, add the denominator number times the whole number to the numerator number.
Put the numerator number into the fraction’s numerator.
Put the denominator number into the fraction’s denominator.
If the string’s first’s target is the dash byte, negate the fraction.

To convert a string to a nibble string:
Clear the nibble string.
Slap a substring on the string.
Loop.
If the substring is blank, exit.
Convert the substring’s first’s target to another nibble string.
Append the other nibble string to the nibble string.
Add 1 to the substring’s first.
Repeat.

To convert a string to a number:
Put 0 into the number.
Slap a substring on the string.
If the substring is blank, exit.
If the substring’s first’s target is any sign, add 1 to the substring’s first.
Loop.
If the substring is blank, break.
Multiply the number by 10.
Put the substring’s first’s target into another number.
Subtract 48 from the other number.
Add the other number to the number.
Add 1 to the substring’s first.
Repeat.

If the string’s first’s target is the dash byte, negate the number.

To convert a string to a number between another number and a third number:
Convert the string to the number.
Limit the number to the other number and the third number.

To convert a string to a pdf string:
Put "(" into the pdf string.
Slap a substring on the string.
Subtract 1 from the substring’s first.
Loop.
Add 1 to the substring’s first.
If the substring is blank, break.
If the substring’s first’s target is the left-paren byte, append "\(" to the pdf string; repeat.
If the substring’s first’s target is the right-paren byte, append "\)" to the pdf string; repeat.
If the substring’s first’s target is the backslash byte, append "\\" to the pdf string; repeat.
Append the substring’s first’s target to the pdf string.
Repeat.
Append ")" to the pdf string.

To convert a string to a pointer: \ assumes pointer is in nibble format
Convert the string as a nibble string to a hex string.
Void the pointer.
Slap a substring on the hex string.
Put 24 into a shift count.
Loop.
If the substring is blank, exit.
If the shift count is less than 0, exit.
Put the substring’s first’s target into a number.
Shift the number left the shift count.
Bitwise or the pointer as a number with the number.
Add 1 to the substring’s first.
Subtract 8 from the shift count.
Repeat.

To convert a string to a query string:
Clear the query string.
Slap a substring on the string.
Loop.
If the substring is blank, break.
Convert the substring’s first’s target to a query byte.
Append the query byte to the query string.
Add 1 to the substring’s first.
Repeat.

To convert a string to some rows:
Slap a rider on the string.
Loop.
Move the rider (text file rules).
If the rider’s token is blank, break.
Create a row given the rider’s token.
Append the row to the rows.
If the row’s string’s last’s target is the linefeed byte, put the return byte into the row’s string’s last’s target. \ *dahn new to handle lines terminated by just linefeed
Repeat.

Renumber the rows.

To convert a string to a uuid:
Convert the string to a wide string and null terminate.
Call "ole32.dll" "CLSIDFromString" with the wide string's first and the uuid's whereabouts.

To convert a string to a wide string:
Clear the wide string.
Slap a substring on the string.
Loop.
If the substring is blank, exit.
Append the substring's first's target to the wide string.
Append the null byte to the wide string.
Add 1 to the substring's first.
Repeat.

To convert a string to a wide string and null terminate:
Convert the string to the wide string.
Null terminate the wide string.

To convert a url to a url record:
Privatize the url.
Null terminate the url.
Put a urlcomponents' magnitude into the urlcomponents' dwstructsize.
Put 1 into the urlcomponents' dwschemelength.
Put 1 into the urlcomponents' dwhostnamelength.
Put 1 into the urlcomponents' dwurlpathlength.
Put 1 into the urlcomponents' dwextrainfolength.
Call "wininet.dll" "InternetCrackUrlA" with the url's first and 0 and 0 and the urlcomponents' whereabouts returning a number.
Convert the urlcomponents' lpszscheme and the urlcomponents' dwschemelength to the url record's scheme.
Convert the urlcomponents' lpszhostname and the urlcomponents' dwhostnamelength to the url record's host name.
Convert the urlcomponents' lpszurlpath and the urlcomponents' dwurlpathlength to the url record's path.
Convert the urlcomponents' lpszextrainfo and the urlcomponents' dwextrainfolength to the url record's extra.
Put the urlcomponents' nport into the url record's port number.

To convert a wyrd to a nibble string:
Split the wyrd into a byte and another byte.
Convert the byte to the nibble string.
Convert the other byte to another nibble string.
Append the other nibble string to the nibble string.

A coord is some twips.

To copy bytes from a pointer to another pointer for a byte count: \ copy handling overlap with 1 byte moves
Intel $8BB508000000. \ mov esi,[ebp+8] \ the pointer
Intel $8B36. \ mov esi,[esi]
Intel $8BBD0C000000. \ mov edi,[ebp+12] \ the other pointer
Intel $8B3F. \ mov edi,[edi]
Intel $8B8D10000000. \ mov ecx,[ebp+16] \ the number
Intel $8B09. \ mov ecx,[ecx]

```
\ check for something to copy
Intel $81F900000000. \ cmp ecx,0
Intel $0F8E39000000. \ jle end
\ check for no overlap
Intel $3BF7. \ cmp esi,edi
Intel $0F8D24000000. \ jge forward
Intel $8BC6. \ mov eax,esi
Intel $03C1. \ add eax,ecx
Intel $3BC7. \ cmp eax,edi
Intel $0F8E18000000. \ jle forward
\ copy backward
Intel $03F1. \ add esi,ecx
Intel $4E. \ dec esi
Intel $03F9. \ add edi,ecx
Intel $4F. \ dec esi
\ backward
Intel $8A16. \ mov dl,[esi]
Intel $8817. \ mov [edi],dl
Intel $4E. \ dec esi
Intel $4F. \ dec edi
Intel $49. \ dec ecx
Intel $0F85F3FFFFFF. \ jnz backward
Intel $E90D000000. \ jmp end
\ forward: copy forward
Intel $8A16. \ mov dl,[esi]
Intel $8817. \ mov [edi],dl
Intel $46. \ inc esi
Intel $47. \ inc edi
Intel $49. \ dec ecx
Intel $0F85F3FFFFFF. \ jnz forward
```

To copy an event into another event:
If the event is nil, void the other event; exit.
Create the other event.
Put the event's kind into the other event's kind.
Put the event's shift flag into the other event's shift flag.
Put the event's ctrl flag into the other event's ctrl flag.
Put the event's alt flag into the other event's alt flag.
Put the event's spot into the other event's spot.
Put the event's key into the other event's key.
Put the event's byte into the other event's byte.

To copy a gpbitmap into another gpbitmap:
If the gpbitmap is nil, void the other gpbitmap; exit.
Call "gdiplus.dll" "GdipCloneBitmapAreaI" with 0 and 0 and the gpbitmap's width and the gpbitmap's height and 0 [pixelformatdontcare] and
The gpbitmap and the other gpbitmap's whereabouts.

To copy the guts of a text into another text:
If the text is nil, exit.
If the other text is nil, exit.
Put the text's box into the other text's box.
Put the text's origin into the other text's origin.
Put the text's pen into the other text's pen.
Put the text's font into the other text's font.

Put the text's alignment into the other text's alignment.
Copy the text's rows into the other text's rows.
Put the text's margin into the other text's margin.
Put the text's scale into the other text's scale.
Put the text's wrap flag into the other text's wrap flag.
Put the text's horizontal scroll flag into the other text's horizontal scroll flag.
Put the text's vertical scroll flag into the other text's vertical scroll flag.
Put the text's selection into the other text's selection.
Put the text's modified flag into the other text's modified flag.
Put the text's last operation into the other text's last operation.
\ don't copy undos and redos

A copy is a number.

To copy a picture into another picture:
If the picture is nil, void the other picture; exit.
Allocate memory for the other picture.
Put the picture's box into the other picture's box.
Put the picture's uncropped box into the other picture's uncropped box.
Put the picture's grayscale flag into the other picture's grayscale flag.
Put the picture's mirror flag into the other picture's mirror flag.
Put the picture's rotate angle into the other picture's rotate angle.
Put the picture's data into the other picture's data.
Copy the picture's gpbitmap into the other picture's gpbitmap.

To copy a polygon into another polygon:
If the polygon is nil, void the other polygon; exit.
Allocate memory for the other polygon.
Copy the polygon's vertices into the other polygon's vertices.

To copy a row into another row:
If the row is nil, void the other row; exit.
Allocate memory for the other row.
Put the row's row# into the other row's row#.
Put the row's string into the other row's string.

To copy some rows into some other rows:
Destroy the other rows.
Loop.
Get a row from the rows.
If the row is nil, exit.
Copy the row into another row.
Append the other row to the other rows.
Repeat.

To copy a text into another text:
If the text is nil, void the other text; exit.
Allocate memory for the other text.
Copy the guts of the text into the other text.

To copy a vertex into another vertex:
If the vertex is nil, void the other vertex; exit.
Allocate memory for the other vertex.
Put the vertex's x into the other vertex's x.
Put the vertex's y into the other vertex's y.

To copy some vertices into some other vertices:
Destroy the other vertices.
Loop.
Get a vertex from the vertices.
If the vertex is nil, exit.
Copy the vertex into another vertex.
Append the other vertex to the other vertices.
Repeat.

The copyright byte is a byte equal to 169.

A count is a number.

A counter is an number.

To create the connect handle of a winhttp request using a url record:
If the winhttp request is nil, exit.
Convert the url record's host name into a wide string called wide host name and null terminate.
Call "winhttp.dll" "WinHttpConnect"
With the winhttp request's session
And the wide host name's first
And the url record's port
And 0
Returning the winhttp request's connection.
If the winhttp request's connection is 0, put "Could not connect." into the i/o error; exit.

To create a console:
Allocate memory for the console.
Put the lighter gray color into the console's border.
Put the lighter gray color into the console's fill.
Put the screen's box into the console's box.
Put the screen's box into a box.
Subtract the default font's height from the box's bottom.
Put the box's height divided by the default font's height times the default font's height into a height.
Put the box's top plus the height into the box's bottom.
Center the box in the screen's box (vertically).
Put the box's top into the box's left.
Subtract the box's top from the box's right.
Create the console's text.
Put the box into the console's text's box.
Set the console's text's wrap flag.
Clear the console's text's horizontal scroll flag.
Set the console's text's vertical scroll flag.
Put the default font's height into the console's grid.
Multiply the console's grid's x by 2.

To create a crypt session with a passphrase string: \ sets i/o error if failure
Clear the i/o error.
Allocate memory for the crypt session.
\ acquire context
Call "advapi32.dll" "CryptAcquireContextA" with the crypt session's hcryptprov's whereabouts and 0 and "Microsoft Enhanced Cryptographic Provider v1.0"'s first
And 1 [prov_rsa_full] and -268435456 [crypt_verifycontext] returning a result number.
If the result is 0, put "Could not acquire context." into the i/o error; destroy the crypt session; exit.

\ create hash
Call "advapi32.dll" "CryptCreateHash" with the crypt session's hcryptprov and 32771 [calg_md5] and 0 and 0
And the crypt session's hcrypthash's whereabouts returning the result number.
If the result is 0, put "Could not create hash." into the i/o error; destroy the crypt session; exit.
\ hash passphrase
Call "advapi32.dll" "CryptHashData" with the crypt session's hcrypthash and the passphrase's first and the passphrase's length and 0 returning the result number.
If the result is 0, put "Could not hash password." into the i/o error; destroy the crypt session; exit.
\ derive session key
Call "advapi32.dll" "CryptDeriveKey" with the crypt session's hcryptprov and 26625 [calg_rc4 stream cipher] and the crypt session's hcrypthash
And 8388608 [128 bit] and the crypt session's hcryptkey's whereabouts returning the result number.
If the result is 0, put "Could not derive session key." into the i/o error; destroy the crypt session; exit.

To create a dyad:
Allocate memory for the dyad.

To create an event:
Allocate memory for the event.

To create a font info given a font:
Create the font info given the font (basic data).
Create the font info's font widths given the font.

To create a font info given a font (basic data):
Allocate memory for the font info.
Put the font into the font info's font.
Get an outlinetextmetric given the font.
Put 32 into the font info's flags. \ could be updated with a more information
Put the outlinetextmetric's otmtextmetrics' tminternalleading into the font info's internal leading.
Put the outlinetextmetric's otmemsquare into the font info's emsquare.
Put the outlinetextmetric's otmtextmetrics' tmascent into the font info's ascent.
Put - the outlinetextmetric's otmtextmetrics' tmdescent into the font info's descent.
Put the outlinetextmetric's otmscapemheight into the font info's capheight.
Put the outlinetextmetric's otmitalicangle into the font info's italicangle.
Put 0 into the font info's stemv. \ don't know where to get this from
Put the outlinetextmetric's otmrcfontbox into the font info's font box.

To create a font info given a font and a string: \ creates widths based on characters in string
Create the font info given the font (basic data).
Create the font info's font widths given the font and the string.

To create some font widths given a font:
Allocate memory for the font widths.
Put the font into the font widths' font.
Put 256 into the font widths' count.
Assign the font widths' data given the font widths' count times a number's magnitude.
Create the hfont of the memory canvas given the font.
Assign an original abc pointer given 256 times an abc's magnitude.
Call "gdi32.dll" "GetCharABCWidthsA" with the memory canvas and 0 and 255 and the original abc pointer.
Destroy the hfont of the memory canvas.
Put the original abc pointer into an abc pointer.
Put the font widths' data into a number pointer.

Loop.
If a counter is past 256, break.
Put the abc pointer's abca into the number pointer's target.
Add the abc pointer's abcb to the number pointer's target.
Add the abc pointer's abcc to the number pointer's target.
Add the abc's magnitude to the abc pointer.
Add the number's magnitude to the number pointer.
Repeat.
Unassign the original abc pointer.

To create some font widths given a font and a string:
Allocate memory for the font widths.
Put the font into the font widths' font.
Put the string's length into the font widths' count.
If the string is blank, exit.
Put a gcpresults' magnitude into the gcpresults' lstructsize.
Put the string's length into the gcpresults' nglyphs.
Assign the gcpresults' lpdx given the string's length times a number's magnitude.
Create the hfont of the memory canvas given the font.
Call "gdi32.dll" "GetCharacterPlacementA" with the memory canvas and the string's first
And the string's length and 0 and the gcpresults' whereabouts and 0.
Destroy the hfont of the memory canvas.
Put the gcpresults' lpdx into the font widths' data.

To create a gpbitmap given a buffer:
Clear the i/o error.
Call "kernel32.dll" "GlobalAlloc" with 2 [gmem_moveable] and the buffer's length returning a handle.
Call "kernel32.dll" "GlobalLock" with the handle returning a pointer.
Copy bytes from the buffer's first to the pointer for the buffer's length.
Call "kernel32.dll" "GlobalUnlock" with the handle.
Call "ole32.dll" "CreateStreamOnHGlobal" with the handle and 1 [true] and an istream's whereabouts.
Call "gdiplus.dll" "GdipCreateBitmapFromStream" with the istream and the gpbitmap's whereabouts returning a number.
If the number is not 0, put "I don't know how to process this kind of picture." into the i/o error; void the gpbitmap.
Call the istream's vtable's release with the istream.

To create a gpimageattributes (grayscale):
Call "gdiplus.dll" "GdipCreateImageAttributes" with the gpimageattributes' whereabouts.
Call "gdiplus.dll" "GdipSetImageAttributesColorMatrix" with the gpimageattributes and 0 [coloradjusttypedefault] and 1
And the grayscale color matrix's first and 0 and 0 [colormatrixflagsdefault].

To create the hbrush of a canvas given a color:
Convert the color to a colorref.
If the color is clear, put the null hbrush into an hbrush.
If the color is not clear, call "gdi32.dll" "CreateSolidBrush" with the colorref returning the hbrush.
Call "gdi32.dll" "SelectObject" with the canvas and the hbrush.

To create the hfont of a canvas given a font:
Convert the font to an hfont.
Call "gdi32.dll" "SelectObject" with the canvas and the hfont.

To create the hpen of a canvas given a color:
Convert the color to a colorref.

If the color is clear, put the null hpen into an hpen.
Put the tpp times the pen size into a number.
If the canvas is the printer canvas, put 1/96 inch times the pen size into the number.
If the color is not clear, call "gdi32.dll" "CreatePen" with 0 [ps_solid] and the number and the colorref returning the hpen.
Call "gdi32.dll" "SelectObject" with the canvas and the hpen.

To create an hrgn given a box:
Privatize the box.
Add the tpp to the box's right-bottom.
Call "gdi32.dll" "BeginPath" with the current canvas.
Call "gdi32.dll" "Rectangle" with the current canvas and the box's left and the box's top and the box's right and the box's bottom.
Call "gdi32.dll" "EndPath" with the current canvas.
Call "gdi32.dll" "PathToRegion" with the current canvas returning the hrgn.

To create an hrgn given an ellipse:
Put the ellipse's box into a box.
Call "gdi32.dll" "BeginPath" with the current canvas.
Call "gdi32.dll" "Ellipse" with the current canvas and the box's left and the box's top and the box's right and the box's bottom.
Call "gdi32.dll" "EndPath" with the current canvas.
Call "gdi32.dll" "PathToRegion" with the current canvas returning the hrgn.

To create an hrgn given a polygon:
If the polygon is nil, put 0 into the hrgn; exit.
Create a vertex array given the polygon's vertices.
Call "gdi32.dll" "LPtoDP" with the current canvas and the vertex array's spot pointer and the vertex array's count.
Call "gdi32.dll" "CreatePolygonRgn" with the vertex array's spot pointer and the vertex array's count and 2 [winding] returning the hrgn.
Destroy the vertex array.

To create an hrgn given a roundy box:
If the roundy box's radius is 0, create the hrgn given the roundy box as a box; exit.
Put the roundy box into a box.
Put the roundy box's radius times 2 into a diameter number.
Call "gdi32.dll" "BeginPath" with the current canvas.
Call "gdi32.dll" "RoundRect" with the current canvas and the box's left and the box's top and the box's right and the box's bottom and the diameter and the diameter.
Call "gdi32.dll" "EndPath" with the current canvas.
Call "gdi32.dll" "PathToRegion" with the current canvas returning the hrgn.

To create an index given a bucket count:
Allocate memory for the index.
Put the bucket count into the index's bucket count.
Put a bucket record's magnitude into a width.
Put the index's bucket count times the width into a number.
Assign the index's first bucket given the number.
Put the index's first bucket plus the number minus the width into the index's last bucket.

To create the lexicon:
Allocate memory for the lexicon.
Create the lexicon's index given 4027.

To create the open handle of a winhttp request:
If the winhttp request is nil, exit.
Convert the module's name to a wide string called wide module name and null terminate.
Call "winhttp.dll" "WinHttpOpen"
With the wide module name's first
And 0 [winhttp_access_type_default_proxy]
And 0 [winhttp_no_proxy_name]
And 0 [winhttp_no_proxy_bypass]
And 0
Returning the winhttp request's session.
If the winhttp request's session is 0, put "Could not open connection." into the i/o error; exit.

To create a path in the file system:
If the path is directory-format, create the path in the file system (directory); exit.
If the path is file-format, create the path in the file system (file); exit.

To create a path in the file system (directory):
Privatize the path.
Remove any trailing backslash from the path.
Null terminate the path.
Call "kernel32.dll" "CreateDirectoryA" with the path's first and 0 returning a number.
Clear the i/o error.
If the number is not 0, exit.
Put "Error creating directory '" then the path then "'." into the i/o error.

To create a path in the file system (file):
Privatize the path.
Null terminate the path.
Call "kernel32.dll" "CreateFileA" with the path's first and 1073741824 [generic_write] and 0 and 0 and 1 [create_new] and 128 [file_attribute_normal] and 0 returning a handle.
Call "kernel32.dll" "CloseHandle" with the handle.
Clear the i/o error.
If the handle is not -1 [invalid_handle_value], exit.
Put "Error creating file '" then the path then "'." into the i/o error.

To create a pdf object given a kind:
Allocate memory for the pdf object.
Put the kind into the pdf object's kind.

To create a pdf outline entry given a title string and a page height and a destination number:
Allocate memory for the pdf outline entry.
Put the title string into the pdf outline entry's title.
Put the page height into the pdf outline entry's page height.
Put the destination into the pdf outline entry's destination.

To create a picture:
Allocate memory for the picture.

To create a picture given a buffer:
Create a gpbitmap given the buffer.
If the gpbitmap is nil, void the picture; exit.
Allocate memory for the picture.
Put the buffer into the picture's data.
Put the gpbitmap into the picture's gpbitmap.
Adjust the picture (extract boxes from gpbitmap).

To create a picture given a gpbitmap:
If the gpbitmap is nil, void the picture; exit.
Allocate memory for the picture.
Put the gpbitmap into the picture's gpbitmap.
Adjust the picture (extract boxes from gpbitmap).

To create a picture given a url:
Read the url into a buffer.
Create the picture given the buffer.

To create a polygon:
Allocate memory for the polygon.

To create a quora:
Allocate memory for the quora.

To create a refer:
Allocate memory for the refer.

To create the request handle of a winhttp request using a url record:
If the winhttp request is nil, exit.
Convert the url record's path into a wide string called wide path and null terminate.
Convert "POST" to a wide string called wide post string and null terminate.
If the url record's scheme is "https", put 8388608 [winhttp_flag_secure] into a secure number.
Call "winhttp.dll" "WinHttpOpenRequest"
With the winhttp request's connection
And the wide post string's first
And the wide path's first
And 0 [L"HTTP/1.1"]
And 0 [winhttp_no_referer]
And 0 [winhttp_default_accept_types]
And the secure number
Returning the winhttp request's request.
If the winhttp request's request is 0, put "Could not open request." into the i/o error; exit.

To create a row given a byte:
Allocate memory for the row.
Put the byte into the row's string.

To create a row given a string:
Allocate memory for the row.
Put the string into the row's string.

To create a socket given a host string and a port number: \ this guy creates and connects, sets i/o error if there is a problem
Clear the i/o error.
\ get sockaddr
Get a sockaddr given the host.
If the i/o error is not blank, exit.
Put 2 [af_inet] into the sockaddr's sin_family.
Put the port into the sockaddr's sin_port.
\ create socket
Call "ws2_32.dll" "socket" with 2 [af_inet] and 1 [sock_stream] and 0 [ipproto_ip] returning the socket.
If the socket is -1, put "Could not create socket." into the i/o error; exit.

\ connect socket
Call "ws2_32.dll" "connect" with the socket and the sockaddr's whereabouts and the sockaddr's magnitude returning a result number.
If the result is not 0, put "Could not connect to socket." into the i/o error; exit.
\ set send timeout 30 seconds
Call "ws2_32.dll" "setsockopt" with the socket and 65535 and 4101 [so_sndtimo] and 30 seconds' whereabouts and 4 returning the result number.
If the result is not 0, put "Could not set receive timeout." into the i/o error; exit.
\ set receive timeout 30 seconds
Call "ws2_32.dll" "setsockopt" with the socket and 65535 and 4102 [so_rcvtimeo] and 30 seconds' whereabouts and 4 returning the result number.
If the result is not 0, put "Could not set receive timeout." into the i/o error; exit.

To create a string thing given a string:
Allocate memory for the string thing.
Put the string into the string thing's string.

To create a terminal in a box:
Allocate memory for the terminal.
Put the box into the terminal's box.
Put the green color into the terminal's output color.
Put the lightest green color into the terminal's input color.

To create a text:
Allocate memory for the text.
Put the black color into the text's pen.
Put the default font into the text's font.
Put "left" into the text's alignment.
Put 1/1 into the text's scale.
Guarantee one row in the text.
Reset the origin of the text.
Reset the caret of the text.
Deselect the text.

To create a vertex:
Allocate memory for the vertex.

To create a vertex array given a count:
Privatize the count.
Allocate memory for the vertex array.
Put the count into the vertex array's count.
Multiply the count by a spot's magnitude.
Assign the vertex array's spot pointer given the count.

To create a vertex array given some vertices:
Create the vertex array given the vertices' count.
Put the vertex array's spot pointer into a spot pointer.
Loop.
Get a vertex from the vertices.
If the vertex is nil, exit.
Put the vertex's spot into the spot pointer's target.
Add the vertex's spot's magnitude to the spot pointer.
Repeat.

To create a vertex given a spot:

Allocate memory for the vertex.
Put the spot into the vertex's spot.

To create a vertex given an x coord and a y coord:
Allocate memory for the vertex.
Put the x into the vertex's x.
Put the y into the vertex's y.

To create a winhttp request for posting to a url:
Allocate memory for the winhttp request.
Convert the url to a url record.
Create the open handle of the winhttp request.
If the i/o error is not blank, destroy the winhttp request; exit.
Create the connect handle of the winhttp request using the url record.
If the i/o error is not blank, destroy the winhttp request; exit.
Create the request handle of the winhttp request using the url record.
If the i/o error is not blank, destroy the winhttp request; exit.

The crlf string is a string equal to $0D0A.

The cross byte is a byte equal to 43.

A crypt session is a thing with
An hcryptprov pointer,
An hcrypthash pointer,
An hcryptkey pointer.

The ctrl key is a key equal to 17.

The currency-sign byte is a byte equal to 164.

The current canvas is a canvas.

The current event is an event.

The current rainbow color number is a number [1 to 6 indicating, respectively, red, orange, yellow, green, blue, purple].

A cursor is a handle.

To cut a number in half:
Divide the number by 2.

The cyan color is a color.

The cyan pen is a pen.

The d-key is a key equal to 68.

The dagger byte is a byte equal to 134.

The dark blue color is a color.

The dark blue pen is a pen.

The dark brown color is a color.

The dark brown pen is a pen.

A dark color is a color.

The dark cyan color is a color.

The dark cyan pen is a pen.

The dark gray color is a color.

The dark gray pen is a pen.

The dark green color is a color.

The dark green pen is a pen.

The dark lime color is a color.

The dark lime pen is a pen.

The dark magenta color is a color.

The dark magenta pen is a pen.

The dark orange color is a color.

The dark orange pen is a pen.

The dark pink color is a color.

The dark pink pen is a pen.

The dark purple color is a color.

The dark purple pen is a pen.

The dark red color is a color.

The dark red pen is a pen.

The dark sky blue color is a color.

The dark sky blue pen is a pen.

The dark sky color is a color.

The dark sky pen is a pen.

The dark teal color is a color.

The dark teal pen is a pen.

The dark violet color is a color.

The dark violet pen is a pen.

The dark yellow color is a color.

The dark yellow pen is a pen.

To darken a color by an amount:
Subtract the amount from the color's lightness.
Limit the color's lightness to 0 and 1000.

To darken a color by some percent;
To darken a color about some percent;
To darken a color by about some percent;
To darken a color some percent:
Put the color's lightness minus the percent into the color's lightness.
Limit the color's lightness to 0 and 1000.

To darken the current color about some percent:
Darken the context's color by the percent.

The darker blue color is a color.

The darker blue pen is a pen.

The darker brown color is a color.

The darker brown pen is a pen.

The darker cyan color is a color.

The darker cyan pen is a pen.

The darker gray color is a color.

The darker gray pen is a pen.

The darker green color is a color.

The darker green pen is a pen.

The darker lime color is a color.

The darker lime pen is a pen.

The darker magenta color is a color.

The darker magenta pen is a pen.

The darker orange color is a color.

The darker orange pen is a pen.

The darker purple color is a color.

The darker purple pen is a pen.

The darker red color is a color.

The darker red pen is a pen.

The darker sky blue color is a color.

The darker sky blue pen is a pen.

The darker sky color is a color.

The darker sky pen is a pen.

The darker teal color is a color.

The darker teal pen is a pen.

The darker violet color is a color.

The darker violet pen is a pen.

The darker yellow color is a color.

The darker yellow pen is a pen.

The darkest blue color is a color.

The darkest blue pen is a pen.

The darkest brown color is a color.

The darkest brown pen is a pen.

The darkest cyan color is a color.

The darkest cyan pen is a pen.

The darkest gray color is a color.

The darkest gray pen is a pen.

The darkest green color is a color.

The darkest green pen is a pen.

The darkest lime color is a color.

The darkest lime pen is a pen.

The darkest magenta color is a color.

The darkest magenta pen is a pen.

The darkest orange color is a color.

The darkest orange pen is a pen.

The darkest purple color is a color.

The darkest purple pen is a pen.

The darkest red color is a color.

The darkest red pen is a pen.

The darkest sky blue color is a color.

The darkest sky blue pen is a pen.

The darkest sky color is a color.

The darkest sky pen is a pen.

The darkest teal color is a color.

The darkest teal pen is a pen.

The darkest violet color is a color.

The darkest violet pen is a pen.

The darkest yellow color is a color.

The darkest yellow pen is a pen.

The dash byte is a byte equal to 45.

The dash key is a key equal to 189.

The data-link-escape byte is a byte equal to 16.

A date/time has
A year number,
A month number,
A week day number,
A day number,
An hour number,
A minute number,
A second number,
A millisecond number.

To de-sign a fraction:
De-sign the fraction's numerator.
De-sign the fraction's denominator.

To de-sign a number:
If the number is the smallest number, put the largest number into the number; exit.
If the number is less than 0, negate the number.

To de-sign a pair:
De-sign the pair's x.
De-sign the pair's y.

To de-sign a string:
If the string is blank, exit.
If the string's first's target is any sign, remove the first byte from the string.

To debug a box:
Clear a string.
Append "left=" to the string.
Append the box's left to the string.
Append ", top=" to the string.
Append the box's top to the string.
Append ", right=" to the string.
Append the box's right to the string.
Append ", bottom=" to the string.
Append the box's bottom to the string.
Debug the string.

To debug a byte:
Put the byte into a number.
Convert the number to a string.
Debug the string.

To debug a color:
Clear a string.
Append "hue=" to the string.
Append the color's hue to the string.
Append ", saturation=" to the string.
Append the color's saturation to the string.
Append ", lightness=" to the string.
Append the color's lightness to the string.
Debug the string.

To debug a flag:
Convert the flag to a string.
Debug the string.

To debug a font:
Clear a string.
Append "name='" to the string then "'".
Append the font's name to the string.
Append ", height=" to the string.
Append the font's height to the string.
Debug the string.

To debug a fraction:
Clear a string.
Append "numerator=" to the string.
Append the fraction's numerator to the string.
Append ", denominator=" to the string.
Append the fraction's denominator to the string.
Debug the string.

```
To debug a line:
Clear a string.
Append "start=" to the string.
Append the line’s start’s x to the string.
Append "," to the string.
Append the line’s start’s y to the string.
Append " end=" to the string.
Append the line’s end’s x to the string.
Append "," to the string.
Append the line’s end’s y to the string.
Debug the string.

To debug a number:
Convert the number to a string.
Debug the string.

To debug a number and another number:
Debug the number then ", " then the other number.

To debug a pair:
Clear a string.
Append "x=" to the string.
Append the pair’s x to the string.
Append ", y=" to the string.
Append the pair’s y to the string.
Debug the string.

To debug a pointer:
Convert the pointer to a nibble string.
Debug "$" then the nibble string.

To debug a rgb:
Clear a string.
Append "red=" to the string.
Put the rgb’s red byte into a number.
Append the number to the string.
Append ", green=" to the string.
Put the rgb’s green byte into the number.
Append the number to the string.
Append ", blue=" to the string.
Put the rgb’s blue byte into the number.
Append the number to the string.
Debug the string.

To debug a string:
Privatize the string.
Null terminate the string.
Call "user32.dll" "MessageBoxA" with 0 and the string’s first and "debug"’s first and 0.

To debug a string (quoted):
Privatize the string.
Prepend the double-quote byte to the string.
Append the double-quote byte to the string.
Debug the string.
```

```
To debug a wyrd:
Put the wyrd into a number.
Convert the number to a string.
Debug the string.

To decide if a box is another box:
If the box's left is not the other box's left, say no.
If the box's top is not the other box's top, say no.
If the box's right is not the other box's right, say no.
If the box's bottom is not the other box's bottom, say no.
Say yes.

To decide if a box is still in another box;
To decide if a box is in another box;
To decide if a box is inside another box:
If the box's left is less than the other box's left, say no.
If the box's top is less than the other box's top, say no.
If the box's right is greater than the other box's right, say no.
If the box's bottom is greater than the other box's bottom, say no.
Say yes.

To decide if a box is touching another box:
If the other box's right is less than the box's left, say no.
If the other box's bottom is less than the box's top, say no.
If the other box's left is greater than the box's right, say no.
If the other box's top is greater than the box's bottom, say no.
Say yes.

To decide if a byte is alphanumeric:
If the byte is any letter, say yes.
If the byte is any digit, say yes.
Say no.

To decide if a byte is another byte:
Intel $C7C001000000. \ mov eax,1 \ assume true
Intel $8B9D08000000. \ mov ebx,[ebp+8] \ the byte
Intel $8A1B. \ mov bl,[ebx]
Intel $8B8D0C000000. \ mov ecx,[ebp+12] \ the other byte
Intel $3A19. \ cmp bl,[ecx]
Intel $0F8406000000. \ je over the next 1 statement
Intel $C7C000000000. \ mov eax,0 \ now it's false

To decide if a byte is another byte or a third byte:
If the byte is the other byte, say yes.
If the byte is the third byte, say yes.
Say no.

To decide if a byte is any consonant:
If the byte is in "bcdfghjklmnpqrstvwxyz", say yes.
Say no.

To decide if a byte is any digit:
If the byte is less than the zero byte, say no.
If the byte is greater than the nine byte, say no.
Say yes.
```

```
To decide if a byte is any letter:
If the byte is between the big-a byte and the big-z byte, say yes.
If the byte is between the little-a byte and the little-z byte, say yes.
If the byte is 131 or 138, say yes.
If the byte is 140 or 142, say yes.
If the byte is 154 or 156, say yes.
If the byte is between 158 and 159, say yes.
If the byte is between 192 and 214, say yes.
If the byte is between 216 and 246, say yes.
If the byte is between 248 and 255, say yes.
Say no.

To decide if a byte is any numeric starter:
If the byte is any digit, say yes.
If the byte is any sign, say yes.
Say no.

To decide if a byte is any punctuation mark: \ *** questionable?
If the byte is the space byte, say no.
If the byte is not alphanumeric, say yes.
Say no.

To decide if a byte is any sign:
If the byte is the dash byte, say yes.
If the byte is the cross byte, say yes.
Say no.

To decide if a byte is any valid drive:
Put the byte into a path.
Append ":\" to the path.
Get a drive kind for the path.
If the drive kind is "", say no.
Say yes.

To decide if a byte is any vowel:
If the byte is in "aeiou", say yes.
\  if the byte is "y", say sometimes. \ ha ha ha
Say no.

To decide if a byte is between another byte and a third byte:
If the byte is less than the other byte, say no.
If the byte is greater than the third byte, say no.
Say yes.

To decide if a byte is between a number and another number:
If the byte is less than the number, say no.
If the byte is greater than the other number, say no.
Say yes.

To decide if a byte is greater than another byte:
Intel $C7C001000000. \ mov eax,1 \ assume true
Intel $8B9D08000000. \ mov ebx,[ebp+8] \ the byte
Intel $8A1B. \ mov bl,[ebx]
Intel $8B8D0C000000. \ mov ecx,[ebp+12] \ the other byte
```

Intel $3A19. \ cmp bl,[ecx]
Intel $0F8706000000. \ ja over the next 1 statement
Intel $C7C000000000. \ mov eax,0 \ now it's false

To decide if a byte is greater than a number:
Intel $C7C001000000. \ mov eax,1 \ assume true
Intel $8B9D08000000. \ mov ebx,[ebp+8] \ the byte
Intel $0FB61B. \ movzx ebx,byte ptr [ebx]
Intel $8B8D0C000000. \ mov ecx,[ebp+12] \ the number
Intel $3B19. \ cmp ebx,[ecx]
Intel $0F8F06000000. \ jg over the next 1 statement
Intel $C7C000000000. \ mov eax,0 \ now it's false

To decide if a byte is greater than or equal to another byte:
Intel $C7C001000000. \ mov eax,1 \ assume true
Intel $8B9D08000000. \ mov ebx,[ebp+8] \ the byte
Intel $8A1B. \ mov bl,[ebx]
Intel $8B8D0C000000. \ mov ecx,[ebp+12] \ the other byte
Intel $3A19. \ cmp bl,[ecx]
Intel $0F8306000000. \ ja over the next 1 statement
Intel $C7C000000000. \ mov eax,0 \ now it's false

To decide if a byte is in a string:
Privatize the byte.
Lowercase the byte.
Slap a substring on the string.
Loop.
If the substring is blank, say no.
Put the substring's first's target into another byte.
Lowercase the other byte.
If the other byte is the byte, say yes.
Add 1 to the substring's first.
Repeat.

To decide if a byte is less than another byte:
Intel $C7C001000000. \ mov eax,1 \ assume true
Intel $8B9D08000000. \ mov ebx,[ebp+8] \ the byte
Intel $8A1B. \ mov bl,[ebx]
Intel $8B8D0C000000. \ mov ecx,[ebp+12] \ the other byte
Intel $3A19. \ cmp bl,[ecx]
Intel $0F8206000000. \ jb over the next 1 statement
Intel $C7C000000000. \ mov eax,0 \ now it's false

To decide if a byte is less than a number:
Intel $C7C001000000. \ mov eax,1 \ assume true
Intel $8B9D08000000. \ mov ebx,[ebp+8] \ the byte
Intel $0FB61B. \ movzx ebx,byte ptr [ebx]
Intel $8B8D0C000000. \ mov ecx,[ebp+12] \ the number
Intel $3B19. \ cmp ebx,[ecx]
Intel $0F8C06000000. \ jl over the next 1 statement
Intel $C7C000000000. \ mov eax,0 \ now it's false

To decide if a byte is less than or equal to another byte:
Intel $C7C001000000. \ mov eax,1 \ assume true
Intel $8B9D08000000. \ mov ebx,[ebp+8] \ the byte

```
Intel $8A1B. \ mov bl,[ebx]
Intel $8B8D0C000000. \ mov ecx,[ebp+12] \ the other byte
Intel $3A19. \ cmp bl,[ecx]
Intel $0F8606000000. \ jbe over the next 1 statement
Intel $C7C000000000. \ mov eax,0 \ now it's false

To decide if a byte is like another byte: \ used internally for word wrap
If the byte is whitespace, set a flag.
If the other byte is whitespace, set another flag.
If the flag is the other flag, say yes.
Say no.

To decide if a byte is noise:
If the byte is less than or equal to the space byte, say yes.
If the byte is the delete byte, say yes.
If the byte is the non-breaking-space byte, say yes.
If the byte is 129, say yes.
If the byte is 141, say  yes.
If the byte is 143, say yes.
If the byte is 144, say yes.
If the byte is 157, say yes.
Say no.

To decide if a byte is a number:
Intel $C7C001000000. \ mov eax,1 \ assume true
Intel $8B9D08000000. \ mov ebx,[ebp+8] \ the byte
Intel $0FB61B. \ movzx ebx,byte ptr [ebx]
Intel $8B8D0C000000. \ mov ecx,[ebp+12] \ the number
Intel $3B19. \ cmp ebx,[ecx]
Intel $0F8406000000. \ je over the next 1 statement
Intel $C7C000000000. \ mov eax,0 \ now it's false

To decide if a byte is a number or another number:
If the byte is the number, say yes.
If the byte is the other number, say yes.
Say no.

To decide if a byte is printable:
If the byte is less than the space byte, say no.
If the byte is the delete byte, say no.
If the byte is 129, say no.
If the byte is 141, say no.
If the byte is 143, say no.
If the byte is 144, say no.
If the byte is 157, say no.
Say yes.

To decide if a byte is a string:
If the string's length is not 1, say no.
Privatize the byte.
Lowercase the byte.
Put the string's first's target into another byte.
Lowercase the other byte.
If the byte is the other byte, say yes.
Say no.
```

```
To decide if a byte is symbolic:
If the byte is between the exclamation byte and the slash byte, say yes.
If the byte is between the colon byte and the at-sign byte, say yes.
If the byte is between the left-bracket byte and the accent byte, say yes.
If the byte is between the left-brace byte and the tilde byte, say yes.
If the byte is 128, say yes.
If the byte is 130, say yes.
If the byte is between 132 and 137, say yes.
If the byte is 139, say yes.
If the byte is between 145 and 153, say yes.
If the byte is 155, say yes.
If the byte is between 161 and 180, say yes.
If the byte is between 183 and 191, say yes.
If the byte is 215, say yes.
If the byte is 247, say yes.
Say no.

To decide if a byte is whitespace:
If the byte is the space byte, say yes.
If the byte is the tab byte, say yes.
If the byte is the return byte, say yes.
If the byte is the linefeed byte, say yes.
Say no.

To decide if the caret of a text is at the beginning:
If the text is nil, say no.
If the text's caret row# is not 1, say no.
If the text's caret column# is not 1, say no.
Say yes.

To decide if the caret of a text is at the end:
If the text is nil, say no.
If the text's caret row# is not the text's rows' count, say no.
Get a row given the text's caret row# and the text.
If the text's caret column# is not the row's string's length, say no.
Say yes.

To decide if the caret of a text is on the first line:
If the text is nil, say no.
If the text's caret row# is not 1, say no.
Say yes.

To decide if the caret of a text is on the last line:
If the text is nil, say  no.
If the text's rows are empty, say no.
If the text's caret row# is not the text's last row's row#, say no.
Say yes.

To decide if a choice is a string:
If the choice is nil, say no.
If the choice's name is the string, say yes.
Say no.

To decide if a color and another color are clear:
```

If the color is not clear, say no.
If the other color is not clear, say no.
Say yes.

To decide if a color is another color:
If the color's hue is not the other color's hue, say no.
If the color's saturation is not the other color's saturation, say no.
If the color's lightness is not the other color's lightness, say no.
Say yes.

To decide if a color is clear:
If the color's hue is less than 0, say yes.
Say no.

To decide if a color is dark:
If the color's lightness is between 250 and 374, say yes.
Say no.

To decide if a color is light:
If the color's lightness is between 625 and 749, say yes.
Say no.

To decide if a color is normal:
If the color's lightness is between 375 and 624, say yes.
Say no.

To decide if a color is very dark:
If the color's lightness is between 125 and 249, say yes.
Say no.

To decide if a color is very light:
If the color's lightness is between 750 and 874, say yes.
Say no.

To decide if a color is very very dark:
If the color's lightness is less than or equal to 124, say yes.
Say no.

To decide if a color is very very light:
If the color's lightness is greater than or equal to 875, say yes.
Say no.

To decide if a counter is past a number:
Add 1 to the counter.
If the counter is greater than the number, say yes.
Say no.

To decide if the current spot is above or below a box:
If the context's spot is above or below the box, say yes.
Say no.

To decide if the current spot is left or right of a box:
If the context's spot is left or right of the box, say yes.
Say no.

```
To decide if the current spot is to the right of a box:
If the context's spot's x is greater than the box's right, say yes.
Say no.

To decide if the current spot is within some twips of a box:
If the context's spot is within the twips of the box, say yes.
Say no.

To decide if a difference is within a grid:
Privatize the difference.
De-sign the difference.
If the difference's x is greater than or equal to the grid's x, say no.
If the difference's y is greater than or equal to the grid's y, say no.
Say yes.

To decide if an event is any shortcut:
If the event is nil, say no.
If the event's kind is not "key down", say no.
If the event is not modified, say no.
If the event's key is between the a-key and the z-key, say yes.
Say no.

To decide if an event is modified:
If the event's ctrl flag is set, say yes.
If the event's alt flag is set, say yes.
Say no.

To decide if a finger is past the end of a string:
If the finger is nil, say yes.
If the finger is greater than the string's last, say yes.
Say no.

To decide if a flag is another flag;
To decide if a pointer is a number;
To decide if a pointer is another pointer;
To decide if a number is another number:
Intel $C7C001000000. \ mov eax,1 \ assume true
Intel $8B9D08000000. \ mov ebx,[ebp+8] \ the number
Intel $8B1B. \ mov ebx,[ebx]
Intel $8B8D0C000000. \ mov ecx,[ebp+12] \ the other number
Intel $3B19. \ cmp ebx,[ecx]
Intel $0F8406000000. \ je over the next 1 statement
Intel $C7C000000000. \ mov eax,0 \ now it's false

To decide if a flag is on: \ switch as another name for flag also ?  ***
If the flag is set, say yes.
Say no.

To decide if a flag is set:
If the flag is yes, say yes.
Say no.

To decide if a fraction is another fraction:
Privatize the fraction.
Privatize the other fraction.
```

```
Normalize the fraction and the other fraction.
If the fraction’s numerator is the other fraction’s numerator, say yes.
Say no.

To decide if a fraction is greater than another fraction:
Privatize the fraction.
Privatize the other fraction.
Normalize the fraction and the other fraction.
If the fraction’s numerator is greater than the other fraction’s numerator, say yes.
Say no.

To decide if a fraction is greater than or equal to another fraction:
Privatize the fraction.
Privatize the other fraction.
Normalize the fraction and the other fraction.
If the fraction’s numerator is greater than or equal to the other fraction’s numerator, say yes.
Say no.

To decide if a fraction is less than another fraction:
Privatize the fraction.
Privatize the other fraction.
Normalize the fraction and the other fraction.
If the fraction’s numerator is less than the other fraction’s numerator, say yes.
Say no.

To decide if a fraction is less than or equal to another fraction:
Privatize the fraction.
Privatize the other fraction.
Normalize the fraction and the other fraction.
If the fraction’s numerator is less than or equal to the other fraction’s numerator, say yes.
Say no.

To decide if a fraction is negative:
If the fraction’s numerator is less than 0, reverse a flag.
If the fraction’s denominator is less than 0, reverse the flag.
Say the flag.

To decide if a fraction is a number:
If the fraction is the number / 1, say yes.
Say no.

To decide if an index is empty:
\  if the index is nil, say yes. \ to make compiler faster
\  loop.
Get a bucket given the index.
If the bucket is nil, say yes.
If the bucket’s refers are not empty, say no.
Repeat.

To decide if an input is from windows telling us to refresh the cursor;
To decide if an input is from windows telling us to set the cursor:
If the input is nil, say no.
If the input’s kind is "set cursor", say yes.
Say no.
```

To decide if an input is from windows telling us to show all our stuff;
To decide if an input is from windows telling us to show all of our stuff;
To decide if an input is from windows telling us to redraw all our stuff;
To decide if an input is from windows telling us to redraw all of our stuff:
If the input is nil, say no.
If the input's kind is "refresh", say yes.
Say no.

To decide if an input is from windows telling us we're done;
To decide if an input is from windows telling us that we're done;
To decide if an input is from windows telling us the user has quit:
If the input is nil, say yes.
Say no.

To decide if an input is a left click:
Void the left click.
If the input's kind is not "left click", say no.
Put the input into the left click.

To decide if an item is found:
If the item's kind is not blank, say yes.
Say no.

To decide if a key is any digit key:
If the key is between 48 and 57, say yes.
Say no.

To decide if a key is any letter key:
If the key is between 65 and 90, say yes.
Say no.

To decide if a key is any modifier key:
If the key is the alt key, say yes.
If the key is the ctrl key, say yes.
If the key is the shift key ,say yes.
Say no.

To decide if a key is any pad key:
If the key is between 96 and 111, say yes.
Say no.

To decide if a key is any printable key:
If the key is the space key, say yes.
If the key is any digit key, say yes.
If the key is any letter key, say yes.
If the key is any pad key, say yes.
If the key is any symbol key, say yes.
Say no.

To decide if a key is any symbol key:
If the key is between 186 and 192, say yes.
If the key is between 219 and 222, say yes.
Say no.

To decide if a key is any wm-char key:

```
If the key is not any printable key , say no.
If the alt key was down, say no.
If the ctrl key was down, say no.
Say yes.

To decide if a key is down:
Call "user32.dll" "GetAsyncKeyState" with the key returning a wyrd.
Put the wyrd into a number.
If the number is less than 0, say yes.
Say no.

To decide if a key is up:
If the key is down, say no.
Say yes.

To decide if a key was down:
Call "user32.dll" "GetKeyState" with the key returning a wyrd.
Put the wyrd into a number.
If the number is less than 0, say yes.
Say no.

To decide if a key was toggled:
Call "user32.dll" "GetKeyState" with the key returning a wyrd.
Put the wyrd into a number.
Bitwise and the number with 1.
If the number is 1, say yes.
Say no.

To decide if a key was up:
If the key was down, say no.
Say yes.

To decide if a key with an l-param is any repeated escape or modifier key:
Put the l-param into a number.
Bitwise and the number with 1073741824 [$40000000].
If the number is 0, say no.
If the key is the escape key, say yes.
If the key is any modifier key, say yes.
Say no.

To decide if the left mouse button is down:
If the mouse's left button is down, say yes.
Say no.

To decide if a line is above a box:
If the line's start's y is greater than or equal to the box's top, say no.
If the line's end's y is greater than or equal to the box's top, say no.
Say yes.

To decide if a line is above a coord:
If the line's start's y is greater than or equal to the coord, say no.
If the line's end's y is greater than or equal to the coord, say no.
Say yes.

To decide if a line is below a box:
```

If the line's start's y is less than or equal to the box's bottom, say no.
If the line's end's y is less than or equal to the box's bottom, say no.
Say yes.

To decide if a line is below a coord:
If the line's start's y is less than or equal to the coord, say no.
If the line's end's y is less than or equal to the coord, say no.
Say yes.

To decide if a line is still in a box;
To decide if a line is in a box:
If the line's start is not in the box, say no.
If the line's end is not in the box, say no.
Say yes.

To decide if a mixed is a number:
Convert the mixed to a fraction.
If the fraction is the number, say yes.
Say no.

To decide if the mouse has been dragged from a spot given a grid:
If the mouse's left button is up, say no.
Put the mouse's spot into another spot.
Get a difference between the other spot and the spot.
If the difference is within the grid, repeat.
Say yes.

To decide if the mouse is in a box:
If the mouse's spot is in the box, say yes.
Say no.

To decide if a number is another number and a string is another string:
If the number is not the other number, say no.
If the string is not the other string, say no.
Say yes.

To decide if a number is between another number and a third number:
If the number is less than the other number, say no.
If the number is greater than the third number, say no.
Say yes.

To decide if a number is even:
If the number is odd, say no.
Say yes.

To decide if a number is evenly divisible by another number:
Privatize the number.
Divide the number by the other number giving a quotient and a remainder.
If the remainder is 0, say yes.
Say no.

To decide if a number is a multiple of another number:
If the number is evenly divisible by the other number, say yes.
Say no.

```
To decide if a number is negative:
If the number is less than 0, say yes.
Say no.

To decide if a number is odd:
Privatize the number.
Bitwise and the number with 1.
If the number is 0, say no.
Say yes.

To decide if a number is positive:
If the number is less than 0, say no.
Say yes.

To decide if a number is prime:
If the number is less than 2, say no.
If the number is 2, say yes.
Put the number minus 1 into another number.
Loop.
If the number is evenly divisible by the other number, say no.
Subtract 1 from the other number.
If the other number is greater than 1, repeat.
Say yes.

To decide if a pair is another pair:
If the pair's x is not the other pair's x, say no.
If the pair's y is not the other pair's y, say no.
Say yes.

To decide if a pair is a number:
If the pair's x is not the number, say no.
If the pair's y is not the number, say no.
Say yes.

To decide if a pair is a number and another number:
If the pair's x is not the number, say no.
If the pair's y is not the other number, say no.
Say yes.

To decide if a path is directory-format:
If the path is blank, say no.
If the path's last's target is the backslash byte, say yes.
Say no.

To decide if a path is drive-format:
If the path starts with "\\", say yes.
If the path's length is not 3, say no.
If the path ends with ":\", say yes.
Say no.

To decide if a path is empty in the file system:
If the path is not in the file system, say yes.
Get a count of items in the path in the file system.
If the count is 0, say yes.
Say no.
```

```
To decide if a path is file-format:
If the path is blank, say no.
If the path's last's target is the colon byte, say no.
If the path's last's target is the backslash byte, say no.
Say yes.

To decide if a path is in the file system:
Privatize the path.
Null terminate the path.
Call "kernel32.dll" "GetFileAttributesA" with the path's first returning a number.
If the number is less than 0, say no.
Say yes.

To decide if a path is read-only:
Privatize the path.
Null terminate the path.
Call "kernel32.dll" "GetFileAttributesA" with the path's first returning a number.
Bitwise and the number with 1 [file_attribute_readonly].
If the number is not 0, say yes.
Say no.

To decide if a pointer can be found;
To decide if a pointer is coming;
To decide if a pointer is found;
To decide if a pointer was found;
To decide if a pointer is there;
To decide if a pointer does exist:
If the pointer is nil, say no.
Say yes.

To decide if a pointer is greater than another pointer;
To decide if a number is greater than another number:
Intel $C7C001000000. \ mov eax,1 \ assume true
Intel $8B9D08000000. \ mov ebx,[ebp+8] \ the number
Intel $8B1B. \ mov ebx,[ebx]
Intel $8B8D0C000000. \ mov ecx,[ebp+12] \ the other number
Intel $3B19. \ cmp ebx,[ecx]
Intel $0F8F06000000. \ jg over the next 1 statement
Intel $C7C000000000. \ mov eax,0 \ now it's false

To decide if a pointer is greater than or equal to another pointer;
To decide if a number is another number or more;
To decide if a number is greater than or equal to another number:
Intel $C7C001000000. \ mov eax,1 \ assume true
Intel $8B9D08000000. \ mov ebx,[ebp+8] \ the number
Intel $8B1B. \ mov ebx,[ebx]
Intel $8B8D0C000000. \ mov ecx,[ebp+12] \ the other number
Intel $3B19. \ cmp ebx,[ecx]
Intel $0F8D06000000. \ jge over the next 1 statement
Intel $C7C000000000. \ mov eax,0 \ now it's false

To decide if a pointer is less than another pointer;
To decide if a number is less than another number:
Intel $C7C001000000. \ mov eax,1 \ assume true
```

Intel $8B9D08000000. \ mov ebx,[ebp+8] \ the number
Intel $8B1B. \ mov ebx,[ebx]
Intel $8B8D0C000000. \ mov ecx,[ebp+12] \ the other number
Intel $3B19. \ cmp ebx,[ecx]
Intel $0F8C06000000. \ jl over the next 1 statement
Intel $C7C000000000. \ mov eax,0 \ now it's false

To decide if a pointer is less than or equal to another pointer;
To decide if a number is another number or less;
To decide if a number is another number or fewer;
To decide if a number is less than or equal to another number:
Intel $C7C001000000. \ mov eax,1 \ assume true
Intel $8B9D08000000. \ mov ebx,[ebp+8] \ the number
Intel $8B1B. \ mov ebx,[ebx]
Intel $8B8D0C000000. \ mov ecx,[ebp+12] \ the other number
Intel $3B19. \ cmp ebx,[ecx]
Intel $0F8E06000000. \ jle over the next 1 statement
Intel $C7C000000000. \ mov eax,0 \ now it's false

To decide if a pointer is missing;
To decide if a pointer is null and void:
If the pointer is nil, say yes.
Say no.

To decide if a polygon is closed:
If the polygon is nil, say no.
If the polygon's vertices' count is less than 3, say no.
If the polygon's first vertex's spot is the polygon's last vertex's spot, say yes.
Say no.

To decide if a reply is something like another reply:
Privatize the reply.
Privatize the other reply.
Simplify the reply.
Simplify the other reply.
If the reply is the other reply, say yes.
Say no.

To decide if a row is blank:
If the row is nil, say yes.
Slap a substring on the row's string.
Loop.
If the substring is blank, say yes.
If the substring's first's target is not noise, say no.
Add 1 to the substring's first.
Repeat.

To decide if a row of a text is selected:
If the text is nil, say no.
If the row is nil, say no.
If nothing is selected in the text, say no.
Put the text's normalized selection into a selection.
If the row's row# is less than the selection's anchor row#, say no.
If the row's row# is greater than the selection's caret row#, say no.
If the row's row# is not the selection's caret row#, say yes.

```
If the selection’s caret column# is 1, say no.
Say yes.

To decide if a row of a text is visible:
If the text is nil, say no.
If the row is nil, say  no.
Get a box given the row and the text.
Put the text’s box into another box.
Adjust the other box given 0 and the tpp and 0 and - the tpp.
If the box is touching the other box, say yes.
Say no.

To decide if a selection is another selection:
If the selection’s anchor is not the other selection’s anchor, say no.
If the selection’s caret is not the other selection’s caret, say no.
Say yes.

To decide if something is selected in a text:
If the text’s anchor is the text’s caret, say no.
Say yes.

To decide if a spot is above a box:
If the spot’s y is less than the box’s top, say yes.
Say no.

To decide if a spot is above a horizontal line:
If the spot’s y is less than the horizontal line’s start’s y, say yes.
Say no.

To decide if a spot is above or below a box:
If the spot is above the box, say yes.
If the spot is below the box, say yes.
Say no.

To decide if a spot is below a box:
If the spot’s y is greater than the box’s bottom, say yes.
Say no.

To decide if a spot is below a horizontal line:
If the spot’s y is greater than the horizontal line’s start’s y, say yes.
Say no.

To decide if a spot is in an ellipse:
Create an hrgn given the ellipse.
Privatize the spot.
Divide the spot by the tpp.
Call "gdi32.dll" "PtInRegion" with the hrgn and the spot’s x and the spot’s y returning a number.
Destroy the hrgn.
If the number is 0, say no.
Say yes.

To decide if a spot is in a picture:
If the picture is nil, say no.
If the spot is in the picture’s box, say yes.
Say no.
```

To decide if a spot is in a polygon:
If the polygon is nil, say no.
Create a vertex array given the polygon's vertices.
Call "gdi32.dll" "CreatePolygonRgn" with the vertex array's spot pointer and the vertex array's count and 2 [winding] returning an hrgn.
Call "gdi32.dll" "PtInRegion" with the hrgn and the spot's x and the spot's y returning a number.
Call "gdi32.dll" "DeleteObject" with the hrgn.
Destroy the vertex array.
If the number is 0, say no.
Say yes.

To decide if a spot is in some polygons:
Get a polygon from the polygons.
If the polygon is nil, say no.
If the spot is in the polygon, say yes.
Repeat.

To decide if a spot is in a roundy box:
Privatize the roundy box.
Add the tpp to the roundy box's right-bottom.
Put the roundy box's radius times 2 into a diameter number.
Call "gdi32.dll" "CreateRoundRectRgn" with the roundy box's left and the roundy box's top and the roundy box's right and the roundy box's bottom
And the diameter and the diameter returning an hrgn.
Call "gdi32.dll" "PtInRegion" with the hrgn and the spot's x and the spot's y returning a number.
Call "gdi32.dll" "DeleteObject" with the hrgn.
If the number is 0, say no.
Say yes.

To decide if a spot is in a text:
If the text is nil, say no.
If the spot is in the text's box, say yes.
Say no.

To decide if a spot is inside a box;
To decide if a spot is within a box;
To decide if a spot is in a box:
If the spot's x is less than the box's left, say no.
If the spot's y is less than the box's top, say no.
If the spot's x is greater than the box's right, say no.
If the spot's y is greater than the box's bottom, say no.
Say yes.

To decide if a spot is to the left of a box:
If the spot's x is less than the box's left, say yes.
Say no.

To decide if a spot is left or right of a box:
If the spot is to the left of the box, say yes.
If the spot is to the right of the box, say yes.
Say no.

To decide if a spot is on a box:
Put the box into another box.

```
Put 2 times the tpp into a number.
Outdent the other box given the number.
If the spot is not in the other box, say no.
Put the box into a third box.
Put 3 times the tpp into another number.
Indent the third box given the other number.
If the spot is in the third box, say no.
Say yes.

To decide if a spot is on an ellipse:
Put the ellipse into another ellipse.
Put 2 times the tpp into a number.
Outdent the other ellipse's box given the number.
If the spot is not in the other ellipse, say no.
Put the ellipse into a third ellipse.
Put 3 times the tpp into another number.
Indent the third ellipse's box given the other number.
If the spot is in the third ellipse, say no.
Say yes.

To decide if a spot is on a line:
Privatize the line.
Put 3 times the tpp into a number.
Loop.
Get a distance between the spot and the line's center (chessboard).
If the distance is less than or equal to the number, say yes.
Get the distance between the line's start and the line's end (chessboard).
If the distance is less than or equal to the tpp, say no.
Split the line into the line and another line.
Get the distance between the spot and the line's center (chessboard).
Get another distance between the spot and the other line's center (chessboard).
If the distance is greater than the other distance, put the other line into the line.
Repeat.

To decide if a spot is on a picture:
If the picture is nil, say no.
If the spot is on the picture's box, say yes.
Say no.

To decide if a spot is on a polygon:
If the polygon is nil, say no.
Loop.
Get a vertex from the polygon's vertices.
If the vertex is nil, say no.
If the vertex's next is nil, say no.
Put the vertex's spot and the vertex's next's spot into a line.
If the spot is on the line, say yes.
Repeat.

To decide if a spot is on a roundy box:
Put the roundy box into another roundy box.
Put 2 times the tpp into a number.
Outdent the other roundy box given the number.
If the spot is not in the other roundy box, say no.
Put the roundy box into a third roundy box.
```

```
Put 3 times the tpp into another number.
Indent the third roundy box given the other number.
If the spot is in the third roundy box, say no.
Say yes.

To decide if a spot is outside a box:
If the spot is inside the box, say no.
Say yes.

To decide if a spot is to the right of a box:
If the spot's x is greater than the box's right, say yes.
Say no.

To decide if a spot is touching a box:
If the spot is in the box, say yes.
Say no.

To decide if a spot is within a grid of another spot:
Get a difference between the other spot and the spot.
If the difference is within the grid, say yes.
Say no.

To decide if a spot is within some twips of another spot:
Put the twips and the twips into a grid.
If the spot is within the grid of the other spot, say yes.
Say no.

To decide if a spot is within some twips of a box:
Privatize the box.
Outdent the box given the twips.
If the spot is within the box, say yes.
Say no.

To decide if the stack has just one thing on it:
If the stack's count is 1, say yes.
Say no.

To decide if a string does end with another string;
To decide if a string ends with another string:
If the other string's length is greater than the string's length, say no.
Slap a substring on the string.
Put the substring's last minus the other string's length plus 1 into the substring's first.
If the substring is the other string, say yes.
Say no.

To decide if a string does start with another string;
To decide if a string starts with another string:
If the other string's length is greater than the string's length, say no.
Slap a substring on the string.
Put the substring's first plus the other string's length minus 1 into the substring's last.
If the substring is the other string, say yes.
Say no.

To decide if a string does start with a byte;
To decide if a string starts with a byte:
```

```
If the string is blank, say no.
If the string's first's target is the byte, say yes.
Say no.

To decide if a string is another string:
Compare the string to the other string given the string's length and the other string's length (equal only).

To decide if a string is another string or a third string:
If the string is the other string, say yes.
If the string is the third string, say yes.
Say no.

To decide if a string is any fraction literal;
To decide if a string is any ratio literal:
Slap a substring on the string.
If the substring is blank, say no.
If the substring's first's target is not any numeric starter, say no.
If the substring's first's target is any sign, add 1 to the substring's first.
Split the substring into a numerator substring and a denominator substring given the slash byte.
If the numerator substring is not any integer literal, say no.
If the denominator substring is not any integer literal, say no.
Say yes.

To decide if a string is any integer:
Slap a substring on the string.
If the substring is blank, say no.
If the substring's first's target is any sign, add 1 to the substring's first.
If the substring is blank, say no.
Loop.
If the substring's first's target is not any digit, say no.
Add 1 to the substring's first.
If the substring is blank, say yes.
Repeat.

To decide if a string is any integer literal:
Slap a substring on the string.
If the substring is blank, say no.
If the substring's first's target is any sign, add 1 to the substring's first.
If the substring is blank, say no.
Loop.
If the substring's first's target is not any digit, say no.
Add 1 to the substring's first.
If the substring is blank, say yes.
Repeat.

To decide if a string is any mixed literal:
Slap a substring on the string.
If the substring is blank, say no.
If the substring's first's target is not any numeric starter, say no.
If the substring's first's target is any sign, add 1 to the substring's first.
Split the substring into an integer substring and a fraction substring given the dash byte.
If the integer substring is not any integer literal, say no.
If the fraction substring is not any fraction literal, say no.
Say yes.
```

```
To decide if a string is any numeric literal:
If the string is blank, say no.
If the string's first's target is not any numeric starter, say no.
If the string is any integer literal, say yes.
If the string is any fraction literal, say yes.
If the string is any mixed literal, say yes.
Say no.

To decide if a string is any sign:
If the string's length is not 1, say no.
If the string's first's target is any sign, say yes.
Say no.

To decide if a string is any word:
If the string's length is less than 2, say no.
Slap a substring on the string.
Subtract 1 from the substring's first.
Loop.
Add 1 to the substring's first.
If the substring is blank, say yes.
If the substring's first's target is any letter, repeat.
If the substring's first's target is the single-quote byte, repeat.
Say no.

To decide if a string is blank:
\ assume true
Intel $B801000000. \ mov eax,1
Intel $8B9D08000000. \ mov ebx,[ebp+8] \ the string
\ if first is 0, exit
Intel $833B00. \ cmp [ebx],0
Intel $0F8410000000. \ je end
\ if last is less than first, exit
Intel $8B5304. \ mov edx,[ebx+4] \ last pointer
Intel $3B13. \ cmp edx,[ebx]
Intel $0F8C05000000. \ jl end
\ SAY NO:
Intel $B800000000. \ mov eax,0
\ END:

To decide if a string is a byte:
If the string's length is not 1, say no.
If the string's first's target is the byte, say yes.
Say no.

To decide if a string is dos-compatible:
If the string is blank, say no.
If the string's first's target is the period byte, say no.
Slap a substring on the string.
Loop.
If the substring is blank, say yes.
If the substring's first's target is the slash byte, say no.
If the substring's first's target is the backslash byte, say no.
If the substring's first's target is the colon byte, say no.
If the substring's first's target is the asterisk byte, say no.
If the substring's first's target is the question-mark byte, say no.
```

```
If the substring's first's target is the double-quote byte, say no.
If the substring's first's target is the left-alligator byte, say no.
If the substring's first's target is the right-alligator byte, say no.
If the substring's first's target is the bar byte, say no.
Add 1 to the substring's first.
Repeat.

To decide if a string is greater than another string:
Compare the string to the other string given the string's length and the other string's length returning a number.
If the number is greater than 0, say yes.
Say no.

To decide if a string is greater than or equal to another string:
Compare the string to the other string given the string's length and the other string's length returning a number.
If the number is greater than or equal to 0, say yes.
Say no.

To decide if a string is in another string:
Slap a substring on the other string.
Put the substring's first plus the string's length minus 1 into the substring's last.
Loop.
If the substring's last is greater than the other string's last, say no.
If the substring is the string, say yes.
Move the substring given 1.
Repeat.

To decide if a string is in an index:
\  if the index is nil, say no. \ to make compiler faster
If the string is blank, say no.
Find a refer given the string and the index.
If the refer is nil, say no.
Say yes.

To decide if a string is less than another string:
Compare the string to the other string given the string's length and the other string's length returning a number.
If the number is less than 0, say yes.
Say no.

To decide if a string is less than or equal to another string:
Compare the string to the other string given the string's length and the other string's length returning a number.
If the number is less than or equal to 0, say yes.
Say no.

To decide if a string is misspelled:
If the lexicon is nil, say no.
If the string is not any word, say no.
Privatize the string.
If the string ends with "'s", remove the last two bytes from the string.
If the string is in the lexicon's index, say no.
Say yes.
```

To decide if a string is wider than a box: \ assumes font is selected on memory canvas
Get an abca and an abcc given the string and the memory canvas.
Get a width given the string and the memory canvas.
Subtract the abca from the width.
Subtract the abcc from the width.
If the width is greater than the box's width, say yes.
Say no.

To decide if a substring is on any contraction:
Put the substring's first plus 1 into a byte pointer.
If the byte pointer is greater than the substring's last, say no.
If the substring's first's target is not the single-quote byte, say no.
If the byte pointer's target is not any letter, say no.
Say yes.

To decide if a terminal is full:
Put the terminal's height divided by 1/4 inch into a number.
If the terminal's quoras' count is greater than the number, say yes.
Say no.

To decide if a text can be redone:
If the text is nil, say no.
If the text's redos' last is nil, say no.
Say yes.

To decide if a text can be undone:
If the text is nil, say no.
If the text's undos' last is nil, say no.
Say yes.

To decide if a text is modified:
If the text is nil, say no.
If the text's modified flag is set, say yes.
Say no.

To decide if there is something to backspace in a text:
If the text is nil, say no.
If something is selected in the text, say yes.
If the text's caret row# is not 1, say yes.
If the text's caret column# is not 1, say yes.
Say no.

To decide if there is something to remove in a text:
If the text is nil, say no.
If something is selected in the text, say yes.
If the text's caret row# is not the text's row count, say yes.
Get a row given the text's caret row# and the text.
If the text's caret column# is not the row's string's length, say yes.
Say no.

To decide if there is text on the windows clipboard:
Call "user32.dll" "IsClipboardFormatAvailable" with 1 [cf_text] returning a number.
If the number is 0, say no.
Say yes.

```
To decide if some things are empty:
If the things' first is nil, say yes.
Say no.

To decide if a token is numeric:
Privatize the token.
Remove any leading noise from the token.
If the token is blank, say no.
Loop.
Get a byte from the token.
If the byte is not any digit, say no.
If the token is blank, say yes.
Repeat.

To decide if the user is clicking in some choices;
To decide if the user has clicked in some choices;
To decide if the user clicked in some choices;
To decide if the user clicks in some choices;
To decide if the user is clicking on some choices;
To decide if the user has clicked on some choices;
To decide if the user clicked on some choices;
To decide if the user clicks on some choices:
Loop.
If the mouse's left button is not down, say no.
Find a choice given the mouse's spot.
If the choice can't be found, say no.
Say yes.

To decide if we ca find a pointer: \ should be "can" not "ca", compiler bug
If the pointer is nil, say no.
Say yes.

To decide if we could find a pointer:
If the pointer is nil, say no.
Say yes.

To decide if we did find a pointer:
If the pointer is nil, say no.
Say yes.

To decide if we're above a box:
If the context's y is less than the box's top, say yes.
Say no.

To decide if we're above a coord:
If the context's y is less than the coord, say yes.
Say no.

To decide if we're above a horizontal line:
If the context's spot is above the horizontal line, say yes.
Say no.

To decide if we're above or below a box:
If the context's y is less than the box's top, say yes.
If the context's y is greater than the box's bottom, say yes.
```

```
Say no.

To decide if we're above a spot:
If the context's y is less than the spot's y, say yes.
Say no.

To decide if we're below a box:
If the context's y is greater than the box's bottom, say yes.
Say no.

To decide if we're below a coord:
If the context's y is greater than the coord, say yes.
Say no.

To decide if we're below a horizontal line:
If the context's spot is below the horizontal line, say yes.
Say no.

To decide if we're below a spot:
If the context's y is greater than the spot's y, say yes.
Say no.

To decide if we're facing north: \ *** need east, south, west
Normalize the context's heading.
If the context's heading is 0, say yes.
Say no.

To decide if we're left of a box:
If the context's x is less than the box's left, say yes.
Say no.

To decide if we're left or right of a box:
If the context's x is less than the box's left, say yes.
If the context's x is greater than the box's right, say yes.
Say no.

To decide if we're outside a box:
If the context's spot is outside the box, say yes.
Say no.

To decide if we're right of a box:
If the context's x is greater than the box's right, say yes.
Say no.

To decide if we're still in a box;
To decide if we're in a box:
If the context's spot is in the box, say yes.
Say no.

To decide if we're within some twips of a box:
If the context's spot is within the twips of the box, say yes.
Say no.

To decide if you feel like it:
Pick a number between 1 and 100.
```

If the number is less than 51, say yes.
Say no.

To decrypt a buffer given a passphrase string: \ sets i/o error if failure
Clear the i/o error.
Create a crypt session given the passphrase.
If the crypt session is nil, exit.
Convert the buffer as a nibble string to a hex string.
Put the hex string's length into a length.
Call "advapi32.dll" "CryptDecrypt" with the crypt session's hcryptkey and 0 and 1 and 0 and the hex string's first
And the length's whereabouts returning a result number.
If the result number is 0, put "Error decrypting data." into the i/o error; destroy the crypt session; exit.
Destroy the crypt session.
Put the hex string into the buffer.

The console is a console.

The default font is a font.

The default smtp server is "localhost".

A degree is a number [0 to 359].

The degree-symbol byte is a byte equal to 176.

The delete byte is a byte equal to 127.

The delete key is a key equal to 46.

A depth is some twips.

A description is a string.

To deselect a text:
If the text is nil, exit.
Put the text's caret into the text's anchor.

A designator is a string. \ rightmost directory with slash = folder2\ OR after the last slash to end of path = file.ext

To destroy a crypt session:
If the crypt session is nil, exit.
Call "advapi32.dll" "CryptDestroyKey" with the crypt session's hcryptkey.
Call "advapi32.dll" "CryptDestroyHash" with the crypt session's hcrypthash.
Call "advapi32.dll" "CryptReleaseContext" with the crypt session's hcryptprov and 0.
Deallocate the crypt session.

To destroy a gpimage:
If the gpimage is nil, exit.
Call "gdiplus.dll" "GdipDisposeImage" with the gpimage.
Void the gpimage.

To destroy a gpimageattributes:
If the gpimageattributes is nil, exit.

Call "gdiplus.dll" "GdipDisposeImageAttributes" with the gpimageattributes.
Void the gpimageattributes.

To destroy the hbrush of a canvas:
Call "gdi32.dll" "SelectObject" with the canvas and the null hbrush returning an hbrush.
Call "gdi32.dll" "DeleteObject" with the hbrush.

To destroy the hfont of a canvas:
Call "gdi32.dll" "SelectObject" with the canvas and the null hfont returning an hfont.
Call "gdi32.dll" "DeleteObject" with the hfont.

To destroy the hpen of a canvas:
Call "gdi32.dll" "SelectObject" with the canvas and the null hpen returning an hpen.
Call "gdi32.dll" "DeleteObject" with the hpen.

To destroy an hrgn:
Call "gdi32.dll" "DeleteObject" with the hrgn.

To destroy an index:
If the index is nil, exit.
Loop.
Get a bucket given the index.
If the bucket is nil, break.
Destroy the bucket's refers.
Repeat.
Unassign the index's first bucket.
Deallocate the index.

To destroy a path in the file system:
Set the path to read-write mode.
If the path is directory-format, destroy the path in the file system (directory).
If the path is file-format, destroy the path in the file system (file).

To destroy a path in the file system (directory):
Loop.
Get an item from the path.
If the item is not found, break.
Put the path into another path.
Append the item's designator to the other path.
Destroy the other path in the file system.
If the i/o error is not blank, exit.
Repeat.
Privatize the path.
Null terminate the path.
Call "kernel32.dll" "RemoveDirectoryA" with the path's first returning a number.
Clear the i/o error.
If the number is not 0, exit.
Put "Error deleting directory '" then the path then "'." into the i/o error.

To destroy a path in the file system (file):
Privatize the path.
Null terminate the path.
Call "kernel32.dll" "DeleteFileA" with the path's first returning a number.
Clear the i/o error.
If the number is not 0, exit.

Put "Error deleting file '" then the path then "'." into the i/o error.

To destroy a picture:
If the picture is nil, exit.
Destroy the picture's gpbitmap.
Deallocate the picture.

To destroy a socket:
Call "ws2_32.dll" "closesocket" with the socket.

To destroy a vertex given a polygon:
If the vertex is nil, exit.
If the polygon is nil, exit.
Privatize the vertex.
Remove the vertex from the polygon's vertices.
Destroy the vertex.

To destroy a winhttp request:
If the winhttp request is nil, exit.
Call "winhttp.dll" "WinHttpCloseHandle" with the winhttp request's request.
Call "winhttp.dll" "WinHttpCloseHandle" with the winhttp request's connection.
Call "winhttp.dll" "WinHttpCloseHandle" with the winhttp request's session.
Deallocate the winhttp request.

The device-control-four byte is a byte equal to 20.

The device-control-one byte is a byte equal to 17.

The device-control-three byte is a byte equal to 19.

The device-control-two byte is a byte equal to 18.

A devmode is a record with
32 bytes called dmdevicename,
A wyrd called dmspecversion,
A wyrd called dmdriverversion,
A wyrd called dmsize,
A wyrd called dmdriverextra,
A number called dmfields,
A wyrd called dmorientation,
A wyrd called dmpapersize,
A wyrd called paperlength,
A wyrd called paperwidth,
A wyrd called dmscale,
A wyrd called dmcopies,
A wyrd called dmdefaultsource,
A wyrd called dmprintquality,
A wyrd called dmcolor,
A wyrd called dmduplex,
A wyrd called ydmresolution,
A wyrd called dmttoption,
A wyrd called dmcollate,
32 bytes called dmformname,
A wyrd called dmlogpixels,
A number called dmbitsperpel,

A number called dmpelswidth,
A number called dmpelsheight,
A number called dmdisplayflags,
A number called dmdisplayfrequency,
A number called dmicmmethod,
A number called dmicmintent,
A number called dmmediatype,
A number called dmdithertype,
A number called dmreserved1,
A number called dmreserved2.

The diaeresis byte is a byte equal to 168.

A difference is a pair.

A directory is a path. \ start of path to last slash inclusive = c:\folder1\folder2\

A directory name is a string. \ rightmost directory with slash = folder2\

A directory name w/o slash is a string. \ rightmost directory without slash = folder2

A distance is a number.

To divide a fraction by another fraction:
Privatize the other fraction.
Flip the other fraction.
Multiply the fraction by the other fraction.

To divide a fraction by a number:
Multiply the fraction's denominator by the number.
Reduce the fraction.

To divide a number by a fraction:
Privatize the fraction.
Flip the fraction.
Multiply the number by the fraction.

To divide a pair by another pair:
Divide the pair's x by the other pair's x.
Divide the pair's y by the other pair's y.

To divide a pair by a number:
Divide the pair's x by the number.
Divide the pair's y by the number.

To divide a pair by a number and another number:
Divide the pair's x by the number.
Divide the pair's y by the other number.

To divide a pointer by a number;
To divide a number by another number:
If the other number is 0, put the largest number into the number; exit.
Intel $8B8508000000. \ mov eax,[ebp+8] \ the number
Intel $8B9D0C000000. \ mov ebx,[ebp+12] \ the other number
Intel $8B00. \ mov eax,[eax]

Intel $99. \ cdq
Intel $F73B. \ div [ebx] \ means div eax,[ebx] but is weird form
Intel $8B9D08000000. \ mov ebx,[ebp+8] \ the number
Intel $8903. \ mov [ebx],eax

To divide a pointer by a number giving a quotient and a remainder;
To divide a number by another number giving a quotient and a remainder:
If the other number is 0, put the largest number into the number; put 0 into the remainder; exit.
Intel $8B8508000000. \ mov eax,[ebp+8] \ the number
Intel $8B9D0C000000. \ mov ebx,[ebp+12] \ the other number
Intel $8B00. \ mov eax,[eax]
Intel $99. \ cdq
Intel $F73B. \ idiv [ebx] \ means idiv eax,[ebx] but is weird form
Intel $8B9D10000000. \ mov ebx,[ebp+16] \ the quotient
Intel $8903. \ mov [ebx],eax
Intel $8B9D14000000. \ mov ebx,[ebp+20] \ the remainder
Intel $8913. \ mov [ebx],edx

The division-symbol byte is a byte equal to 247.

A docinfo is a record with
A number called cbsize,
A pointer called lpszdocname,
A pointer called lpszoutput,
A pointer called lpszdatadtype,
A number called fwtype.

The dollar-sign byte is a byte equal to 36.

A dot is an ellipse.

To double a number:
Add the number to the number.

The double-dagger byte is a byte equal to 135.

The double-quote byte is a byte equal to 34.

The down-arrow key is a key equal to 40.

To draw and fill a box with a color:
Draw the box with the color and the color.

To draw any visible rows of a text:
If the text is nil, exit.
Loop.
Get a row from the text’s rows.
If the row is nil, exit.
If the row of the text is not visible, repeat.
Draw the row of the text.
Repeat.

To draw any visible rows of a text (editing):
If the text is nil, exit.
Loop.

Get a row from the text's rows.
If the row is nil, exit.
If the row of the text is not visible, repeat.
Draw the row of the text (editing).
Repeat.

To draw a box:
Draw the box with the black color and the clear color.

To draw a box (focus style):
Privatize the box.
Add the tpp to the box's right-bottom.
Call "gdi32.dll" "LPtoDP" with the current canvas and the box's whereabouts and 2.
Convert the white color to a colorref.
Call "gdi32.dll" "SetBkColor" with the current canvas and the colorref.
Convert the black color to another colorref.
Call "gdi32.dll" "SetTextColor" with the current canvas and the other colorref.
Call "gdi32.dll" "SetMapMode" with the current canvas and 1 [mm_text].
Call "user32.dll" "DrawFocusRect" with the current canvas and the box's whereabouts.
Normalize the current canvas.

To draw a box in a color;
To draw a box with a color:
Draw the box with the color and the clear color.

To draw a box with a border color and a fill color:
If the pdf state's page flag is set, draw the box with the border and the fill (pdf style); exit.
Privatize the box.
Create the hpen of the current canvas given the border.
Create the hbrush of the current canvas given the fill.
If the border is clear, add the tpp to the box's left-top.
Call "gdi32.dll" "Rectangle" with the current canvas and the box's left and the box's top and the box's right and the box's bottom.
Destroy the hbrush of the current canvas.
Destroy the hpen of the current canvas.

To draw a box with a border color and a fill color (pdf style):
If the border and the fill are clear, exit.
Output setcolor given the border and the fill.
Output the box's left-bottom without advancing.
Output the box's x-extent without advancing.
Output the box's y-extent without advancing.
Output "re".
Output stroke and fill given the border and the fill.

To draw the caret in a text:
If the text is nil, exit.
Get a box for the caret in the text.
If the box is not touching the text's box, exit.
Put the box's left-top and the box's right-bottom into a line.
Draw the line with the black color.

To draw a circle about some twips wide;
To draw a circle given a width;
To draw a circle some twips in width;

To draw a circle some twips wide:
Put the twips times the pizza pie divided by 48 into a segment length.
Add 1 to the segment length.
Loop.
Stroke the segment length.
Turn right 1/48 of the way.
Add 1 to a count. If the count is 48, break.
Repeat.

To draw a circle some twips wide (backwards);
To draw a circle some twips wide (counterclockwise):
Put the twips times the pizza pie divided by 48 into a segment length.
Add 1 to the segment length.
Loop.
Stroke the segment length.
Turn left 1/48 of the way.
Add 1 to a count. If the count is 48, break.
Repeat.

To draw a console:
If the console is nil, exit.
Save the current canvas.
Mask only outside the console's box.
Draw the console's box with the console's border and the console's fill.
Draw the console's text.
Restore the current canvas.

To draw a dot some twips wide on the current spot with a color:
Make the dot the twips wide.
Center the dot on the context's spot.
Draw the dot with the color.

To draw a dot some twips wide on a spot with a color:
Make the dot the twips wide.
Center the dot on the spot.
Draw the dot with the color.

To draw an ellipse:
Draw the ellipse with the black color and the clear color.

To draw an ellipse on a spot with a color:
Center the ellipse on the spot.
Draw the ellipse with the color and the clear color.

To fill an ellipse on a spot with a color:
Center the ellipse on the spot.
Draw the ellipse with the clear color and the color.

To draw an ellipse with a border color and a fill color:
If the pdf state's page flag is set, draw the ellipse with the border and the fill (pdf style); exit.
Privatize the ellipse.
Create the hpen of the current canvas given the border.
Create the hbrush of the current canvas given the fill.
If the border is clear, add the tpp to the ellipse's left-top.
Call "gdi32.dll" "Ellipse" with the current canvas and the ellipse's left and the ellipse's top and the ellipse's

right and the ellipse’s bottom.
Destroy the hbrush of the current canvas.
Destroy the hpen of the current canvas.

To draw an ellipse with a border color and a fill color (pdf style):
If the border and the fill are clear, exit.
Output setcolor given the border and the fill.
Put the ellipse’s left and the ellipse’s top into a spot.
Add the ellipse’s y-extent divided by 2 to the spot’s y.
Output moveto given the spot.
Output the arc of the ellipse given "left-top".
Output the arc of the ellipse given "right-top".
Output the arc of the ellipse given "right-bottom".
Output the arc of the ellipse given "left-bottom".
Output stroke and fill given the border and the fill.

To draw an ellipse with a color:
Draw the ellipse with the color and the color.

To draw a fancy arrow some twips long with a color;
To stroke a fancy arrow some twips long with a color:
Put the color into the context’s color.
Save the context.
Stroke the twips.
Turn around.
Move the twips divided by 8.
Save the context.
Turn left 1/8 of the way.
Draw a spiral starting with the twips divided by 4.
Restore the context.
Turn right 1/8 of the way.
Draw another spiral backwards starting with the twips divided by 4.
Restore the context.

To draw a figure with a color:
Draw the figure with the color and the clear color.

To draw a figure with some sides about some twips wide:
Put 1 and the side count into a fraction.
Put the twips times the pizza pie divided by the sides into a segment length.
If the sides are 4, put the twips times 7/8 into the segment length. \ what is this? ***
Loop.
Stroke the segment length.
Turn the fraction.
Add 1 to a count. if the count is the sides, break.
Repeat.

To draw a figure with some sides some twips long;
To draw a figure with some sides and some twips:
Put 1 and the sides into a fraction.
Loop.
Stroke the twips.
Turn the fraction.
Add 1 to a count. If the count is the sides, break.
Repeat.

To draw a figure with some sides some twips long using a color;
To draw a figure with some sides and some twips using a color:
Put the color into the context's color.
Draw the figure with the sides and the twips.

To draw a gpbitmap at a spot (pdf style):
If the gpbitmap is nil, exit.
\ add xobject
Add an xobject pdf object given "image object".
Put "I" then the xobject's number into a name.
\ add to image resources in current page
Put "/" then the name then " " then the xobject's number then " 0 R" into a string.
Create a new string thing given the string.
Append the new string thing to the pdf state's current page's image strings.
\ finish setting up xobject
Append the xobject's number then " 0 obj" to the xobject.
Append "<<" to the xobject.
Append "/Type /XObject" to the xobject.
Append "/Subtype /Image" to the xobject.
Append "/ColorSpace /DeviceRGB" to the xobject.
Append "/Width " then the gpbitmap's width to the xobject.
Append "/Height " then the gpbitmap's height to the xobject.
Append "/BitsPerComponent 8" to the xobject.
Convert the gpbitmap to a buffer (pdf style).
Convert the buffer to a nibble string.
Append "/Filter /ASCIIHexDecode" to the xobject.
Append "/Length " then the nibble string's length to the xobject.
Append ">>" to the xobject.
Append "stream" to the xobject.
Append the nibble string to the xobject.
Append "endstream" to the xobject.
Append "endobj" to the xobject.
\ draw the image in the current contents
Put the gpbitmap's width times the tpp into a width.
Put the gpbitmap's height times the tpp into a height.
Put the spot's x into an x number.
Put the pdf state's current height minus the spot's y minus the height into a y number.
Output "q".
Output the width then " 0 0 " then the height then " " then the x then " " then the y then " cm".
Output "/" then the name then " Do".
Output "Q".

To draw a half circle about some twips wide;
To draw a half circle given a size:
Put the size times the pizza pie divided by 48 into a segment length.
Add 1 to the segment length.
Loop.
Stroke the segment length.
Turn right 1/48 of the way.
Add 1 to a count.
If the count is 24, exit.
Repeat.

To draw a half circle given a size (backwards);

```
To draw a half circle given a size (counterclockwise):
Put the size times the pizza pie divided by 48 into a segment length.
Add 1 to the segment length.
Loop.
Stroke the segment length.
Turn left 1/48 of the way.
Add 1 to a count.
If the count is 24, exit.
Repeat.

To draw a hexagon given a side length:
Stroke the side length.
Turn right 1/6 of the way.
Add 1 to a count. If the count is 6, break.
Repeat.

To draw a line:
Draw the line with the context’s color.

To draw a line around some twips long; \ "around" is significant here
To draw a line about some twips long; \ "about" is significant here
To stroke a line around some twips long; \ "around" is significant here
To stroke a line about some twips long: \ "about" is significant here
Put the twips divided by 2 into some other twips.
Pick some third twips between the twips minus the other twips and the twips plus the other twips.
Stroke the line the third twips long.

To draw a line as high as a box with a color;
To draw a line as tall as a box with a color:
Put the color into the context’s color.
Stroke the box’s height.

To draw a line as wide as a box with a color:
Put the color into the context’s color.
Stroke the box’s width.

To draw a line between some twips and some other twips long;
To draw a line between some twips to some other twips long;
To draw a line some twips to some other twips long;
To stroke a line some twips to some other twips long:
Pick some third twips between the twips and the other twips.
Stroke the line the third twips long.

To draw a line some twips long;
To stroke a line some twips long:
\Wait for the delay. ***
Put the context’s spot into the line’s start.
Put the context’s spot into the line’s end.
Get a rise and a run given the context’s heading.
Add the run times the twips divided by 10000 to the line’s end’s x.
Add the rise times the twips divided by 10000 to the line’s end’s y.
Put the line’s end into the context’s spot.
Draw the line with the context’s color.
\If the delay is positive, refresh the screen.
```

To draw a line using some twips as the length;
To draw a line using some twips for the length:
Stroke the twips.

To draw a line with a color:
If the pdf state's page flag is set, draw the line with the color (pdf style); exit.
Create the hpen of the current canvas given the color.
Call "gdi32.dll" "MoveToEx" with the current canvas and the line's start's x and the line's start's y and nil.
Call "gdi32.dll" "LineTo" with the current canvas and the line's end's x and the line's end's y.
Convert the color to a colorref.
Call "gdi32.dll" "SetPixelV" with the current canvas and the line's end's x and the line's end's y and the colorref.
Destroy the hpen of the current canvas.

To draw a line with a color (pdf style):
If the color is clear, exit.
Output setcolor given the color and the clear color.
Output moveto given the line's start.
Output lineto given the line's end.
Output stroke and fill given the color and the clear color.

To draw a line with a color around some twips long; \ "around" is significant here
To draw a line with a color about some twips long; \ "about" is significant here
To stroke a line with a color around some twips long; \ "around" is significant here
To stroke a line with a color about some twips long: \ "about" is significant here
Put the twips divided by 2 into some other twips.
Pick some third twips between the twips minus the other twips and the twips plus the other twips.
Stroke the line with the color the third twips long.

To draw a line with a color some twips long;
To stroke a line with a color some twips long:
Put the color into the context's color.
Put the context's spot into the line's start.
Stroke the twips.
Put the context's spot into the line's end.

To draw a line with a color some twips to some other twips long;
To stroke a line with a color some twips to some other twips long:
Pick some third twips between the twips and the other twips.
Stroke the line with the color the third twips long.

To draw a number in a box with a color:
Put the number into a string.
Draw the string in the box with the color and "center".

To draw only within a box;
To draw only inside a box;
To draw only in a box;
To draw only within a box from now on;
To draw only inside a box from now on;
To draw only in a box from now on;
To only draw within a box from now on;
To only draw inside a box from now on;
To only draw in a box from now on;
To only draw within a box;

To only draw inside a box;
To only draw in a box;
To draw within a box only;
To draw inside a box only;
To draw in a box only;
To put masking tape all around a box;
To put masking tape around a box: \ note that this masks the box's border as well
Privatize the box.
Indent the box 1 pixel.
Mask outside the box.
Put the original box into the context's box. \ *** original box? or indented one?

To draw a picture:
If the pdf state's page flag is set, draw the picture (pdf style); exit.
If the picture is nil, exit.
Save the current canvas.
Mask outside the picture's box.
Call "gdiplus.dll" "GdipCreateFromHDC" with the current canvas and a gpgraphic's whereabouts.
Call "gdiplus.dll" "GdipSetPageUnit" with the gpgraphic and 2 [pixels].
Draw the picture on the gpgraphic at the picture's uncropped box's left and the picture's uncropped box's top.
Call "gdiplus.dll" "GdipDeleteGraphics" with the gpgraphic.
Restore the current canvas.

To draw a picture (pdf style):
If the picture is nil, exit.
Put the picture's box's left minus the picture's uncropped box's left divided by the tpp into an x number.
Put the picture's box's top minus the picture's uncropped box's top divided by the tpp into an y number.
Put the picture's box's width divided by the tpp into a width.
Put the picture's box's height divided by the tpp into a height.
Call "gdiplus.dll" "GdipCreateBitmapFromScan0" with the width and the height and 0 and 137224 [pixelformat24bpprgb] and 0 and a gpbitmap's whereabouts.
Call "gdiplus.dll" "GdipGetImageGraphicsContext" with the gpbitmap and a gpgraphic's whereabouts.
If the picture's grayscale flag is set, create a gpimageattributes (grayscale).
Call "gdiplus.dll" "GdipDrawImageRectRectI" with the gpgraphic and the picture's gpbitmap
And 0 and 0 and the width and the height
And the x and the y and the width and the height
And 2 [unitpixel] and the gpimageattributes and nil and 0.
If the gpimageattributes is not nil, destroy the gpimageattributes.
Call "gdiplus.dll" "GdipDeleteGraphics" with the gpgraphic.
Draw the gpbitmap at the picture's box's left-top (pdf style).
Call "gdiplus.dll" "GdipDisposeImage" with the gpbitmap.

To draw a picture on a gpgraphic at an x coord and a y coord:
If the picture is nil, exit.
If the picture's grayscale flag is set, create a gpimageattributes (grayscale).
Call "gdiplus.dll" "GdipDrawImageRectRectI" with the gpgraphic and the picture's gpbitmap
And the x and the y and the picture's uncropped box's width and the picture's uncropped box's height
And 0 and 0 and the picture's gpbitmap's width and the picture's gpbitmap's height
And 2 [unitpixel] and the gpimageattributes and nil and 0.
If the gpimageattributes is not nil, destroy the gpimageattributes.

To draw a polygon:
Draw the polygon with the black color and the clear color.

To draw a polygon with a border color and a fill color:
If the pdf state's page flag is set, draw the polygon with the border and the fill (pdf style); exit.
If the polygon is nil, exit.
Draw the polygon with the fill (fill only).
Draw the polygon with the border (border only).

To draw a polygon with a border color and a fill color (pdf style):
If the polygon is nil, exit.
If the border and the fill are clear, exit.
Output setcolor given the border and the fill.
Get a vertex from the polygon's vertices.
If the vertex is nil, exit.
Output moveto given the vertex's spot.
Loop.
Get the vertex from the polygon's vertices.
If the vertex is nil, break.
Output lineto given the vertex's spot.
Repeat.
Output stroke and fill given the border and the fill.

To draw a polygon with a color:
Draw the polygon with the color and the color.

To draw a polygon with a color (border only):
If the polygon is nil, exit.
If the color is clear, exit.
Create the hpen of the current canvas given the color.
Get a vertex from the polygon's vertices.
If the vertex is nil, exit.
Call "gdi32.dll" "MoveToEx" with the current canvas and the vertex's x and the vertex's y and nil.
Loop.
Get the vertex from the polygon's vertices.
If the vertex is nil, break.
Call "gdi32.dll" "LineTo" with the current canvas and the vertex's x and the vertex's y.
Repeat.
Destroy the hpen of the current canvas.
If the polygon's vertices' first's spot is the polygon's vertices' last's spot, exit.
Convert the color to a colorref.
Call "gdi32.dll" "SetPixelV" with the current canvas and the polygon's vertices' last's x and the polygon's vertices' last's y and the colorref.

To draw a polygon with a color (fill only):
If the polygon is nil, exit.
If the color is clear, exit.
Create the hpen of the current canvas given the clear color.
Create the hbrush of the current canvas given the color.
Call "gdi32.dll" "SetPolyFillMode" with the current canvas and 2 [winding].
Create a vertex array given the polygon's vertices.
Call "gdi32.dll" "Polygon" with the current canvas and the vertex array's spot pointer and the vertex array's count.
Destroy the vertex array.
Destroy the hbrush of the current canvas.
Destroy the hpen of the current canvas.

To draw a quarter circle about some twips wide;

To draw a quarter circle given a size:
Put the size times the pizza pie divided by 48 into a segment length.
Add 1 to the segment length.
Loop.
Stroke the segment length.
Turn right 1/48 of the way.
Add 1 to a count.
If the count is 12, exit.
Repeat.

To draw a quarter circle between some twips and some other twips wide:
Pick some third twips between the twips and the other twips.
Draw a quarter circle given the third twips.

To draw a quarter circle given a size (counterclockwise):
Put the size times the pizza pie divided by 48 into a segment length.
Add 1 to the segment length.
Loop.
Stroke the segment length.
Turn left 1/48 of the way.
Add 1 to a count.
If the count is 12, exit.
Repeat.

To draw a rectangle some twips by some other twips;
To draw a rectangle some twips wide by some other twips high:
Stroke the other twips.
Turn right.
Stroke the twips.
Turn right.
Stroke the other twips.
Turn right.
Stroke the twips.
Turn right.

To draw a roundy box:
Draw the roundy box with the black color and the clear color.

To draw a roundy box with a border color and a fill color:
If the pdf state’s page flag is set, draw the roundy box with the border and the fill (pdf style); exit.
Privatize the roundy box.
Create the hpen of the current canvas given the border.
Create the hbrush of the current canvas given the fill.
If the border is clear, add the tpp to the roundy box’s left-top.
Put the roundy box’s radius times 2 into a diameter number.
Call "gdi32.dll" "RoundRect" with the current canvas and the roundy box’s left and the roundy box’s top and the roundy box’s right and the roundy box’s bottom
And the diameter and the diameter.
Destroy the hbrush of the current canvas.
Destroy the hpen of the current canvas.

To draw a roundy box with a border color and a fill color (pdf style):
If the border and the fill are clear, exit.
If the roundy box’s radius is 0, draw the roundy box as a box with the border and the fill (pdf style); exit.
Put the roundy box’s radius into a radius.

Put the radius times 2 into an offset.
Put the roundy box into a box.
Output setcolor given the border and the fill.
\ initial moveto
Output moveto given the box’s left and the box’s top plus the radius.
\ left-top
Put the box’s left and the box’s top and the box’s left plus the offset and the box’s top plus the offset into an ellipse.
Output the arc of the ellipse given "left-top".
Output lineto given the box’s right minus the radius and the box’s top.
\ right-top
Put the box’s right minus the offset and the box’s top and the box’s right and the box’s top plus the offset into the ellipse.
Output the arc of the ellipse given "right-top".
Output lineto given the box’s right and the box’s bottom minus the radius.
\ right-bottom
Put the box’s right minus the offset and the box’s bottom minus the offset and the box’s right and the box’s bottom into the ellipse.
Output the arc of the ellipse given "right-bottom".
Output lineto given the box’s left plus the radius and the box’s bottom.
\ left-bottom
Put the box’s left and the box’s bottom minus the offset and the box’s left plus the offset and the box’s bottom into the ellipse.
Output the arc of the ellipse given "left-bottom".
\ finish up
Output "h".
Output stroke and fill given the border and the fill.

To draw a roundy box with a color:
Draw the roundy box with the color and the color.

To draw a row of a text:
If the text is nil, exit.
If the row is nil, exit.
Get a box given the row and the text.
Draw the row’s working string in the box with the text’s pen and the text’s font and the text’s alignment.

To draw a row of a text (editing):
If the text is nil, exit.
If the row is nil, exit.
Draw the selection box for the row of the text.
Get a box given the row and the text.
Draw the row’s working string in the box with the text’s pen and the text’s font and the text’s alignment.

To draw the selection box for a row of a text:
If the text is nil, exit.
If the row is nil, exit.
If the row of the text is not selected, exit.
Get a selection box given the row and the text.
Draw the selection box with the hilite color and the hilite color.

To draw a spiral backward given some twips;
To draw a spiral backward starting with some twips;
To draw a spiral given a size (backwards):
Privatize the size.

```
Loop.
Draw a half circle given the size (backwards).
Divide the size by 2.
Add 1 to a count.
If the count is 5, break.
Repeat.

To draw a spiral starting with some twips;
To draw a spiral given a size:
Privatize the size.
Loop.
Draw a half circle given the size.
Divide the size by 2.
Add 1 to a count. If the count is 5, break.
Repeat.

To draw a spot:
Draw the spot with the black color.

To draw a spot with a color:
Convert the color to a colorref.
Call "gdi32.dll" "SetPixelV" with the current canvas and the spot's x and the spot's y and the colorref.

To draw a star given a point count and a size:
Put 1 and the point count into a fraction.
Loop.
Turn right 1/48 of the way.
Stroke the size.
Turn around.
Turn left 1/24 of the way.
Stroke the size.
Turn around.
Turn right 1/48 of the way.
Turn right the fraction of the way.
Add 1 to a count. If the count is the point count, break.
Repeat.

To draw a string at the left of a box:
Draw the string at the left of the box with the black color and the default font.

To draw a string at the left of a box with a color:
Draw the string at the left of the box with the color and the default font.

To draw a string at the left of a box with a color and a font:
Draw the string in the box with the color and the font and "left".

To draw a string at the left of a box with a font:
Draw the string at the left of the box with the black color and the font.

To draw a string at the right of a box:
Draw the string at the right of the box with the black color and the default font.

To draw a string at the right of a box with a color:
Draw the string at the right of the box with the color and the default font.
```

To draw a string at the right of a box with a color and a font:
Draw the string in the box with the color and the font and "right".

To draw a string at the right of a box with a font:
Draw the string at the right of the box with the black color and the font.

To draw a string at a spot with a color:
Draw the string at the spot with the color and the default font.

To draw a string at a spot with a color and a font:
If the pdf state's page flag is set, draw the string at the spot with the color and the font (pdf style); exit.
Set the colorref of the current canvas given the color.
Create the hfont of the current canvas given the font.
Adjust spacing given the string.
Put the string's first into a substring's first.
Put the substring's first plus the text cutoff minus 1 into the substring's last.
Privatize the spot.
Loop.
If the substring is blank, break.
If the substring's last is greater than the string's last, put the string's last into the substring's last.
Call "gdi32.dll" "TextOutA" with the current canvas and the spot's x and the spot's y and the substring's first and the substring's length.
Get a width given the substring and the current canvas.
Add the width to the spot's x.
Move the substring given the text cutoff.
Repeat.
Destroy the hfont of the current canvas.

To draw a string at a spot with a color and a font (pdf style):
If the string is blank, exit.
Privatize the spot.
Include the font in the current pdf.
Include the font in the pdf state's current page.
Find a definition pdf object given the font's name and the pdf state's font index.
If the definition is nil, exit. \ error
Create a font info given the font and the string.
Output the pdf border given the color.
Output the pdf fill given the color.
Output "BT".
Output "/" then the definition's font name then " " then the font's adjusted height then " Tf".
\ add the font's adjusted height minus the font info's internal leading to the spot's y. \ just plain wrong
Add the font info's ascent to the spot's y. \ fix for line above
Output the spot without advancing.
Output "Td".
Output "[" without advancing.
Convert the font info to pdf em units.
Convert the string and the font info and the definition's font info into a buffer for pdf output.
Output the buffer without advancing.
Output "]" without advancing.
Output " TJ".
Output "ET".
Destroy the font info.

To draw a string in a box:
Draw the string in the box with the black color and the default font and "left".

To draw a string in a box over a number with a color and a font and an alignment:
Privatize the box.
If the alignment is "left", add the number to the box's left.
If the alignment is "right", subtract the number from the box's right.
Draw the string in the box with the color and the font and the alignment.

To draw a string in a box with an alignment:
Draw the string in the box with the black color and the default font and the alignment.

To draw a string in a box with a color:
Draw the string in the box with the color and the default font and "left".

To draw a string in a box with a color and an alignment:
Draw the string in the box with the color and the default font and the alignment.

To draw a string in a box with a color and a font and an alignment:
Get an offset pair given the string and the box and the font and the alignment.
Draw the string at the box's left-top plus the offset pair with the color and the font.

To draw a string in a box with a font and an alignment:
Draw the string in the box with the black color and the font and the alignment.

To draw a string in the center of a box:
Draw the string in the center of the box with the black color and the default font.

To draw a string in the center of a box with a color:
Draw the string in the center of the box with the color and the default font.

To draw a string in the center of a box with a color and a font:
Draw the string in the box with the color and the font and "center".

To draw a string in the center of a box with a font:
Draw the string in the center of the box with the black color and the font.

To draw a terminal:
If the terminal is nil, exit.
Save the current canvas.
Mask only outside the terminal's box.
Draw and fill the terminal's box with the black color.
Write the terminal's quoras in the terminal's box.
Restore the current canvas.

To draw a text:
If the text is nil, exit.
Save the current canvas.
Mask outside the text's box.
Draw any visible rows of the text.
Restore the current canvas.

To draw a text (editing):
If the text is nil, exit.
Save the current canvas.
Mask outside the text's box.
Draw any visible rows of the text (editing).

Draw the caret in the text.
Restore the current canvas.

A drive is a string. \ start of path to first slash = c:\ OR start of path to fourth slash = \\computer\share\

A drive kind is a string.

A drive name is a string.

To duplicate a path to another path in the file system:
If the path is directory-format, duplicate the path to the other path in the file system (directory).
If the path is file-format, duplicate the path to the other path in the file system (file).

To duplicate a path to another path in the file system (directory):
If the path is in the other path, put "Error duplicating directory '" then the path then "' - invalid recursion." into the i/o error; exit.
If the path is not in the file system, put "Error duplicating directory '" then the path then "'." into the i/o error; exit.
If the other path is not in the file system, create the other path in the file system.
Loop.
Get an item from the path.
If the item is not found, exit.
Put the path into a third path.
Append the item's designator to the third path.
Put the other path into a fourth path.
Append the item's designator to the fourth path.
Duplicate the third path to the fourth path in the file system.
Repeat.

To duplicate a path to another path in the file system (file):
Privatize the path.
Null terminate the path.
Privatize the other path.
Null terminate the other path.
Call "kernel32.dll" "CopyFileA" with the path's first and the other path's first and 0 returning a number.
Clear the i/o error.
If the number is not 0, set the path to read-write mode; exit.
Put "Error duplicating file '" then the path then "'." into the i/o error.

A dyad is a thing with
A name,
A value string.

The e-key is a key equal to 69.

The eight byte is a byte equal to 56.

The eight key is a key equal to 56.

An ellipse has a box.

The ellipsis byte is a byte equal to 133.

The em-dash byte is a byte equal to 151.

An email has
A smtp server,
A sender,
A recipient,
A subject,
A message.

The en-dash byte is a byte equal to 150.

To encrypt a buffer given a passphrase string: \ sets i/o error if failure
Clear the i/o error.
Create a crypt session given the passphrase.
If the crypt session is nil, exit.
Put the buffer into a temp buffer.
Put the temp buffer's length into a length.
Call "advapi32.dll" "CryptEncrypt" with the crypt session's hcryptkey and 0 and 1 and 0 and the temp buffer's first
And the length's whereabouts and the length returning a result number.
If the result number is 0, put "Error encrypting data." into the i/o error; destroy the crypt session; exit.
Destroy the crypt session.
Convert the temp buffer to a nibble string.
Put the nibble string into the buffer.

The end key is a key equal to 35.

To end printing:
If the pdf state's document flag is set, end printing (pdf style); exit.
Call "gdi32.dll" "EndDoc" with the printer canvas.
Finalize the printer canvas.

To end printing (pdf style):
If the pdf state's document flag is not set, exit.
End printing the pdf state's pdf pointer's target.

To end printing a pdf:
End printing the pdf (finish the parent).
End printing the pdf (append the outline).
End printing the pdf (finish the root).
Clear the pdf.
End printing the pdf (append header).
End printing the pdf (offset and append objects).
End printing the pdf (append xref table).
End printing the pdf (append trailer).
End printing the pdf (append footer).
Destroy the pdf state's font index.
Destroy the pdf state's outline entries.
Destroy the pdf state's objects.
Clear the pdf state's document flag.

To end printing a pdf (append footer):
Append "startxref" then the crlf string to the pdf.
Append the pdf state's xref offset then the crlf string to the pdf.
Append "%%EOF" to the pdf.

To end printing a pdf (append header):

Append "%PDF-1.3" then the crlf string to the pdf.
Append "% " then the crlf string to the pdf.
Append the crlf string to the pdf.

To end printing a pdf (append the outline entries - create the objects):
Get a pdf outline entry from the pdf state's outline entries.
If the pdf outline entry is nil, exit.
Add an entry pdf object given "outline entry".
Put the entry into the pdf outline entry's pdf object.
Repeat.

To end printing a pdf (append the outline entries):
If the pdf state's outline entries are empty, exit.
End printing the pdf (append the outline entries - create the objects).
Loop.
Get a pdf outline entry from the pdf state's outline entries.
If the pdf outline entry is nil, exit.
Put the pdf outline entry's pdf object into an object pdf object.
Append the object's number then " 0 obj" to the object.
Append "<<" to the object.
Convert the pdf outline entry's title to a pdf string.
Append "/Title " then the pdf string to the object.
Append "/Parent " then the pdf state's outline's number then " 0 R" to the object.
If the pdf outline entry's next is not nil, append "/Next " then the pdf outline entry's next's pdf object's number then " 0 R" to the object.
If the pdf outline entry's previous is not nil, append "/Prev " then the pdf outline entry's previous' pdf object's number then " 0 R" to the object.
Append "/Dest [" then the pdf outline entry's destination then " 0 R /XYZ null " then the pdf outline entry's page height then " null]" to the object.
Append ">>" to the object.
Append "endobj" to the object.
Repeat.

To end printing a pdf (append the outline):
Void the pdf state's outline.
If the pdf state's outline entries are empty, exit.
Add an outline pdf object given "outline".
Put the outline into the pdf state's outline.
End printing the pdf (append the outline entries).
Append the outline's number then " 0 obj" to the outline.
Append "<<" to the outline.
Append "/Type /Outlines" to the outline.
Append "/First " then the pdf state's outline entries' first's pdf object's number then " 0 R" to the outline.
Append "/Last " then the pdf state's outline entries' last's pdf object's number then " 0 R" to the outline.
Append "/Count " then the pdf state's outline entries' count to the outline.
Append ">>" to the outline.
Append "endobj" to the outline.

To end printing a pdf (append trailer):
Append "trailer" then the crlf string to the pdf.
Append "<<" then the crlf string to the pdf.
Put the pdf state's objects' count plus 1 into a count.
Append "/Size " then the count then the crlf string to the pdf.
Append "/Root " then the pdf state's root's number then " 0 R" then the crlf string to the pdf.
Append ">>" then the crlf string to the pdf.

Append the crlf string to the pdf.

To end printing a pdf (append xref table):
Put the pdf's length into the pdf state's xref offset.
Append "xref" then the crlf string to the pdf.
Put the pdf state's objects' count plus 1 into a count.
Append "0 " then the count then the crlf string to the pdf.
Append "0000000000 65535 f" then the crlf string to the pdf.
Loop.
Get a pdf object from the pdf state's objects.
If the pdf object is nil, break.
Zero fill the pdf object's offset given 10 and append it to the pdf.
Append " 00000 n" then the crlf string to the pdf.
Repeat.
Append the crlf string to the pdf.

To end printing a pdf (finish the parent):
Put the pdf state's parent into a parent pdf object.
Append "/Kids [" to the parent without advancing.
Loop.
Get a pdf object from the pdf state's objects.
If the pdf object is nil, break.
If the pdf object's kind is not "page", repeat.
If a flag is set, append " " to the parent without advancing.
Append the pdf object's number then " 0 R" to the parent without advancing.
Set the flag.
Add 1 to a count.
If the count is evenly divisible by 20, append the crlf string then " " to the parent without advancing.
Repeat.
Append "]" to the parent.
Append "/Count " then the count to the parent.
Append ">>" to the parent.
Append "endobj" to the parent.

To end printing a pdf (finish the root):
Put the pdf state's root into a root pdf object.
Append "/Pages " then the pdf state's parent's number then " 0 R" to the root.
Find a pdf object given "page".
Append "/OpenAction [" then the pdf object's number then " 0 R /XYZ null null 1]" to the root.
Append "/PageMode /UseNone" to the root.
If the pdf state's outline is not nil, append "/Outlines " then the pdf state's outline's number then " 0 R" to the root.
Append ">>" to the root.
Append "endobj" to the root.

To end printing a pdf (offset and append objects):
Get a pdf object from the pdf state's objects.
If the pdf object is nil, break.
Put the pdf's length into the pdf object's offset.
Append the pdf object's data to the pdf.
Append the crlf string to the pdf.
Repeat.

To end a sheet:
If the pdf state's document flag is set, end the sheet (pdf style); exit.

Call "gdi32.dll" "EndPage" with the printer canvas.
Put the memory canvas into the current canvas.
Put the saved tpp into the tpp.

To end a sheet (pdf style - finish the current contents):
Put the pdf state’s current contents into a content pdf object.
Put the content’s data into a buffer.
Clear the content’s data.
Append the content’s number then " 0 obj" to the content.
Append "<</Length " then the buffer’s length then " >>" to the content.
Append "stream" to the content.
Append the buffer to the content’s data.
Append "endstream" to the content.
Append "endobj" to the content.
Clear the pdf state’s page flag.

To end a sheet (pdf style - finish the current page - font resources):
Put the pdf state’s current page into a page pdf object.
If the page’s font strings are empty, exit.
Append "/Font <<" to the page without advancing.
Loop.
Get a string thing from the page’s font strings.
If the string thing is nil, break.
If a flag is set, append " " to the page without advancing.
Append the string thing’s string to the page without advancing.
Set the flag.
Repeat.
Append ">>" to the page.

To end a sheet (pdf style - finish the current page - image resources):
Put the pdf state’s current page into a page pdf object.
If the page’s image strings are empty, exit.
Append "/XObject <<" to the page without advancing.
Loop.
Get a string thing from the page’s image strings.
If the string thing is nil, break.
If a flag is set, append " " to the page without advancing.
Append the string thing’s string to the page without advancing.
Set the flag.
Repeat.
Append ">>" to the page.

To end a sheet (pdf style - finish the current page):
Put the pdf state’s current page into a page pdf object.
Append "/Resources" to the page.
Append "<<" to the page.
Append "/ProcSet [/PDF /Text /ImageC]" to the page.
End the sheet (pdf style - finish the current page - font resources).
End the sheet (pdf style - finish the current page - image resources).
Append ">>" to the page. \ end resources
Append ">>" to the page. \ end page
Append "endobj" to the page.

To end a sheet (pdf style):
End the sheet (pdf style - finish the current page).

End the sheet (pdf style - finish the current contents).

The end-of-medium byte is a byte equal to 25.

The end-of-text byte is a byte equal to 3.

The end-of-transmission byte is a byte equal to 4.

The end-of-transmission-block byte is a byte equal to 23.

To enlarge a box by some twips:
Subtract the twips from the box's left-top.
Add the twips to the box's right-bottom.

To enlarge an ellipse by some twips:
Subtract the twips from the ellipse's left-top.
Add the twips to the ellipse's right-bottom.

To enque an event:
Append the event to the event queue.

The enquiry byte is a byte equal to 5.

The enter key is a key equal to 13.

The equal-sign byte is a byte equal to 61.

The equal-sign key is a key equal to 187.

To erase the insides of a box;
To erase inside a box;
To clear inside a box:
Draw the box with the clear color and the black color.

The escape byte is a byte equal to 27.

The esc key is a key equal to 27.

The escape key is a key equal to 27.

To estimate a rise and a run given a heading:
Put the heading into a low heading.
Round the low heading down to the nearest multiple of 20.
Get a low rise and a low run given the low heading.
Put the heading into a high heading.
Round the high heading up to the nearest multiple of 20.
Get a high rise and a high run given the high heading.
Put the low rise plus the high rise divided by 2 into the rise.
Put the low run plus the high run divided by 2 into the run.

The euro-sign byte is a byte equal to 128.

An event is a thing with
A kind,
A shift flag,

A ctrl flag,
An alt flag,
A spot,
A key, a byte.

The event queue is an event queue.

An event queue is some events.

The exclamation byte is a byte equal to 33.

The exclamation-mark byte is a byte equal to 33.

To extend any selection in a text given a spot:
If the text is nil, exit.
Get the text's caret given the spot and the text.
Clear the text's last operation.

To extend a box to include another box:
If the other box's left is less than the box's left, put the other box's left into the box's left.
If the other box's top is less than the box's top, put the other box's top into the box's top.
If the other box's right is greater than the box's right, put the other box's right into the box's right.
If the other box's bottom is greater than the box's bottom, put the other box's bottom into the box's bottom.

An extension is a string. \ last dot to end of path = .ext

Some extra points are some points.

To extract a designator from a path:
Clear the designator.
Extract a drive from the path.
Slap a path substring on the path.
Add the drive's length to the path substring's first.
If the path substring is blank, put the drive into the designator; exit.
Slap a substring on the last byte of the path substring.
If the substring's first's target is the backslash byte, subtract 1 from the substring's first.
Loop.
If the substring's first is less than the path substring's first, break.
If the substring's first's target is the backslash byte, break.
Subtract 1 from the substring's first.
Repeat.
Add 1 to the substring's first.
Put the substring into the designator.

To extract a directory from a path:
Clear the directory.
Extract a drive from the path.
If the drive is blank, exit.
Slap a substring on the path.
Add the drive's length to the substring's first.
If the substring is blank, exit.
If the substring's last's target is the backslash byte, subtract 1 from the substring's last.
Loop.
If the substring is blank, break.

If the substring’s last’s target is the backslash byte, break.
Subtract 1 from the substring’s last.
Repeat.
Put the drive then the substring into the directory.

To extract a directory name from a path:
Clear the directory name.
If the path is not directory-format, exit.
Extract the directory name as a designator from the path.

To extract a directory name w/o slash from a path:
Extract the directory name w/o slash as a directory name from the path.
If the directory name w/o slash is blank, exit.
Remove the last byte from the directory name w/o slash.

To extract a drive from a path:
Clear the drive.
If the path’s length is less than 3, exit.
Slap a substring on the first byte of the path.
Add 2 to the substring’s last.
If the substring ends with ":\", put the substring into the drive; exit.
If the substring does not start with "\\", exit.
Slap the substring on the first byte of the path.
Loop.
If the substring’s last is greater than the path’s last, exit.
If the substring’s last’s target is the backslash byte, add 1 to a count.
If the count is 4, break. \ "\\computer\share\"
Add 1 to the substring’s last.
Repeat.
Put the substring into the drive.

To extract an extension from a path:
Clear the extension.
If the path is blank, exit.
Slap a substring on the last byte of the path.
Loop.
If the substring’s first is less than the path’s first, exit.
If the substring’s first’s target is the colon byte, exit.
If the substring’s first’s target is the backslash byte, exit.
If the substring’s first’s target is the period byte, break.
Subtract 1 from the substring’s first.
Repeat.
Put the substring into the extension.

To extract a file name from a path:
Clear the file name.
If the path is not file-format, exit.
Extract the file name as a designator from the path.

To extract a file name w/o extension from a path:
Extract the file name w/o extension as a file name from the path.
Extract an extension from the path.
Remove trailing bytes from the file name w/o extension given the extension’s length.

To extract a picture given a box:

Put the box's width divided by the tpp into a width.
Put the box's height divided by the tpp into a height.
Call "gdiplus.dll" "GdipCreateBitmapFromScan0" with the width and the height and 0 and 137224 [pixelformat24bpprgb] and 0 and a gpbitmap's whereabouts.
Call "gdiplus.dll" "GdipGetImageGraphicsContext" with the gpbitmap and a gpgraphic's whereabouts.
Call "gdiplus.dll" "GdipGetDC" with the gpgraphic and a bitmap canvas' whereabouts.
Normalize the bitmap canvas.
Call "gdi32.dll" "BitBlt" with the bitmap canvas and 0 and 0 and the box's width and the box's height
And the current canvas and the box's left and the box's top and 13369376 [srccopy].
Call "gdiplus.dll" "GdipReleaseDC" with the gpgraphic and the bitmap canvas.
Call "gdiplus.dll" "GdipDeleteGraphics" with the gpgraphic.
Create the picture given the gpbitmap.
Put the box into the picture's box.
Put the box into the picture's uncropped box.

To extract a string from a text:
If the text is nil, clear the string; exit.
Convert the text's rows to the string.
Remove any trailing linefeed byte from the string.
Remove any trailing return byte from the string.

To extract a string from a text (no linefeed additions):
If the text is nil, clear the string; exit.
Convert the text's rows to the string (no linefeed additions).
Remove any trailing return byte from the string.

To extract a string from a text (selected bytes):
Clear the string.
If the text is nil, exit.
Loop.
Get a row from the text's rows.
If the row is nil, exit.
Slap a substring on any selected bytes in the row of the text.
If the substring is blank, repeat.
Append the substring to the string.
If the substring's last's target is the return byte, append the linefeed byte to the string.
Repeat.

The f-key is a key equal to 70.

The f1-key is a key equal to 112.

The f10-key is a key equal to 121.

The f11-key is a key equal to 122.

The f12-key is a key equal to 123.

The f2-key is a key equal to 113.

The f3-key is a key equal to 114.

The f4-key is a key equal to 115.

The f5-key is a key equal to 116.

The f6-key is a key equal to 117.

The f7-key is a key equal to 118.

The f8-key is a key equal to 119.

The f9-key is a key equal to 120.

To face any way you want;
To face any which way:
Pick a heading.

To face east:
Put 960 into the context's heading.

To face north:
Put 0 into the context's heading.

To face south:
Put 1920 into the context's heading.

To face west:
Put 2880 into the context's heading.

A fancy arrow is a figure.

The feminine byte is a byte equal to 170.

A figure is a polygon.

The figures are some polygons.

A file is a handle.

A file name is a string. \ after the last slash to end of path = file.ext

A file name w/o extension is a string. \ after the last slash to last dot or end of path = file

The file-separator byte is a byte equal to 28.

A filetime is a record with
A number called dwlowdatetime,
A number called dwhighdatetime.

To fill a box with a color:
Draw the box with the clear color and the color.

To fill bytes with a byte starting at a pointer for a byte count:
Intel $8BBD0C000000. \ mov edi,[ebp+12] \ the pointer
Intel $8B3F. \ mov edi,[edi]
Intel $8B8D10000000. \ mov ecx,[ebp+16] \ the count
Intel $8B09. \ mov ecx,[ecx]
Intel $8B8508000000. \ mov eax,[ebp+8] \ the byte
Intel $8A00. \ mov al,[eax]

Intel $FC. \ cld
Intel $F3AA. \ rep stosb

To fill a string with a byte given a count:
Reassign the string's first given the count.
Fill bytes with the byte starting at the string's first for the count.
Put the string's first plus the count minus 1 into the string's last.

To finalize after run:
If the heap count is 0, exit.
Put the heap count into a count.
Put the count then " drip(s)." into a string.
Debug the string.

To finalize the canvases:
Finalize the memory canvas.
Finalize the screen canvas.

To finalize the cgi:
Call "kernel32.dll" "FreeConsole".

To finalize the colors:

To finalize com:
Call "ole32.dll" "CoUninitialize".

To finalize a context:
Destroy the choices.
Destroy the figures.
Destroy the context stack.
Destroy the context.

To finalize the cursors:
Call "user32.dll" "DestroyCursor" with the i-beam cursor.
Call "user32.dll" "DestroyCursor" with the hand cursor.
Call "user32.dll" "DestroyCursor" with the arrow cursor.

To finalize the fonts:
Call "gdi32.dll" "RemoveFontMemResourceEx" with the osmosian font resource.

To finalize gdi+:
Call "gdiplus.dll" "GdiplusShutdown" with the gptoken.

To finalize the memory canvas:
Call "gdi32.dll" "SelectObject" with the memory canvas and the saved memory hbitmap returning an hbitmap.
Call "gdi32.dll" "DeleteObject" with the hbitmap.
Call "gdi32.dll" "DeleteDC" with the memory canvas.

To finalize the module:

To finalize the mouse:

To finalize the printer canvas:
Call "kernel32.dll" "GlobalFree" with the printer device mode handle.

Put 0 into the printer canvas.

To finalize the screen:

To finalize the screen canvas:

To finalize a talker:
If the talker is nil, exit.
Call the talker's vtable's release with the talker.
Put nil into the talker.

To finalize the window:
Call "user32.dll" "DestroyWindow" with the main window.
Loop.
Call "user32.dll" "GetMessageA" with an msg's whereabouts and 0 and 0 and 0 returning a number.
If the number is less than 1, break.
Call "user32.dll" "TranslateMessage" with the msg's whereabouts.
Call "user32.dll" "DispatchMessageA" with the msg's whereabouts.
Repeat.
Flush the event queue.
Destroy the current event.

To finalize winsock:
Call "ws2_32.dll" "WSACleanup".

The find anchor is an anchor.

To find a choice given a spot:
Start with nothing in the choice.
Loop.
Get the [first/next] choice from the choices.
If the choice is missing, exit.
If the spot is in the choice's box, break.
Repeat.

To find a dyad given some dyads and a name:
Void the dyad.
Loop.
Get the dyad from the dyads.
If the dyad is nil, exit.
If the dyad's name is the name, exit.
Repeat.

To find next given a row and a text and a flag:
Clear the flag.
If the text is nil, exit.
If the row is nil, exit.
Slap a substring on the row's string.
Put the substring's first plus the find string's length minus 1 into the substring's last.
If the row's row# is the find anchor's row#, move the substring given the find anchor's column# minus 1.
Loop.
If the substring's last is greater than or equal to the row's string's last, exit.
If the substring is the find string, break.
Move the substring given 1.
Repeat.

Set the flag.
Put the substring's first minus the row's string's first plus 1 into the text's anchor column#.
Put the row's row# into the text's anchor row#.
Put the substring's last minus the row's string's first plus 2 into the text's caret column#.
Put the row's row# into the text's caret row#.

To find next given a text and a flag:
If the text's wrap flag is set, find next given the text and the flag (wrapped text); exit.
Clear the flag.
If the text is nil, exit.
If the find string's length is 0, exit.
Loop.
Get a row from the text's rows.
If the row is nil, exit.
If the row's row# is less than the find anchor's row#, repeat.
Find next given the row and the text and the flag.
If the flag is set, exit.
Repeat.

To find next given a text and a flag (wrapped text):
Clear the flag.
If the text is nil, exit.
Convert the find anchor to an absolute position called offset given the text.
Extract a string from the text (no linefeed additions).
Put the string's first plus the offset minus 1 into a substring's first.
Put the substring's first plus the find string's length minus 1 into the substring's last.
Loop.
If the substring's last is greater than the string's last, exit.
If the substring is the find string, break.
Move the substring given 1.
Repeat.
Set the flag.
Put the substring's first minus the string's first plus 1 into an anchor absolute position.
Put the substring's last minus the string's first plus 2 into a caret absolute position.
Convert the anchor absolute position to the text's anchor given the text.
Convert the caret absolute position to the text's caret given the text.

To find the next misspelling given a row and a text and a flag:
Clear the flag.
If the text is nil, exit.
If the row is nil, exit.
Slap a rider on the row's string.
If the row's row# is the find anchor's row#, add the find anchor's column# minus 1 to the rider's source's first.
If the rider's source's first is not the row's string's first, skip word characters in the rider's source.
Loop.
Move the rider (spell checking rules).
If the rider's token is blank, exit.
If the rider's token is not misspelled, repeat.
Set the flag.
Put the rider's token's first minus the row's string's first plus 1 into the text's anchor column#.
Put the row's row# into the text's anchor row#.
Put the rider's token's last minus the row's string's first plus 2 into the text's caret column#.
Put the row's row# into the text's caret row#.

```
To find the next misspelling in a text given a flag:
Clear the flag.
If the text is nil, exit.
Loop.
Get a row from the text's rows.
If the row is nil, exit.
If the row's row# is less than the find anchor's row#, repeat.
Find the next misspelling given the row and the text and the flag.
If the flag is set, exit.
Repeat.

To find a pdf object given a kind:
Void the pdf object.
Loop.
Get the pdf object given the pdf state's objects.
If the pdf object is nil, break.
If the pdf object's kind is the kind, break.
Repeat.

To find a pointer given a string and an index:
Find a refer given the string and the index.
If the refer is not nil, put the refer's pointer into the pointer; exit.
Void the pointer.

To find a refer given a string and an index:
\  if the index is nil, exit. \ to make compiler faster
If the string is blank, void the refer; exit.
Get a bucket given the string and the index.
Find the refer given the string and the bucket's refers.

To find a refer given a string and some refers:
Void the refer.
Loop.
Get the refer from the refers.
If the refer is nil, exit.
If the string is the refer's string, exit.
Repeat.

To find a sector given a grid and a spot:
Put the spot's x divided by the grid's x times the grid's x into the sector's x.
Put the spot's y divided by the grid's y times the grid's y into the sector's y.

To find a square root of a number: \ rounds down
Privatize the number.
De-sign the number.
If the number is 0, put 0 into the square root; exit.
If the number is 1, put 1 into the square root; exit.
Put 1 into a square number.
Put 3 into a delta number.
Loop.
If the square is greater than the number, break.
Add the delta to the square.
Add 2 to the delta.
Repeat.
Put the delta divided by 2 minus 1 into the square root.
```

To find a string given some string things and a string#:
Clear the string.
Loop.
Get a string thing from the string things.
If the string thing is nil, exit.
Add 1 to a count.
If the count is not the string#, repeat.
Put the string thing's string into the string.

The find string is a string.

To find a string thing given a string and some string things:
Void the string thing.
Loop.
Get the string thing given the string things.
If the string thing is nil, break.
If the string thing's string is the string, break.
Repeat.

To find a substring in a string given another string:
Slap the substring on the string.
Loop.
If the substring is blank, exit.
If the substring starts with the other string, break.
Add 1 to the substring's first.
Repeat.
Put the substring's first plus the other string's length minus 1 into the substring's last.

To find a value string given some dyads and a name:
Find a dyad given the dyads and the name.
If the dyad is nil, clear the value; exit.
Put the dyad's value into the value.

To find a value string given the environment variables and a name:
Privatize the name.
Null terminate the name.
Put 32767 into a length. \ max size for environment variable
Reassign the value's first given the length.
Call "kernel32.dll" "GetEnvironmentVariableA" with the name's first and the value's first and the length returning the length.
Put the value's first plus the length minus 1 into the value's last.

A finger is a byte pointer.

The five byte is a byte equal to 53.

The five key is a key equal to 53.

A flag has 4 bytes.

To flip a fraction:
Swap the fraction's numerator with the fraction's denominator.

To flip the gpbitmap in a picture:

If the picture is nil, exit.
Reverse the picture's mirror flag.
Add 1800 to the picture's rotate angle.
Normalize the picture's rotate angle.
Flip the picture's gpbitmap.

To flip a gpimage:
Call "gdiplus.dll" "GdipImageRotateFlip" with the gpimage and 6 [rotatenoneflipy aka rotate180flipx].

To flip a picture:
If the picture is nil, exit.
Put the picture's box's center's y minus the picture's uncropped box's center's y into a pair's y.
Multiply the pair's y by 2.
Move the picture's uncropped box given the pair.
Flip the gpbitmap in the picture.

To flip a polygon:
If the polygon is nil, exit.
Put the polygon's box into a box.
Loop.
Get a vertex from the polygon's vertices.
If the vertex is nil, exit.
Subtract the box's top from the vertex's y.
Put the box's bottom minus the vertex's y into the vertex's y.
Repeat.

To flush all input;
To flush all inputs;
To flush all events:
Flush any messages.
Flush the event queue.

To flush any messages:
Call "user32.dll" "PeekMessageA" with an msg's whereabouts and 0 and 0 and 0 and 1 [pm_remove] returning a number.
If the number is 0, exit.
If the msg's message is 15 [wm_paint], call "user32.dll" "ValidateRect" with the main window and 0.
Repeat.

To flush an event queue:
Get an event from the event queue.
If the event is nil, exit.
Remove the event from the event queue.
Destroy the event.
Repeat.

A font has a name and a height.

A font height is some twips. \ indicates line height - the letters will fit nicely in a box of this height

A font info is a thing with \ used for pdf conversion
A font,
An emsquare number,
An internal leading number,
A flags number,

An ascent number,
A descent number,
A capheight number,
An italicangle number,
A stemv number,
A font box,
Some font widths.

A font resource is a handle.

Some font widths is a thing with \ used for pdf conversion
A font,
A count,
A number pointer called data.

A foot is 12 inches.

The form-feed byte is a byte equal to 12.

To format a number and a singular string or a plural string into a string:
Convert the number to the string.
Append the space byte to the string.
If the number is 1, append the singular to the string.
If the number is not 1, append the plural to the string.

The four byte is a byte equal to 52.

The four key is a key equal to 52.

A fraction has a numerator number and a denominator number, and a top number at the numerator and a bottom number at the denominator.

A fraction pair has a fraction and another fraction.

The g-key is a key equal to 71.

A gcd is a number.

A gcpresults is a record with
A number called lstructsize,
A pchar called lpoutstring,
A number pointer called lporder,
A number pointer called lpdx,
A number pointer called lpcaretpos,
A pointer called lpclass,
A pointer called lpglyphs,
A number called nglyphs,
A number called maxfit.

A gdiplusstartupinput is a record with
A number called gdiplusversion,
A pointer called debugeventcallback,
A number called suppressbackgroundthread,
A number called suppressexternalcodecs.

A geometric figure is a figure.

To get an abca and an abcc given a string and a canvas:
Put 0 into the abca.
Put 0 into the abcc.
If the string's length is less than 1, exit.
Call "gdi32.dll" "GetCharABCWidthsA" with the canvas and the string's first's target and the string's first's target and an abc's whereabouts.
Put the abc's abca into the abca.
Call "gdi32.dll" "GetCharABCWidthsA" with the canvas and the string's last's target and the string's last's target and another abc's whereabouts.
Put the other abc's abcc into the abcc.

To get an abca given a string and a canvas:
Put 0 into the abca.
If the string's length is less than 1, exit.
Call "gdi32.dll" "GetCharABCWidthsA" with the canvas and the string's first's target and the string's first's target and an abc's whereabouts.
Put the abc's abca into the abca.

To get some addrinfo routines:
Clear the i/o error.
Call "kernel32.dll" "LoadLibraryA" with "ws2_32.dll"'s first returning a handle.
If the handle is 0, put "Could not load ws2_32.dll" into the i/o error; exit.
Call "kernel32.dll" "GetProcAddress" with the handle and "getaddrinfo"'s first returning a pointer.
If the pointer is nil, put "Sorry these routines only work on Windows XP and up." into the i/o error; exit.
Put the pointer into the addrinfo routines' getaddrinfo pointer.
Call "kernel32.dll" "GetProcAddress" with the handle and "freeaddrinfo"'s first returning the pointer.
If the pointer is nil, put "Sorry these routines only work on Windows XP and up." into the i/o error; exit.
Put the pointer into the addrinfo routines' freeaddrinfo pointer.

To get a box for the caret in a text:
If the text is nil, zero the box; exit.
Get a spot given the text's caret and the text.
Put the spot and the spot into the box.
Add the text's row height to the box's bottom.
Adjust the box given 0 and the tpp and 0 and - the tpp.
Put the text's globalized origin into an origin.
If the box's left is less than the origin's x, put the origin's x into the box's left; put the origin's x into the box's right.
If the text's wrap flag is not set, exit.
Limit the box's left to the text's left and the text's right.
Limit the box's right to the text's left and the text's right.

To get a box for a line:
Put the line's start into the box's left-top.
Put the line's end into the box's right-bottom.
Normalize the box.

To get a box given a row and a text:
If the text is nil, zero the box; exit.
If the row is nil, zero the box; exit.
Put the text's globalized origin into the box's left-top.
Add the row's row# minus 1 times the text's row height to the box's top.
Put the text's right into the box's right.

```
Put the box's top plus the text's row height into the box's bottom.

To get a bucket given a bucket# and an index:
\  if the index is nil, void the bucket; exit. \ to make compiler faster
Put the index's first bucket into the bucket.
Add the bucket# times a bucket record's magnitude to the bucket.

To get a bucket given an index:
\  if the index is nil, void the bucket; exit. \ to make compiler faster
If the bucket is nil, put the index's first bucket into the bucket; exit.
If the bucket is the index's last bucket, void the bucket; exit.
Add a bucket record's magnitude to the bucket.

To get a bucket given a string and an index:
\  if the index is nil, void the bucket; exit. \ to make compiler faster
Get a bucket# given the string and the index.
Get the bucket given the bucket# and the index.

To get a bucket# given a string and an index: \ based on the djb2 algorithm
\  if the index is nil, put 0 into the bucket#; exit. \ to make compiler faster
Put the string's length into the bucket#.
If the bucket# is 0, exit.
Add 5381 to the bucket#.
Slap a substring on the string.
Loop.
Put the substring's first's target into a byte.
Lowercase the byte.
Put the bucket# into a number.
Shift the bucket# left 5 bits.
Add the number to the bucket#.
Add the byte to the bucket#.
Add 3 to the substring's first.
If the substring is blank, break.
Repeat.
Bitwise and the bucket# with the largest number.
Divide the bucket# by the index's bucket count giving a quotient and the bucket#.

To get a byte from a string:
If the string is blank, put the null byte into the byte; exit.
Put the string's first's target into the byte.
Remove the first byte from the string.

To get a byte from a string (backwards):
If the string is blank, put the null byte into the byte; exit.
Put the string's last's target into the byte.
Remove the last byte from the string.

To get a center spot given a spot and another spot:
Put the spot and the other spot into a line.
Put the line's center into the center.

To get a color given a spot:
Call "gdi32.dll" "GetPixel" with the current canvas and the spot's x and the spot's y returning a colorref.
Convert the colorref to the color.
```

To get a column# given a row and a spot and a text:
Put 0 into the column#.
If the text is nil, exit.
If the row is nil, exit.
Get a box given the row and the text.
If the spot's y is greater than the box's bottom, put the row's string's length into the column#; exit. \ only happens on last row of text
Create the hfont of the memory canvas given the text's font.
Get a start width and a substring given the row and the spot and the text (for "get a column# given a row...").
Loop.
If the substring's last is the row's string's last, break.
Get a width given the substring and the memory canvas.
Add the start width to the width.
Get another width given the substring's last's target and the memory canvas.
Divide the other width by 2.
Subtract the other width from the width.
If the spot's x is less than the width, break.
Add 1 to the substring's last.
Repeat.
Put the substring's last minus the row's string's first plus 1 into the column#.
Destroy the hfont of the memory canvas.

To get a count of items in a path in the file system:
Put 0 into the count.
Loop.
Get an item from the path.
If the item is not found, exit.
Add 1 to the count.
Repeat.

To get a description for a path:
Clear the description.
Get a drive kind for the path.
Put the drive kind into the description.
Get a drive name for the path.
If the drive name is not blank, put the drive name into the description.
Lowercase the description.

To get a difference between a pair and another pair:
Put the pair into the difference.
Subtract the other pair from the difference.

To get a difference between a pair and another pair given a grid pair:
Get the difference between the pair and the other pair.
Round the difference to the grid.

To get a distance between a spot and another spot (approximate):
Put the spot's x minus the other spot's x into a number.
De-sign the number.
Put the spot's y minus the other spot's y into another number.
De-sign the other number.
Put the number times the number into the distance.
Add the other number times the other number to the distance.
Find a square root of the distance.

```
Put the square root into the distance.
If the d-key is down, debug the distance.

To get a distance between a spot and another spot (chessboard):
Put the spot's x minus the other spot's x into a number.
De-sign the number.
Put the spot's y minus the other spot's y into another number.
De-sign the other number.
Put the number into the distance.
If the other number is greater than the number, put the other number into the distance.

To get a drive kind for a path:
Privatize the path.
Null terminate the path.
Call "kernel32.dll" "GetDriveTypeA" with the path's first returning a number.
If the number is 2 [drive_removable], put "removable drive" into the drive kind; exit.
If the number is 3 [drive_fixed], put "hard disk / flash drive" into the drive kind; exit.
If the number is 4 [drive_remote], put "network drive" into the drive kind; exit.
If the number is 5 [drive_cdrom], put "cd-rom / dvd drive" into the drive kind; exit.
If the number is 6 [drive_ramdisk], put "virtual drive in memory" into the drive kind; exit.
Put "" into the drive kind.

To get a drive name for a path:
Privatize the path.
Null terminate the path.
Put 512 into a length.
Reassign a buffer's first given the length.
Call "kernel32.dll" "GetVolumeInformationA" with the path's first and the buffer's first and the length and 0
and 0 and 0 and 0 and 0 returning a number.
If the number is 0, clear the drive name; exit.
Convert the buffer's first as a pchar to the drive name.

To get the first-eighth equivalent of a heading:
Get the first-quarter equivalent of the heading.
If the heading is less than 480, exit.
Subtract 960 from the heading.
De-sign the heading.

To get the first-quarter equivalent of a heading:
If the heading is less than 960, exit.
Subtract 960 from the heading.
Repeat.

To get fresh random numbers;
To get new random numbers;
To seed the random number generator:
Put the system's tick count into the seed.

To get a gcd given a number and another number:
Put the number into a dividend number.
Put the other number into the gcd.
De-sign the dividend number.
De-sign the gcd.
If the dividend is less than the gcd, swap the dividend with the gcd.
If the gcd is 0, put 1 into the gcd; exit.
```

Loop.
Divide the dividend by the gcd giving a quotient and a remainder.
If the remainder is 0, exit.
Put the gcd into the dividend.
Put the remainder into the gcd.
Repeat.

To get an ip address given a host string:
Clear the ip address.
Get a sockaddr given the host string.
If the i/o error is not blank, exit.
Put the sockaddr's sin_addr's s_addr into the ip address' number.
Call "ws2_32.dll" "inet_ntoa" with the ip address' number returning a pchar.
Convert the pchar to the ip address' string.

To get an item (not first time):
Clear the i/o error.
Call "kernel32.dll" "FindNextFileA" with the item's handle and the item's win32finddata's whereabouts returning a number.
If the number is not 0, adjust the item; exit.
Clear the item's kind.
Call "kernel32.dll" "FindClose" with the item's handle.

To get an item from a path:
If the path is not directory-format, exit.
Put the path into the item's directory.
If the item's kind is blank, get the item from the path (first time); exit.
Get the item (not first time).

To get an item from a path (first time):
Clear the i/o error.
Privatize the path.
Append "*.*" to the path.
Null terminate the path.
Clear the item's kind.
Call "kernel32.dll" "FindFirstFileA" with the path's first and the item's win32finddata's whereabouts returning the item's handle.
If the item's handle is -1 [invalid_handle_value], exit.
Adjust the item.

To get an lcm given a number and another number:
Get a gcd given the number and the other number.
Call "kernel32.dll" "MulDiv" with the number and the other number and the gcd returning the lcm.

To get a letter from the alphabet:
Put the next letter into the letter.
Bump the next letter limiting it to the big-a byte and the big-z byte.
Add 1 to the next letter.
If the next letter is greater than the big-z byte, put the big-a byte into the next letter.

To get a number from the stack:
Put 0 into the number.
Get a stack entry from the stack.
If the stack entry is nil, exit.
Convert the stack entry's string to the number.

Remove the stack entry from the stack.
Destroy the stack entry.

To get an offset pair given a string and a box and a font and an alignment:
Create the hfont of the memory canvas given the font.
Get the offset pair given the string and the box and the font and the alignment (fast).
Destroy the hfont of the memory canvas.

To get an offset pair given a string and a box and a font and an alignment (fast):
If the alignment is "left", get the offset pair's x given the string and the box (fast - left).
If the alignment is "right", get the offset pair's x given the string and the box (fast - right).
If the alignment is "center", get the offset pair's x given the string and the box (fast - center).
Call "gdi32.dll" "GetTextMetricsA" with the memory canvas and a textmetric's whereabouts.
Add the box's height minus the textmetric's tmheight divided by 2 to the offset pair's y.

To get an outlinetextmetric given a font:
Create the hfont of the memory canvas given the font.
Call "gdi32.dll" "GetOutlineTextMetricsA" with the memory canvas and 0 and 0 returning a result number.
Assign a poutlinetextmetric given the result.
Call "gdi32.dll" "GetOutlineTextMetricsA" with the memory canvas and the result and the poutlinetextmetric.
Put the poutlinetextmetric's target into the outlinetextmetric.
Destroy the hfont of the current canvas.
Unassign the poutlinetextmetric.

To get a position given a spot and a text:
If the text is nil, clear the position; exit.
Get a row given the spot and the text.
Put the row's row# into the position's row#.
Get the position's column# given the row and the spot and the text.

To get a rgb pointer from a bitmapdata at a row# and a column#: \ 1 based
Put the bitmapdata's scan0 into the rgb pointer.
Add the row# minus 1 times the bitmapdata's stride to the rgb pointer.
Add the column# minus 1 times a rgb's magnitude to the rgb pointer.

To get a rise and a run given a heading: \ see Madhava's Numbers
Privatize the heading.
Normalize the heading.
Normalize the original heading.
If the heading is not evenly divisible by 20, estimate the rise and the run given the heading; exit.
\ special cases
If the heading is 0, put -10000 in the rise; put -0 in the run; exit. \ 0 degrees
If the heading is 960, put 10000 in the run; put -0 in the rise; exit. \ 90 degrees
If the heading is 1920, put 10000 in the rise; put 0 in the run; exit. \ 180 degrees
If the heading is 2880, put -10000 in the run; put 0 in the rise; exit. \ 270 degrees
If the heading is 3840, debug "invalid heading"; debug the heading; debug the original heading.
\ force it into the first eighth of the circle
Get the first-eighth equivalent of the heading.
\ find the first eighth of the circle unsigned values
If the heading is 20, put 0327 in the run; put 9995 in the rise; break. \ 1.875 degrees
If the heading is 40, put 0654 in the run; put 9979 in the rise; break. \ 3.75 degrees
If the heading is 60, put 0980 in the run; put 9952 in the rise; break. \ 5.625 degrees = 1/64 of the way
If the heading is 80, put 1305 in the run; put 9914 in the rise; break. \7.5 degrees
If the heading is 100, put 1629 in the run; put 9866 in the rise; break. \ 9.375 degrees

If the heading is 120, put 1951 in the run; put 9808 in the rise; break. \ 11.25 degrees = 2/64 of the way
If the heading is 140, put 2271 in the run; put 9739 in the rise; break. \ 13.125 degrees
If the heading is 160, put 2588 in the run; put 9659 in the rise; break. \ 15 degrees
If the heading is 180, put 2903 in the run; put 9569 in the rise; break. \ 16.875 degrees = 3/64 of the way
If the heading is 200, put 3214 in the run; put 9469 in the rise; break. \ 18.75 degrees
If the heading is 220, put 3523 in the run; put 9359 in the rise; break. \ 20.625 degrees
If the heading is 240, put 3827 in the run; put 9239 in the rise; break. \ 22.5 degrees = 4/64 of the way
If the heading is 260, put 4127 in the run; put 9109 in the rise; break. \ 24.375 degrees
If the heading is 280, put 4423 in the run; put 8969 in the rise; break. \ 26.25 degrees
If the heading is 300, put 4714 in the run; put 8819 in the rise; break. \ 28.125 degrees = 5/64 of the way
If the heading is 320, put 5000 in the run; put 8660 in the rise; break. \ 30 degrees
If the heading is 340, put 5281 in the run; put 8492 in the rise; break. \ 31.875 degrees
If the heading is 360, put 5556 in the run; put 8315 in the rise; break. \ 33.75 degrees = 6/64 of the way
If the heading is 380, put 5825 in the run; put 8128 in the rise; break. \ 35.625 degrees
If the heading is 400, put 6088 in the run; put 7934 in the rise; break. \ 37.5 degrees
If the heading is 420, put 6344 in the run; put 7730 in the rise; break. \ 39.375 degrees = 7/64 of the way
If the heading is 440, put 6593 in the run; put 7518 in the rise; break. \ 41.25 degrees
If the heading is 460, put 6836 in the run; put 7299 in the rise; break. \ 43.125 degrees
If the heading is 480, put 7071 in the run; put 7071 in the rise; break. \ 45 degrees = 8/64 of the way
Repeat. \ not really a repeat, just a label for the above breaks.
\ adjust for other eighths of the circle
If the original heading is between 0 and 480, negate the rise; exit. \ 1st eighth (12:00 to 1:30)
If the original heading is between 480 and 960, swap the run with the rise; negate the rise; exit. \ 2nd eighth (1:30 to 3:00)
If the original heading is between 960 and 1440, swap the run with the rise; exit. \ 3rd eighth (3:00 to 4:30)
If the original heading is between 1440 and 1920, exit. \ 4th eighth (4:30 to 6:00)
If the original heading is between 1920 and 2400, negate the run; exit. \ 5th eighth (6:00 to 7:30)
If the original heading is between 2400 and 2880, swap the run with the rise; negate the run; exit. \ 6th eighth (7:30 to 9:00)
If the original heading is between 2880 and 3360, swap the run with the rise; negate the run; negate the rise; exit. \ 7th eighth (9:00 to 10:30)
If the original heading is between 3360 and 3840, negate the run; negate the rise; exit. \ 8th eighth (10:30 to 12:00)

To get a row given a row# and a text:
Void the row.
If the text is nil, exit.
Loop.
Get the row from the text's rows.
If the row is nil, exit.
If the row's row# is the row#, exit.
Repeat.

To get a row given a spot and a text:
If the text is nil, void the row; exit.
Put the spot's y into a y coord.
Limit the y to the text's top and the text's bottom.
Put the y minus the text's globalized origin's y divided by the text's row height plus 1 into a row#.
Limit the row# to 1 and the text's row count.
Get the row given the row# and the text.

To get a selection box given a row and a text:
Clear the selection box.
If the text is nil, exit.

If the row is nil, exit.
Get a box given the row and the text.
Put the box into the selection box.
Get the selection box given the row and the text (left side).
Get the selection box given the row and the text (right side).

To get a selection box given a row and a text (left side):
Put the text's normalized selection into a selection.
Put the text's globalized origin's x into the selection box's left.
If the selection's anchor row# is the row's row#, get a spot given the selection's anchor and the text; put the spot's x into the selection box's left.
Limit the selection box's left to the text's left and the text's right.

To get a selection box given a row and a text (right side):
Put the text's normalized selection into a selection.
Put the text's right into the selection box's right.
If the selection's caret row# is the row's row#, get a spot given the selection's caret and the text; put the spot's x into the selection box's right.
Limit the selection box's right to the text's left and the text's right.

To get a size given a path in the file system:
If the path is directory-format, get the size given the path in the file system (directory).
If the path is file-format, get the size given the path in the file system (file).

To get a size given a path in the file system (directory):
Put 0 into the size.
Loop.
Get an item from the path.
If the item is not found, exit.
If the item's kind is "file", add the item's size to the size; repeat.
Put the path into another path.
Append the item's designator to the other path.
Get another size given the other path in the file system.
Add the other size to the size.
Repeat.

To get a size given a path in the file system (file):
Privatize the path.
Null terminate the path.
Call "kernel32.dll" "GetFileAttributesExA" with the path's first and 0 and a win32finddata's whereabouts.
Put the win32finddata's nfilesizelow into the size.

To get a sockaddr given a host string:
Clear the i/o error.
\ prepare strings
Privatize the host string.
Null terminate the host string.
\ get the function addresses
Get some addrinfo routines.
If the i/o error is not blank, exit.
\ get the sockaddr
Put 2 [af_inet] into a addrinfo's ai_family.
Put 1 [sock_stream] into the addrinfo's ai_sockettype.
Put 6 [ipproto_tcp] into the addrinfo's ai_protocol.
Call the addrinfo routines' getaddrinfo with the host string's first and 0 and the addrinfo's whereabouts and

a addrinfoptr's whereabouts returning a result number.
If the result number is not 0, put "Could not resolve host name '" then the host then "'." into the i/o error; exit.
If the addrinfoptr is nil, put "Could not resolve host name '" then the host then "'." into the i/o error; exit.
Put the addrinfoptr's ai_addr's target into the sockaddr.
Call the addrinfo routines' freeaddrinfo with the addrinfoptr.

To get a spot given a position and a text:
Clear the spot.
If the text is nil, exit.
Get a row given the position's row# and the text.
Get a box given the row and the text.
Put the box's top into the spot's y.
Put the row's string's first into a substring's first.
Put the substring's first plus the position's column# minus 2 into the substring's last.
Get a width given the substring and the memory canvas and the text's font.
Put the box's left plus the width into the spot's x.
Get an offset pair given the row's working string and the box and the text's font and the text's alignment.
Add the offset pair's x to the spot's x.

To get a start width and a substring given a row and a spot and a text (for "get a column# given a row..."):
Clear the start width.
Clear the substring.
If the text is nil, exit.
If the row is nil, exit.
Get a box given the row and the text.
Slap the substring on the row's working string.
Get an offset pair given the substring and the box and the text's font and the text's alignment (fast).
Put the text's globalized origin's x plus the offset pair's x into the start width.
Put the substring's first plus the text cutoff minus 1 into the substring's last.
Loop.
If the substring's last is greater than or equal to the row's string's last, break.
Get a width given the substring and the memory canvas.
Put the start width plus the width into another width.
If the spot's x is less than or equal to the other width, break.
Add the width to the start width.
Move the substring given the text cutoff.
Repeat.
Put the substring's first into the substring's last.

To get a string from the stack:
Clear the string.
Get an stack entry from the stack.
If the stack entry is nil, put "ERROR" into the string; exit.
Put the stack entry's string to the string.
Remove the stack entry from the stack.
Destroy the stack entry.

To get a string from the windows clipboard:
Clear the string.
Call "user32.dll" "OpenClipboard" with the main window.
Call "user32.dll" "GetClipboardData" with 1 [cf_text] returning a handle.
If the handle is 0, call "user32.dll" "CloseClipboard"; exit.
Call "kernel32.dll" "GlobalLock" with the handle returning a pchar.
Convert the pchar to the string.

```
Call "kernel32.dll" "GlobalUnlock" with the handle.
Call "user32.dll" "CloseClipboard".

To get a thing from some things:
If the things are empty, void the thing; exit.
If the thing is nil, put the things' first into the thing; exit.
Put the thing's next into the thing.

To get a thing from some things (backwards):
If the things are empty, void the thing; exit.
If the thing is nil, put the things' last into the thing; exit.
Put the thing's previous into the thing.

To get a token from a reply:
Remove any leading noise from the reply.
Clear the token.
Loop.
If the reply is blank, exit.
Get a byte from the reply.
If the byte is the space byte, exit.
Append the byte to the token.
Repeat.

To get a width given a byte and a canvas:
Call "gdi32.dll" "GetTextExtentPoint32A" with the canvas and the byte's whereabouts and 1 and a pair's whereabouts.
Put the pair's x into the width.

To get a width given a byte and some font widths:
Put the byte into a number.
Get the width given the number and the font widths.

To get a width given a number and some font widths: \ indexes are 0 based
If the font widths are nil, clear the width; exit.
Put the font widths' data into a number pointer.
Add the number times the number's magnitude to the number pointer.
Put the number pointer's target into the width.

To get a width given a string and a canvas: \ assumes font is already selected in canvas
Call "gdi32.dll" "GetTextExtentPoint32A" with the canvas and the string's first and the string's length and a pair's whereabouts.
Put the pair's x into the width.

To get a width given a string and a canvas and a font:
Create the hfont of the canvas given the font.
Get the width given the string and the canvas.
Destroy the hfont of the canvas.

To get a width given a string and a font: \ assumes memory canvas
Get the width given the string and the memory canvas and the font.

To get an x coord given a string and a box (fast - center):
Get a width given the string and the memory canvas.
Get an abca and an abcc given the string and the memory canvas.
Put the width minus the abca minus the abcc into the width.
```

Put the box's width minus the width divided by 2 minus the abca into the x.

To get an x coord given a string and a box (fast - left):
Get an abca given the string and the memory canvas.
Put - the abca into the x.

To get an x coord given a string and a box (fast - right):
Get an abca and an abcc given the string and the memory canvas.
Get a width given the string and the memory canvas.
Put the width minus the abca minus the abcc into the width.
Put the box's width minus the width minus the abca into the x.

A gigabyte is 1024 megabytes.

To globalize a box given a pair:
Move the box given the pair.

To globalize a spot given a pair:
Move the spot given the pair.

To go back to where we were:
Restore the context.

The gold color is a color.

The gold pen is a pen.

A gpbitmap is a gpimage.

A gpgraphic is a pointer.

A gpimage is a pointer.

A gpimageattributes is a pointer.

A gprect is a record with
A number called x,
A number called y,
A number called width,
A number called height.

The gptoken is a gptoken.

A gptoken is a number.

A grain is 10 milliseconds.

The gray color is a color.

The gray pen is a pen.

The grayscale color matrix is a hex string equal to
$8716993E8716993E8716993E0000000000000000A245163FA245163FA245163F0000000000000000D578E93DD578E93DD578E93D0000000000000000000000000000000000000000803F0000000000000000000000000000000000000000000000000000803F.

The greater-than-sign byte is a byte equal to 62.

The green color is a color.

The green pen is a pen.

A greenish color is a color.

A grid is a pair.

The group-separator byte is a byte equal to 29.

To guarantee one row in a text:
If the text is nil, exit.
If the text's rows are not empty, exit.
Create a row given the return byte.
Append the row to the text's rows.
Renumber the text's rows.

A guid is a uuid.

The h-key is a key equal to 72.

The hand cursor is a cursor.

To handle align given a text and an alignment:
If the text is nil, exit.
Remember the text with "alignment".
Align the text given the alignment.

To handle any wm-activate with a w-param:
Split the w-param into a wyrd and another wyrd.
Put the other wyrd into a number.
If the number is 0, handle any wm-activate with the w-param (deactivate); exit.
Handle any wm-activate with the w-param (activate).

To handle any wm-activate with a w-param (activate):
Call "user32.dll" "SetFocus" with the main window.
Call "user32.dll" "ClipCursor" with 0.
\ seterrormode(sem_failcriticalerrors) \ keeps certain disk errors from appearing
Create an event.
Put "activate" into the event's kind.
Enque the event.
Call "user32.dll" "PostMessageA" with the main window and 0 [wm_null] and 0 and 0.

To handle any wm-activate with a w-param (deactivate):
Create an event.
Put "deactivate" into the event's kind.
Enque the event.
Call "user32.dll" "PostMessageA" with the main window and 0 [wn_null] and 0 and 0.

To handle any wm-char with a w-param and an l-param:
If the alt key was down, exit.
If the ctrl key was down, exit.

Put the w-param into a byte.
If the byte is not printable, exit.
Create an event.
Put "key down" into the event's kind.
If the shift key was down, set the event's shift flag.
Put the byte into the event's byte.
Convert the l-param to the event's key.
Enque the event.

To handle any wm-create with a window:
Put the window into the main window.

To handle any wm-destroy:
Call "user32.dll" "PostQuitMessage" with 0.

To handle any wm-lbuttondblclk with a l-param:
Create an event.
Put "left double click" into the event's kind.
If the alt key was down, set the event's alt flag.
If the ctrl key was down, set the event's ctrl flag.
If the shift key was down, set the event's shift flag.
Convert the l-param to the event's spot.
Enque the event.

To handle any wm-lbuttondown with a l-param:
Create an event.
Put "left click" into the event's kind.
If the alt key was down, set the event's alt flag.
If the ctrl key was down, set the event's ctrl flag.
If the shift key was down, set the event's shift flag.
Convert the l-param to the event's spot.
Enque the event.

To handle any wm-paint with a window:
Call "user32.dll" "BeginPaint" with the window and a paintstruct's whereabouts.
Call "user32.dll" "EndPaint" with the window and the paintstruct's whereabouts.
Create an event.
Put "refresh" into the event's kind.
Enque the event.

To handle any wm-rbuttondblclk with a l-param:
Create an event.
Put "right double click" into the event's kind.
If the alt key was down, set the event's alt flag.
If the ctrl key was down, set the event's ctrl flag.
If the shift key was down, set the event's shift flag.
Convert the l-param to the event's spot.
Enque the event.

To handle any wm-rbuttondown with a l-param:
Create an event.
Put "right click" into the event's kind.
If the alt key was down, set the event's alt flag.
If the ctrl key was down, set the event's ctrl flag.
If the shift key was down, set the event's shift flag.

```
Convert the l-param to the event's spot.
Enque the event.

To handle any wm-setcursor:
Refresh the cursor.

To handle any wm-syskeydown with a w-param and an l-param;
To handle any wm-keydown with a w-param and an l-param:
Put the w-param into a key.
If the key with the l-param is any repeated escape or modifier key, exit.
If the key is any wm-char key, exit.
Create an event.
Put "key down" into the event's kind.
If the alt key was down, set the event's alt flag.
If the ctrl key was down, set the event's ctrl flag.
If the shift key was down, set the event's shift flag.
Put the key into the event's key.
Enque the event.

To handle capitalize given a text:
If the text is nil, exit.
If nothing is selected in the text, exit.
Remember the text with "capitalize".
Capitalize any selected rows in the text.
Square up any selection in the text.
Wrap the text.

To handle copy given a text:
If the text is nil, exit.
If nothing is selected in the text, exit.
Extract a string from the text (selected bytes).
Put the string on the windows clipboard.

To handle cut given a text:
If the text is nil, exit.
If nothing is selected in the text, exit.
Remember the text.
Extract a string from the text (selected bytes).
Put the string on the windows clipboard.
Remove any selected bytes in the text.
Wrap the text.
Scroll the text to the caret.

To handle an event given a console:
If the console is nil, exit.
If the event's kind is "key down", handle the event given the console (key down); exit.
If the event's kind is "refresh", handle the event given the console (refresh); exit.
If the event's kind is "right click", handle the event given the console (right click); exit.
If the event's kind is "set cursor", handle the event given the console (set cursor); exit.

To handle an event given a console (backspace key):
If the console's reply is blank, exit.
If the event is modified, exit.
Handle the event given the console's text (backspace key).
Remove the last byte from the console's reply.
```

Show the console.

To handle an event given a console (down-arrow key):
Scroll the console's text down one line.
Show the console.

To handle an event given a console (end key):
Scroll the console's text to the bottom.
Show the console.

To handle an event given a console (enter key):
Handle the event given the console's text (enter key).
Relinquish control.

To handle an event given a console (home key):
Scroll the console's text to the top.
Show the console.

To handle an event given a console (key down):
If the event's key is the backspace key, handle the event given the console (backspace key); exit.
If the event's key is the down-arrow key, handle the event given the console (down-arrow key); exit.
If the event's key is the end key, handle the event given the console (end key); exit.
If the event's key is the enter key, handle the event given the console (enter key); exit.
If the event's key is the home key, handle the event given the console (home key); exit.
If the event's key is the page-down key, handle the event given the console (page-down key); exit.
If the event's key is the page-up key, handle the event given the console (page-up key); exit.
If the event's key is the up-arrow key, handle the event given the console (up-arrow key); exit.
If the event's byte is not printable, exit.
Append the event's byte to the console's reply.
Handle the event given the console's text (printable key).
Show the console.

To handle an event given a console (page-down key):
Scroll the console's text down one page.
Show the console.

To handle an event given a console (page-up key):
Scroll the console's text up one page.
Show the console.

To handle an event given a console (refresh):
Show the console.

To handle an event given a console (right click):
Show the hand cursor.
Scroll the console given the event.
Refresh the cursor.

To handle an event given a console (set cursor):
Show the arrow cursor.

To handle an event given a console (up-arrow key):
Scroll the console's text up one line.
Show the console.

To handle an event given a terminal:
If the terminal is nil, exit.
If the event’s kind is "key down", handle the event given the terminal (key down); exit.
If the event’s kind is "refresh", handle the event given the terminal (refresh); exit.
If the event’s kind is "set cursor", handle the event given the terminal (set cursor); exit.
If the event’s kind is "left click", relinquish control. \ *** added for invisible turtle book questionable

To handle an event given a terminal (backspace key):
If the terminal’s reply is blank, exit.
If the event is modified, exit.
Remove the last byte from the terminal’s reply.
Remove the last byte from the terminal’s quoras’ last’s string.
Show the terminal.

To handle an event given a terminal (enter key):
Relinquish control.

To handle an event given a terminal (key down):
If the event’s key is the backspace key, handle the event given the terminal (backspace key); exit.
If the event’s key is the enter key, handle the event given the terminal (enter key); exit.
If the event’s byte is not printable, exit.
Append the event’s byte to the terminal’s reply.
Append the event’s byte to the terminal’s quoras’ last’s string.
Show the terminal.

To handle an event given a terminal (refresh):
Show the terminal.

To handle an event given a terminal (set cursor):
Show the arrow cursor.

To handle an event given a text (backspace key):
If the text is nil, exit.
If there is nothing to backspace in the text, exit.
Remember the text with "backspace".
If the event is modified, remove bytes from the text (backspace with jump).
If the event is not modified, remove bytes from the text (backspace).
Wrap the text.
Scroll the text to the caret.

To handle an event given a text (delete key):
If the text is nil, exit.
If there is nothing to remove in the text, exit.
Remember the text with "delete".
If the event is modified, remove bytes from the text (forward delete with jump).
If the event is not modified, remove bytes from the text (forward delete).
Wrap the text.
Scroll the text to the caret.

To handle an event given a text (down-arrow key):
If the text is nil, exit.
If the caret of the text is on the last line, set a flag.
If the flag is set, move the caret to the last byte of the text.
If the flag is not set, move the caret down in the text.
If the event’s shift flag is not set, deselect the text.

Clear the text's last operation.
Scroll the text to the caret.

To handle an event given a text (end key):
If the text is nil, exit.
If the event is modified, move the caret to the last byte of the text.
If the event is not modified, move the caret to the last byte of the current row of the text.
If the event's shift flag is not set, deselect the text.
Clear the text's last operation.
Scroll the text to the caret.

To handle an event given a text (enter key):
If the text is nil, exit.
Remember the text with "insert return".
Remove any selected bytes in the text.
Insert the return byte into the text.
Wrap the text.
Scroll the text to the caret.

To handle an event given a text (escape key):
If the text is nil, exit.
Deselect the text.

To handle an event given a text (home key):
If the text is nil, exit.
If the event is modified, move the caret to the first byte of the text.
If the event is not modified, move the caret to the first byte of the current row of the text.
If the event's shift flag is not set, deselect the text.
Clear the text's last operation.
Scroll the text to the caret.

To handle an event given a text (left double click):
If the text is nil, exit.
Deselect the text.
Move the caret right to any non-alphanumeric byte in the text.
Move the anchor left to any non-alphanumeric byte in the text.

To handle an event given a text (left-arrow key):
If the text is nil, exit.
If the event is modified, jump the caret left in the text.
If the event is not modified, move the caret left in the text.
If the event's shift flag is not set, deselect the text.
Clear the text's last operation.
Scroll the text to the caret.

To handle an event given a text (page-down key):
If the text is nil, exit.
Scroll the text down one page.
Move the caret down one page in the text.
If the event's shift flag is not set, deselect the text.
Clear the text's last operation.

To handle an event given a text (page-up key):
If the text is nil, exit.
Scroll the text up one page.

```
Move the caret up one page in the text.
If the event’s shift flag is not set, deselect the text.
Clear the text’s last operation.

To handle an event given a text (printable key):
Remember the text with "insert".
Remove any selected bytes in the text.
Insert the event’s byte into the text.
Wrap the text.
Scroll the text to the caret.

To handle an event given a text (right-arrow key):
If the text is nil, exit.
If the event is modified, jump the caret right in the text.
If the event is not modified, move the caret right in the text.
If the event’s shift flag is not set, deselect the text.
Clear the text’s last operation.
Scroll the text to the caret.

To handle an event given a text (tab key):
If the text is nil, exit.
Remember the text with "insert".
Remove any selected bytes in the text.
Insert the space byte into the text.
Divide the text’s caret column# by 2 giving a quotient and a remainder.
If the remainder is 0, insert the space byte into the text.
Scroll the text to the caret.

To handle an event given a text (up-arrow key):
If the text is nil, exit.
Move the caret up in the text.
If the event’s shift flag is not set, deselect the text.
Clear the text’s last operation.
Scroll the text to the caret.

To handle events given a console:
If the console is nil, exit.
Loop.
Deque an event.
If the event is nil, exit.
Handle the event given the console.
Repeat.

To handle events given a terminal:
If the terminal is nil, exit.
Loop.
Deque an event.
If the event is nil, exit.
Handle the event given the terminal.
Repeat.

To handle font height given a text and a box and a font height:
If the text is nil, exit.
Remember the text with "font height".
Change the text given the box.
```

```
Change the text given the font height.

To handle font height given a text and a font height:
If the text is nil, exit.
Remember the text with "font height".
Change the text given the font height.

To handle font name given a text and a font name:
If the text is nil, exit.
Remember the text with "font name".
Change the text given the font name.

To handle indent given a text:
If the text is nil, exit.
If nothing is selected in the text, exit.
Remember the text with "dent".
Indent any selected rows in the text.
Square up any selection in the text.
Wrap the text.

A handle is a number.

To handle lowercase given a text:
If the text is nil, exit.
If nothing is selected in the text, exit.
Remember the text with "case".
Lowercase any selected bytes in the text.
Wrap the text.

To handle outdent given a text:
If the text is nil, exit.
If nothing is selected in the text, exit.
Remember the text with "dent".
Outdent any selected rows in the text.
Square up any selection in the text.
Wrap the text.

To handle paste given a text:
If the text is nil, exit.
If there is not text on the windows clipboard, exit.
Remember the text.
Remove any selected bytes in the text.
Get a string from the windows clipboard.
Insert the string into the text.
Wrap the text.
Scroll the text to the caret.

To handle pen given a text and a color:
If the text is nil, exit.
Remember the text with "pen".
Put the color into the text's pen.

To handle redo given a text:
If the text is nil, exit.
If the text's redos' last is nil, exit.
```

```
Copy the text into another text.
Append the other text to the text's undos.
Put the text's redos' last into a third text.
Remove the third text from the text's redos.
Copy the guts of the third text into the text.
Destroy the third text.
Set the text's modified flag.

To handle reverse given a text:
If the text is nil, exit.
If nothing is selected in the text, exit.
Remember the text with "reverse".
Reverse any selected rows of the text.
Square up any selection in the text.
Wrap the text.

To handle select all given a text:
If the text is nil, exit.
Select every byte in the text.

To handle sort any selected rows given a text:
If the text is nil, exit.
If nothing is selected in the text, exit.
Remember the text with "sort selected rows".
Sort any selected rows in the text.
Square up any selection in the text.
Wrap the text.

To handle undo given a text:
If the text is nil, exit.
If the text's undos' last is nil, exit.
Copy the text into another text.
Append the other text to the text's redos.
Put the text's undos' last into a third text.
Remove the third text from the text's undos.
Copy the guts of the third text into the text.
Destroy the third text.
Set the text's modified flag.

To handle uppercase given a text:
If the text is nil, exit.
If nothing is selected in the text, exit.
Remember the text with "case".
Uppercase any selected bytes in the text.
Wrap the text.

The hash-tag byte is a byte equal to 163.

The hashtag byte is a byte equal to 163.

An hbitmap is a handle.

An hbrush is a handle.

An hdc is a handle.
```

A heading is some points.

The heap count is a number.

The heap pointer is a pointer.

A height is some twips.

A hex string is a string.

An hfont is a handle.

An hicon is a handle.

To hide the cursor:
Call "user32.dll" "ShowCursor" with 0 returning a number.
If the number is less than 0, exit.
Repeat.

The hilite color is a color.

The home key is a key equal to 36.

A horizontal line is a line.

An hour is 60 minutes.

An hpen is a handle.

An hrgn is a handle.

A hue is some precise degrees [0 to 3599].

A hundred is 100 units.

The i-beam cursor is a cursor.

The i-key is a key equal to 73.

The i/o error is a string.

A iid is a uuid.

To imagine a box some twips by some other twips;
To make a box some twips by some other twips:
Put 0 into the box's left.
Put 0 into the box's top.
Put the twips into the box's right.
Put the other twips into the box's bottom.

To imagine a box some twips high by some other twips wide;
To make a box some twips high by some other twips wide:
Put 0 and 0 and the other twips and the twips into the box.

To imagine a box some twips smaller than another box;
To make a box some twips smaller than another box:
Put the other box into the box.
Indent the box by the twips divided by 2.

To imagine a box some twips smaller than another box on every side:
Put the other box into the box.
Indent the box by the twips.

To imagine a box some twips wide by some other twips high;
To make a box some twips wide by some other twips high:
Put 0 and 0 and the twips and the other twips into the box.

To imagine a box with a left coord and a top coord and a right coord and a bottom coord;
To make a box with a left coord and a top coord and a right coord and a bottom coord:
Put the left coord and the top coord and the right coord and the bottom coord into the box.

To imagine a box with a spot and another spot;
To make a box with a spot and another spot:
Put the spot and the other spot into the box.

To imagine a color from a hue and a saturation and a lightness;
To make a color from a hue and a saturation and a lightness:
Put the hue and the saturation and the lightness into the color.

To imagine a dot about some twips wide;
To make a dot about some twips wide;
To make a dot some twips wide:
Make the dot the twips by the twips.

To imagine a dot between some twips and some other twips wide;
To make a dot between some twips and some other twips wide:
Pick some third twips between the twips and the other twips.
Make the dot the third twips wide.

To imagine an ellipse given a box;
To make an ellipse given a box:
Put the box into the ellipse's box.

To imagine an ellipse some twips by some other twips;
To make an ellipse some twips by some other twips:
Put 0 into the ellipse's left.
Put 0 into the ellipse's top.
Put the twips into the ellipse's right.
Put the other twips into the ellipse's bottom.

To imagine an ellipse with a left coord and a top coord and a right coord and a bottom coord;
To make an ellipse with a left coord and a top coord and a right coord and a bottom coord:
Put the left coord and the top coord and the right coord and the bottom coord into the ellipse.

To imagine an ellipse with a spot and another spot;
To make an ellipse with a spot and another spot:
Put the spot and the other spot into the ellipse.

To imagine a figure using a string and a spot;

```
To make a figure using a string and a spot;
To create a figure using a string and a center spot:
Create the figure.
Append the figure to the figures.
Privatize the string.
Lowercase the string.
Slap a substring on the string.
Loop.
Skip any leading noise in the substring.
If the substring's length is less than 2, exit.
Put the substring's first's target into a byte.
Put the byte minus the little-a byte into a spot's y.
Add 1 to the substring's first.
Put the substring's first's target into the byte.
Put the byte minus the little-a byte into the spot's x.
Multiply the spot by 1/4 inch.
Add the center spot's x minus 3-1/8 inches plus 1 pixel to the spot's x.
Add the center spot's y minus 3-1/8 inches plus 1 pixel to the spot's y.
Append the spot to the figure.
Add 1 to the substring's first.
Repeat.

To imagine a horizontal line a fraction of the way up from the bottom of a box;
To make a horizontal line a fraction of the way up from the bottom of a box:
Imagine the horizontal line across the box the fraction of the way up from the bottom.

To imagine a line across the bottom of a box;
To make a line across the bottom of a box;
To imagine a line along the bottom of a box;
To make a line along the bottom of a box:
Put the box's bottom line into the line.

To imagine a line across a box a fraction of the way up from the bottom;
To make a line across a box a fraction of the way up from the bottom:
Put the box's left into the line's start's x.
Put the box's right into the line's end's x.
Put the box's bottom times the fraction into some twips.
Put the box's bottom minus the twips into the line's start's y.
Put the box's bottom minus the twips into the line's end's y.

To imagine a line across the top of a box;
To make a line across the top of a box;
To imagine a line along the top of a box;
To make a line along the top of a box:
Put the box's top line into the line.

To imagine a line in the middle of a box;
To make a line in the middle of a box;
To imagine a line across the middle of a box;
To make a line across the middle of a box;
To imagine a line in the center of a box;
To make a line in the center of a box;
To imagine a line across the center of a box;
To make a line across the center of a box:
Put the box's left into the line's start's x.
```

Put the box's right into the line's end's x.
Put the box's center's y into the line's start's y.
Put the box's center's y into the line's end's y.

To imagine a line some twips up from the bottom of a box;
To make a line some twips up from the bottom of a box:
Put the box's left into the line's start's x.
Put the box's right into the line's end's x.
Put the box's bottom minus the twips into the line's start's y.
Put the box's bottom minus the twips into the line's end's y.

To imagine a line with a spot and another spot;
To make a line with a spot and another spot:
Put the spot and the other spot into the line.

To imagine a line with an x coord and a y coord and another x coord and another y coord;
To make a line with an x coord and a y coord and another x coord and another y coord:
Put the x coord and the y coord and the other x coord and the other y coord into the line.

To imagine a roundy box from a box and a radius;
To make a roundy box from a box and a radius:
Put the box and the radius into the roundy box.

To imagine a roundy box some twips by some other twips;
To make a roundy box some twips by some other twips:
Put 0 into the roundy box's left.
Put 0 into the roundy box's top.
Put the twips into the roundy box's right.
Put the other twips into the roundy box's bottom.

To imagine a roundy box with a left coord and a top coord and a right coord and a bottom coord and a radius;
To make a roundy box with a left coord and a top coord and a right coord and a bottom coord and a radius:
Put the left coord and the top coord and the right coord and the bottom coord and the radius into the roundy box.

To imagine a roundy box with a spot and another spot and a radius;
To make a roundy box with a spot and another spot and a radius:
Put the spot and the other spot and the radius into the roundy box.

To imagine a spot with an x coord and a y coord;
To make a spot with an x coord and a y coord:
Put the x coord and the y coord into the spot.

An inch is 1440 twips.

To include a font in the current pdf:
Find a pdf object given the font's name and the pdf state's font index.
If the pdf object is not nil, exit.
Create a font info given the font.
Convert the font info to pdf em units.
\ stream
Put the actual data of the font into a buffer.
Convert the buffer to a nibble string.

Add a stream pdf object given "font stream".
Append the stream's number then " 0 obj" to the stream.
Append "<<" to the stream without advancing.
Append "/Filter /ASCIIHexDecode" to the stream without advancing.
Append " /Length " then the nibble string's length to the stream without advancing.
Append " /Length1 " then the buffer's length to the stream without advancing.
Append ">>" to the stream.
Append "stream" to the stream.
Append the nibble string to the stream.
Append "endstream" to the stream.
Append "endobj" to the stream.
\ descriptor
Add a descriptor pdf object given "font descriptor".
Put "F" then the descriptor's number into a font name.
Put the font's name into a font base name.
Replace the space byte with the underscore byte in the font base name.
Append the descriptor's number then " 0 obj" to the descriptor.
Append "<<" to the descriptor.
Append "/Type /FontDescriptor" to the descriptor.
Append "/FontName /" then the font base name to the descriptor.
Append "/FontFile2 " then the stream's number then " 0 R" to the descriptor.
Append "/Flags " then the font info's flags to the descriptor.
Append "/FontBBox [" then the font info's font box then "]" to the descriptor.
Append "/Ascent " then the font info's ascent to the descriptor.
Append "/Descent " then the font info's descent to the descriptor.
Append "/CapHeight " then the font info's capheight to the descriptor.
Append "/ItalicAngle " then the font info's italicangle to the descriptor.
Append "/StemV " then the font info's stemv to the descriptor.
Append ">>" to the descriptor.
Append "endobj" to the descriptor.
\ definition
Add a definition pdf object given "font definition".
Put the font name into the definition's font name.
Append the definition's number then " 0 obj" to the definition.
Append "<<" to the definition.
Append "/Type /Font" to the definition.
Append "/Subtype /TrueType" to the definition.
Append "/Name /" then the font name to the definition.
Append "/BaseFont /" then the font base name to the definition.
Append "/Encoding /WinAnsiEncoding" to the definition.
Append "/FontDescriptor " then the descriptor's number then " 0 R" to the definition.
Append "/FirstChar 0" to the definition.
Append "/LastChar 255" to the definition.
Put the font info's font widths into another buffer.
Append "/Widths [" to the definition.
Append the other buffer then "]" to the definition.
Append ">>" to the definition.
Append "endobj" to the definition.
Put the font info into the definition's font info.
Index the definition given the font's name and the pdf state's font index.

To include a font in a pdf object:
If the pdf object is nil, exit.
Find a font pdf object given the font's name and the pdf state's font index.
If the font pdf object is nil, exit.

Put "/" then the font pdf object's font name then " " then the font pdf object's number then " 0 R" into a string.
Find a string thing given the string and the pdf object's font strings.
If the string thing is not nil, exit.
Create a new string thing given the string.
Append the new string thing to the pdf object's font strings.

To indent any selected rows in a text:
If the text is nil, exit.
Loop.
Get a row from the text's rows.
If the row is nil, exit.
If the row of the text is not selected, repeat.
If the row is blank, repeat.
Prepend the space byte to the row's string.
Prepend the space byte to the row's string.
Repeat.

An indent is an count. \ *** a count?

An index is a thing with
A bucket count,
A first bucket and a last bucket.

To index a pointer given a string and an index:
\ if the index is nil, exit. \ to make compiler faster
If the string's length is 0, exit.
Get a bucket given the string and the index.
Create a refer.
Append the refer to the bucket's refers.
Put the string into the refer's string.
Put the pointer into the refer's pointer.

To index a string in an index:
\ if the index is nil, exit. \ to make compiler faster
Index nil given the string and the index.

To initalize the terminal:
Create the terminal in the screen's box.

To initialize before run:
Call "user32.dll" "DisableProcessWindowsGhosting".
Call "kernel32.dll" "GetProcessHeap" returning the heap pointer.
Call "kernel32.dll" "LoadLibraryA" with "kernel32.dll"'s first returning a handle.
If the handle is not 0, call "kernel32.dll" "GetProcAddress" with the handle and "HeapSetInformation"'s first returning a pointer.
If the pointer is not nil, call the pointer with the heap pointer and 0 and 2's whereabouts and 4.

To initialize the canvases:
Initialize the screen canvas.
Initialize the memory canvas.
Put the memory canvas into the current canvas.

To initialize the cgi:
Call "kernel32.dll" "AllocConsole".

Call "kernel32.dll" "GetStdHandle" with -10 [std_input_handle] returning the stdin handle.
Call "kernel32.dll" "GetStdHandle" with -11 [std_output_handle] returning the stdout handle.

To initialize the colors:
Put 1 into the pen size.
Call "gdi32.dll" "GetStockObject" with 8 [null_pen] returning the null hpen.
Call "gdi32.dll" "GetStockObject" with 5 [null_brush] returning the null hbrush.
Put -1 and 0 and 0 into the clear color.
Put 0 and 0 and 1000 into the white color.
Put 0 and 0 and 875 into the lightest gray color.
Put 0 and 0 and 750 into the lighter gray color.
Put 0 and 0 and 625 into the light gray color.
Put 0 and 0 and 500 into the gray color.
Put 0 and 0 and 375 into the dark gray color.
Put 0 and 0 and 250 into the darker gray color.
Put 0 and 0 and 125 into the darkest gray color.
Put 0 and 0 and 0 into the black color.
Put 0 and 1000 and 875 into the lightest red color.
Put 0 and 1000 and 750 into the lighter red color.
Put 0 and 1000 and 625 into the light red color.
Put 0 and 1000 and 500 into the red color.
Put 0 and 1000 and 375 into the dark red color.
Put 0 and 1000 and 250 into the darker red color.
Put 0 and 1000 and 125 into the darkest red color.
Put 300 and 1000 and 875 into the lightest orange color.
Put 300 and 1000 and 750 into the lighter orange color.
Put 300 and 1000 and 625 into the light orange color.
Put 300 and 1000 and 500 into the orange color.
Put 300 and 1000 and 375 into the dark orange color.
Put 300 and 1000 and 250 into the darker orange color.
Put 300 and 1000 and 125 into the darkest orange color.
Put 600 and 1000 and 875 into the lightest yellow color.
Put 600 and 1000 and 750 into the lighter yellow color.
Put 600 and 1000 and 625 into the light yellow color.
Put 600 and 1000 and 500 into the yellow color.
Put 600 and 1000 and 375 into the dark yellow color.
Put 600 and 1000 and 250 into the darker yellow color.
Put 600 and 1000 and 125 into the darkest yellow color.
Put 900 and 1000 and 875 into the lightest lime color.
Put 900 and 1000 and 750 into the lighter lime color.
Put 900 and 1000 and 625 into the light lime color.
Put 900 and 1000 and 500 into the lime color.
Put 900 and 1000 and 375 into the dark lime color.
Put 900 and 1000 and 250 into the darker lime color.
Put 900 and 1000 and 125 into the darkest lime color.
Put 1200 and 1000 and 875 into the lightest green color.
Put 1200 and 1000 and 750 into the lighter green color.
Put 1200 and 1000 and 625 into the light green color.
Put 1200 and 1000 and 500 into the green color.
Put 1200 and 1000 and 375 into the dark green color.
Put 1200 and 1000 and 250 into the darker green color.
Put 1200 and 1000 and 125 into the darkest green color.
Put 1500 and 1000 and 875 into the lightest teal color.
Put 1500 and 1000 and 750 into the lighter teal color.
Put 1500 and 1000 and 625 into the light teal color.

Put 1500 and 1000 and 500 into the teal color.
Put 1500 and 1000 and 375 into the dark teal color.
Put 1500 and 1000 and 250 into the darker teal color.
Put 1500 and 1000 and 125 into the darkest teal color.
Put 1800 and 1000 and 875 into the lightest cyan color.
Put 1800 and 1000 and 750 into the lighter cyan color.
Put 1800 and 1000 and 625 into the light cyan color.
Put 1800 and 1000 and 500 into the cyan color.
Put 1800 and 1000 and 375 into the dark cyan color.
Put 1800 and 1000 and 250 into the darker cyan color.
Put 1800 and 1000 and 125 into the darkest cyan color.
Put 2100 and 1000 and 875 into the lightest sky color.
Put 2100 and 1000 and 750 into the lighter sky color.
Put 2100 and 1000 and 625 into the light sky color.
Put 2100 and 1000 and 500 into the sky color.
Put 2100 and 1000 and 375 into the dark sky color.
Put 2100 and 1000 and 250 into the darker sky color.
Put 2100 and 1000 and 125 into the darkest sky color.
Put 2400 and 1000 and 875 into the lightest blue color.
Put 2400 and 1000 and 750 into the lighter blue color.
Put 2400 and 1000 and 625 into the light blue color.
Put 2400 and 1000 and 500 into the blue color.
Put 2400 and 1000 and 375 into the dark blue color.
Put 2400 and 1000 and 250 into the darker blue color.
Put 2400 and 1000 and 125 into the darkest blue color.
Put 2700 and 1000 and 875 into the lightest purple color.
Put 2700 and 1000 and 750 into the lighter purple color.
Put 2700 and 1000 and 625 into the light purple color.
Put 2700 and 1000 and 500 into the purple color.
Put 2700 and 1000 and 375 into the dark purple color.
Put 2700 and 1000 and 250 into the darker purple color.
Put 2700 and 1000 and 125 into the darkest purple color.
Put 3000 and 1000 and 875 into the lightest magenta color.
Put 3000 and 1000 and 750 into the lighter magenta color.
Put 3000 and 1000 and 625 into the light magenta color.
Put 3000 and 1000 and 500 into the magenta color.
Put 3000 and 1000 and 375 into the dark magenta color.
Put 3000 and 1000 and 250 into the darker magenta color.
Put 3000 and 1000 and 125 into the darkest magenta color.
Put 3300 and 1000 and 875 into the lightest violet color.
Put 3300 and 1000 and 750 into the lighter violet color.
Put 3300 and 1000 and 625 into the light violet color.
Put 3300 and 1000 and 500 into the violet color.
Put 3300 and 1000 and 375 into the dark violet color.
Put 3300 and 1000 and 250 into the darker violet color.
Put 3300 and 1000 and 125 into the darkest violet color.
Put 0 and 0 and 800 into the hilite color.
\ special colors
Put the lightest orange color into the tan color.
Put the dark orange color into the brown color.
Put the darker orange color into the dark brown color.
Put the darkest orange color into the darker brown color.
Put the darkest orange color into the darkest brown color.
Put the lightest red color into the pink color.
Put the lighter red color into the dark pink color.

Put 500 and 1000 and 500 into the gold color.
\ colors as pens
Put the clear color into the clear pen.
Put the white color into the white pen.
Put the black color into the black pen.
Put the lightest gray color into the lightest gray pen.
Put the lighter gray color into the lighter gray pen.
Put the light gray color into the light gray pen.
Put the gray color into the gray pen.
Put the dark gray color into the dark gray pen.
Put the darker gray color into the darker gray pen.
Put the darkest gray color into the darkest gray pen.
Put the lightest red color into the lightest red pen.
Put the lighter red color into the lighter red pen.
Put the light red color into the light red pen.
Put the red color into the red pen.
Put the dark red color into the dark red pen.
Put the darker red color into the darker red pen.
Put the darkest red color into the darkest red pen.
Put the lightest orange color into the lightest orange pen.
Put the lighter orange color into the lighter orange pen.
Put the light orange color into the light orange pen.
Put the orange color into the orange pen.
Put the dark orange color into the dark orange pen.
Put the darker orange color into the darker orange pen.
Put the darkest orange color into the darkest orange pen.
Put the lightest yellow color into the lightest yellow pen.
Put the lighter yellow color into the lighter yellow pen.
Put the light yellow color into the light yellow pen.
Put the yellow color into the yellow pen.
Put the dark yellow color into the dark yellow pen.
Put the darker yellow color into the darker yellow pen.
Put the darkest yellow color into the darkest yellow pen.
Put the lightest lime color into the lightest lime pen.
Put the lighter lime color into the lighter lime pen.
Put the light lime color into the light lime pen.
Put the lime color into the lime pen.
Put the dark lime color into the dark lime pen.
Put the darker lime color into the darker lime pen.
Put the darkest lime color into the darkest lime pen.
Put the lightest green color into the lightest green pen.
Put the lighter green color into the lighter green pen.
Put the light green color into the light green pen.
Put the green color into the green pen.
Put the dark green color into the dark green pen.
Put the darker green color into the darker green pen.
Put the darkest green color into the darkest green pen.
Put the lightest teal color into the lightest teal pen.
Put the lighter teal color into the lighter teal pen.
Put the light teal color into the light teal pen.
Put the teal color into the teal pen.
Put the dark teal color into the dark teal pen.
Put the darker teal color into the darker teal pen.
Put the darkest teal color into the darkest teal pen.
Put the lightest cyan color into the lightest cyan pen.

Put the lighter cyan color into the lighter cyan pen.
Put the light cyan color into the light cyan pen.
Put the cyan color into the cyan pen.
Put the dark cyan color into the dark cyan pen.
Put the darker cyan color into the darker cyan pen.
Put the darkest cyan color into the darkest cyan pen.
Put the lightest sky color into the lightest sky pen.
Put the lighter sky color into the lighter sky pen.
Put the light sky color into the light sky pen.
Put the sky color into the sky pen.
Put the dark sky color into the dark sky pen.
Put the darker sky color into the darker sky pen.
Put the darkest sky color into the darkest sky pen.
Put the lightest blue color into the lightest blue pen.
Put the lighter blue color into the lighter blue pen.
Put the light blue color into the light blue pen.
Put the blue color into the blue pen.
Put the dark blue color into the dark blue pen.
Put the darker blue color into the darker blue pen.
Put the darkest blue color into the darkest blue pen.
Put the lightest purple color into the lightest purple pen.
Put the lighter purple color into the lighter purple pen.
Put the light purple color into the light purple pen.
Put the purple color into the purple pen.
Put the dark purple color into the dark purple pen.
Put the darker purple color into the darker purple pen.
Put the darkest purple color into the darkest purple pen.
Put the lightest magenta color into the lightest magenta pen.
Put the lighter magenta color into the lighter magenta pen.
Put the light magenta color into the light magenta pen.
Put the magenta color into the magenta pen.
Put the dark magenta color into the dark magenta pen.
Put the darker magenta color into the darker magenta pen.
Put the darkest magenta color into the darkest magenta pen.
Put the lightest violet color into the lightest violet pen.
Put the lighter violet color into the lighter violet pen.
Put the light violet color into the light violet pen.
Put the violet color into the violet pen.
Put the dark violet color into the dark violet pen.
Put the darker violet color into the darker violet pen.
Put the darkest violet color into the darkest violet pen.
\ special color pens
Put the tan color into the tan pen.
Put the brown color into the brown pen.
Put the dark brown color into the dark brown pen.
Put the darker brown color into the darker brown pen.
Put the darkest brown color into the darkest brown pen.
Put the pink color into the pink pen.
Put the dark pink color into the dark pink pen.
Put the gold color in the gold pen.
\ "sky" renamed "sky blue"
Put the lightest sky color into the lightest sky blue color.
Put the lighter sky color into the lighter sky blue color.
Put the light sky color into the light sky blue color.
Put the sky color into the sky blue color.

Put the dark sky color into the dark sky blue color.
Put the darker sky color into the darker sky blue color.
Put the darkest sky color into the darkest sky blue color.
Put the lightest sky color into the lightest sky blue pen.
Put the lighter sky color into the lighter sky blue pen.
Put the light sky color into the light sky blue pen.
Put the sky color into the sky blue pen.
Put the dark sky color into the dark sky blue pen.
Put the darker sky color into the darker sky blue pen.
Put the darkest sky color into the darkest sky blue pen.

To initialize com:
Call "ole32.dll" "CoInitializeEx" with 0 and 2 [coinit_apartmentthreaded].

To initialize a context:
Allocate memory for the context.
Put the screen's center into the context's spot.
Put 0 into the context's heading.
Put the green color into the context's color.
Put the small letter height into the context's letter height.
\Put 1/60 second into the delay. ***
Seed the random number generator.

To initialize the cursors:
Initialize the cursors (arrow cursor).
Initialize the cursors (hand cursor).
Initialize the cursors (i-beam cursor).
Hide the cursor.

To initialize the cursors (arrow cursor):
Append $00000000000000004000000060000000 to an xor-mask.
Append $70000000780000007C0000007E000000 to the xor-mask.
Append $7F0000007F8000007C0000006C000000 to the xor-mask.
Append $46000000060000000300000003000000 to the xor-mask.
Append $01800000018000000000000000000000 to the xor-mask.
Append $00 to the xor-mask given 48.
Append $7FFFFFFF3FFFFFFF1FFFFFFF0FFFFFFF to an and-mask.
Append $07FFFFFF03FFFFFF01FFFFFF00FFFFFF to the and-mask.
Append $007FFFFF003FFFFF001FFFFF01FFFFFF to the and-mask.
Append $10FFFFFF30FFFFFF787FFFFF87FFFFF to the and-mask.
Append $FC3FFFFFFC3FFFFFFE7FFFFFFFFFFFFF to the and-mask.
Append $FF to the and-mask given 48.
Call "user32.dll" "CreateCursor" with the module's handle and 0 and 0 and 32 and 32 and the and-mask's first and the xor-mask's first returning the arrow cursor.

To initialize the cursors (hand cursor):
Append $00000000180000019B0000019B00000 to an xor-mask.
Append $0DB200000DB6000007F6000067FE0000 to the xor-mask.
Append $7FFC00003FFC00001FFC00001FF80000 to the xor-mask.
Append $0FF8000007F0000003F0000003F00000 to the xor-mask.
Append $00 to the xor-mask given 64.
Append $FE7FFFFFE40FFFFFC007FFFFC005FFFF to an and-mask.
Append $E000FFFFE000FFFF9000FFFF0000FFFF to the and-mask.
Append $0001FFFF8001FFFFC001FFFFC003FFFF to the and-mask.
Append $E003FFFFF007FFFFF807FFFFF807FFFF to the and-mask.

Append $FF to the and-mask given 64.
Call "user32.dll" "CreateCursor" with the module's handle and 2 and 1 and 32 and 32 and the and-mask's first and the xor-mask's first returning the hand cursor.

To initialize the cursors (i-beam cursor):
Append $EE00000010000000100000010000000 to an xor-mask.
Append $10000000100000001000000010000000 to the xor-mask.
Append $10000000100000001000000010000000 to the xor-mask.
Append $10000000100000010000000EE000000 to the xor-mask.
Append $00 to the xor-mask given 64.
Append $FF to an and-mask given 128.
Call "user32.dll" "CreateCursor" with the module's handle and 3 and 7 and 32 and 32 and the and-mask's first and the xor-mask's first returning the i-beam cursor.

To initialize the fonts:
Call "gdi32.dll" "GetStockObject" with 11 [ansi_fixed_font] returning the null hfont.
Call "gdi32.dll" "AddFontMemResourceEx" with the osmosian font source's first and the osmosian font source's length and 0 and a number's whereabouts
Returning the osmosian font resource.
Put "osmosian" and 1/4 inch into the default font.
\ stroked fonts below
Put 1/8 inch into the small letter height.
Put 1/4 inch into the medium letter height.
Put 1/2 inch into the large letter height.

To initialize gdi+:
Put 1 into a gdiplusstartupinput's gdiplusversion.
Call "gdiplus.dll" "GdiplusStartup" with the gptoken's whereabouts and the gdiplusstartupinput's whereabouts and 0.

To initialize the memory canvas:
Call "gdi32.dll" "CreateCompatibleDC" with the screen canvas returning the memory canvas.
Call "gdi32.dll" "GetCurrentObject" with the memory canvas and 7 [obj_bitmap] returning the saved memory hbitmap.
Call "gdi32.dll" "CreateCompatibleBitmap" with the screen canvas and the screen's pixel width and the screen's pixel height returning an hbitmap.
Call "gdi32.dll" "SelectObject" with the memory canvas and the hbitmap.
Normalize the memory canvas.

To initialize the module:
\ temp path
Put 512 into a length.
Reassign the temp path's first given the length.
Call "kernel32.dll" "GetTempPathA" with the length and the temp path's first returning the length.
Put the temp path's first plus the length minus 1 into the temp path's last.
Null terminate the temp path.
\ module handle
Call "kernel32.dll" "GetModuleHandleA" with 0 returning the module's handle.
\ module name
Put 512 into the length.
Reassign the module's path's first given the length.
Call "kernel32.dll" "GetModuleFileNameA" with the module's handle and the module's path's first and the length returning the length.
Put the module's path's first plus the length minus 1 into the module's path's last.
If the module's path starts with "\\?\", remove leading bytes from the module's path given 4.

Lowercase the module's path.
Null terminate the module's path.
\ module's other path pieces
Extract the module's name from the module's path.
Null terminate the module's name.
Extract the module's directory from the module's path.
Null terminate the module's directory.
Extract the module's root directory from the module's directory.
Null terminate the module's root directory.

To initialize the mouse:
Put 1 into the mouse's left button.
Put 2 into the mouse's right button.
Call "user32.dll" "GetSystemMetrics" with 23 [sm_swapbutton] returning a number.
If the number is 0, exit.
Swap the mouse's left button with the mouse's right button.

To initialize the printer canvas:
Put a printdlgex's magnitude into the printdlgex's lstructsize.
Put the main window into the printdlgex's hwndowner.
Put 1288 [pd_returndc + pd_returndefault + pd_nopageenums] into the printdlgex's flags.
Put -1 [start_page_general] into the printdlgex's nstartpage.
Call "comdlg32.dll" "PrintDlgExA" with the printdlgex's whereabouts.
Call "kernel32.dll" "GlobalFree" with the printdlgex's hdevnames.
Put the printdlgex's hdevmode into the printer device mode handle.
Put the printdlgex's hdc into the printer canvas.

To initialize the screen:
Call "user32.dll" "GetSystemMetrics" with 0 [sm_cxscreen] returning the screen's pixel width.
Call "user32.dll" "GetSystemMetrics" with 1 [sm_cyscreen] returning the screen's pixel height.
Put 96 into the ppi.
Put the tpi divided by the ppi into the tpp.
Put the screen's pixel width times the tpp into a width.
Put the screen's pixel height times the tpp into a height.
Put 0 and 0 and the width and the height into the screen's box.
Subtract the tpp from the screen's right-bottom.

To initialize the screen canvas:
Call "user32.dll" "GetDC" with the main window returning the screen canvas.
Normalize the screen canvas.

To initialize a talker:
Convert "{96749377-3391-11D2-9EE3-00C04F797396}" [clsid_spvoice] to a clsid.
Convert "{6C44DF74-72B9-4992-A1EC-EF996E0422D4}" [iid_ispvoice] to an iid.
Call "ole32.dll" "CoCreateInstance" with the clsid's whereabouts and 0 and 7 [clsctx_all] and the iid's whereabouts and the talker's whereabouts.

To initialize the window:
Put a window class's magnitude into the window class' cbsize.
Put 40 [cs_owndc + cs_dblclks] into the window class' style.
Point the window class' lpfnwndproc to routine handle any message with a window a message number a w-param and a l-param.
Put the module's handle into the window class' hinstance.
Put the module's name's first into the window class' lpszclassname.
Call "user32.dll" "RegisterClassExA" with the window class's whereabouts.

Call "user32.dll" "CreateWindowExA" with 0 and the module's name's first and the module's name's first and -2147483648 [ws_popup]
And 0 and 0 and the screen's pixel width and the screen's pixel height and 0 and 0 and the module's handle and 0.
Call "user32.dll" "ShowWindow" with the main window and 1 [sw_shownormal].

To initialize winsock:
Call "ws2_32.dll" "WSAStartup" with 2 and a wsadata's whereabouts.

An input is an event.

To insert a byte into a text:
If the text is nil, exit.
Put the byte into a string.
Insert the string into the text.

The insert key is a key equal to 45.

To insert a spot into a polygon after a vertex:
If the polygon is nil, exit.
Create another vertex given the spot.
Insert the other vertex into the polygon's vertices after the vertex.

To insert a string into another string before a byte#:
If the string's length is 0, exit.
Privatize the byte#.
Limit the byte# to 1 and the other string's length plus 1.
Slap a substring on the other string. \ left side
Put the substring's first plus the byte# minus 2 into the substring's last.
Slap another substring on the other string. \ right side
Put the other substring's first plus the byte# minus 1 into the other substring's first.
Put the other string's length plus the string's length into a combined length.
Reassign a pointer given the combined length.
Put the pointer into a third substring's first.
Copy bytes from the substring's first to the third substring's first for the substring's length.
Add the substring's length to the third substring's first.
Copy bytes from the string's first to the third substring's first for the string's length.
Add the string's length to the third substring's first.
Copy bytes from the other substring's first to the third substring's first for the other substring's length.
Unassign the other string's first. \ don't use put a string into a string to prevent extra allocating and copying
Put the pointer into the other string's first.
Put the other string's first plus the combined length minus 1 into the other string's last.

To insert a string into a text:
If the text is nil, exit.
Get a row given the text's caret row# and the text.
Put the row's string's length minus the text's caret column# into a number.
Put the row's string into another string.
Insert the string into the other string before the text's caret column#.
Convert the other string to some rows.
Put the rows' last into another row.
Insert the rows into the text's rows before the row.
Remove the row from the text's rows.
Destroy the row.

Renumber the text's rows.
Put the other row's row# into the text's caret row#.
Put the other row's string's length minus the number into the text's caret column#.
Deselect the text.

To insert a thing into some things after another thing:
If the thing is nil, exit.
If the other thing is nil, prepend the thing to the things; exit.
Insert the thing into the things before the other thing's next.

To insert a thing into some things before another thing:
If the thing is nil, exit.
If the things are empty, append the thing to the things; exit.
If the other thing is nil, append the thing to the things; exit.
If the other thing is the things' first, prepend the thing to the things; exit.
Put the thing into a new thing.
Put the other thing into a previous thing.
Put the new thing into the previous thing's previous' next.
Put the previous thing into the new thing's next.
Put the previous thing's previous into the new thing's previous.
Put the new thing into the previous thing's previous.

To insert some things into some other things after a thing:
If the thing is nil, prepend the things to the other things; exit.
Insert the things into the other things before the thing's next.

To insert some things into some other things before a thing:
Privatize the thing.
Loop.
Put the things' first into another thing.
If the other thing is nil, exit.
Remove the other thing from the things.
Insert the other thing into the other things before the thing.
Repeat.

To insert a vertex into a polygon after another vertex:
If the polygon is nil, exit.
If the vertex is nil, exit.
Insert the vertex into the polygon's vertices after the other vertex.

To insert a vertex into a polygon at a spot:
If the polygon is nil, exit.
If the vertex is nil, exit.
Loop.
Get another vertex from the polygon's vertices.
If the other vertex is nil, exit.
If the other vertex's next is nil, exit.
Put the other vertex's spot and the other vertex's next's spot into a line.
If the spot is not on the line, repeat.
Insert the vertex into the polygon's vertices after the other vertex.

To insert a vertex into a polygon before another vertex:
If the polygon is nil, exit.
If the vertex is nil, exit.
Insert the vertex into the polygon's vertices before the other vertex.

To intersect a box with another box giving a third box:
\ boxes do not touch
Clear the third box.
If the box's left is greater than the other box's right, exit.
If the box's top is greater than the other box's bottom, exit.
If the box's right is less than the other box's left, exit.
If the box's bottom is less than the other box's top, exit.
\ boxes touch
Put the box into the third box.
If the box's left is less than the other box's left, put the other box's left into the third box's left.
If the box's top is less than the other box's top, put the other box's top into the third box's top.
If the box's right is greater than the other box's right, put the other box's right into the third box's right.
If the box's bottom is greater than the other box's bottom, put the other box's bottom into the third box's bottom.

To invert a flag:
If the flag is yes, put no into the flag; exit.
Put yes into the flag.

The inverted-exclamation-mark byte is a byte equal to 161.

The inverted-question-mark byte is a byte equal to 191.

A in_addr is a record with
A byte called s_b1,
A byte called s_b2,
A byte called s_b3,
A byte called s_b4,
A wyrd [unsigned] called s_w1 at the s_b1,
A wyrd [unsigned] called s_w2 at the s_b3,
A number called s_addr at the s_b1.

An ip address has
A number,
A string.

An istream is a pointer to an istream object.

An istream object is a record with an istream vtable called vtable.

An istream vtable is a pointer to an istream vtable record.

An istream vtable record is a record with
\ iunknown
A pointer called queryinterface,
A pointer called addref,
A pointer called release, \ function(this:istream):number; stdcall;
\ istream
A pointer called read,
A pointer called write,
A pointer called seek,
A pointer called setsize,
A pointer called copyto,
A pointer called commit,

A pointer called revert,
A pointer called lockregion,
A pointer called unlockregion,
A pointer called stat,
A pointer called clone.

An item has
A kind,
A path, a directory, a designator, an extension,
A size,
A win32finddata and a handle.

The j-key is a key equal to 74.

To jump the caret left in a text:
If the text is nil, exit.
Move the caret left to any non-noise byte in the text.
If the text's caret column# is 1, exit.
Get a row given the text's caret row# and the text.
Put the row's string's first plus the text's caret column# minus 2 into a byte pointer.
If the byte pointer's target is alphanumeric, move the caret left to any non-alphanumeric byte in the text.
If the byte pointer's target is not alphanumeric, move the caret left to any non-symbolic byte in the text.
Move the caret left to any non-noise byte in the text.

To jump the caret right in a text:
If the text is nil, exit.
Move the caret right to any non-noise byte in the text.
Get a row given the text's caret row# and the text.
If the text's caret column# is the row's string's length, exit.
Put the row's string's first plus the text's caret column# minus 1 into a byte pointer.
If the byte pointer's target is alphanumeric, move the caret right to any non-alphanumeric byte in the text.
If the byte pointer's target is not alphanumeric, move the caret right to any non-symbolic byte in the text.
Move the caret right to any non-noise byte in the text.

The k-key is a key equal to 75.

A key is a number.

A kilobyte is 1024 units.

A kind is a string.

The l-key is a key equal to 76.

A l-param is a number.

A landscape sheet is a sheet.

The large letter height is a letter height.

The largest number is 2147483647.

An lcm is a number.

A left click is an input.

A left is some twips.

The left-alligator byte is a byte equal to 60.

The left-alligator-quote byte is a byte equal to 139.

The left-arrow key is a key equal to 37.

The left-brace byte is a byte equal to 123.

The left-bracket byte is a byte equal to 91.

The left-double-alligator-quote byte is a byte equal to 171.

The left-double-quote byte is a byte equal to 147.

The left-paren byte is a byte equal to 40.

The left-single-quote byte is a byte equal to 145.

The left-window key is a key equal to 91.

A length is some twips.

The less-than-sign byte is a byte equal to 60.

A letter height is some twips. \ indicates actual height of a typical uppercase letter

A letter is a byte.

The lexicon is a thing with an index.

The light blue color is a color.

The light blue pen is a pen.

A light color is a color.

The light cyan color is a color.

The light cyan pen is a pen.

The light gray color is a color.

The light gray pen is a pen.

The light green color is a color.

The light green pen is a pen.

The light lime color is a color.

The light lime pen is a pen.

The light magenta color is a color.

The light magenta pen is a pen.

The light orange color is a color.

The light orange pen is a pen.

The light purple color is a color.

The light purple pen is a pen.

The light red color is a color.

The light red pen is a pen.

The light sky blue color is a color.

The light sky blue pen is a pen.

The light sky color is a color.

The light sky pen is a pen.

The light teal color is a color.

The light teal pen is a pen.

The light violet color is a color.

The light violet pen is a pen.

The light yellow color is a color.

The light yellow pen is a pen.

To lighten a color by an amount:
Add the amount to the color's lightness.
Limit the color's lightness to 0 and 1000.

To lighten a color by some percent;
To lighten a color about some percent;
To lighten a color by about some percent;
To lighten a color some percent:
Put the color's lightness plus the percent into the color's lightness.
Limit the color's lightness to 0 and 1000.

To lighten the current color about some percent:
Lighten the context's color by the percent.

To lighten a hue by some degrees:
Add the degrees to the hue.

To lighten a hue by some points:
Convert the hue to some other points.

Add the points to the other points.
Convert the other points to the hue.

The lighter blue color is a color.

The lighter blue pen is a pen.

The lighter cyan color is a color.

The lighter cyan pen is a pen.

The lighter gray color is a color.

The lighter gray pen is a pen.

The lighter green color is a color.

The lighter green pen is a pen.

The lighter lime color is a color.

The lighter lime pen is a pen.

The lighter magenta color is a color.

The lighter magenta pen is a pen.

The lighter orange color is a color.

The lighter orange pen is a pen.

The lighter purple color is a color.

The lighter purple pen is a pen.

The lighter red color is a color.

The lighter red pen is a pen.

The lighter sky blue color is a color.

The lighter sky blue pen is a pen.

The lighter sky color is a color.

The lighter sky pen is a pen.

The lighter teal color is a color.

The lighter teal pen is a pen.

The lighter violet color is a color.

The lighter violet pen is a pen.

The lighter yellow color is a color.

The lighter yellow pen is a pen.

The lightest blue color is a color.

The lightest blue pen is a pen.

The lightest cyan color is a color.

The lightest cyan pen is a pen.

The lightest gray color is a color.

The lightest gray pen is a pen.

The lightest green color is a color.

The lightest green pen is a pen.

The lightest lime color is a color.

The lightest lime pen is a pen.

The lightest magenta color is a color.

The lightest magenta pen is a pen.

The lightest orange color is a color.

The lightest orange pen is a pen.

The lightest purple color is a color.

The lightest purple pen is a pen.

The lightest red color is a color.

The lightest red pen is a pen.

The lightest sky blue color is a color.

The lightest sky blue pen is a pen.

The lightest sky color is a color.

The lightest sky pen is a pen.

The lightest teal color is a color.

The lightest teal pen is a pen.

The lightest violet color is a color.

The lightest violet pen is a pen.

The lightest yellow color is a color.

The lightest yellow pen is a pen.

A lightness is a number [0 to 1000].

The lime color is a color.

The lime pen is a pen.

To limit a box to another box:
Limit the box's left to the other box's left and the other box's right.
Limit the box's top to the other box's top and the other box's bottom.
Limit the box's right to the other box's left and the other box's right.
Limit the box's bottom to the other box's top and the other box's bottom.

To limit the caret in a text:
If the text is nil, exit.
Limit the text's caret row# to 1 and the text's row count.
Get a row given the text's caret row# and the text.
Limit the text's caret column# to 1 and the row's string's length.

To limit a number to another number and a third number:
If the number is less than the other number, put the other number into the number; exit.
If the number is greater than the third number, put the third number into the number.

To limit the origin of a text:
If the text is nil, exit.
Limit the text's x to the smallest number and the text's margin.
Put the text's row count minus 1 times the text's row height into a number.
Limit the text's y to - the number and 0.

To limit a spot to a box:
If the spot's x is less than the box's left, put the box's left into the spot's x.
If the spot's y is less than the box's top, put the box's top into the spot's y.
If the spot's x is greater than the box's right, put the box's right into the spot's x.
If the spot's y is greater than the box's bottom, put the box's bottom into the spot's y.

To limit some texts to a count:
Put the texts' count into another count.
Loop.
If the other count is less than or equal to the count, exit.
Put the texts' first into a text.
Remove the text from the texts.
Destroy the text.
Subtract 1 from the other count.
Repeat.

A line has a start spot and an end spot.

The linefeed byte is a byte equal to 10.

To list some choices in a box;
To draw some choices in a box:

\Draw really fast. ***
Get a [first/next] choice from the choices.
If the choice is missing [because we've drawn them all], exit.
Put the box's left plus 1/4 inch into the choice's left.
Put the box's right minus 1/4 inch into the choice's right.
If the choice is the choices' first, put the box's top plus 1/4 inch into the choice's top.
If the choice is not the choices' first, put the choice's previous' bottom into the choice's top.
Put the choice's top plus 1/4 inch into the choice's bottom.
\Draw the choice's box with the purple color. \ temp ***
Stroke the choice's name in the choice's box with the context's color.
Repeat.

To list some choices in a box with a color;
To draw some choices in a box with a color:
Put the color into the context's color.
Draw the choices in the box.

The little-a byte is a byte equal to 97.

The little-a-acute byte is a byte equal to 225.

The little-a-circumflex byte is a byte equal to 226.

The little-a-diaeresis byte is a byte equal to 228.

The little-a-grave byte is a byte equal to 224.

The little-a-ring byte is a byte equal to 229.

The little-a-tilde byte is a byte equal to 227.

The little-ae byte is a byte equal to 230.

The little-b byte is a byte equal to 98.

The little-c byte is a byte equal to 99.

The little-c-cedilla byte is a byte equal to 231.

The little-d byte is a byte equal to 100.

The little-e byte is a byte equal to 101.

The little-e-acute byte is a byte equal to 233.

The little-e-circumflex byte is a byte equal to 234.

The little-e-diaeresis byte is a byte equal to 235.

The little-e-grave byte is a byte equal to 232.

The little-eth byte is a byte equal to 240.

The little-f byte is a byte equal to 102.

The little-f-hook byte is a byte equal to 131.

The little-g byte is a byte equal to 103.

The little-h byte is a byte equal to 104.

The little-i byte is a byte equal to 105.

The little-i-acute byte is a byte equal to 237.

The little-i-circumflex byte is a byte equal to 238.

The little-i-diaeresis byte is a byte equal to 239.

The little-i-grave byte is a byte equal to 236.

The little-j byte is a byte equal to 106.

The little-k byte is a byte equal to 107.

The little-l byte is a byte equal to 108.

The little-m byte is a byte equal to 109.

The little-n byte is a byte equal to 110.

The little-n-tilde byte is a byte equal to 241.

The little-o byte is a byte equal to 111.

The little-o-acute byte is a byte equal to 243.

The little-o-circumflex byte is a byte equal to 244.

The little-o-diaeresis byte is a byte equal to 246.

The little-o-grave byte is a byte equal to 242.

The little-o-stroke byte is a byte equal to 248.

The little-o-tilde byte is a byte equal to 245.

The little-oe byte is a byte equal to 156.

The little-p byte is a byte equal to 112.

The little-q byte is a byte equal to 113.

The little-r byte is a byte equal to 114.

The little-s byte is a byte equal to 115.

The little-s-caron byte is a byte equal to 154.

The little-t byte is a byte equal to 116.

The little-thorn byte is a byte equal to 254.

The little-tilde byte is a byte equal to 152.

The little-u byte is a byte equal to 117.

The little-u-acute byte is a byte equal to 250.

The little-u-circumflex byte is a byte equal to 251.

The little-u-diaeresis byte is a byte equal to 252.

The little-u-grave byte is a byte equal to 249.

The little-v byte is a byte equal to 118.

The little-w byte is a byte equal to 119.

The little-x byte is a byte equal to 120.

The little-y byte is a byte equal to 121.

The little-y-acute byte is a byte equal to 253.

The little-y-diaeresis byte is a byte equal to 255.

The little-z byte is a byte equal to 122.

The little-z-caron byte is a byte equal to 158.

To load the lexicon:
If the lexicon is not nil, exit.
Extract a directory from the module's path.
Loop.
If the directory is blank, exit.
Put the directory then "lexicon\" into a path.
If the path is in the file system, load the lexicon given the path; exit.
Extract the directory from the directory.
Repeat.

To load the lexicon given a buffer:
If the lexicon is nil, create the lexicon.
Slap a rider on the buffer.
Loop.
Move the rider (index lexicon rules).
If the rider's token is blank, exit.
Index the rider's token in the lexicon's index.
Repeat.

To load the lexicon given a path:
Get an item from the path.
If the item is not found, exit.
If the item's kind is not "file", repeat.
Read the item's path into a buffer.

If the i/o error is not blank, repeat.
Load the lexicon given the buffer.
Repeat.

To localize a box given a pair:
Privatize the pair.
Negate the pair.
Move the box given the pair.

To localize a spot given a pair:
Privatize the pair.
Negate the pair.
Move the spot given the pair.

To lock a gpbitmap given a bitmapdata (24-bit rgb):
Put the gpbitmap's gprect into a gprect.
Call "gdiplus.dll" "GdipBitmapLockBits" with the gpbitmap and the gprect's whereabouts and 3 [imagelockmoderead or imagelockmodewrite]
And 137224 [pixelformat24bpprgb] and the bitmapdata's whereabouts.

A logbrush has
A number called lbstyle,
A colorref called lbcolor,
A number called lbhatch.

The lower-double-quote byte is a byte equal to 132.

The lower-single-quote byte is a byte equal to 130.

To lowercase any selected bytes in a text:
If the text is nil, exit.
Loop.
Get a row from the text's rows.
If the row is nil, exit.
If the row of the text is not selected, repeat.
Slap a substring on any selected bytes in the row of the text.
Lowercase the substring.
Repeat.

To lowercase a byte:
Intel $8B8508000000. \ mov eax,[ebp+8] \ the byte
Intel $803841. \ cmp byte ptr [eax],'A'
Intel $0F820C000000. \ jb END
Intel $80385A. \ cmp byte ptr [eax],'Z'
Intel $0F8703000000. \ ja END
Intel $800020. \ add byte ptr [eax],$20
\ END

To lowercase the character under a finger and put it into a string:
If the finger is nil, exit.
Put the finger's target into the string.
Lowercase the string.

To lowercase a string:
Slap a substring on the string.

Loop.
If the substring is blank, exit.
Lowercase the substring's first's target.
Add 1 to the substring's first.
Repeat.

To lowercase a text:
If the text is nil, exit.
Loop.
Get a row from the text's rows.
If the row is nil, break.
Lowercase the row's string.
Repeat.
Wrap the text.

The m-key is a key equal to 77.

The magenta color is a color.

The magenta pen is a pen.

The main window is a window.

To make a fraction with a number and another number:
Put the number into the fraction's numerator.
Put the other number into the fraction's denominator.

To make a ratio pair given a box and another box;
To make a fraction pair given a box and another box:
Put the box's x-extent into the fraction pair's fraction's numerator.
Put the other box's x-extent into the fraction pair's fraction's denominator.
Put the box's y-extent into the fraction pair's other fraction's numerator.
Put the other box's y-extent into the fraction pair's other fraction's denominator.

A margin is a number.

The masculine byte is a byte equal to 186.

To mask inside a box:
Create an hrgn given the box.
Mask inside the hrgn.
Destroy the hrgn.

To mask inside an ellipse:
Create an hrgn given the ellipse.
Mask inside the hrgn.
Destroy the hrgn.

To mask inside an hrgn:
Call "gdi32.dll" "ExtSelectClipRgn" with the current canvas and the hrgn and 4 [rgn_diff].

To mask inside a polygon:
Create an hrgn given the polygon.
Mask inside the hrgn.
Destroy the hrgn.

To mask inside a roundy box:
Create an hrgn given the roundy box.
Mask inside the hrgn.
Destroy the hrgn.

A mask is a hex string.

To mask only inside a box:
Unmask everything.
Mask inside the box.

To mask only inside an ellipse:
Unmask everything.
Mask inside the ellipse.

To mask only inside a polygon:
Unmask everything.
Mask inside the polygon.

To mask only inside a roundy box:
Unmask everything.
Mask inside the roundy box.

To mask only outside a box:
Unmask everything.
Mask outside the box.

To mask only outside an ellipse:
Unmask everything.
Mask outside the ellipse.

To mask only outside a polygon:
Unmask everything.
Mask outside the polygon.

To mask only outside a roundy box:
Unmask everything.
Mask outside the roundy box.

To mask outside a box:
Create an hrgn given the box.
Mask outside the hrgn.
Destroy the hrgn.

To mask outside an ellipse:
Create an hrgn given the ellipse.
Mask outside the hrgn.
Destroy the hrgn.

To mask outside an hrgn:
Call "gdi32.dll" "ExtSelectClipRgn" with the current canvas and the hrgn and 1 [rgn_and].

To mask outside a polygon:
Create an hrgn given the polygon.

Mask outside the hrgn.
Destroy the hrgn.

To mask outside a roundy box:
Create an hrgn given the roundy box.
Mask outside the hrgn.
Destroy the hrgn.

The max text undos is a count equal to 32.

The medium letter height is a letter height.

A megabyte is 1024 kilobytes.

The memory canvas is a canvas.

The menu key is a key equal to 93.

A message is a string.

The micro byte is a byte equal to 181.

A million is 1000 thousands.

A millisecond is a number.

To minimize a window:
Call "user32.dll" "ShowWindow" with the window and 6 [sw_minimize].

The minus-sign byte is a byte equal to 45.

A minute is 60 seconds.

To mirror the gpbitmap in a picture:
If the picture is nil, exit.
Reverse the picture's mirror flag.
Mirror the picture's gpbitmap.

To mirror a gpimage:
Call "gdiplus.dll" "GdipImageRotateFlip" with the gpimage and 4 [rotatenoneflipx].

To mirror a picture:
If the picture is nil, exit.
Put the picture's box's center's x minus the picture's uncropped box's center's x into a pair's x.
Multiply the pair's x by 2.
Move the picture's uncropped box given the pair.
Mirror the gpbitmap in the picture.

To mirror a polygon:
If the polygon is nil, exit.
Put the polygon's box into a box.
Loop.
Get a vertex from the polygon's vertices.
If the vertex is nil, exit.
Subtract the box's left from the vertex's x.

Put the box's right minus the vertex's x into the vertex's x.
Repeat.

A mixed has a whole number and a ratio and a fraction at the ratio.

The module has
A handle,
A path,
A directory,
A root directory, \ one up from the directory that exe is run from
A file name w/o extension called name.

The mouse has
A key called left button,
A key called right button.

To move the anchor left to any non-alphanumeric byte in a text:
If the text is nil, exit.
Get a row given the text's anchor row# and the text.
Loop.
If the text's anchor column# is 1, exit.
Put the row's string's first plus the text's anchor column# minus 2 into a byte pointer.
If the byte pointer's target is not alphanumeric, exit.
Subtract 1 from the text's anchor column#.
Repeat.

To move back some twips:
Turn around.
Move the twips.
Turn around.

To move a box to the bottom of another box:
Move the box down the other box's bottom minus the box's bottom.

To move a box close to the left side of another box;
To move a box close to the left of another box:
Move the box to the left of the other box.
Pick a length between 0 and the box's width times 3/4.
Move the box right the length.

To move a box down some twips:
Move the box given 0 and the twips.

To move a box given a pair:
Move the box given the pair's x and the pair's y.

To move a box given a ratio pair and a spot;
To move a box given a fraction pair and a spot:
Get a difference between the box's left-top and the spot.
Put the difference into another difference.
Scale the other difference given the fraction pair.
Subtract the difference from the other difference.
Move the box given the other difference.

To move a box given some x twips and some y twips:

Add the x twips to the box's left.
Add the y twips to the box's top.
Add the x twips to the box's right.
Add the y twips to the box's bottom.

To move a box left to a coord:
Move the box left the box's left minus the coord.

To move a box to the left side of another box;
To move a box to the left of another box:
Move the box left the box's left minus the other box's left.

To move a box left some twips:
Move the box given - the twips and 0.

To move a box to the right side of another box;
To move a box to the right of another box:
Move the box right the other box's right minus the box's right.

To move a box right some twips:
Move the box given the twips and 0.

To move a box to a spot:
Get a difference between the spot and the box's left-top.
Move the box given the difference.

To move a box to the top left corner of another box:
Move the box to the other box's left-top.

To move a box to the top of another box:
Move the box up the box's top minus the other box's top.

To move a box up some twips:
Move the box given 0 and - the twips.

To move the caret down in a text:
If the text is nil, exit.
Add 1 to the text's caret row#.
Limit the caret in the text.

To move the caret down one page in a text:
If the text is nil, exit.
Add the text's rows/box to the text's caret row#.
Subtract 1 from the text's caret row#.
Limit the caret in the text.

To move the caret to the first byte of the current row of a text:
If the text is nil, exit.
Put 1 into the text's caret column#.

To move the caret to the first byte of a text:
If the text is nil, exit.
Put 1 and 1 into the text's caret.

To move the caret to the last byte of the current row of a text:

```
If the text is nil, exit.
Get a row given the text's caret row# and the text.
Put the row's string's length into the text's caret column#.

To move the caret to the last byte of a text:
If the text is nil, exit.
Put the text's row count into the text's caret row#.
Get a row given the text's caret row# and the text.
Put the row's string's length into the text's caret column#.

To move the caret left to any non-alphanumeric byte in a text:
If the text is nil, exit.
Get a row given the text's caret row# and the text.
Loop.
If the text's caret column# is 1, exit.
Put the row's string's first plus the text's caret column# minus 2 into a byte pointer.
If the byte pointer's target is not alphanumeric, exit.
Subtract 1 from the text's caret column#.
Repeat.

To move the caret left to any non-noise byte in a text:
If the text is nil, exit.
Get a row given the text's caret row# and the text.
Loop.
If the text's caret column# is 1, exit.
Put the row's string's first plus the text's caret column# minus 2 into a byte pointer.
If the byte pointer's target is not noise, exit.
Subtract 1 from the text's caret column#.
Repeat.

To move the caret left to any non-symbolic byte in a text:
If the text is nil, exit.
Get a row given the text's caret row# and the text.
Loop.
If the text's caret column# is 1, exit.
Put the row's string's first plus the text's caret column# minus 2 into a byte pointer.
If the byte pointer's target is not symbolic, exit.
Subtract 1 from the text's caret column#.
Repeat.

To move the caret left in a text:
If the text is nil, exit.
Subtract 1 from the text's caret column#.
Limit the caret in the text.

To move the caret right to any non-alphanumeric byte in a text:
If the text is nil, exit.
Get a row given the text's caret row# and the text.
Loop.
If the text's caret column# is the row's string's length, exit.
Put the row's string's first plus the text's caret column# minus 1 into a byte pointer.
If the byte pointer's target is not alphanumeric, exit.
Add 1 to the text's caret column#.
Repeat.
```

```
To move the caret right to any non-noise byte in a text:
If the text is nil, exit.
Get a row given the text's caret row# and the text.
Loop.
If the text's caret column# is the row's string's length, exit.
Put the row's string's first plus the text's caret column# minus 1 into a byte pointer.
If the byte pointer's target is not noise, exit.
Add 1 to the text's caret column#.
Repeat.

To move the caret right to any non-symbolic byte in a text:
If the text is nil, exit.
Get a row given the text's caret row# and the text.
Loop.
If the text's caret column# is the row's string's length, exit.
Put the row's string's first plus the text's caret column# minus 1 into a byte pointer.
If the byte pointer's target is not symbolic, exit.
Add 1 to the text's caret column#.
Repeat.

To move the caret right in a text:
If the text is nil, exit.
Add 1 to the text's caret column#.
Limit the caret in the text.

To move the caret up in a text:
If the text is nil, exit.
Subtract 1 from the text's caret row#.
Limit the caret in the text.

To move the caret up one page in a text:
If the text is nil, exit.
Subtract the text's rows/box from the text's caret row#.
Add 1 to the text's caret row#.
Limit the caret in the text.

To move an ellipse down some twips:
Move the ellipse given 0 and the twips.

To move an ellipse given a pair:
Move the ellipse given the pair's x and the pair's y.

To move an ellipse given some x twips and some y twips:
Move the ellipse's box given the x twips and the y twips.

To move an ellipse left some twips:
Move the ellipse given - the twips and 0.

To move an ellipse right some twips:
Move the ellipse given the twips and 0.

To move an ellipse to a spot:
Get a difference between the spot and the ellipse's left-top.
Move the ellipse given the difference.
```

To move an ellipse up some twips:
Move the ellipse given 0 and - the twips.

To move a finger over a number:
If the finger is nil, exit.
Add 1 to the finger.

To move to the left some twips and down some other twips;
To move left some twips and down some other twips:
To move some twips to the left and some other twips down;
To move some twips left and some other twips down:
Move the context's spot left the twips.
Move the context's spot down the other twips.

To move to the left some twips and up some other twips;
To move left some twips and up some other twips;
To move some twips to the left and some other twips up;
To move some twips left and some other twips up:
Move the context's spot left the twips.
Move the context's spot up the other twips.

To move a line down some twips:
Move the line given 0 and the twips.

To move a line given a pair:
Move the line given the pair's x and the pair's y.

To move a line given some x twips and some y twips:
Add the x twips to the line's start's x.
Add the y twips to the line's start's y.
Add the x twips to the line's end's x.
Add the y twips to the line's end's y.

To move a line left some twips:
Move the line given - the twips and 0.

To move a line to a spot:
Get a difference between the spot and the line's start.
Move the line given the difference.

To move a line some twips to the right;
To move a line right some twips:
Move the line given the twips and 0.

To move a line up some twips:
Move the line given 0 and - the twips.

To move to the middle;
To start in the middle;
To move to the center;
To start in the center:
Put the context's box's center into the context's spot.

To move to the middle of a box;
To start in the middle of a box;

```
To move to the center of a box;
To start in the center of a box:
Put the box's center into the context's spot.

To move a picture down some twips:
Move the picture given 0 and the twips.

To move a picture given a pair:
Move the picture given the pair's x and the pair's y.

To move a picture given some x twips and some y twips:
If the picture is nil, exit.
Move the picture's box given the x twips and the y twips.
Move the picture's uncropped box given the x twips and the y twips.

To move a picture left some twips:
Move the picture given - the twips and 0.

To move a picture right some twips:
Move the picture given the twips and 0.

To move a picture to a spot:
If the picture is nil, exit.
Get a difference between the spot and the picture's left-top.
Move the picture given the difference.

To move a picture up some twips:
Move the picture given 0 and - the twips.

To move a polygon down some twips:
Move the polygon given 0 and the twips.

To move a polygon given a pair:
Move the polygon given the pair's x and the pair's y.

To move a polygon given some x twips and some y twips:
If the polygon is nil, exit.
Loop.
Get a vertex from the polygon's vertices.
If the vertex is nil, exit.
Move the vertex given the x twips and the y twips.
Repeat.

To move a polygon left some twips:
Move the polygon given - the twips and 0.

To move a polygon left some twips and up down other twips:
Move the polygon left the twips.
Move the polygon down the other twips.

To move a polygon left some twips and up some other twips:
Move the polygon left the twips.
Move the polygon up the other twips.

To move a polygon right some twips:
```

```
Move the polygon given the twips and 0.

To move a polygon right some twips and up down other twips:
Move the polygon right the twips.
Move the polygon down the other twips.

To move a polygon right some twips and up some other twips:
Move the polygon right the twips.
Move the polygon up the other twips.

To move a polygon to a spot:
If the polygon is nil, exit.
Get a difference between the spot and the polygon's box's left-top.
Move the polygon given the difference.

To move a polygon up some twips:
Move the polygon given 0 and - the twips.

To move a rider (index lexicon rules):
Skip any leading noise in the rider's source.
Position the rider's token on the rider's source.
Loop.
If the rider's source is blank, exit.
Bump the rider.
If the rider's source's first's target is noise, exit.
Repeat.

To move a rider (quoted string rules):
Bump the rider.
If the rider's source is blank, exit.
If the rider's source's first's target is not the double-quote byte, repeat.
If the rider's source's first is the rider's source's last, bump the rider; exit.
Bump the rider.
If the rider's source's first's target is not the double-quote byte, exit.
Repeat.

To move a rider (spell checking rules):
Skip any non-alphanumeric bytes in the rider's source.
Position the rider's token on the rider's source.
Loop.
If the rider's source is blank, exit.
Bump the rider.
If the rider's source is on any contraction, bump the rider; repeat.
If the rider's source's first's target is not alphanumeric, exit.
Repeat.

To move a rider (text file rules):
Position the rider's token on the rider's source.
Loop.
If the rider's source is blank, exit.
If the rider's source's first's target is the return byte, bump the rider; break.
If the rider's source's first's target is the linefeed byte, bump the rider; exit. \ *dahn new to handle lines terminated by just linefeed
Bump the rider.
Repeat.
```

If the rider's source is blank, exit.
If the rider's source's first's target is the linefeed byte, add 1 to the rider's source's first.

To move a rider (word wrapping rules):
Position the rider's token on the rider's source.
If the rider's source is blank, exit.
If the rider's source's first's target is the return byte, bump the rider; exit.
Loop.
If the rider's source is blank, exit.
If the rider's source's first's target is the return byte, exit.
If the rider's token is blank, bump the rider; repeat.
If the rider's source's first's target is like the rider's token's last's target, bump the rider; repeat.

To move a rider given a box (word wrapping rules):
Skip any leading linefeed byte in the rider's source.
Position the rider's token on the rider's source.
If the rider's source is blank, exit.
Slap another rider on the rider.
Loop.
If the rider's source is blank, exit.
Move the other rider (word wrapping rules).
If the other rider's token is blank, exit.
If the other rider's token's first's target is the return byte, bump the rider; exit.
If the other rider's token's first's target is whitespace, bump the rider by the other rider's token's length; repeat.
If the rider's token is blank, bump the rider by the other rider's token's length; repeat.
If the rider's token then the other rider's token is wider than the box, exit.
Bump the rider by the other rider's token's length.
Repeat.

To move a rider given a separator byte:
Position the rider's token on the rider's source.
Loop.
If the rider's source is blank, exit.
If the rider's source's first's target is the separator byte, add 1 to the rider's source's first; exit.
Bump the rider.
Repeat.

To move to the right some twips and down some other twips;
To move right some twips and down some other twips;
To move some twips to the right and some other twips down;
To move some twips right and some other twips down:
Move the context's spot right the twips.
Move the context's spot down the other twips.

To move to the right some twips and up some other twips;
To move right some twips and up some other twips;
To move some twips to the right and some other twips up;
To move some twips right and some other twips up:
Move the context's spot right the twips.
Move the context's spot up the other twips.

To move to a spot:
Put the spot into the context's spot.

To move a spot about some twips in any direction:
Pick another spot within the twips of the spot.
Put the other spot into the spot.

To move a spot to another spot:
Put the other spot into the spot.

To move a spot given a pair:
Move the spot given the pair's x and the pair's y.

To move a spot given some x twips and some y twips:
Add the x twips to the spot's x.
Add the y twips to the spot's y.

To move a spot some twips down;
To move a spot down some twips:
Move the spot given 0 and the twips.

To move a spot some twips to the left;
To move a spot some twips left;
To move a spot left some twips:
Move the spot given - the twips and 0.

To move a spot some twips right;
To move a spot some twips to the right;
To move a spot right some twips:
Move the spot given the twips and 0.

To move a spot some twips to the right and some other twips down;
To move a spot some twips right and some other twips down:
Add the twips to the spot's x.
Add the other twips to the spot's y.

To move a spot some twips up;
To move a spot up some twips:
Move the spot given 0 and - the twips.

To move some squares:
Move the square size times the squares divided by 1 square. \ squares are scaled up for precision hence the division at the end

To move some squares diagonally;
To move some squares slantways:
Move the square size times the squares times the squirt o' two divided by 1 square. \ squares are scaled up for precision hence the division at the end

To move a substring given a number:
Add the number to the substring's first.
Add the number to the substring's last.

To move a text down some twips:
Move the text given 0 and the twips.

To move a text given a pair:
Move the text given the pair's x and the pair's y.

```
To move a text given some x twips and some y twips:
If the text is nil, exit.
Move the text's box given the x twips and the y twips.

To move a text left some twips:
Move the text given - the twips and 0.

To move a text right some twips:
Move the text given the twips and 0.

To move a text to a spot:
If the text is nil, exit.
Get a difference between the spot and the text's left-top.
Move the text given the difference.

To move a text up some twips:
Move the text given 0 and - the twips.

To move a thing from some things to some other things:
If the thing is nil, exit.
Privatize the thing.
Remove the thing from the things.
Append the thing to the other things.

To move some things to some other things:
Put the things' first into the other things' first.
Put the things' last into the other things' last.
Clear the things.

To move some twips:
\Wait for the delay. ***
Put the context's spot into a line's start.
Put the context's spot into the line's end.
Get a rise and a run given the context's heading.
Add the run times the twips divided by 10000 to the line's end's x.
Add the rise times the twips divided by 10000 to the line's end's y.
Put the line's end into the context's spot.

To move some twips down;
To move down some twips:
Add the twips to the context's y.

To move some twips to the left;
To move some twips left;
To move left some twips:
Subtract the twips from the context's x.

To move some twips to the right;
To move some twips right;
To move right some twips:
Add the twips to the context's x.

To move some twips up;
To move up some twips:
```

Subtract the twips from the context's y.

To move a vertex down some twips:
Move the vertex given 0 and the twips.

To move a vertex given a pair:
Move the vertex given the pair's x and the pair's y.

To move a vertex given some x twips and some y twips:
If the vertex is nil, exit.
Add the x twips to the vertex's x.
Add the y twips to the vertex's y.

To move a vertex left some twips:
Move the vertex given - the twips and 0.

To move a vertex right some twips:
Move the vertex given the twips and 0.

To move a vertex to a spot:
If the vertex is nil, exit.
Put the spot into the vertex's spot.

To move a vertex up some twips:
Move the vertex given 0 and - the twips.

To move a window left:
Call "user32.dll" "GetWindowRect" with the main window and a box's whereabouts.
Subtract the screen's pixel width from the box's left.
Call "user32.dll" "MoveWindow" with the window and the box's left and the box's top and the screen's pixel width and the screen's pixel height and 1.

To move a window right:
Call "user32.dll" "GetWindowRect" with the main window and a box's whereabouts.
Add the screen's pixel width to the box's left.
Call "user32.dll" "MoveWindow" with the window and the box's left and the box's top and the screen's pixel width and the screen's pixel height and 1.

A ms is 1 millisecond.

A msg is a record with
A window called hwnd,
A number called message,
A w-param called wparam,
A l-param called lparam,
A number called time,
A spot called pt.

A multiple is a number.

The multiplication-symbol byte is a byte equal to 215.

To multiply a fraction by a number:
Multiply the fraction's numerator by the number.
Reduce the fraction.

To multiply a number by a fraction;
To scale a number given a ratio;
To scale a number given a fraction:
If the fraction’s denominator is 0, exit.
Call "kernel32.dll" "MulDiv" with the number and the fraction’s numerator and the fraction’s denominator returning the number.

To multiply a pair by another pair:
Multiply the pair’s x by the other pair’s x.
Multiply the pair’s y by the other pair’s y.

To multiply a pair by a number:
Multiply the pair’s x by the number.
Multiply the pair’s y by the number.

To multiply a pair by a number and another number:
Multiply the pair’s x by the number.
Multiply the pair’s y by the other number.

To multiply a pointer by a number;
To multiply a number by another number:
Intel $8B8508000000. \ mov eax,[ebp+8] \ the number
Intel $8B00. \ mov eax,[eax]
Intel $8B9D0C000000. \ mov ebx,[ebp+12] \ the other number
Intel $F72B.\ mul [ebx] \ means mul eax,[ebx] but is weird form
Intel $8B9D08000000. \ mov ebx,[ebp+8] \ the number
Intel $8903. \ mov [ebx],eax

The n-key is a key equal to 78.

A name is a string.

To negate a fraction:
Negate the fraction’s numerator.

To negate a number:
Intel $8B8508000000. \ mov eax,[ebp+8] \ the number
Intel $F718.\ neg [eax]

To negate a pair:
Negate the pair’s x.
Negate the pair’s y.

The negative-acknowledge byte is a byte equal to 21.

The next letter is a letter equal to 65 [the big-a byte].

A nibble is a byte. \ only low order 4 bits are valid

A nibble string is a string. \ $0010A0...

The nine byte is a byte equal to 57.

The nine key is a key equal to 57.

The non-breaking-space byte is a byte equal to 160.

To non-destructively resize a picture given a ratio pair;
To non-destructively resize a picture given a fraction pair:
If the picture is nil, exit.
Move the picture's uncropped box given the fraction pair and the picture's box's left-top.
Resize the picture's uncropped box given the fraction pair.
Resize the picture's box given the fraction pair.
If the picture's right is less than the picture's left, mirror the gpbitmap in the picture.
If the picture's bottom is less than the picture's top, flip the gpbitmap in the picture.

To non-destructively resize a picture given a twip pair:
If the picture is nil, exit.
Put the picture's box into a box.
Resize the box given the twip pair.
Make a fraction pair given the box and the picture's box.
Non-destructively resize the picture given the fraction pair.

To normalize an angle: divide the angle by 3600 giving a quotient and the angle.

To normalize a box:
If the box's right is less than the box's left, swap the box's left with the box's right.
If the box's bottom is less than the box's top, swap the box's top with the box's bottom.

To normalize a canvas:
Call "gdi32.dll" "SetGraphicsMode" with the canvas and 2 [gm_advanced].
Call "gdi32.dll" "SetBkMode" with the canvas and 1 [transparent].
Call "gdi32.dll" "SetMapMode" with the canvas and 8 [mm_anisotropic].
Call "gdi32.dll" "SetViewportOrgEx" with the canvas and 0 and 0 and nil.
Call "gdi32.dll" "SetViewportExtEx" with the canvas and the ppi and the ppi and nil.
Call "gdi32.dll" "SetWindowOrgEx" with the canvas and 0 and 0 and nil.
Call "gdi32.dll" "SetWindowExtEx" with the canvas and the tpi and the tpi and nil.

To normalize an ellipse:
Normalize the ellipse's box.

To normalize a fraction and another fraction:
Get a lcm given the fraction's denominator and the other fraction's denominator.
Multiply the fraction's numerator by the lcm divided by the fraction's denominator.
Put the lcm into the fraction's denominator.
Multiply the other fraction's numerator by the lcm divided by the other fraction's denominator.
Put the lcm into the other fraction's denominator.

To normalize a heading:
Divide the heading by 3840 giving a quotient and a remainder.
Put the remainder into the heading.
If the heading is less than 0, add 3840 to the heading.

To normalize a horizontal line:
If the line's end is less than the line's start, swap the line's end with the line's start.

To normalize a hue:
Divide the hue by 3600 giving a quotient and a remainder.
Put the remainder into the hue.

If the hue is less than 0, add 3600 to the hue.

To normalize a picture:
If the picture is nil, exit.
Normalize the picture's box.
Normalize the picture's uncropped box.

To normalize a selection:
If the selection's anchor row# is less than the selection's caret row#, exit.
If the selection's anchor row# is greater than the selection's caret row#, swap the selection's anchor with the selection's caret; exit.
If the selection's anchor column# is greater than the selection's caret column#, swap the selection's anchor column# with the selection's caret column#.

To normalize a text:
If the text is nil, exit.
Normalize the text's box.

To normalize a vertical line:
If the line's end is less than the line's start, swap the line's end with the line's start.

The not byte is a byte equal to 172.

The null byte is a byte equal to 0.

The null hbrush is an hbrush.

The null hfont is an hfont.

The null hpen is an hpen.

To null terminate a string:
Put the string's length into a saved length.
Reassign the string's first given the saved length plus 1.
Put the string's first plus the saved length minus 1 into the string's last.
Put the string's last plus 1 into a byte pointer.
Put the null byte into the byte pointer's target.

To null terminate a wide string:
Put the wide string's length into a saved length.
Reassign the wide string's first given the saved length plus 2.
Put the wide string's first plus the saved length minus 1 into the wide string's last.

A number has
A first byte,
A second byte,
A third byte,
A fourth byte,
A low wyrd at the first byte,
A high wyrd at the third byte.

A number pointer is a pointer to a number.

The number-sign byte is a byte equal to 35.

The numlock key is a key equal to 144.

The numpad-astrisk key is a key equal to 106.

The numpad-cross key is a key equal to 107.

The numpad-dash key is a key equal to 109.

The numpad-eight key is a key equal to 104.

The numpad-five key is a key equal to 101.

The numpad-four key is a key equal to 100.

The numpad-nine key is a key equal to 105.

The numpad-one key is a key equal to 97.

The numpad-period key is a key equal to 110.

The numpad-seven key is a key equal to 103.

The numpad-six key is a key equal to 102.

The numpad-slash key is a key equal to 111.

The numpad-three key is a key equal to 99.

The numpad-two key is a key equal to 98.

The numpad-zero key is a key equal to 96.

The o-key is a key equal to 79.

An offset is a number.

The one byte is a byte equal to 49.

The one key is a key equal to 49.

The one-half byte is a byte equal to 189.

The one-quarter byte is a byte equal to 188.

To open a file given a path:
Clear the i/o error.
Extract a directory from the path.
If the directory is not in the file system, put "Directory '" then the directory then "' doesn't exist." into the i/o error; exit.
Set the path to read-write mode.
Privatize the path.
Null terminate the path.
Call "kernel32.dll" "CreateFileA" with the path's first and -1073741824 [generic_read+generic_write] and 0 and 0 and 4 [open_always]
And -2147483520 [file_flag_write_through or file_attribute_normal] and 0 returning the file.

If the file is -1 [invalid_handle_value], put "Error opening file '" then the path then "'." into the i/o error; exit.

To open a file given a path and some milliseconds:
Start a timer.
Loop.
Open the file given the path.
If the i/o error is blank, exit.
If the timer's ticks are greater than the milliseconds, exit.
Repeat.

An operation is a string.

The orange color is a color.

The orange pen is a pen.

An origin is a spot.

The osmosian font resource is a font resource.

The osmosian font source is a hex string equal to
$00010000000B0080000300304F532F32BB41B2760000013800000056636D6170E2B9EDE900000544
0000032867617370FFFF000300008BDC00000008676C79664268E45D00000A4800007A6868656164E
4394351000000BC0000003668686561 0D54057A000000F400000024686D747868C6405100000190000
003B46C6F6361A327C3220000086C000001DC6D6178700559021500000118000000206E616D659DA
64946000084B0000004FA706F73745544E6B3000089AC000002300001000000001000091EDF0B35F0F
3CF5000B080000000000BF91DAB800000000C0031E39FFBAFE4805CC071A0000000900010000000
000000010000073EFE4E0043063D0005000105CC000100000000000000000000000000000ED000100
0000ED00DF000701040004000200100 02F00420000040C0000000200010001 02FB019000050008059A
05330000011B059A0533000003D1006602120000020000000000000000A00002AF500078FB000000
0000000000484C20200040002025CA05D3FE510133073E01B26000019FDFF70000000003E8007F00C
8000000C8000001900000025800CE02BC007804B0002F03B6004603E8005D04B00050019000780284
006402840064046D00220467004001900057 03E8006401900064032000 6B03E8003603E8006B03E800
3503E8003203E8002803E8003C03E8006403E8006C03E8007203E8005A01D400640190005604B0002
E04B0005704A9002104B000C804B00064043F002803E8003B0460003C03AC00050460000E03C00064
046700210474001 1039E0006039F000D03D4004B03CD004A047B0022044C003504B0005603E800640
514004904030042 03B600350453000D038C00360377002805350014 03DB001B03690028041D0021025
800490320002202580049038400420 4A9004A0258004303E8002F03EF005A0320002803D4002803630
02F0334002103E1004303A5005001980064021C000D033B00190161005405350064038A00500495004
30341002803D7004302AF0038033B002102A1001E037700360320001404170036034800 1B031300140
409001B025800420190004902580 04E03E8005D043F0028043F002804600036046000 0E044C003504B
00056038C003603E8002203E8003203E8002F03E8002F03E8002F03E8002F0320002803630 02
02F0363002F0363002F0190FFBA0190FFD90190FFD90190FFBE038A00200495003604950043049500
430495004304950043037700360377003603770036037700360377003603770036034F00420258004903200 07E046D00210
438007802580063049C0078041D008505CB005D0517004904F3006A02AE006B02E4005E04EC004305
1400560495005E0564005D048F006404590042048F0042042B003603AC006403B3006B048F005004B0
006403E800720319008B0320006A051E001A046C007E04B0005004B000C80190007104600042044C0
07203E8006404B0005604B0005104B0007804B0006B02FF0042043F0036043F003D04B00056051A00
67042B002F04670056063D008C0320005D02C3000701ED007802010078046700500453006B03130014
0369002804B0005D0438005602BC006402BC006403E5007F0190002F01F400560320005D053C00690
43F00280460000E043F00280460000E0460000E039E002F039E001A039E0006039E002E04B0005604
B0005D04B0005D038C0036038C0036038C00360190007803200057 03E8006502BC007801F4004902C
3004903B60035033B0021041D00210409001B025800DC0453004A041D006403690028031300 1403A50
050038B005704380057039F005002580071032000A0032000AE04B0006A0507006B0578007F03E8007

904B0004A0000000200010000000000140003000100000011A000001060000010000000000000000102000000002000000000000000000000000000000000100000304050607080900A0B0C0D0E0F10111213141516
1718191A1B1C1D1E1F202122232425262728292A2B2C2D2E2F303132333435363738393A3B3C3D3E
3F404142434445464748494A4B4C4D4E4F505152535455565758595A5B5C5D5E5F606100626364656
66768696A6B6C6D6E6F707172737475767778797A7B7C7D7E7F808182838485868788898A8B8C8D8
E8F909192939495969798999A009B9C9D9E9FA0A1A2A3A4A5A6A7A8A9AA03ABACADAEAFB0B1B2
B3B4B5B6B7B8B9BABBBCBD0000BEBFC0C1C2C3C4C5C6C7C8C9CACBCCCD00CECFD0D1D2D3
D4000000D5D60000D70004020E0000004800400005000800 7E00FF013101530161017801 7E019202C
702DA02DC037E2014201A201E202220262030203A204420AC212221262202220622 0F22122215221A
221E222B22482260226525CAFFFF0000002000A00131015201600178017D019202C602DA02DC037E
20132018201C2020202620302039204420AC212221262202220622 0F221122152219221E222B224822
60226425CAFFFFFFE30000FFA1FF5CFF78FF41FF5DFF130000FDFBFDF8FCA0E09D000000000000
E084E092E083E076E00FDF6ADF78DE96DEA1DE8B0000DEA50000DE74DE70DE5EDE2FDE30DAE
D000100000046000000000000000000000000F8000000000000000000F200F600FA00000000000000
0000000000000000000000000000EA000000EA0000000000000000000000000300A2008400850 0
0EB009600DC0086008E008B009C00A800A30010008A00EC0083009300E600E7008D0097008800BF0
0D600E5009D00A900E900E800EA00A100AB00C500C300AC00620063009000 6400C7006500C400C6
00CB00C800C900CA00DD006600CE00CC00CD00AD006700E4009100D100CF00D0006800DF00E100
89006A0069006B006D006C006E009F006F00710070007200730075007400 76007700DE0078007A0079
007B007D007C00B600A0007F007E0080008100E000E200B800D300D700B400B500C000B200B300C1
008200BE0087009900E300BF00A40000004E004E004E004E007E00AC011A01A0021C028202A202E0
031C037803AE03D203F2040E043A04B404EA054805A8060C068E06FA073807C60844087608AE08E8
0928096209C00A5C0ADC0B600BBE0C380CAC0CFC0D8E0DFE0E380E7C0ED60F1A0F9C0FFE1130
11941232134E13B013EC1446149E152C159815DA162C165A168616B416E8170617241794180E186A1
8D0194619A41A461AB21AE41B281B8C1BB41C2E1C741CE41D401DB21DEA1E3E1E881ECE1F201F
8C1FD8201E207820CC20EC213E217821842190219C21A821B421C021CC21D821E421F021FC22082
2142220222C223822442250225C226822762282228E229A22A622B222BE22CA22D622E222EE22FA2
33A237023B6244824D024F0255425C2265C26DC273C275C278627E2284428EA295E29B029FA2A442
A9A2AE22B362B8A2BC82C102C502C942CFA2D782DF82E562E842EA82EF22F5C2FC43010306430B
830E430F030FC3108318631FA32123228327032B832DC32FE3336339833A433B033F63456348234AE
34FA3518353A3582362A36363642364E365A36663672367E368A369636A236AE36BA36C636D236DE
3702372A3762379637C037EA37F63802380E381A384A38B439343940394C399A39F63A1C3A643A94
3AD23B183B943C103CA83D163D340002007F0000036B0560001E00340000133317331705333215111
4071507111615062B012627222735342711271136171117111 61D011617143B01113735363D01272325
27BF2C2C3420011C24C00C0810193BE02AE67C140C080878080C1EF21C98080C2828FEC42005600
8080454FEE0AD93202CFE3C15175C0B0544CCAD9301142C01804088FED42CFEE093AD8C0804080
1D82C1493ADF0040408000200CEFF8F017105FB0011001C0000011615110715171506232 63D012735
37113613161D010623222734373401314010080838400810082940083 83B191405FB0732FE367DA172
E5390732E5728F7D01DC39FA74083860405C10284C0000000002007803060230058200 0D001B00001
33217140711062326351127343705321707110623263511263534 37D03F09140838400C44012C3F090
C08384014440582601A16FE5440083801B4302F25046030FE704008380198161A2F250002002FFFFA
048F04BA00470050000001161507141721273536331 61D011733253217142307231 51F013215062B012
22715171506232352227262316 1D010623263D013427232635363B013526352326353 63B0126353413
1733321735262735014F40041C010404083840042401 0C270D74CC1C20BA580838201C86200838440
818714F2008384028E8400838C830D8400838B418BC38103C800C0C04BA0D27407B6D30944008389
43014404410 10DC0F40400314AF204064E31CE32960400838604BCD0838400486220838 40767A6CF
E24B418047F2D1C00000000030046FFE6034605600047005200 5C000001161D0107153633161514270
6071536333437161D01140706072207060F011617060726372227070 60F012635343F0136352627060F
0222273536353637363337363735363334033637343337350607061505173337323732372207 01E24008
40287589272D537D1C40240C140E161622501403165938280903444162184093801C0C241D6073183
B19182C2847192040280A1AF488380C041C406401402404480B0906122D5705600838202C38282338
390C2408D0240701083840153B0F3D3C184070958A3F0828964C341C1804083839175C0A067A962F4
233045C60502C5A3258143F05847409FD1456490804B01D23713328B07C30241C0005005D002E03A
1056200190028003100450053000001161507060703072207150623263D01363F011237323736353605
321F011506232627263534373437071633363735230607013332171617151407062322 2F013536332635

3417151407171507151734373526270 2CD402C693F5C4807090838400A263C4454061A5C17FE956A1
E08414F5448382C6814201C15132C150F01CC20315F310334153BA8180C0B0908840C0408242C053
305620838A4A4ECFEB0B84434400838404D57A0011CE844945044DC4C2C40D808382C4441234434
DC28213F28063EFE6C40352B202557604C2C4044153F3E76081D0704102C24081848100C1C000300
500006044005C2002E00350044000001333 21F0115061507161F013336373633161514070 60F0116150
623223527230714072322 2F013637363F010335361715173337352603140F011514173332373637262F
010138203A8A2420382C2C4404DE4E2A2240403BBD50280838401404B848209F350440486117146C0
88C2804081A0E7C4C3834178125274430100 5C2885C806D437462B29CB1532C083829274696406E5
A405C5C680C0C704CCE7AA8402002182040B404C8286440FD900EDAC814110B500E227BA144000
1007803260114057A0012000013331615140715140F012635363D0127363334C0045010301C400C0C0
E16057A0E3A1715F0E9030408383369E43058090000000 10064FFB0022005BC002600000116151407 0
215070615111417161 7161D0106232227262F012635113437363736373237363736 01E04034A044243C
23157C0838283854204814401B19263607213319 2105BC08382B19FEEF0F88761EFEEC885C501C83
210440544B5594822E0100659F213B595B44501418000000010064FFB0022005BC0026000017263534
37123537363511342726272 63D0136333217161F011615111407060706072207060706A44034A044243
C23157C0838283854204814401B19263607213319215008382B1901110F88761E0114885C501C83210
440544B5594822EFF0064A0213B595B44501418000100 22009A042204D6003D000001161514070607 1
537363316151407060F0101170623222F012306150706232635363736352322 0F01140726353437363F
013526273536333 21F0136373602E64018101CC020144038515364014804083 82B45D80448201B1D40
201C20040B8DEC284098AA4A64D60A08382C3C8024141C04D608380F316860048C1408382E164533
48FE80184064F8C15F941C0A32CE7A4A0E60540B05083843213E423004F61610405C9085776000000
0010040FFC004200420002000000 1062B010713171407223503060723263536 3B01363F012603363332
171337331604200 83874D44C044044488A72A04008389465830C28380838430D48E0804002504030FE
5440270D740188141C0838401814089201 0E4080FEBC34080000010057FEB9010F00BD00150000371
615321715060714232 63D013637353427263D013697680B050C1840401C08281008BD0C3028C04A4A
4C08382058346C071915172040000000000 1006401C2038402AA00110000011615140714050607232 6
35363B013625340344406 4FEE0C7554040083834A3018102AA0838371D081C1C1408384024280C000
100640000012C0124000E000013321716170623140726 3D01343736C43A0A160E0C3428601C0E0124
4C1751600B05093F20611744000000010 06BFFB002EF05C800170000001715070 20F01020706071423
222734371312373413363302E7085090045C60440D0B48380824645048CC233505C84008ACFECC38
F4FEB6965F097C401C9C011801169E4401B06000000003003600000 3AE0584002A002E00560000013
316150623153332171615321D010715140714070623072322272627262 7263D0134373637343734 3736
171533350115141714171617163B0136353635363D0137352627262B010623263534373523060714070
60F0101BE2040091B8C507858080C2844745080207C3C17612C182828174D58641B5508FEC02C581
C34151334A474240C081C543C64151740401028384017313805840838380468685 81C60F030518B367
2A41C44065E406C3D7FE06266736D13650943287C0404FDE8E05C5C41530B2D0C1C18874D812744
F054472D5C1008383018042420094 7324AC40000000001006BFFBD038705A5002300000116151136 3
33217142304150607232635343736331 10623062B01263534333F02353601C740E8602D0B5CFE246E2
E0840CC222E7D0B1E1620405C5854140805A50838FB282040402E12181808384D1F1403C47C14083
8445888302040000001 0035FFD703C7055B003F0000001716171615140706072207060733361733321F
013217062322273427232407140723222734373637363 7363D013427260706070607062326353437363
7363702A92F30101F6B1F6611D8391704D7EA0F612C490A05083520344122FEF3ED351F351B3219A
DBE1FB32A95A428371D1815193D41622B8B7C054435314F905495A33672F0271D4828203C2840301
10328641A0A5848282597D709DE7E1472866B8B17351C40140838334D620A5612000100320000039 6
057300420000013332171615140714070607061532171617161D011407060706072322072635343736 3
B01363F013534272627262726353437363736372307212635222736 3B01172102B2402B15205058167A
2C3197973D0C30315340BC5421AB407C4B5934D63630443AB248A824EC77351622048CFEA074090
30838202C016805733214262645086 02F5D16074F63AA4B3522693445332129 2E083A440F1D2943424
F863C60351E0B19264D4C515D0B2F110B1A2543090001002800000 3D705400045000001321707150 6
151F013637363316151407060715161533361736331615140706170607062322273534272627070623 22
272635113633343716150711172535263534373534330 20C2B150414040CAC67271940A02DBA340403
082D1740342A0B141019472D0B24060AE42E2631271C0C202440100401140C143C05403050404F91
D00448311C08384523254C04E4501A132008382B191D1A0E4E204050B2A62808603C24685C0200B4
09030A3270FDD428740459B3836134800000001003C000003C00570005C000001321714230607060714
2314071507151417153336373637363736373733321 7161D010607060706071407263534373637363736 3

D0134272322072 20F011407140723170623222735263D013437351233343716150607323F0136373633
036C270D74F64E804C30240C140410242A366E3246664C51333C0553AA06127E3C4078241CCF010
C3C383860158738283C080808385A12140C1F1940401C101351780E46FC5805704044442C4341080B
09104C94445C58703056327D174C14588A4278477DB71D1E92210B083824882739DF2D0E1A683D6B
609C68315727092440FC346B41803933140144320A083856663C400C1C5C00000000200640000037
B0573002C00470000011615140706070607220732373 63F01321F011507060706070607140722722272 27
26352227353437123F013637032315141F023317333637363736373237 23F013527060706150706019040 4
00B29301806060F8C10DC5458241C03371D124E3B3DCC0C244C543C0A0624343C501F11880C1C20
309030042A1A3C1C141812010C142E169CDB140573083A1089264A84AE2E7F395A154F74844BD429
77585F150C11196776458021658701427 5B63901FC9B4F457543210C1634318E17697529644A0A1337
22BD2900000001006C000103840540002600000132171615140306070 60F0206232635363F023437363
F0123260F022635343736373237032C301414A82E0654594D27181C40131449615C17454008AA667A
9E40AA6B4B0FE505403C161A18FEC88B01CDB0A4BE180838923898C912C65E7A7C0B1B30AE0838
42A031130800000000020072000003620574004E006200000116151407140706071407061506 0F01151
61F01333437343F013437161514070607060716171417161514070607060714072723222722 2F013437
363736352627263D013437363734373637363334131407060715163317161733363736373527262702 3
2405C48142C2C1437210423912808485870 3C406867052F1D495B340838064A132D3C2C405F351A3
E14383F414012BE681C173D38422E2D3F2084461A130D10012B7020243C2430085005740838 3D0B0
B051010071D0F112D4704142C2C1C1173228288210B08383068893F32522B811858223664641E6226
22210B08288868684C78484B152143561A0C611734481D1B390B2007FD45179D7840186C200C0C0D
433E864460234D0002005AFFB9039E0592003C0054000001321F023217161715140715060706070607
0607220F022635363B0137343F013637363706070 60F01232635263D013437353637363736 3F013E010
115071516331417333637363726272222 2F0126070607220702AE392F241810100A221C1C0C0B0D3D23
2880132D7440400838246C34782424201858682977382 09C540C1C262D27497F403240FE700C31272
8046969646E03190F191222485086134F0592686060CA212F10142C68515F0C80C3102F5D1C240408
3840240C0C641EA1D9375D6B293F0C32224D6B0C393334B34B433B2D6324241AFD82304C40580C0
818684C7E89737C2C2C212F66A400000002006400140170 03D0000E001E000013217321715060706
23263534373603161D01140722070623263D0136373601041C2C1B091E422014403C15015C38092331
0F400D272B03D028382064281408386B6510FD94193B403D373018083820634944000002 0056FEB90
11E034000150024000037161532171506071423263D013637353427263D013613321716170623140726
3D0134373697680B050C1840401C08281008573A0A160E0C3428601C0EBD0C3028C04A4A4C08382
058346C07191517204002834C1751600B05093F20611744000001002EFFF9043E04390022000005222
72627262726272635342536373637363316151405060515161732171617161 50603BA39637D878CDC2
838240180CE626A622E2640FEC48CFEA477CD089459674008075449735 4C810041824634934303345
3C0838539D404C047E8A78354B0838400000000200570 0DC0437030C0012002800000133161506 2B0
10607220F012635343F012536133316150 62B010723072306232635343337333637360 3B74040083854
FEA290805C40689C0138E12F4040083840E0548C748844405CA49448D44F030C08384030102010083
8410B181C30FE940838401410200838402010041000000000010021000004310440002200001332171 6
171617161716151405060706070623263534253625352627222726275 27263536A539637D878CDC283824
FE80CE626A622E2640013C8C015C77CD089459674008044054497354C810041824634934303345 3C
0838539D404C047E8A78354B0838400000000200C8FF4F040005A90033003F00000132173217161D0
106070607060715141F01142B012635373427263534 3F013637363D013427232207061506232635343 7
362536033217161506232707222736032 05E16124A100D634E4AB41034083C084004241854583A6A7
C4414DE36D8181C403C2E0126509B5B1D10083818202F190B05A924A05739206F5D612F6A02042E9
E38800D274C265A534534343012627A3E147C582C451F1808382B2D21630CFA7D3E1D215B06067D
5A000000010064000004680 4B800710000013317321716151 61D011407060723222F010723262726353
6333633321F010623263D01232215141F013237353633161D01363F013534272627262726 23342F0123
220706070615061D0116171633163B013637363F011615140706230 62B0122272627263D01343736373
6373637363734027060 3C50484C3C503054043440043020 6C384C182434745A2A08083840145C443C
13210838404315042C130507111A12580854922A3143481447796232223 6705E3A38342440783F15721
A809E5A5B69703806220933204C1E4A04B8145480406888208E3E461A400C0C163652967C38385C4
0083814407226102 41C40083878382C2C346B49372D012738090F085026A67E3A432180873140083A
121242080838364A20483C2B518B45646D9B015704783B5920240F0000020028FFCE03FC05E5004B0
0580000011615140615321714 1F01163B01371615140F011417161716171617321506232227262F0122
272635230607140706232207060722070623263534373 43F012227343736371336 3F013437360B01060

72207333F0127342735020C402C0D23444C120610804060304808302004120608083841 0B13090C1B4
928080D539838482222320614081824402430200B053420086C13191838281C6814080701108C8C504
805E50A4337501B651287BB2C220A434B0B0F14B120365F27035D224D7D234721CF501E0D190C2E
22C07B3B642C0D3B5441197C86303220785B012E44304817AC1DFE69FED63960183039C116831800
02003BFF8803B505C80053005F000013161D013733041732171506071407153332 1F01161517151407
061507061507062326353437363536 3F01363F013526232627262B010723222715171516 1D01161D010
623263D01263D012735273527113735361311171532373437363526238340042C0145096810097F342
C7183543C08985C3C3C101B1D404C2C1E0E68223A0C052F2735493B908A181715081410083840101
40808080870 08AC564C28081805C808384C041206BC0446960F2504483C4B2D5064328A5014542A2
A381C0A32546011231E1E64283018447C29131C08105060BC4F2D381517304008381C1517448CAC6
0F82C01802C3440FEFCFEF82C145409573A16300001003C0000041D055F003E000001331615142706
2B0106070607220706071407061513171615141F013717323733161514072524352227262726352703 3
437343736373637323736333437340 2D018618120340813393F01154F2341203C06042084849EEC0E4
F084095FEF8FEFE1468590F50080630402C4C14480F21443C84055F31404E280E100B3B0A45085D0
B1E8859FEF7450F1F1D3E3C04042E0A404F2805091C373514722A4E011C58841962473A073D2A401
20E0800000002000500000 38D05C0003300520000131 61D013320151617161F0116171507140607140 7
140706070607062723170623263537352627361726272227353F01273F013617150715140715171617 16
373637363536373637263635262326272227267 23 9D4070010457212157241F09142B4830341C79582C5
B29080808385408571D1058230D070110143F142116741410180B0D5B677116441D1F3523050C1319
26520F291DCF05C00838381C14200761304050189C858454112B0A2A30633A1709022440153B2C040
E42490A95CC3CE0A0952A4B1575F804602C6B35E4FC318B302B6A16540C13352C5893751D6C4325
1418000001000E000004160520005300000116151407232723072315071105333217062B01222723072
3150715161F01363B01173215062B01222723222723220F012327072322272627353723220F01263534
373637113723263534373637173 63B01173303564058D82CA4381414017C843A0E0838406E56D02C0
808102008AF41ACCC48083830171508273DA437B9140C04041052161C100804093F1C406044040810
4054174534232DAC2CAC052008383B11080C1850FEF004504010080C2C30FD57446C204040101074
040404CC5AF64C20400408382E460F0D0108480838491B0804040C0800000000010064000003 9C060
60035000013161D013305331617331615062B012627232523111407171536373637343716151 40F0105
14071517150623263D012735273537133 4B8404001082C286C5C400838646D3320FEF83C09013672A
26C58407884FEFE5008083840080810040606083 83E200C08083840080C20FEC2275D643C150F2410
0D0B0838420E202C100440C0E6400838DEC0FC603454020C2D000000000100210000042105 8C0065
000001161514071407060706070607060F011514070607151617161733321733363736 3F0235263D012
3060F012635343736373637343716151407060F011F011106071407060706071407232623222726272 6
27263D0136373237323F013637343736333437360 25D405C2C012738283C241D131028141044383547
04325A281963380C241C4C048B2D2440B4313F0F75544030176908 5C0C0818301B6D4020287845173
A1A2E623F510814280E0E0D0F3C431D48391F343B058C0838351F0A0E090F3F153153167E1418193
F2B4D906775411B181B252B2D3C68F0462A0C4745080838476520 0C1923250F08382B152A1A08683
4FEF84C20244053450D270B0518180A6A7B81112F94605C5464706F0506324409132800000 10011FF
AC04640581004B000013161506150615111725352603113633161511121715373316151407151617321
7150623263D010326270523151715160706170623222726372735272 20F012635343736332711343736
3736E74006040801FF100C0838400C10780840B8180C0808083840240B05FE1904120D10070308383
A060414090E302C18403430340C04020C1B05800A32354B5544FEE068421041010B0100400838FF00
FEF333082C083848240C8FE19C204008382001B0173D3E04D734854A283540443E6D7134CB240408
382B19208401206D4C364A1C000000000100060000036605A400270000013316150 62B012215161D01
131133161506232126353633211134273526230723263536 3B0137330246C0400838D434181CF440083
8FD40400838014C1C10148CA0400838A0A03405A40838400C534514FEF4FD2008384008384002E03
2CE34800C0838400C0001000D000003450580002C0000013217321F011617110223220706150607062
B012635263D013633321716173336373437343F013526273527360 24E453F09032122243E120E0E5C2
470883020C04408383E062C64102FAD602830382F500805809524558290FF00FEEC28622A2E2A4036
223B2920405432160C5024682020C8E4F75C108940000001004B000203BC0600003B0000131615171
60722171536133F011615140F021617161716150623222726272627 2715161D011617320715160714072 2
3F01262F013536351127372736 8B400D010409121BFB94404044C8A097CD96224408381E4AC3246D
EC081306070B122140440D06030210020A040D080600083846394969FF09010E881408383117BCA02
0D6C61933254044DF288A3804153F146CD6426A3A312D0B907181B35034B61F010BDB103A4000000
001004A000003BE0574002B000013161514071507150312171617153733373637363716151427062306

0723062322273427031335373534373 6B6401810040C1C0F091458BE3B65147C5AAA34604ED01C1A
2A40382818041020140574083 81C18686C04FE40FEED4D173908041E120E0C2429384207201B17349
42B75013401C0106C34601C2000000100220000044405540 05B000001321716173217321733373633 3
6373437363B011615161D011407111217321715062326 3D01262F011137352207220706 0F01060F0115
0715062322352627032723140715070615220F01150623263D0112373637363D0137343734011C353B2
1130B2107290440161205439030181 85C0C141C04060A083840140C1014093B1331291F18240C080C
0838441044681004 1C2034060C040838401C1E10140C1C1C05546423D17C8CC464165226722418400
E1A1C2167FEB0FED28E844C40083844B6D6B4016C68183C601A6A506C4408087874407450B00140
88204C3CC08973F6103440083848015688178 50E1A58681E5638000000000010035FFA303F3059E004
0000013161D01161F011417161716171617351311342735363332173217111407111617062722726 27
2627342722272627263523150711131106232635110311371136814026529084081832 3E114B1C1C083
82E160C0C1C271333453416173719495C151B16DE3C040C04083840040C0805400838 58056390156B
0127295B2044640120016E10300C403848FE8646C6FEE28D7045439C106825700B753005EB3212483
0FEBCFEFFFE984008380158012501403001304000000100560000045A057800DE0000011737331737
33161F0116173217161D010717150717150717151403060706070607060706232227222726272635332
7353727353727353727353727353727353727353727353727353727353727353727353727353437363B0116151
40F011715062317150717150717150717150717150717150717150717150717150716171615161
7161716373637363734371235273537273537342734272627262335072327220706072207220722072207231
7150717150717150717150717150717150717150623222726353327353727353727353727353727353727353
6373437363736370 2EE04041C040410364624290F151330040404040404101418104E15614C8C60501
933395F25174C040404040404040404040404040404040404040C0404040404243C200440240404091B04
04040C040404040404040404040404040404040418281C1C142014695F605D5F30201804040404341805
271D2F041C041E76081C0612130D0E120404040404040404040404045008382E164804040404040404040
0404040414184007452997057804040404063 22C38285038380C04041C04041C04040461FEE63F2540
854334511B0D28AC5D0F9977041C04041C04041C04041C04041C04041C04041C04041C0404103404
1C04041C0404045319480838 1B290C041038041C04041034041C04041C04041C04041C04041C04041
C04041C04041C04A52B26121F41232538062E455D970E36013B40041C04041C041A4E1D131933340
404043C0553245C58041C04041C04041C04041C04041C04377D1040386B5D041C04041C04041C040
41C04041C04041058303C48404C340C0000020064FFE60390060C0034004400000116151407140715 3
3363F0133161515161715140F022207062317151423151617061707062322272637263D01372635113F02
3613230711363736373637363735262B0106070100401814048 75D68847C1E0ED0906417891E16040C160
C11111C08383014030E0D041014141C164A0C18911F0967D5371428508147060C0838142845170C1C
241018181A4A485FB5685C94141C100824CA1E383846403C5D9A962134182D37019488649C70FE90
90FE9496061D478E76381C0B2500010049FF7B049D05BF006E0000013316151423060706070 60F031
517151417141F01163B013732373527342726353633321732 1F0137363F01262F012306071407263534
3F01363B011617161307060714071617161732170623263526272627222706230607232227262734272
63D012735373637123736 3F010259204064163A6A2A261E381C1008404060643824580A2A804018083
829230B415C307A0614146034407A0638405080153F2050506814141C5C60810B290B210B0838583B
1917110A265E1A372920EA721A4A2030082018245840 6B454005BF083844140C47555761989844242
C34424A21473226403004880 83C2117403C505C40C262B2A5350C450B1A0A083830384408083867F
EFF7CA781107C863A261A3C40182837450E2630603602780474162649 6B142C40908F45011438770D
1C000000010042FFC403EA056E00CF000013321733363315373317373332171617150717150717150 7
1715071715140706070607140701161716150623222701222734373237363736353727353734 2F010623
3506230717150717150717150717150717150717150717150717150717150717231715071715071715071507
1715071715071715071715071715071715071715071615231715071715071715062326 3D013727353727353727332 73
5372735372735372735372735372735372635332735372735372733273353727353727353727353727353
7273537273537273537273537273536338E231D0442325C0C040404986C45370404040404040404042C5
3675B1B180190522E1408382F81FE88301C680E26903820040404F8103B316C1C0404040404040404
0404040404040404040C040404040410040404040404040C040404040418040404040404083840040404
0404041804040404040C040404040404041004040404040C040404040404040404040404040404040404
04040D33056E291404110404362C541D04041D04051C05041D0404101E318E3B460B0C19FEA45413
171B4280014367306E2E3383292104041D045527051104191 4041D04051D04041D04041D04041D040
41D04051D04041D046F051D04041D045F0C04051C05041D04041D043A0C04051C05041D048832041
D04041D04041177096E0D04041D04041D04BA041D04041D043A0D04041D04041D04041D043D3304
1D04041D0470041D04041D04051C05041D04041D04041D04041D04051C05041D0404108D00000001

00350000037D056C00400000011615062706OF0106230607151714171617F011617161D0114070607062
3072225273633321F01363F01363D012627252726232627263D013637363732373237360243B11AB66
F1547202B400C5942228BFC302C1B353A1E7F4D1F75FEFF09093D1FE5425A30351F1E67FEE1051A
2D3340580D2423470F2F30356F056C5F389EB51020243C45533C3C0D0B1A22481A361C5840256739
4F9404BD20409508366E403F15602B115404141B3521574061334335 3C3428000001000DFFA304170
5440026000001330415161506232523150711140716151706232227352637113706072207062326353 43
7343702A7380120180A32FEFC18140307100838401 00E0A14F16D1B352E16406CF20544220A181C40
200478FDB471A2774F40407008ABF602A05C140A2820083832360E1C0000010036FFF20350055D003
9000001161D010717150717031607060F0106070607232627262F023327113633161D01071307141716
171617363F013637323727132735372735360310400404040406021B063A2C3814202040D44F2C4C20
080404083840040404 3C214302B94711241D0A1107041004040408055D08380C04041C0404FD63F01
6414F5D2D0A0C20437C1DDB6C640402504008380C04FDC4043EB2634D154A0C30441073940402C
C041C040410400001002800000 33C04D8003A000001161514070607060F0214070607060F010623222
722273537273327353726272 62F02363332173217161F021533343F01133637363F013602FC404C241
C0B0D24281C18280E0E18161A390F0903040804040410281D4F4C200838380C060E50504008082428
3C0F091311581604D808381779698F174D986C0E3A5E52145C2C1454240C0448041C04A2568898E4
9C40486CF5ABE02414204C6C010C2355264E941400010014FFF704F80573005800000116170F02030
603070E010726272627262726272623030607060706072627262726272627262726372627273736171 60
F011617160716171617061716 1F02363F033617160716071617161F023312371337363704D81D031408
0C2A08541E0728384404111F3E0C0A17043A053B292F19353E0E0C183A112C241A1E3B04180C0B0
1060E383F0604090D20053710022F024C022919045605321615 0E383D022003130D161F1C3D044708
2A11082805711C3854542BFE4E5BFEB66E3B310307633A89C7510973FEED3A9586273918073D0B7
D63554D8319918D311B650128403F050E38340E542E359050136218AC24444301E52CF08BDF3F050
65455231F6021857BCB01125801BE5F6E440000000001001BFFF9039B05C90047000011615140706
0F011716171617161716171615142B0122272227262F012627262706070607060703060706232635343736 3
7363736373526272635363332171617321733343736373633032F4054673514301 10B35331321484418
30240D13212310271517154B1D2030162E5C0F19181C404048283937313F0B49500838331507310D3
3044C4F3D1D2705C908382C74F52F2844091B256F113F05375C4C3C284418192B14245F411490465
EFF00093F1808381B75E044BF3D5D47041B458F154044025E400C6CC3514C000000010028FF81034
405A400270000011615140706 0F0106070607060706071407263526 3F013E01272627263D0136333217
161712373603044 05C2C284C2A1A17191A16260140400E452C2E1A1381707C0A327076415F9C321B
05A408381E8A5577D47D5F4197455F38CF320A0838C483A87FB726E688123A044074959D01B14B44
000000000010021FFD103FC0547003400000132171615140714070306072207061507153605171615140
726250722353437343736373437363F012304072207263534373237240311 1B21103C20B92C23076560
046101928C543873FE68D85C3070552F6E523C1004FEB0941F4D40781B5D018E05473C15171C501A
42FEEA7544D87D530404281C2136382E1657042C4412823DAFB070129E6390381B34140838430D20
3B00000000010049FF1002290648001D000013211615062B011517110711331615062321223D0134371
13711273534 37C90100400838D8080CFC400838FF00A0240C084406480838402C2CFD40A0FD80083
8403C0C1A2E0270A002B42C602F2500010022FFB002A605C800170000133217121516 1B0116150623
223526272603272603273536623523CC485064240838480B0D44605C0490500805C860FE50449EFEE
AFEE89C1C407C095F96014AF4380134AC0840000000000010049FF1002290648001D00000521263536
3B013527113711232635363321321D011407110711171514070 1A9FF00400838D8080CFC4008380100
A0240C0844F00838402C2C02C0A002800838403C0C192FFD90A0FD4C2C602E260001004203CD02F
E0585001F000001321732171633161506232635262723060706070623263534373637363736373702 1E37
150917382418083854263E04446C337514144040078558441953058554588 4181C40191B1AA2404C16
560C08383018124A3C4034240000000 1004A0000044A00A800100000251615140723050723263536 3B
01372533040A4080F0FE70A02040083820A00190E4A80838430D0C0C0838400C0C000100430460023
305DC00100000133217161716333217062322 2F01263536831D532381331540140838285 0A4940805D
C481060285C403C785137400002002FFFD0039303E80039004B000001330415161716 1D01161506231
707170623222F01352207060F012326272627353637363736213334273427232207060714072635343 73
6333603151633141733363F013527232007060706 01A76001002C2C1C14060E0C04100838410B080A6
28B3D5080315F23191B3D7D4B3B013D1C40A8681E4E260E40402C4242269A1B1D2498257BB80430
FEDD254E5A1F03E82D3F33A9B53388161A2C68045840683818403F2D040349245420904 0891F4432
861A1E180D27270D0A324A1A4818FCC804400C08243070B4283C26762E0000000001005AFFAA03B
C05FE00510000131617140715061716173 63F0136373637363F013316171617161D0114070607060706

0F01060F022326353433363736373637373237363D01262F0123060F010615060F020607222726270335373627 26AC401426160803260B1844262E41431C3C14604D3B1D0B0C200A0A091B164A5C4D833C18204058C04040241612090B20101C203C283428742E1A10431439391A0C1A010C150A0205FE08671CC5FCB2375AB12854DC2D574E1E0A2604104C1448154B40662E13390460167A8C585C2004083844728E53450B5940352F745410141C141CA533A11740824C0F5193800185AC9C12AD4D0000010028000002E80448003D0000011615062B01070607060706070607060F01151617161716171417333636536371615140706230607232227262326272635223D013637363734373637363732370 2A8400838344434406513123A211F10140814101814382478 3F3540501E2E295B402C581D07101C340C172133454042322957104804480838401C2B196E0A27494C24605457050C24082C0C10133137050838341 84C181040300735294B5C40AB096359182C541C0C401C0000020028FF9C03C80610002D00410000011607031417151737343716150623140 71F011407223526272306070 62B012235273637363736373637363F01171336011F01333237363727352F013727060706150706 02EE5C52180408603C40142C94280440441408043ABA3B1920C4280D1F30300F314E42726A212EFDF9184020089E4268040804040CAC2080402306104F45FDE874A4887C2427090838600533C4242D0B641B651DA32C6C99AFE34458172948303B0B01BB98FAAC702A7D3245043080A07C571137711764CE00000001002FFFCB032B040B00510000013332171 41F011516151407060706071407060F012635343734373637363736373427342723220722070607061511141716 3B013637363736373633161514070607220706 2B012227262711343736353637363701B780354B30044078063E555F58015710406C6008554348282C58243C132536222438254F0C1B5528403309272940454F300F752E1A20B6363808585825231 94F040B7C1476180C08383F15091733150C0C090F0408383810110B0B10202527315A24485027834A1AFEE23D2F100F26133112494008382560462 83D24505050015029C36C143010331900000001002 1FF6A031305D1003E000001333217161506232272227230615061507173336333615060F01220715061D011607160306232637362736273537060F012635343727363F013637323736021B2461175C08381D2F0A12405450100E04644B8C0845532E710C0E0106041D6038250B0905060442421840E40E14081C23350B393B05D11C2D33402C102D075C80509216213F40140E1604251F2C71A475FEF168367993A88C8B243017310408384B49B6625A4C3F4524240000000000300 43FE5503CB04390046005800710000013217161507161506070E01070E013315370516171417161D0114070607060706232227232 22F013534373437363335263D013637352227262F013437363732373335262735360115163B013637363536353 42F010706070603151617333 21F01333237363F013526232527230607220714024621 6B180C440C4834503C50071C14010C3933402C282F5124382A6A3E8E405983081C8830141007142 94F3F15047C6779147C04460A08FED862222C255F44443408C83D2B50233D2F1831933C040E2E7F35104F15FEF804044C3C2D4F04394C310F30364E53F13C29073A460404040830094F3C1C205F35412B19133C249424406117286C18041517202632044C273D4C3F895D1B240424201040FDF41C5C0C0C2507F01C0E0A0838212365FD59544907200428353F2834780404111F640A00010050FFB7035805C3004A00001316150F011507113237323F013637363736331615161D01161D010715170607062326352 7323D013437353427353427230607060706070607060714070607150623223527263511123736C840240C080B090D371C2E22113FA421602C140C1B090B181C401B080C141404195B412C07150A16181050081C08384004100A220B05C30838909F3480FDCC3CC0448937342828142C4F79146B41A04C54DB021E180838CF1430393394445C203D47074019940143095B4C182484028348405CA1224A0280018132700000000000020064FFDF010805B80014002000001316 1D010711171517070623223726 3D012711363337223D01343B011615071714B0400C1410100838471B1414062E26443C0840040404000838204CFE944C74DB2E40C5663250500164BCBC740C800D274C4C270002000DFE7001BD0540000B002C000001321D01142B01263537273413161D01141F021514071407062326353437363736373637353427032227353601214 43C0840040440401428206 0A0452B40343F4D12262A0E1C300907080540740C800D274C4C27FEAD0838203C48E4DCDC877530745008382B194731104C2A4EE0458301188020400000000010019FFBA031D05C70044000013161D010615111733363F013437323736333437161514070623060706072207321716171615161F01062322272227262F011514071407062F013637363D013711343735368540010404261E1C5809671D1F5440382C2C294B1C200D23165E4B0DA92B2108083840801E5B2B55611308104D3203141301010805C7083840EB7CFECC1C1731181947501414180838 2E16282A2A1C3434705C23D1263F1D2040FEAC1E6230543A8278643C023B43889236588C01407AF93440000000010054FFB6011005AD00150000131617071106151314170623222737033437113427 36A2402E1814040808384616140414021505AD084560FEC03F3DFDBE38D440A0B00246424201106756100000000010064FF7B04E203D4005300000132173217153637343733321732173217111617072 63F011127342723061506072207151407142326352637112735232207060F010607060716070623263735263526271136331615111417363732373637363702548818090338 6C28602A360B150C0C140A62631B0C182030990C04100F0C4C400410084C34540A1E181C1418101414083868281C080C0838400C140406420D1F4F3903D44C24302C100B055

06048FDE46A503738594C022448251726360D37A4A011567E0838743501182CF094054B6C2C5C851
767394014C028315793290140400838FECC1D0349337C063A5E0E00000000010050FF96033803F200
2E000013161511373637363B013217141F011613141714072235342726272635230607060706151407 1
506232635110335369040 4C48443E1E203E2A3424202C084044400F1D3C046E3E182C301008384008
0803F20838FEE88C6B3130781A82C42BFEA79D2B2D0B70EAF613C1862A4A961A8659372212F4400
83801B00100D0400002004300000437041400250 04A0000011615073316 1F011615161D010607140706
2B01262F012627262726273534373637363736373633011514171417 1E0117333637363F013526272623272
3220F012635343736332723072207 20706026B600454753B4018141C28708A2A807867613E32172121034
046724C0C5D3FFE94283C2856B664644C1420141804520E2C44085C3840482616084C6C2E8620041
4093F1C1E4E582A3E155B608C303D6358142E3E2C4C095F352F80228A644C270D40FE246813491C
3C3C3A261D5F3C34547454286C144C0C0838380C300848882F00000000020028FE4D031F044A0027
003A0000131615140F01153637363B011617161115060714070607062716071607262736372736351 13
437361311171637363736373423262723060706 0706D2402808107174641 44F552C2A3A4C2E777E6E0
61B1B4F5701100D3222381C2C04817C6937171D2C171D1C5728384C48044A083810603C04183A6C0
F6D2EFEFE0CDA47105C2A524F22CE7478231948AED649396601B065C358FE60FE2C3022635A6A2
0A9F81E0E2B2436254400000 10043FE4E03D6043C004E00000116151407220F010607060F01151417
163B0136373637363D0136373633161D011407140706150706173 63F0116151407060F01232227263F0
1230607062326272 62F01353437363F013637363334037340701E32B45048A03C08B40E1A1477316E1
614121E291F40382018271B3517351C40182E375C0C39132C1B27041C802266A828412B1470293F98
50A45C54043C0838410B285024304D8F701467250C245081872D13407616280838201 19715633FB1E
7D2A035440408380F313F350E907DEAC740402027211C585020CC64342C582C44440700010038FFC
602A804260021000001161706270 60F020617061507062326353734373403273437321 5173637363736
020C7E1E3478800C541C1911090D0838400D0524184048101B1D2490180426246A6B7174449828589
88D485B4008385B33BE310147E42D0B8C84184C5E7216000000010021FFB1030D042A00390000011
6151427060F010607163B0105161732171514071407140714070623222722273633321732373637343 7
3527232635222735343736373633360 23B4D913B9D1C2E342F1984013C41130B055E58246C4242124
6320A08382A3A1E52364E46F0C07831135629977A4E27042A3638441E066E1876401804192B284026
B11B3108241351282840402848522A0F981C040B1D6020229062524C14000001001EFFB00272057C0
030000001161514071136 3B011615062B01220714171417061F010706232237222735263726272637 06
232635343736331136373601264010 6E4E20400838206256050F0312070D15384B2007010E020C0410
1367194038641C100418057C0838462AFEA41C0838402427A55D682F433D6E8AF41C300C902D871C
644408382E16440198324A240000000010036FF95033603DD002F000013161D010715161F01361337
27353633321715071715141714171506232227222726352302232227263D01343F013536A640300CB82
8296F200408383F09041C281808382E1609171C0462526BB96428080803DD0838C0C0C4495F105101
53604C60406060489850AFB1352F084038B47E62FEBC8C51778C467E28B4400000010014000002F80
3F000350000011615140714070 60F010607220723171506232227262726272227262726352635363332 1
716171617161716173437363736373637360 2B840283C1C1814190F0B09040408382E161E2A1834112
B3014303408384020130D240C261E363228243C140C0B211803F008381E5E49BB557B541E62341404
40380F3D0B5178453F475D8C1040943B553335265E5B211E5EBAAE496F026A1800010036FFCE03E
E041C0047000001161506150207060706070623222722272635262723060F01060F0115062322272627
262F01353633161D011633173637363736373637363F013633321715171 21F013336373637123303AE40303
80C141C26121428390F0F11380C0C041418385410080838 3D0B1C380B1D100838400913403513 1C10
1F050E0E10152B3B1928281C0C042008181C2F19041C083BD838FECF6E61537A042A587EC63D268
6306B8A933F082E436083A72F3EB08643093A86ACCD51452B5C5141179558326037C4FEC44E3794
6355E60108000001001BFFDE032B03F60031000001161514070 60F01153217161D010623222F010706
0706070607062326353437343736 3F012627263536333217161736373602 9B40701E32080AB65808382
B4DA440334D2329162615174040682B4D2C6448540838126639573D632503F608382E561F790804A0
531D04406488604B9921731749100838 1E621FA1626E48541C2830403C1D439F3D380000010014FE4
802F103EC002A000001161707060F01061507020F0106232637343637362627262726272626273633161F
011417373637353637360 2B73901290814251824462424181C4007242F18242835633D59200D0838545
BAC3418141D141E1803EC1552662283BE3C50C0FED46E761808600F64E12173305E8641C1243040
14E2F41044985A903462626E0001001BFFC803FC0421003B000001321714071714070 60F0114070615
373637361734371615060726070607060706232635343F013437343F02060F01060F012635343736373
6333402DC2D0B0C043C712F3C6C20E0063E7469246F0E59545AB8AB2F552014404040 74487018735
A5815A318405C5ACE955C0421461517272521E1495C0FB23E0D650A193D080A03093D501F3729644

72A2016093D36637716B8106ACE35312834045605093E30411D69540D0000010042FF3E01FE064200
3A000001331615142306150715141716171611D011423071516150714OF011F011615062B01222F01123
33734272326353431F013637352F01263D013437360192204070880C381523101C245C71141C4293400
83840A42849230D71286040543010182C401450A806420838443E0610347947443C2311207034084CB
CA00F29D0741408384082920l0C9079270838390F2C261A30608C4F4940364254000001004 9FF7401
2D06340011000013161511161514071407263511222734373 4C54028284040320A2406340838FA5C1D
231D43320A083805B440113F230000000001004EFF3E01FE0642003A000017232635343336353735 34
272627263D013433373526353734 3F0127232635363B01321F010223071417331615140F010607151F0
1161D01140706AE2040706E0C1D1623101C245C71141C5D6440083840A41B42220E7128604054301
0182C341444A8C20838443F0510347947443C2311207034084CB6A01028D08E08384081A0FEFB9073
270838390F2C261A30607F4E4A403550540001005D0168037D02B40026000001161D01062314072326
2F01262B0106150623263D0134373633321F011417333635323532373603 3D40251B5C606B29305622
105C18244070362275279 03C4C140807051802B4083820AC2319112B2450434D240838203371303070
0709251F144C24000000FFFF0028FFCE03FC070602220024000000003008E009D0145FFFF0028FF5D
03FC070902220024008F000300D500A50198FFFF0036FE5D041705CA02220026FA6B000300D6016A
FF8FFFFF000E000041607060222002800000003008D00A20152FFFF0035FFA303F30711022200310
000000300D4000A01CBFFFF00560000045A070D022200320000003008E00EA014CFFFF0036FFF20
350070D022200380000000 3008E0070014CFFFF0022FFD0038605C802220044F3000002008D5C1400
00FFFF0032FFD0039605BA022200440300000 3004300AAFFDEFFFF002FFFD0039305B20222004400
00000200D357FA0000FFFF002FFFD00393057802220044000000002008E74B70000FFFF002FFFD0039
30589022200440000000200D4DE430000FFFF002FFFD00393059902220044000000300D5008E0028
FFFF0028FE5D02E804B302220046006B000200D6388F0000FFFF0022FFCB031E05A702220048F300
0002008D33F30000FFFF002FFFCB032B05AD022200480000000 3004300A0FFD1FFFF002FFFCB032
B05B8022200480000000200D331000000FFFF002FFFCB032B057E022200480000000 2008E48BD0000
FFFFFFBAFFDF01AA05AD022200D2F3000003008DFF4FFFF9FFFFFFD9FFDF01C905A0022200D200
000002004396C40000FFFFFFD9FFDF01B605B8022200D20000004200D39400333340000000FFFFFF
BEFFDF01C20571022200D200000003008EFF60FFB0FFFF0020FF96033C0582022200510000000200
D4BB3C0000FFFF00360000042A05C802220052F3000003008D00D60014FFFF00430000043705CF02
22005200000000300430112FFF3FFFF00430000043705B8022200520000000300D300C00000FFFF0043
000004370585022200520000000 3008E0112FFC4FFFF00430000043705890222005200000000200D443
430000FFFF0036FF95033605A702220058000000002008D66F30000FFFF0036FF95033605A002220058
000000030043009 3FFC4FFFF0036FF95033605B8022200580000000200D338000000FFFF0036FF9503
360571022200580000000 2008E57B0000000010042FF95030205B5002A00000116150F0115373337321
714230607171516131506 2B012227343735032735072326353 63B01373534373401CA401C041404C82
D0B708E1A040C40163A083F0920400CAC4040083840A82405B50A32883CE4041040440C0814049EF
DE244A0601A264002348C081008384010F85B7527000000002004903CE020905A600160021000001321
732171617150623062322272627263D01343F0136071514173F013534270706016123 1D2E162103111B
2A325771243C1054583561982008284030 05A62838382C40785C2C03411652202D473C30F4300E263
42C281B352C16000001007E003C02A20528003100000116 1D013316150623222723060706 1D0114331
43B013237363316150623140723150623263D012227263D01343F01273536019640505C08381A16205
262408450143B1515174018306818083840 7A461CC01C040805280838CC193B401411776A52283C08
2C1008385C0D13F0400838FC58332120E4A01444C84000000000010021FFAA041D058A00680000013
217321732170623222726232706070615072207151616F012533161514072207171506070607220733173
217161F013326353633321F01151407062B012227222726272 62F01232207062B012635343736373237
352307232635343336372734273536333637363702996226 1F2D1B0908382636290F28AA16543010081
A1A2801040840701B89040913211F0606346431B33F2D2808140838313B043820202014701040285C1
B790864202C2117204094283C070904CC20406408842830 1B1D171D4296058A243C38403C18040917
1937443444223E40340838410B201CA05D172D3F1020683A0A18161A406014202E16243C302626161
A0830180838317B21734490340838440 81C4C0C4C707C1D435123000000000020078FF88039805B800
4B006000005262726353633321716173336373526272627263D013437363735 2635223D01343F01363
B0132171615062322 2F0123220F011516331617161716171617151423140715161517140706071413 34
27262F0123060F011514171617153637363F010214964618083 81D2F235118625E52C656C64C70445C
40084848614F1C41472808382632240848681 80D1B1D7B028A6404160E1C6C6404607D2BA89C0C38
0C808E1610386CC40C581B3504781A42181C402C160E2890344C48156F4F9920384C1C1004365A1C
402D633440442C1C404408583C24641947084C5820175140701E3204453334625E7D070C02F458540

90B0C1523100C83113E42040C0418140400010063O1CF01F3038D00120000013332171610140F01
23263D013437363334012425463F2545459373402A37038D454834504E46190C702380375E09000000
020078FF310424059D00320041000001321736170715171417110623263511 2F0111230715161516150
3110623263511343734272326232272272263534373637360114171617321735033506070607060 30C
76224D0B080828083840280888080C1C0C0838400C0C14682C3F4D3480207C32BEADFE6770451F5
54F0C69672B3140059D0804404CE8EC40BCFD5C4008380294F0FC01041CF0ADB78D4FFEF4FEAC
400838014039E73349242C983B79AC44355738FE608A5A190F2004013CF4164214282700000000010
0850021037905510 04B000001331615161517140706231516173 21F011506070607062322273433363 7
3637363D012623262726353437363736353735342726 2B010607060F0111171615062322 2F011134373
637363734022D207458089026167 15B2B29040953CB71E1432D0B64C672B410140E224167A0341E5E
6C04300828383B4527451018240838 3B2918582D1F6B29055118185321507E5E28042F499C3C604B5
58B194040442C2C691F2010705444201646 2A1A34205C10101C311F18122A0C5828FE7C74261E407
480016486463A0A44041400000000040 05D003C0549055800250048006500 6E0000013332171617 1617
32171407060706072322272627262726 72635263D013437363536331615333415062322070607220715141
7161F01163B013637363F013403272627 262B012207263517321F011506230607231F0132170623222F
02150623263511343334171533323735342B01033920607C4F1D7E120F09381A828662A0959B38942D
2F2C0C789C88403C04053713C57507090320153B60B4809C70901632046C2010783038281E4E3C3C
B232141C241C6404745C281808382B6D445C083840641C0436726440055890403CF8949C824658585
E12500E7E22523A0E2B3D80AC84A60A8C0927606848DC734D3CA8361638344C6819831375449001
04403A5E48180927E86440205C20185C485840743040DC400838024044078764340434000003004900
5E04C50532001E003B00590000013316171615321716171617161514070 60F01232227262F013534373
6373601151617163B013237323736353427262734272627342723220706150625321506232723061507 1
51633163B013716151407222722726 3D01343702A9205692301D3F37250B1110B459A78020BBE931
470C6439CB83FE952745C0683482562884302439532407694428466AC85401A4A0083840108410773
9163A082C40605E4633A11C6C05320A56291B685D6711634F41E18B53350C941B8930C089A37DBB
48FD68C46F21683C9041735E7E9769150F15270E0A40D72590945040101D1B101880180808383F092
4A4231940315700020 06A0313047605770026004200000132171617150623223534270607060706232 2
2F01150623263D013437343732171417333637360533161514230623161D011706232227220327072 32
635343336333603C2430D491B083840401C08090B181C234124083840402435372004 3C1C18FE4420
40701B11101408383A06130908642040706A5617057780EB9124405C18E41F35021E18481CD840083
8E0763A0903580B19725E1808083844107583246C4044014C2C10083844140F0000000001006B04380
25B05B4001000000017140F0106232227363332373637363302530894A450283808144015338123531D
05B4403751783C405C2860104800000000020 05E04E5026205C1000D0019000001321D010623171506
2322273536071615140F0123263D01363301EA78050B04193B3B150BDF4C54182040155305C140042
804105C5C40400411372F4104083820580000000 10043FF24047B0474003E0000011615140F0133161
7161506232723060717331615062B012F0107140722070623263534371327232227263536331F013317
132325232635363B01053337360377402C4434A147180838CC501448F84840083840D854443807091B
1D40285C04D07379140838804090 6C6004FEE0C0400838C00120245434047408382028BC161A181C
402462CE1C0838401808E8198B381C0A322F5101280438161A40240404012C1008384010E85800000
200560064049E0524003B00420000013215173325333217142307231516 1F01331615062B0127151321
161506232114072 63D010335210607060F0226353437343F013637363736373403333427060F0102824
004200144042D0B8CF40C3014E44840083864B4240108400838FEF0384024FED4432D101C30244024
58504B31333D2B0978D4401A265405245C34184048100495870408384004 10FEA40838401A0A08386
00160049C40125A4C080838163E2490AC8F4570643F452DFDCF37D1225A880003005EFF24043A05F
00042005D0071000001161D01060F011714071516173217161D010207140706070 62B01222706151407
140F01263534373437343F01263D013734373637363736373637363B01321F013336373601151417363736 3
7363734373637272 62B010607061506070615060114 0F01060F013336373637363736 3D0127262703CA
402430140424311B1133281329500D67AA36804A1618284C204030281C04A8283C181C10249B29431
5402F652804420618FD386C3B593B310D4F1C2222046D1B245A2A50210F5018024864502C34689834
7843112E261848172105F0083820802828241F1104393B783E1A60FEEC1C16863359540C290F0B3D2
7510808381B3D183C1A2A187FBDA0B0498728580F5DAB0120441C505C24FC4CA8615BA5C3D23A3
77D0B295127045017413E323838906C3A016A0AAAAC9F69FC04403F3943618A42548C11470000030
05D011E04ED03BA002D003C004E0000013217161716 1F01151407062B012627262706070623072627
342F013536333437321736373437333 21F013736331706070615161F0133323735262F0105231514171
417323736372627230615060703954A2E8004282C084C415B204C64702C521E622A3C5C5050081828

2C1C185F3530404C341878451728412F385282302844102735 6CFD4810445026362252381428 6C293703BA448C20443420602E4220073948545A425C081B7112AA2860580E06182E0205076C28804C7C3F3D2818833108203C256F7C8014108C243C443D5F6D13130D0B2100020064000004200 43C00230036000001161D012533161506232272305140715062326 3D01343F012306072635343736373536013316150623222723220706072635343725360228400 11C405C08381A162CFEE428083840200804E35D406823F908018810600838171504B1DB77AD4070016068043C0838C008193B401408817F40400838203A7664181C0838410B0C1CCC40FCA4153B4010300C240838410B381C0000020042003C041603F0001E002C000001161514070607060F020417161506232425262726353 43F013637363736013305331615062B012523263536031E40346B3D6854702401E3911C0838FEDBFEF5765A40749884204C486EFDC2C00180E0400838E0FE80C0400803F008382B19681C68204C201C181B1D401C0C14180838392B684E1A482870FCD4080838400808384000000002004200 3C041603F0001E002C000001716171 61F011615140706070405222734373625 2F012627262726353437001714072305232227343733253301646E484C20849874405A76FEF5FEDB38081C9101E3247054683D6B344002A00840C0FE80E0380840E00180C003F07028481A4E682B39380818140C1C401D1B181C204C20681C68192B3808FCD4403808084038080800000010036FFC403DA0584003C000133215161716333736373633161514072 20F0115331615062B011517211615062B011711062326351127232235363317333527232635363B010027353676442672B50BEC23511513407C105C8CB8400838B80C0110400838F004083840088CA0083840900CC8400838ACFE9D31080584645183ACD80C6C0C08382D73607C0C083840047C08384050FEA040083801743C504010047C0838400126CA20400000010064FF1D031C03250030000001161D011615321716150623222722270607062322272303142326 3D011332373637363B01161D0114173337363F013536029C40140606200838331D0A062020342C5038046C4040680A0E2513093B04402C044029130C080325083834739D30281C4050241F3D4460FE4C50083820019CE09527640838E08C2064417748344000000002006BFFC4032F0490002800390000013217161516 1D011407062314072322272635343736373637332F01262B010715062326 3D01343736131433163B01323F013527230607140706 01A3833184543C465230205A8E38440A724070243C342F2114C00838403C22A21C583808582C0C0C346F5D2810049060EA46C9A760B0486805078C2692924A264E1F19B46C600850400838602B2D20FC786460688460800C500A3622000000010050005704100 54B00380000013316151423042316171617161514070607060706071516171 63B011615062B0122272627263D01343736373637363736352726272635363337039C204064FDCC2836AA5C5C60540864497F327A63FD7FC13040083830C28EE458C4A04E52564A063E407DAB80191BCC054B08384494305C4430442C3E060B25255B0D7F04231D280838402820143A2A1039834C24461E0917042057616034542800010064FF95044803FD00280000131633213633161514070 61D0113150623263D0103353437212227071110070623263536351136333 4C8AD5301B44D3F4034140C0838400C18FE3045830824161A4014101403FD241008381656497B24FE70F0400838F0019010A74D1820FE4CFE8E3A140A32A2E201C08C0E000000010072FEE1035A0651002D0000013217140F010607060706151116151114 0F011407060F01263534373637363F01113427113437363736373403222D0B4844035D40180820183488365A184048063E0F590D2F0C205C3F29205406514038 0C38174558801222FE987F6DFEE0581C740B8515470408382C280E1E045C027E0C014C627E01608BB9570D37391A000002008B035C02A705C80020002A00000133321F011517140722270723222726 3D01363F0133161735342F012307263534131514 1F013332372623011F60C23E240440301C3840693B74134544A03913581C7030404C6024481D070E3205C8A08C709C270D480C28275528720A040810044B29040C08383EFEAA180D23040C400002006A034802B60610001A002C0000013217141716 1D010623060723263526272 63D013437363733353617140727060706 1D011617331733323735260 1B6343850442331313360942666147018701408403818333120452B242C44390F19061050155F932580A025070F15156B290F609E5A22260C40B01A0A040A3E2838644A1A085C647500000001001AFFD804A6042C004700000133321732171615321732171514073316150623212635343736373636 3D012627262722272227230723061507151617161716171615062321263536 3B012F0235343736373636 3B0101F28017212765940616090350C4400838FEA040382C18300C1808640E3A2A12604C4C3C402A2202562F15280838FEA0400838DC683430480844296B14042C184C995FB824C024B808384008382E16184C651BA425BB33752C14102C0C5CC0B632156F38301428400838409078BCCC394F2C30300000000003007EFFDE040A03FE003F004B005A000001321F0233363B013217161715140706 0F01141F013633161D010623222717150623263D010722272635343736 3B013733262726272306151423263534373601151733363537352327232201231514171633373534273 42B01060172363E203004502020B6361008241C5C98A4581123400838BF6D0C08384080647C383C2014344C601C301C1418504040642201 5E0410C4140480140BFE77107C1513601C08643303FE503C783834133940153B4329202282141C08382040789420400838340 8A03E722B2D140CD547440C13654808389D372CFE7048141937202808FED40C3B690C080C3C68100C0000030050FEA00428051C00330042005500000133161514070607321716

17161D01140F010607062B012227030623263534 3F012627263D013637363F013637331733373637360
11516173613373527230615061506250307151F013332373237363F01353427263503C8204044385021
271D37302C382755A523402D73C40D3F4078703F59180C1407318413AD60401458323E15FD53224A
71872820448C642E0206C8745C142818842C300A2E045438051C08383A06448C783D4F4F6160144C
6848244848FE886C083845CFDC1F6D181CC0571D235980191F10944E460CFD28983C30D00114440
8081715541850C4FE7CD80830043C60076504702B75700C00000200C8FF4F040005A90033003F0000
0522272227263D013637363736373534 2F01343B011615071417161514 0F010607061D0114173332373
6353633161514070605061322272635363317373217060 1A85E16124A100D634E4AB41034083C08400
4241854583A6A7C4414DE36D8181C403C2EFEDA509B5A1E10083818202F190BB124A05739206F5D
612F6A02042E9E38800D274C265A534534343012627A3E147C582C451F1808382B2D20640C05833E
1E205B06067C5B00000000020071FF8F010D05FB0011001C000016271127353735343732171507151 7
11140702352635363332171514079508100840380808104048141 93B380840713D01F6849879F234083
CF279AA85FE1D3508058C4C28105C406038080000010042 00FD03EE030500140000132105331615 14
17150623263D012723252126353682014001806040 0C0838400C20FE80FEC040080305080838859B60
40083860E008083840000001007 2FFD803E605AC003100000133161514230607142316 1D0103062307
22272627262723061514232 63D013437363F013217141317333711340335343736038620405C21C7183
4041030143F290C503A1E04544040445721203C20800C040434D08405AC0838400B0908E9FFC0FE2
CB804940FD56761DB214808382013C1CF0D085C32FEF6243C01BCEB010520590B04000000010064
FF4603A00642004B000001331615321706232 22F012307140F01140F011517331615062B01271506150
6070607060F012322272635273437161D013217333736353735343735232635363B0135343734373437
36373602EC20800E0608382418240C28441C1C048004400838048018 0A12294F0F4918401D5318044
0400E12183C4C14147C4008387C20204804542E0642142C2C4024083C175D7C2C68408C040838400
4345FD110DC6F552A2E04403014242D0B0838101050429A9C54416B280838408C596739671F69255
B18000000000200560 0AE0469032000240049000001333217163B013237363316151407062B01222726
2F012306070607140726353437363334133332171 63B013237363316151407062B012227262F0123060
7060714072635343736333401A560334D6038781F25242C40543F396071471F551C7C10302F3538407
03C346160334D6038781F25242C40543F396071471F551C7C10302F353840703C34032044684444083
8206C3C3C14540811072F191A0A0838383C3C14FEBA446844440838206C3C3C14540811072F191A0
A0838383C3C140000020051FFC50431042D001D002E000013215161716171613321706230F0123072
3060723263534371336373637170607030615333733323726272627262702 3D441A323C7037710B0514
406C809080D44F25084040A44440035D0434408C20E480A4761E494F5E364A02042D707E423DEB41
FEF1285C1004080A16083818980198D9672375E85DC7FEA44A0E080CBF71C53F6729000000000200
780000042C03F0001B00350000011615140706070615161716150623222726272635343736373635360
5161514070306071617161716150623222726272635343713360 3BC4044693784C672600838206859BF
44A435475408FEB04034E41C3806AA462E1008382D9F534D184CF84803F00A32465285639B11856B
23354048578133254AB65D5B6C14401008381C38FEBC1C580B853F19151740903A4A28141C640160
70000002006B0000041F03F0001B00350000001714171617161514070607062322273437363734272 62
7263534370417131615140706070623222734373637363726270326353437011308544735A444BF5968
2038086072C6843769444001B848F84C184D539F2D3808102E46AA06381CE4344003F040146C5B5D
B64A25338157484035236B85119B63855246320A1070FEA0641C14284A3A90401715193F850B581C0
144381C380800000300420013029A00BA00080011001A0000371615140726 3D01361736333217062B0
1260526353437161D01068260604008D4153B3B1508382040013C60604008BA153B3B1508382040606
06040080F153B3B1508382040FFFF0036FF6A040A071A022200240E9C0003004300FD013EFFFF003
DFF92041107170222002415C4000300D4003601D1FFFF00560000045A07030222003200000000300D4
007401BD00020067005704AB05330039005800000116172116150623211617113337331615062B01072
327230607060F011721321D01062327212723062B0126273427263D0137363F01363B01341714070607
06072207220715163316 1F013336353637363536351126232623 01D34D5B0170400838FEE01424C02C
40400838342CCC040418100B211008016CA00A3264FEC0800C5020405662442C1C090B4C64480C2
C38511B12160705070118143C242C241C3F0D3C1C3729261E0533064A083840247CFEF4080838400
8045765095710083C0C4008083812760D5F6C5CA0900864C8981B77121A597F23894C14A48C661A1
C0C0C3E265A524F550110BC400003002F005703C303A70033004300500000013217161517343734373
33321716 1D010607062B01161F0236331615140F012227263523060706 2B012227263511343736373 61
70F0115161733323F01363D012623342515073336353637353427230601 4B1824741C702C60424E202
6324C2078253740281424404C2C619B2004175536661C246C48382448403080141C481813213C1C16
0E010410683010143 04C3003A71C59232066360E064C392320873D3C93293014200838322608AC3B1

1573D60504E2601002F692C7C4888C844E42D2734382A1AD4641F0D306818240C480424101800000
00001005601AA03F6022E000B000013172116150623212722273486F00240400838FDC0EC270D022E
0408384004402B000001008C01B905CC023900090000132116150623212635 36CC04C0400838FB404
008023908384008384000000002005D037002A105D8001C00300000011615140714070607220715373 3
16150623222F013534373437363336051615140706071533321706232227353637363301 2D403028101
80C04041040193B41330C242C3F0D2001484050160A083E0E153B5B29281C442005D808382B150A2
E1E5244640408385C5038605E4623495C141008381C684B0D78605C74D07E26680002000703410 24B
05A9001C00300000012635343734373637323735072326353633321F011514071407062306252635343
736373523222736333217150607062301 7B40302810180C04041040193B41330C242C3F0D20FEB8405
0160A083D0F153B5B29281C4420034108382B150A2E1E5244640408385C5038605E4623495C141008
381C684B0D78605C74D07E2668000100780333018405A700140000013316150623070 61D013217062
B01222F013437360124204008383 01C4014093F204C1C04581F05A70838407046661C5C608C4CAD9
B5400000000010078033501840 5A900140000132326353633373 63D012227363B01321F01140706D82
0400838301C4014093F204C1C04581F03350838407046661C5C608C4CAD9B5400030050003C03F00
3C0000A00150021000001171615140723222735360117211615062321263536013217142B01222F0134
333402106C2054283B1508FEB82C02F4400838FD00600801E0361A44202F25043403C028151F621E5
C4040FE8C08083840093F40FECC786444147C0500000002006B000003C705880028003E000001161D
011617161716171617F011423060306072317150623222726272627262726 3D013437123736371706070
607151617161516173312373527262726 2F010247641010251F1E461D27101C4157150B04040838373
D224259335D1B6C48FD0361170823612E86501484356F0477452C156F173514058812523813313157 1
9734335387065FED92D67041040781D6F815F5F417616084430012B259612C8537142960458309C20
41B701924A18543B8D47592C00FFFF0014FE4802F105770222005C00000002008E3CB60000FFFF00
28FF81034406FA0222003C00000003008E006801390001005DFFDE04550572002B0000011615140F0
106072207060706150607060714070607062326353437363F013637363736373637363732373604154 02
C6C2B3908582010705917 1C54742C041B1D40445E6648521638243E322739164E0721250572083819
37D03769801B319A0E55372F61097F342C1C0838355766827053353759415 32D6B0FA944300000010
05600AD03BA050D00420000013316150 62B010607060F012116150623210721161506232114171633 1
417161506232227222726352635232635363B01363F012726272635363317333637363336370 2F64040
0838343A564C2820019C400838FE282001F4400838FE00B4304C441C0A322438566290306040083864
0616040454442408388C28104479332E5E050D0838400133264A4C083840600838405E861C0B0D1B
1D4014507F1D39570838400157040410041824401011 9F84290F0001006400A0023C03940019000001
1615140F01153217161F010623222726272635343F0136373601FC40A890226E77210408381F4D4157
886C685A2A15039408382FD1800468442C18403C25533E3E2B555C7147300000000 1006400A0023C0
3940019000012171 61F011615140706070623222737363736333527263534 37CF152A5A686C8857414
D1F38080421776E2290A84003943047715C552B3E3E53253C40182C44680480D12F38080000000001
007FFFF90383059900330000011 61D011333363316151423072316172116150623211 21D010623263D
01340323223D013633173303232722273633173503353601BF40088C3339405C449009130118400838
FEF8340838403458A00A3264441804D42D0B0B2DD008080599083820FEE40C0838400CB15B083840
FEF34B80400838803A011E3C0C4008010C044040040401182040000000001002F022A0147033E00100
000133332153217140F012227222F01343736B9265011075117123A312E053E20033E6E383B2E051E5
51A3520320000000010056FF250162019900140000172326353633373 63D012227363B01321F01140706
B620400838301C4014093F204C1C04581FDB0838407046661C5C608C4CAD9B54000002005DFED40
2A1013C001C00300000012635343734373637323735072326353633321F011514071407062306252635
343736373523222736333217150607062301 D140302810180C04041040193B41330C242C3F0D20FEB
84050160A083D0F153B5B29281C4420FED408382C140A2E1F5144640408385C5038605E4623495C1
41008381D674B0D78605C74D07D276800070069006A04EC04AA00190033003F00520065006D007400
00011615140F010607060706231407263536373637363 7F0136053217141F021407140723222 7222
F01373437363B011733341706150733321733352734230 1321F011514231407232272227353437323 7
051737161F011514070623222726 3D013633360515173335272306051517373527060377 40A0649D0F3
2A645072440 09BB581C02BA2B496415FDCD2050282C0820584061172C30041C4015170C040414181
8040F196440140323325E144028401850260E4C2113FE6818242F550460290F6751141B1D3E01C230
141C0417FE3B3C28283C04AA08382DE788AB3147B570090308384EBE6D330ED2495FAC10483C112
740383824161A1C4C3854313710040B770D23381410540CFE285C4040A00B05385C803226180C0404
086418602C5C18602711207C5088541C681810500C2034302433FFFF0028FF6403FC0718022002 40
096000300D300560160FFFF000E00000416070A022200280000000300D300740152FFFF0028FF7703F

C070D0222002400A90003008D00970159FFFF000E000041607070222002800000003008E00910146F
FFF000E000041607070222002800000003004300DD012BFFFF002FFFA3038F07140222002C29A300
03008DFFEB0160FFFF001AFF95037A070B0222002C1495000300D3FFE60153FFFF00060000036607
000222002C00000003008E0027013FFFFF002EFFB0038E07070222002C28B0000300430053012BFFF
F0056FFCB045A06FA0222003200CB0003008D00E70146FFFF005DFFBD046107040222003207BD000
300D300CC014CFFFF005DFF8F046106DE02220032078F00030043017E0102FFFF0036FFF2035006F
2022200380000000003008D006E013EFFFF0036FFF203500718022200380000000300D300440160FFFF0
036FFF203500700022200380000000030043006A012400010078FFDF011C0400001400001316 1D01071
11715170706232237263D0127113633C4400C1410100838471B1414062E04000838204CFE944C74DB
2E40C5663250500164BC0000000100570424 02AB05B800160000013217161716150623222726270706
232635343F01363701DB2E162A4A1808382B194616C84F1D403490495B05B8387153181C40344844A
44008382B19783163000000010065043E038105460024000001331 6171617163B01363F011615140706
2B012227222F012306072207062326 3D013437360175202B5D302C3B19081C2C244068271120378110
3C38047A061C041824406816054603350B25201B41080838345C145024141117342408382032 3E1900
02007803F502500571001300200000011617161514070 62B012627263D01343F0136330715173337352
623272306150601685F31583C38306047612C3034164A405C4830283C0404381605710C24442C77254
008382F1D2022364824D0041C3014340411072A0000010049FECE017D00D6001A0000251615073217
140722070623263D013437363D012326353437363701254014 1D0F9C1513161A4044702840481F11D6
083844788F4128140838203B052F2D2008382B45370100000100490 3FC027105740019000013321716
173437323536331615062306071407222726272632653689 2E162187780818244019 1B51333C30683C3C
24080574380A96146C0C24083854454B210B804527321A4000FFFF0035FFB6037D07090222003600B
6000300D7006C0195FFFF0021FFB1030D05D10222005600000000200D7275D0000FFFF0021FFD103F
C07090222003D0000000300D7008A0195FFFF001BFFC803FC05E50222005D0000000200D77271000
0000200DCFF7A01780606000F001D000001161514071507150623263D0137353613161D01171506232
63D012735360138401 00C0838400C151F400C0838400C08060608381715D0A0A0400838A0A0E05CF
BF40838A0A0C0400838C0A0A04000000002004A006B03FA053F0030004A000001161D013733373332
17161F011514070607060706232227343335332735230607062326353437363736 3D01342F013536172
3061D01161D0136371615140F011136373637363D01022B01013E401C348C60466A503C044C7D9B6
5CB3F212D0B580404042B4115174048293F2C200808B4102420AD4B4070C8A781304C30488468053
F0838080410545FCD4CC026A67553393710404408 3CC40D2F100838311B26120A0A4C9BED684040
C4070920C8B4282C0C0838410B30FEF0315F184C7410D4012C0000030064000003B004980021003F
00580000011615140706071716150623222F010623263534373633262726352227363332173603321736
3332171417161D01140706230723202F013437363736373237361723070607060F0115163B013237363
F013534273427263506028040383A12BC180A322E6A7090244034410B021E600B050838199F8E4223
1115135379403C5079375080FED149041C19378C20092F2236204C1C404B110432CA743755300C145
85C4814049808382E16271544181C4030287808382B1934090B360628405C84FE401C0C6C07355331
4077755C14C84861173F2DA212280880342A464331045864301F2D40542F3D1B2912120C00FFFF002
8FF4B034407000222003C00CA0003008D0045014CFFFF0014FE4802F105A00222005C00000002008
D40EC000000020050FFF9034C0579002400330000131615071533321732171617321715140706071 40
52315062307263D013735273711271136131517113336373637352726272623984008B49D731359152B
09033C2567FED474062E18400C08081409771458F81C641828133571530579083 82CC0687402A2244
0791B291F1814A8BC040838204CD02C2C012CE0014060FE5404D0FEE80917202058840B4D580002
0057FE70031B05100030003E000013321513153332171615171407060706072315060715070623263 53
437353637352326353 63B010335232635363335033413151333253637353427262326 23B7441C80CD2
F84047C5389057F18100C1018245C280C1008400838080808400838 20A8080C01043D27482E3A153F
051070FED00468DA6E3C3D573414091B7041BF205C24163A241C10BC505C0838400138BC0838400
401682DFDE7BCFED0441E2E04617F700800000000010057017C03B702400015000013160533173316
171615062322272327232427263536976001004 02CB433492408383933B42C34FECD511C0802401818
080C041824400C081C1C1B1D400000010050009903380 3A9002F000001161514070607151714171417
161506232227262707140706 2B012635343734373637262726353633321716173736370 2F8404402B28
43020140838375D3E2E885060140C405068017F20C4300838232D9A3E88453303A9083820441BA904
6C093B0612161A40882828880A427008382947085C08781C80321E4038623288632D00000100710334
01F105BC001D000001161D011615173316150623212635363B01372F010623263534373637340131400
0C0430400838FF00400838480404041E1E4030074105BC083880A89C040838400838401 0DC1C20083
81D3316422D000000000100A0034802B805C4002A00000116153217321D011407060F011533321D010

623272126353437363F0135262B01060706232635343736330160B01D1710206A3220A0A00A3264FF0
05C8832421C0B69200533141440281C5405C40917346420601C701814043C0C4008193B386C1E3E20
70101C500C08381D4358000100AE035502A605D5003000000133321F0114073217161D01140F01140
7232635363B01373237323734272627263534373437352723220F012635363334016240932114343 62E
0C5C7844A04008389460210B07115421471878141C80191F1C40184005D554542329741414202A6230
090B08384024201C38141315181C3C1C09031004240408385C0C000003006A000004520554001D003
500570000011615111733161506 2B0107222734333527352306232635343734373536051615140706070
607060706232635363F023637363736033332151615140F013316150623210722273437363F01352322
070623263536333401 4E40082440083854742D0B5C0804181C4034440802B840B88331296F8F7D193
B40046C84B43A5EB44815A5207C146C18D0400838FEEC542D0B6023450C281808181C401C240554
0838FE6C280838401040440424B41808382B190A4E284018083828F8B0504D8FCBC57008384B95C8
F86F79F3750CFD5434155B538D300838400C4024684163381820180838 5C100003006BFFFA048B056
E001D00350058000001161511173316150 62B0107222734333527352306232635343734373536051615
140706070607060706232635363F023637363736131615113215062B0127161D010623263D013427232
2352227353633161D0117331136014F40082440083854742D0B5C0804181C4034440802B840B883312
96F8F7D193B40046C84B43A5EB448152740680838041 40C0838400C68B807010838402870080 56E08
38FE6C280838401040440424B41808382B190A4E284018083828F8B0504D8FCBC57008384B95C8F8
6F79F3750CFDE40838FEDC4440044389504008385 0834D301C804008384804012040000 3007FFFD10
5030565002D004B006B000001321F011506231516151714071407060726353437363532373527230722
35343F01353423220F012326353437340516151407140714010306150607062326353437363736013437
363736330316150711321 7140F01150623263D013723263537353633161507150733113602 1F3B35141
3255C04908C4282406084556 7106C3C649064103C4844084098032840 60B0FE64E8442711181C4058
7C28CC0138AC024A1F296840082D0B3C0808384008E45C20093F40080CB00905654840205C083642
0C6D57112B1A1208383F091113700C1010443E2A4C0418201C083841270E020838267220A818FE68F
EF4341C3F391808382C947642DC01380EB21A5244FDB408382CFEDC4030146C804008388064153B
70706008382C7430013C600000020079014D038D043D0039004B000001321737363316151 40F01161D
010607161716150623222722273523062322270623060726353437363526352736 3F0126352736333217
363336031516333 63F013527262F010727230607060215336D701513406034380B293C20240838271D
14440447414B611C2C313340443840041C14046C040838275950581 1BD6F7D73110C2C172D10242C
20481C12043D70500C083830302C742C2445333C102E1A403054083C341825070838321617093 72D3
8781C0C640C1440604C1CFE88149449172C1074174D0C08082D371C0001004A04BD044A056500100
0001333051733321714072327252326353437D6E40190A02038084020A0FE70F0804005550C0C40380
80C0C0D43380800000028 01E60001000000000000050000000100000000000100080057000100000 0
000002000700500001000000000030015005700010000000000400080057000100000000005002E00
6C000100000000000600080057000100000000000A0040009A000300010403000 2000C02CA00030001
04050002001000DA000300010406000 2000C00EA000300010407000 2001000F600030001040800020 0
100106000300010409000000700116000300010409000100100194000300010409000 2000E0186000300
0104090003002A019400030001040900040010019400030001040900050 05C01BE00030001040900060
0100194000300010409000A0080021A00030001040A0002000C02CA00030001040B00020010029A000
30001040C0002000C02CA00030001040E0002000C02E800030001041000 02000E02AA000300010413
0002001202B800030001041400 02000C02CA000300010415000 2001002CA000300010416000 2000C02
CA000300010419000 2000E02DA00030001041B0002001002E800030001041D0002000C02CA0003000
1041F0002000C02CA000300010424000 2000E02F800030001042D0002000E030600030001080A00020
00C02CA000300010816000 2000C02CA000300010C0A0002000C02CA000300010C0C0002000C02CA
4F736D6F7369616E20536C6F70707920A92032303035 2052656C6174696F6E616C2053797374656D7
320436F72706F726174696F6E20323030352E20416C6C20526967687473205265736572766564 2E526
567756C61726F736D6F7369616E3A56657273696F6E20312E303056657273696F6E20312E3030204E
6F76656D62657220362C20323030352C20696E697469616C2072656C656173656546869732066 6F6E74
207761732063726561746564207573696E6720466F6E742043726561746F7220352E302066726F6D204
86967682D4C6F6769632E636F6D006F00620079010D0065006A006E00E9006E006F0072006D006100
6C005300740061006E006400610072006403 9A03B103BD03BF03BD03B903BA03AC004F0073006D00
6F007300690061006E002000A900200032003000300036002000540068006500200 04F0073006D006F0
073006900610 06E0020004F007200640065007 2002E00200041006C006C002000520069006700680074
007300200052006500730065007200760065006 4002E005200650067007 5006C00610072006F0073006
D006F007300690061006E003A005600650072007 30069006F006E00200033002E00300030005600650

07200730069006F006E00200033002E003000300020004E006F00760065006D00620065007200200036
002C00200032003000300035002C00200069006E006900740069006100 6C002000720065006C006500
610073006500540068006900730020006600 6F006E00740020007700610073002000630072006500610
074006500640020007500730069006E006700200046006F006E00740020004300720065006100 74006F
007200200035002E003000200066007200 6F006D002000480069006700 68002D004C006F0067006900
63002E0063006F006D004E006F0072006D00610061006C0069004E006F0072006D0061006C0065005
300740061006E006400610061007200 64004E006F0072006D0061006C006E0079041E0431044B04470
43D044B0439004E006F0072006D00E1006C006E0065004E006100760061006400 6E006F0041007200
720075006E007400610000000200000000000FF270096000000000000000000000000000000000000
00000ED000001020103000300040005000600070008000900 0A000B000C000D000E000F00100011001
200130014001500160017001800190 01A001B001C001D001E001F002000210022002300240025002600
270028002900 2A002B002C002D002E002F003000310032003300340035003600370038003900 3A003
B003C003D003E003F004000410042004300440045004600470048004900 4A004B004C004D004E004F
005000510052005300540055005600570058005900 5A005B005C005D005E005F006000610062006300
640065006600670068006900 6A006B006C006D006E006F007000710072007300740075007600770078
0079007A007B007C007D007E007F008000810082008300840085008600870088008900 8A008B008C00
8D008E008F009000910092009300940095009600970098009900 9A009C009D009E009F00A000A100A
200A300A400A500A600A700A800A900AA00AB00AD00AE00AF00B000B100B200B300B400B500B600
B700B800B900BA00BB00BC010400BE00BF00C200C300C400C500C600C700C800C900CA00CB00C
C00CD00CE00CF00D000D100D300D400D500D600D700D800D900DD00DE00E100E400E500E600E7
00E800E900EA00EB00EC00ED00EE00EF00F001050106010700F400F500F600BD00DA052E6E756C6
C106E6F6E6D61726B696E6772657475726E044575726F07756E693030423907756E693030423207756
E693030423300000001FFFF0002.

To outdent any selected rows in a text:
If the text is nil, exit.
Loop.
Get a row from the text's rows.
If the row is nil, exit.
If the row of the text is not selected, repeat.
If the row's string's first's target is the space byte, remove the first byte from the row's string.
If the row's string's first's target is the space byte, remove the first byte from the row's string.
Repeat.

To outdent a box some twips;
To outdent a box given some twips:
Subtract the twips from the box's left.
Subtract the twips from the box's top.
Add the twips to the box's right.
Add the twips to the box's bottom.

An outdent is a number.

To outline a box with a color:
Draw the box with the color and the clear color.

A outlinetextmetric is a record with
A number called otmsize,
A textmetric called otmtextmetrics,
3 bytes, \ needed to align structure
A byte called otmfiller,
A panose called otmpanosenumber,
1 bytes, \ needed to align structure
A number called otmfsselection,
A number called otmfstype,

A number called otmscharsloperise,
A number called otmscharsloperun,
A number called otmitalicangle,
A number called otmemsquare,
A number called otmascent,
A number called otmdescent,
A number called otmlinegap,
A number called otmscapemheight,
A number called otmsxheight,
A box called otmrcfontbox,
A number called otmmacascent,
A number called otmmacdescent,
A number called otmmaclinegap,
A number called otmusminiumppem,
A spot called otmptsubscriptsize,
A spot called otmptsubscriptoffset,
A spot called otmptsuperscriptsize,
A spot called otmptsuperscriptoffset,
A number called otmsstrikeoutsize,
A number called otmsstrikeoutposition,
A number called otmsunderscoresize,
A number called otmsunderscoreposition,
A pointer called otmpfamilyname,
A pointer called otmpfacename,
A pointer called otmpstylename,
A pointer called otmpfullname.

To output the arc of an ellipse given a string:
Put 2761/10000 into a fraction. \ 2/3*(sqrt(2)-1)
Put the ellipse’s center into a center spot.
Put the ellipse’s x-extent divided by 2 into a half width.
Put the ellipse’s y-extent divided by 2 into a half height.
Put the ellipse’s x-extent times the fraction into an x offset.
Put the ellipse’s y-extent times the fraction into a y offset.
\ control point 1
If the string is "left-top", put the ellipse’s left and the center’s y minus the y offset into a first control spot.
If the string is "right-top", put the center’s x plus the x offset and the ellipse’s top into the first control spot.
If the string is "right-bottom", put the ellipse’s right and the center’s y plus the y offset into the first control spot.
If the string is "left-bottom", put the center’s x minus the x offset and the ellipse’s bottom into the first control spot.
\ control point 2
If the string is "left-top", put the center’s x minus the x offset and the ellipse’s top into a second control spot.
If the string is "right-top", put the ellipse’s right and the center’s y minus the y offset into the second control spot.
If the string is "right-bottom", put the center’s x plus the x offset and the ellipse’s bottom into the second control spot.
If the string is "left-bottom", put the ellipse’s left and the center’s y plus the y offset into the second control spot.
\ ending point
If the string is "left-top", put the ellipse’s left plus the half width and the ellipse’s top into an ending spot.
If the string is "right-top", put the ellipse’s right and the ellipse’s top plus the half height into the ending spot.
If the string is "right-bottom", put the ellipse’s right minus the half width and the ellipse’s bottom into the

ending spot.
If the string is "left-bottom", put the ellipse’s left and the ellipse’s bottom minus the half height into the ending spot.
\ spit it out
Output the first control spot without advancing.
Output the second control spot without advancing.
Output the ending spot without advancing.
Output "c".

To output a color without advancing:
Convert the color to a rgb.
Put the rgb’s red byte / 255 into a fraction.
Convert the fraction to a red string given 4.
Put the rgb’s green byte / 255 into the fraction.
Convert the fraction to a green string given 4.
Put the rgb’s blue byte / 255 into the fraction.
Convert the fraction to a blue string given 4.
Output the red string then " " then the green string then " " then the blue string without advancing.

To output lineto given a spot:
Output the spot without advancing.
Output "l".

To output lineto given an x number and a y number:
Put the x and the y into a spot.
Output lineto given the spot.

To output moveto given a spot:
Output the spot without advancing.
Output "m".

To output moveto given an x number and a y number:
Put the x and the y into a spot.
Output moveto given the spot.

To output a number without advancing:
Convert the number to a string.
Output the string without advancing.
Output " " without advancing.

To output the pdf border given a color:
If the color is the pdf state’s current border, exit.
Output the color without advancing.
Output " RG".
Put the color into the pdf state’s current border.

To output the pdf fill given a color:
If the color is the pdf state’s current fill, exit.
Output the color without advancing.
Output " rg".
Put the color into the pdf state’s current fill.

To output setcolor given a border color and a fill color:
If the fill is not clear, output the pdf fill given the fill.
If the border is not clear, output the pdf border given the border.

To output a spot without advancing:
Output the spot's x without advancing.
Output the pdf state's current height minus the spot's y without advancing.

To output a string:
Append the string to the pdf state's current contents.

To output a string without advancing:
Append the string to the pdf state's current contents without advancing.

To output stroke and fill given a border color and a fill color:
Put "B" into a string. \ stroke and fill
If the fill is clear, put "S" into the string. \ stroke
If the border is clear, put "f" into the string. \ fill
Output the string.

The p-key is a key equal to 80.

A pabc is a pointer to an abc.

The page-down key is a key equal to 34.

The page-up key is a key equal to 33.

A paintstruct is a record with
An hdc called hdc,
A number called ferase,
A box called rcpaint,
A number called frestore,
A number called fincupdate,
32 bytes.

A pair has an x number and a y number.

A panose is a record with
A byte called bfamilytype,
A byte called bserifstyle,
A byte called bweight,
A byte called bproportion,
A byte called bcontrast,
A byte called bstrokevariation,
A byte called barmstyle,
A byte called bletterform,
A byte called bmidline,
A byte called bxheight.

The paragraph byte is a byte equal to 182.

A pastel color is a color.

A path is a string. \ complete name = c:\folder1\folder2\file.ext

The pause key is a key equal to 19.

A pchar is a byte pointer.

A pdevmode is a pointer to a devmode.

A pdf is a buffer.

A pdf object is a thing with
A kind [contents, font definition, font descriptor, font streamoutline, image object, outline entry, page, parent, root],
A number,
An offset,
A data buffer,
A font name [font definition],
A font info [font definition],
Some string things called font strings [page],
Some string things called image strings [page].

A pdf outline entry is a thing with
A pdf object (reference),
A title string,
A page height,
A destination number.

A pdf pointer is a pointer to a pdf.

A pdf state has
A pdf pointer,
A document flag,
A page flag,
An object number,
Some pdf objects called objects,
An xref offset,
An outline pdf object (reference),
Some pdf outline entries called outline entries,
A root pdf object (reference),
A parent pdf object (reference),
A current contents pdf object (reference),
A current page pdf object (reference),
A current height,
A current border color,
A current fill color,
A font index.

The pdf state is a pdf state.

A pdf string is a string. \ string surrounded by ( ) and has \(for left paren, \) for right paren, \\ for backslash

An pen is a color.

The pen size is a number.

The per-mille-sign byte is a byte equal to 137.

A percent is a number. \ a scale with 100 in the denominator

The percent-sign byte is a byte equal to 37.

The period byte is a byte equal to 46.

To pick a brightness between a percent and another percent;
To vary a lightness between a percent and another percent;
To pick a lightness between a percent and another percent:
Pick a number between the percent and the other percent.
Put the number times 10 into the lightness.
Put the lightness into the context's lightness.

To pick a brownish color:
Pick the brownish color's hue between 250 and 350.
Pick the brownish color's saturation between 500 and 1000.
Pick the brownish color's brightness between 125 and 375.
Put the brownish color into the context's color.

To pick a brownish color about some percent of the time: \ *** generalize this for all colors
Pick a number between 1 and 100.
If the number is greater than the percent, exit.
Pick the brownish color.
Put the brownish color into the context's color.

To pick a color:
Pick the color's hue between 0 and 3600.
Pick the color's saturation between 0 and 1000.
Pick the color's lightness between 0 and 1000.
Put the color into the context's color.

To pick a color between another color and a third color:
Pick the color's hue between the other color's hue and the third color's hue.
Pick the color's saturation between the other color's saturation and the third color's saturation.
Pick the color's lightness between the other color's lightness and the third color's lightness.
Put the color into the context's color.

To pick a color like another color:
Put the other color into the color.
Pick a number between -100 and 100.
Add the number to the color's hue.
Limit the color's hue to 0 and 3599.
Set the color's saturation to something between 100 and 1000.
Set the color's lightness to something between 0 and 800.
Put the color into the context's color.

To pick a dark color:
Pick the dark color's hue between 0 and 3599.
Put 1000 into the dark color's saturation.
Put 375 into the dark color's lightness.
Put the dark color into the context's color.

To pick a greenish color:
Pick the greenish color's hue between 900 and 1200.
Pick the greenish color's saturation between 500 and 1000.
Pick the greenish color's brightness between 250 and 875.

```
Put the greenish color into the context's color.

To pick a greenish color about some percent of the time: \ *** generalize this for all colors
Pick a number between 1 and 100.
If the number is greater than the percent, exit.
Pick the greenish color.
Put the greenish color into the context's color.

To pick a heading:
Pick the heading between 0 and 3839.
Put the heading into the context's heading.

To pick a letter height between some twips and some other twips:
Pick a random number between the twips and the other twips.
Put the random number into the letter height.
Put the random number into the context's letter height.

To pick a letter of the alphabet:  \ put letter into context? ***
Pick a number between 65 and 90.
Put the number into the letter.

To pick a light color:
Pick the light color's hue between 0 and 3599.
Put 1000 into the light color's saturation.
Put 625 into the light color's lightness.
Put the light color into the context's color.

To pick a number:
Pick the number between 0 and the largest number.
Put the number into the context's number.

To pick a number within an amount of another number:
Pick the number between the other number minus the amount and the other number plus the amount.
Put the number into the context's number.

To pick a pastel color:
Pick the pastel color's hue between 0 and 3599.
Put 1000 into the pastel color's saturation.
Put 875 into the pastel color's lightness.
Put the pastel color into the context's color.

To pick a rainbow color:
Add 1 to the current rainbow color number.
If the current rainbow color number is greater than 6, put 1 into the current rainbow color number.
If the current rainbow color number is 1, put the red color into the rainbow color.
If the current rainbow color number is 2, put the orange color into the rainbow color.
If the current rainbow color number is 3, put the yellow color into the rainbow color.
If the current rainbow color number is 4, put the green color into the rainbow color.
If the current rainbow color number is 5, put the blue color into the rainbow color.
If the current rainbow color number is 6, put the purple color into the rainbow color.
Put the rainbow color into the context's color.

To pick a solid color:
Pick the solid color's hue between 0 and 3599.
Put 1000 into the solid color's saturation.
```

```
Put 500 into the solid color's lightness.
Put the solid color into the context's color.

To pick a spot anywhere in the bottom a fraction of a box;
To pick a spot in the bottom a fraction of a box:
Privatize the box.
Put the box's height times the fraction into some twips.
Put the box's bottom minus the twips into the box's top.
Pick the spot in the box.

To pick a spot anywhere in a box:
Pick the spot's x between the box's left and the box's right.
Pick the spot's y between the box's top and the box's bottom.
Put the spot into the context's spot.

To pick a spot anywhere in the middle a fraction of a box;
To pick a spot in the middle a fraction of a box:
Privatize the box.
Put the box's center's y into a coord.
Put the box's height times the fraction divided by 2 into a number.
Put the coord minus the number into the box's top.
Put the coord plus the number into the box's bottom.
Pick the spot in the box.

To pick a spot anywhere in the top half of a box;
To pick a spot in the top half of a box:
Privatize the box.
Put the box's center's y into the box's bottom.
Pick the spot in the box.

To pick a spot anywhere in the top middle a fraction of a box;
To pick a spot in the top middle a fraction of a box:
Privatize the box.
Put the box's center's y into a coord.
Put the box's height times the fraction into a number.
Put the coord minus the number into the box's top.
Put the coord into the box's bottom.
Pick the spot in the box.

To pick a spot anywhere on a horizontal line;
To pick a spot on a horizontal line:
Pick the spot's x between the horizontal line's start's x and the horizontal line's end's x.
Put the horizontal line's y into the spot's y.

To pick a spot in a box:
Pick the spot's x between the box's left and the box's right.
Pick the spot's y between the box's top and the box's bottom.
Put the spot into the context's spot.

To pick a spot in a box about some twips above the middle;
To pick a spot in a box about some twips above the center:
Put the box into a bounding box.
Put the twips divided by 2 into some other twips.
Put the box's center's y minus the other twips into the bounding box's bottom.
Put the bounding box's bottom minus the twips into the bounding box's top.
```

Pick the spot anywhere in the bounding box.

To pick a spot in a box about some twips below the middle;
To pick a spot in a box about some twips below the center:
Put the box into a bounding box.
Put the twips divided by 2 into some other twips.
Put the box's center's y plus the other twips into the bounding box's top.
Put the bounding box's top plus the twips into the bounding box's bottom.
Pick the spot anywhere in the bounding box.

To pick a spot in a box some twips to some other twips above the middle;
To pick a spot in a box some twips to some other twips above the center:
Put the box into a bounding box.
Put the box's center's y minus the twips into the bounding box's bottom.
Put the bounding box's bottom minus the other twips into the bounding box's top.
Pick the spot anywhere in the bounding box.

To pick a spot in a box some twips to some other twips below the middle;
To pick a spot in a box some twips to some other twips below the center:
Put the box into a bounding box.
Put the box's center's y plus the twips into the bounding box's top.
Put the bounding box's top plus the other twips into the bounding box's bottom.
Pick the spot anywhere in the bounding box.

To pick a spot within a distance of another spot:
Pick the spot's x within the distance of the other spot's x.
Pick the spot's y within the distance of the other spot's y.
Put the spot into the context's spot.

To pick some twips between some min twips and some other twips; \ are all these necessary? ***
To pick a number between some min twips and some other twips;
To pick a number between a min number and a max number;
To pick a number from a min number to a max number;
To set a number to something between another number and a third number;
To pick a random number between a min number and a max number:
Put the seed's whereabouts into eax.
\ put address of randseed into ecx
Intel $8BC8. \ mov ecx,eax
\ calculate zero based max
Intel $8B8510000000. \ mov eax,[ebp+16] \ the max
Intel $8B00. \ mov eax,[eax]
Intel $8B9D0C000000. \ mov ebx,[ebp+12] \ the min
Intel $2B03. \ sub eax,[ebx]
Intel $40. \ inc eax
\ adjust randseed
Intel $691105840808. \ imul edx,[ecx],134775813
Intel $42. \ inc edx
Intel $8911. \ mov [ecx],edx
\ mul adjusted randseed by the zero based max
Intel $F7E2. \ mul edx
\ add the min to un-zero base the number
Intel $0313. \ add edx,[ebx] the min
\ store the result
Intel $8B9D08000000. \ mov ebx,[ebp+08] \ the random number
Intel $8913. \ mov [ebx],edx

Put the random number into the context’s number.

To pick a very dark color;
To pick a really dark color:
Pick the really dark color’s hue between 0 and 3599.
Put 500 into the really dark color’s saturation.
Put 250 into the really dark color’s lightness.
Put the really dark color into the context’s color.

To pick a very light color;
To pick a really light color:
Pick the really light color’s hue between 0 and 3599.
Put 1000 into the really light color’s saturation.
Put 750 into the really light color’s lightness.
Put the really light color into the context’s color.

To pick a very very dark color;
To pick a really really dark color:
Pick the really really dark color’s hue between 0 and 3599.
Put 500 into the really really dark color’s saturation.
Put 125 into the really really dark color’s lightness.
Put the really really dark color into the context’s color.

To pick a very very light color;
To pick a really really light color:
Pick the really really light color’s hue between 0 and 3599.
Put 1000 into the really really light color’s saturation.
Put 875 into the really really light color’s lightness.
Put the really really light color into the context’s color.

A picture is a thing with
\ all boxes are in twits
A box [location of cropped picture on the page],
A uncropped box [location of entire picture on the page],
A grayscale flag,
A mirror flag,
A rotate angle, \ rotation is clockwise
A hex string called data [original bytes in original format],
A gpbitmap.

The pink color is a color.

The pink pen is a pen.

A pixel is 15 twips.

The pizza pie is a fraction equal to 355/113.

To play a wave:
Call "winmm.dll" "PlaySound" with the wave’s first and 0 and 5 [snd_memory+snd_async].

To play a wave and wait:
Call "winmm.dll" "PlaySound" with the wave’s first and 0 and 4 [snd_memory+snd_sync].

To play a wave file:

Privatize the wave file.
Null terminate the wave file.
Call "winmm.dll" "PlaySound" with the wave file's first and 0 and 131073 [snd_filename+snd_async].

To play a wave file and wait:
Privatize the wave file.
Null terminate the wave file.
Call "winmm.dll" "PlaySound" with the wave file's first and 0 and 131072 [snd_filename+snd_sync].

The plus-or-minus byte is a byte equal to 177.

The plus-sign byte is a byte equal to 43.

A point is a number [0 to 3839; for dividing a circle into compass points; 0 is noon; 960 points per quarter].

A pointer has 4 bytes.

A polygon is a thing with some vertices.

A portrait sheet is a sheet.

A position is a pair with a column# and a row#.

To position a substring on a string:
Intel $8B8508000000. \ mov eax,[ebp+8] \ the substring
Intel $8B9D0C000000. \ mov ebx,[ebp+12] \ the string
\ put the string's first into the substring's first
Intel $8B8B00000000. \ mov ecx,[ebx+0] \ the string's first
Intel $898800000000. \ mov [eax+0],ecx \ the substring's first
\ put the string's first minus 1 into the substring's last
Intel $49. \ dec ecx
Intel $898804000000. \ mov [eax+4],ecx \ the substring's last

To post a data string to a url and receive a response string: \ only works with http and https
Clear the response.
Clear the i/o error.
Create a winhttp request for posting to the url.
If the i/o error is not blank, exit.
Send the data to the winhttp request.
If the i/o error is not blank, destroy the winhttp request; exit.
Receive the response from the winhttp request.
If the i/o error is not blank, destroy the winhttp request; exit.
Read the response from the winhttp request.
If the i/o error is not blank, destroy the winhttp request; exit.
Destroy the winhttp request.

The pound-sign byte is a byte equal to 163.

A poutlinetextmetric is a pointer to an outlinetextmetric.

The ppi is some twips.

A precise degree is a number [0 to 3599].

To prepend a byte to a string:
Put the string's length into a saved length.
Reassign the string's first given the string's length plus 1.
Copy bytes from the string's first to the string's first plus 1 for the saved length.
Put the string's first plus the saved length into the string's last.
Put the byte into the string's first's target.

To prepend a string to another string:
Put the other string's length plus the string's length into a combined length.
Reassign a pointer given the combined length.
Put the pointer into a substring's first.
Copy bytes from the string's first to the substring's first for the string's length.
Add the string's length to the substring's first.
Copy bytes from the other string's first to the substring's first for the other string's length.
Unassign the other string's first. \ to avoid extra allocates and copies
Put the pointer into the other string's first.
Put the other string's first plus the combined length minus 1 into the other string's last.

To prepend a thing to some things:
If the thing is nil, exit.
Put the things' first into the thing's next.
If the things is not empty, put the thing into the things' first's previous.
If the things are empty, put the thing into the things' last.
Put the thing into the things' first.

To prepend some things to some other things:
Get a thing from the things (backwards).
If the thing is nil, exit.
Remove the thing from the things.
Prepend the thing to the other things.
Repeat.

The print-screen key is a key equal to 44.

A printdlgex is a record with
A number called lstructsize,
A window called hwndowner,
A handle called hdevmode,
A handle called hdevnames,
A canvas called hdc,
A number called flags,
A number called flags2,
A number called exclusionflags,
A number called npageranges,
A number called nmaxpageranges,
A pointer called lppageranges,
A number called nminpage,
A number called nmaxpage,
A number called ncopies,
A handle called hinstance,
A pointer called lpprinttemplatename,
A pointer called lpcallback,
A number called npropertypages,
A pointer called lphpropertypages,
A number called nstartpage,

A number called dwresultaction.

The printer canvas is a canvas.

The printer device mode handle is a handle.

A process is a handle.

A process pointer is a pointer to a process.

A processinfo is a record with
A handle called hprocess,
A handle called hthread,
A number called dwprocessid,
A number called dwthreadid.

A punch line is a string.

The purple color is a color.

The purple pen is a pen.

To put the actual data of a font into a buffer: \ only works with true/open type fonts
Create the hfont of the memory canvas given the font.
Call "gdi32.dll" "GetFontData" with the memory canvas and 0 and 0 and nil and 0 returning a count.
Reassign the buffer's first given the count.
Call "gdi32.dll" "GetFontData" with the memory canvas and 0 and 0 and the buffer's first and the count.
Put the buffer's first plus the count minus 1 into the buffer's last.
Destroy the hfont of the memory canvas.

To put the bottom of a box into a horizontal line;
To put the bottom edge of a box into a horizontal line;
To put the bottom side of a box into a horizontal line:
Put the box's left-bottom into the horizontal line's start.
Put the box's right-bottom into the horizontal line's end.

To put the bottom of a box into a line:
Make the line with the box's left-bottom and the box's right-bottom.

To put a box and a radius into a roundy box:
Put the box's left into the roundy box's left.
Put the box's top into the roundy box's top.
Put the box's right into the roundy box's right.
Put the box's bottom into the roundy box's bottom.
Put the radius into the roundy box's radius.

To put a box in the center of another box;
To center a box in another box:
Center the box in the other box (horizontally).
Center the box in the other box (vertically).

To put a box in the center of the screen;
To center a box on the screen:
Center the box in the screen's box.

To put a box into another box:
Put the box's left into the other box's left.
Put the box's top into the other box's top.
Put the box's right into the other box's right.
Put the box's bottom into the other box's bottom.

To put a box on a spot;
To center a box on a spot:
Get a difference between the spot and the box's center.
Round the difference to the nearest multiple of the tpp.
Move the box given the difference.

To put a box some twips by some other twips in the center of another box;
To center a box some twips by some other twips in another box:
Make the box the twips by the other twips.
Center the box in the other box.

To put a box's bottom line into a horizontal line: \ and "vertical" for left and right
Put the box's left and the box's bottom into the horizontal line's start.
Put the box's right and the box's bottom into the horizontal line's end.

To put a box's bottom-center into a spot: \ *** need these without dashs too
Put the box's center's x into the spot's x.
Put the box's bottom into the spot's y.

To put a box's center into a spot:
Put the box's left plus the box's right into the spot's x.
Put the box's top plus the box's bottom into the spot's y.
Divide the spot by 2.

To put a box's center-bottom into a spot:
Put the box's center's x into the spot's x.
Put the box's bottom into the spot's y.

To put a box's center-top into a spot:
Put the box's center's x into the spot's x.
Put the box's top into the spot's y.

To put a box's height into a height:
Put the box's bottom into the height.
Subtract the box's top from the height.
Add the tpp to the height.

To put a box's left line into a line:
Put the box's left and the box's top into the line's start.
Put the box's left and the box's bottom into the line's end.

To put a box's left-bottom into a spot:
Put the box's left into the spot's x.
Put the box's bottom into the spot's y.

To put a box's left-center into a spot:
Put the box's left into the spot's x.
Put the box's center's y into the spot's y.

To put a box's right line into a line:
Put the box's right and the box's top into the line's start.
Put the box's right and the box's bottom into the line's end.

To put a box's right-center into a spot:
Put the box's right into the spot's x.
Put the box's center's y into the spot's y.

To put a box's right-top into a spot:
Put the box's right into the spot's x.
Put the box's top into the spot's y.

To put a box's top line into a horizontal line:
Put the box's left and the box's top into the horizontal line's start.
Put the box's right and the box's top into the horizontal line's end.

To put a box's top-center into a spot:
Put the box's center's x into the spot's x.
Put the box's top into the spot's y.

To put a box's width into a width:
Put the box's right into the width.
Subtract the box's left from the width.
Add the tpp to the width.

To put a box's x-extent into a width:
Put the box's right into the width.
Subtract the box's left from the width.

To put a box's y-extent into a height:
Put the box's bottom into the height.
Subtract the box's top from the height.

To put a byte and a number into a fraction:
Put the byte into the fraction's numerator.
Put the number into the fraction's denominator.

To put a byte into another byte:
Intel $8B8508000000. \ mov eax,[ebp+8] \ the byte
Intel $8A00. \ mov al,[eax]
Intel $8B9D0C000000. \ mov ebx,[ebp+12] \ the other byte
Intel $8803. \ mov [ebx],al

To put a byte into eax:
Intel $8B9D08000000. \ mov ebx,[ebp+8] \ the byte
Intel $0FB603. \ movzx eax,byte ptr [ebx]

To put a byte into a number:
Intel $8B8508000000. \ mov eax,[ebp+8] \ the byte
Intel $0FB600. \ movzx eax,byte ptr [eax]
Intel $8B9D0C000000. \ mov ebx,[ebp+12] \ the number
Intel $8903. \ mov [ebx],eax

To put a byte into a string:
Put 1 into a length.

Reassign the string's first given the length.
Put the byte into the string's first's target.
Put the string's first into the string's last.

To put a byte into a wyrd:
Intel $8B8508000000. \ mov eax,[ebp+8] \ the byte
Intel $660FB600. \ movzx eax,byte ptr [eax]
Intel $8B9D0C000000. \ mov ebx,[ebp+12] \ the wyrd
Intel $668903. \ mov [ebx],ah

To put the character under a finger into a character:
If the finger is nil, clear the character; exit.
Put the finger's target into the character.

To put a color into another color:
Put the color's hue into the other color's hue.
Put the color's saturation into the other color's saturation.
Put the color's lightness into the other color's lightness.

To put a date/time into another date/time:
Put the date/time's year into the other date/time's year.
Put the date/time's month into the other date/time's month.
Put the date/time's week day into the other date/time's week day.
Put the date/time's day into the other date/time's day.
Put the date/time's hour into the other date/time's hour.
Put the date/time's minute into the other date/time's minute.
Put the date/time's second into the other date/time's second.
Put the date/time's millisecond into the other date/time's millisecond.

To put a date/time's string into a string:
Clear the string.
Append the date/time's year to the string.
Append "/" to the string.
Zero fill the date/time's month given 2 and append it to the string.
Append "/" to the string.
Zero fill the date/time's day given 2 and append it to the string.
Append " " to the string.
Zero fill the date/time's hour given 2 and append it to the string.
Append ":" to the string.
Zero fill the date/time's minute given 2 and append it to the string.
Append ":" to the string.
Zero fill the date/time's second given 2 and append it to the string.
Append ":" to the string.
Zero fill the date/time's millisecond given 3 and append it to the string.

To put eax into a byte:
Intel $8B9D08000000. \ mov ebx,[ebp+8] \ the number
Intel $8803. \ mov [ebx],al

To put eax into a flag;
To put eax into a pointer;
To put eax into a number:
Intel $8B9D08000000. \ mov ebx,[ebp+8] \ the number
Intel $8903. \ mov [ebx],eax

To put eax into a wyrd:
Intel $8B9D08000000. \ mov ebx,[ebp+8] \ the wyrd
Intel $668903. \ mov [ebx],ax

To put an ellipse in the middle of a box;
To center an ellipse in a box:
Center the ellipse in the box (horizontally).
Center the ellipse in the box (vertically).

To put an ellipse into another ellipse:
Put the ellipse's box into the other ellipse's box.

To put an ellipse on a spot;
To center an ellipse on a spot:
Center the ellipse's box on the spot.

To put a finger on the first character of a string:
Put the string's first into the finger.

To put a flag into another flag;
To put a flag into a number;
To put a pointer into a number;
To put a pointer into another pointer;
To put a number into a flag;
To put a number into a pointer;
To put a number into another number:
Intel $8B8508000000. \ mov eax,[ebp+8] \ the number
Intel $8B00. \ mov eax,[eax]
Intel $8B9D0C000000. \ mov ebx,[ebp+12] \ the other number
Intel $8903. \ mov [ebx],eax

To put a flag into eax;
To put a pointer into eax;
To put a number into eax:
Intel $8B9D08000000. \ mov ebx,[ebp+8] \ the number
Intel $8B03. \ mov eax,[ebx]

To put a flag into a string;
To convert a flag to a string:
If the flag is set, put "yes" into the string; exit.
Put "no" into the string.

To put a font into another font:
Put the font's name into the other font's name.
Put the font's height into the other font's height.

To put some font widths into a buffer: \ used for pdf conversion
Clear the buffer.
If the font widths are nil, exit.
Put the font widths' data into a number pointer.
Loop.
If a counter is past the font widths' count, break.
Append the number pointer's target then " " to the buffer.
Add 1 to a count.
If the count is evenly divisible by 16, append the crlf string to the buffer.

Add a number's magnitude to the number pointer.
Repeat.

To put a font's adjusted height into a height:
Put the font's height times 3/4 into the height.

To put a fraction into another fraction:
Put the fraction's numerator into the other fraction's numerator.
Put the fraction's denominator into the other fraction's denominator.

To put a fraction into a string;
To convert a fraction to a string:
Clear the string.
Privatize the fraction.
If the fraction is negative, put "-" into the string; de-sign the fraction.
Reduce the fraction.
Convert the fraction to a mixed.
If the mixed is 0, append "0" to the string; exit.
If the mixed's whole number is not 0, append the mixed's whole number to the string.
If the mixed's fraction is 0, exit.
If the string is not blank, append the dash byte to the string.
Append the mixed's numerator then "/" then the mixed's denominator to the string.

To put a fraction pair into another fraction pair:
Put the fraction pair's fraction into the other fraction pair's fraction.
Put the fraction pair's other fraction into the other fraction pair's other fraction.

To put a fraction's absolute value into another fraction:
Put the fraction into the other fraction.
De-sign the other fraction.

To put a gpimage's gprect into a gprect:
Put 0 into the gprect's x.
Put 0 into the gprect's y.
Put the gpimage's width into the gprect's width.
Put the gpimage's height into the gprect's height.

To put a gpimage's height into a height:
If the gpimage is nil, put 0 into the height; exit.
Call "gdiplus.dll" "GdipGetImageHeight" with the gpimage and the height's whereabouts.

To put a gpimage's width into a width:
If the gpimage is nil, put 0 into the width; exit.
Call "gdiplus.dll" "GdipGetImageWidth" with the gpimage and the width's whereabouts.

To put a gprect into another gprect:
Put the gprect's x into the other gprect's x.
Put the gprect's y into the other gprect's y.
Put the gprect's width into the other gprect's width.
Put the gprect's height into the other gprect's height.

To put a hue and a saturation and a lightness into a color:
Put the hue into the color's hue.
If the color's hue is not -1, limit the color's hue to 0 and 3599. \ -1 is clear
Put the saturation into the color's saturation.

Limit the color's saturation to 0 and 1000.
Put the lightness into the color's lightness.
Limit the color's lightness to 0 and 1000.

To put an index's count into a count:
Put 0 into the count.
If the index is nil, exit.
Loop.
Get a bucket given the index.
If the bucket is nil, exit.
Add the bucket's refers' count to the count.
Repeat.

To put an index's used bucket count into a count:
Put 0 into the count.
If the index is nil, exit.
Loop.
Get a bucket given the index.
If the bucket is nil, exit.
If the bucket's refers are empty, repeat.
Add 1 to the count.
Repeat.

To put a left coord and a top coord and a right coord and a bottom coord and a radius into a roundy box:
Put the left into the roundy box's left.
Put the top into the roundy box's top.
Put the right into the roundy box's right.
Put the bottom into the roundy box's bottom.
Put the radius into the roundy box's radius.

To put a left coord and a top coord and a right coord and a bottom coord into a box:
Put the left into the box's left.
Put the top into the box's top.
Put the right into the box's right.
Put the bottom into the box's bottom.

To put a left coord and a top coord and a right coord and a bottom coord into an ellipse:
Put the left into the ellipse's left.
Put the top into the ellipse's top.
Put the right into the ellipse's right.
Put the bottom into the ellipse's bottom.

To put the left of a box into a vertical line;
To put the left edge of a box into a vertical line;
To put the left side of a box into a vertical line:
Put the box's left-top into the vertical line's start.
Put the box's left-bottom into the vertical line's end.

To put a line in the middle of a box;
To center a line in a box:
Center the line in the box (horizontally).
Center the line in the box (vertically).

To put a line into another line:
Put the line's start into the other line's start.

```
Put the line's end into the other line's end.

To put a line's bottom into a coord:
Put the line's start's y into the coord.
If the line's end's y is greater than the line's start's y, put the line's end's y into the coord.

To put a line's box into a box:
Put the line's start into the box's left-top.
Put the line's end into the box's right-bottom.
Normalize the box.

To put a line's center into a spot:
Put the line's start's x plus the line's end's x into the spot's x.
Put the line's start's y plus the line's end's y into the spot's y.
Divide the spot by 2.

To put a line's left into a coord:
Put the line's start's x into the coord.
If the line's end's x is less than the line's start's x, put the line's end's x into the coord.

To put a line's right into a coord:
Put the line's start's x into the coord.
If the line's end's x is greater than the line's start's x, put the line's end's x into the coord.

To put a line's top into a coord:
Put the line's start's y into the coord.
If the line's end's y is less than the line's start's y, put the line's end's y into the coord.

To put masking tape below a figure:
If the figure is nil, exit.
If the figure's vertices' count is less than 2, exit.
Copy the figure to another figure.
Put the screen's bottom into a spot's y.
Put the figure's last's x into the spot's x.
Append the spot to the other figure.
Put the figure's first's x into the spot's x.
Append the spot to the other figure.
Append the figure's first's spot to the other figure.
Mask inside the other figure.
Destroy the other figure.

To put the middle of a line on a spot;
To center a line on a spot:
Get a difference between the spot and the line's center.
Round the difference to the nearest multiple of the tpp.
Move the line given the difference.

To put the mouse's spot into a spot:
Call "user32.dll" "GetCursorPos" with the spot's whereabouts.
Call "user32.dll" "ScreenToClient" with the main window and the spot's whereabouts. \ in case window is
on another monitor.
Call "gdi32.dll" "DPtoLP" with the screen canvas and the spot's whereabouts and 1.

To put a name and a height into a font:
Put the name into the font's name.
```

Put the height into the font's height.

To put a name into a font:
Put the name into the font's name.

To put a number and another number into a pair:
Put the number into the pair's x.
Put the other number into the pair's y.

To put a number into a big-endian unsigned wyrd:
Put the number into a wyrd.
Put the wyrd into the big-endian unsigned wyrd.

To put a number into a byte:
Intel $8B8508000000. \ mov eax,[ebp+8] \ the number
Intel $8B00. \ mov eax,[eax]
Intel $8B9D0C000000. \ mov ebx,[ebp+12] \ the byte
Intel $8803. \ mov [ebx],al

To put a number into a fraction:
Put the number into the fraction's numerator.
Put 1 into the fraction's denominator.

To put a number into a pair:
Put the number into the pair's x.
Put the number into the pair's y.

To put a number into a string;
To convert a number to a string:
Clear the string.
Privatize the number.
De-sign the number.
Loop.
Divide the number by 10 giving a quotient and a remainder.
Add 48 to the remainder.
Put the remainder into a byte.
Prepend the byte to the string.
If the quotient is 0, break.
Put the quotient into the number.
Repeat.
If the original number is less than 0, prepend the dash byte to the string.

To put a number into a wyrd:
Intel $8B8508000000. \ mov eax,[ebp+8] \ the number
Intel $8B00. \ mov eax,[eax]
Intel $8B9D0C000000. \ mov ebx,[ebp+12] \ the wyrd
Intel $668903. \ mov [ebx],ax

To put a number on the stack:
Convert the number to a token.
Put the token on the stack.

To put a number over another number in a fraction;
To put a number and another number into a fraction:
Put the number into the fraction's numerator.

Put the other number into the fraction's denominator.

To put a number's absolute value into another number:
Put the number into the other number.
De-sign the other number.

To put an outlinetextmetric into another outlinetextmetric:
Copy bytes from the outlinetextmetric's whereabouts to the other outlinetextmetric's whereabouts for the outlinetextmetric's magnitude.

To put a pair into another pair:
Put the pair's x into the other pair's x.
Put the pair's y into the other pair's y.

To put a pair's absolute value into another pair:
Put the pair into the other pair.
De-sign the other pair.

To put a picture in the middle of a box;
To center a picture in a box:
If the picture is nil, exit.
Center the picture in the box (horizontally).
Center the picture in the box (vertically).

To put a picture on a spot;
To center a picture on a spot:
If the picture is nil, exit.
Get a difference between the spot and the picture's box's center.
Round the difference to the nearest multiple of the tpp.
Move the picture given the difference.

To put a polygon in the middle of a box;
To center a polygon in a box:
If the polygon is nil, exit.
Center the polygon in the box (horizontally).
Center the polygon in the box (vertically).

To put a polygon in the middle of the screen;
To center a polygon on the screen:
Center the polygon in the screen's box.

To put a polygon on a spot;
To center a polygon on a spot:
If the polygon is nil, exit.
Get a difference between the spot and the polygon's box's center.
Round the difference to the nearest multiple of the tpp.
Move the polygon given the difference.

To put a polygon's box into a box:
If the polygon is nil, zero the box; exit.
If the polygon's vertices are empty, zero the box; exit.
Put the largest number and the largest number and the smallest number and the smallest number into the box.
Loop.
Get a vertex from the polygon's vertices.

If the vertex is nil, break.
If the vertex's x is less than the box's left, put the vertex's x into the box's left.
If the vertex's y is less than the box's top, put the vertex's y into the box's top.
If the vertex's x is greater than the box's right, put the vertex's x into the box's right.
If the vertex's y is greater than the box's bottom, put the vertex's y into the box's bottom.
Repeat.

To put a polygon's center into a spot:
If the polygon is nil, clear the spot; exit.
Put the polygon's box's center into the spot.

To put a polygon's center-bottom into a spot:
If the polygon is nil, clear the spot; exit.
Put the polygon's box's center-bottom into the spot.

To put a polygon's center-top into a spot:
If the polygon is nil, clear the spot; exit.
Put the polygon's box's center-top into the spot.

To put a polygon's height into a height:
If the polygon is nil, clear the height; exit.
Put the polygon's box's height into the height.

To put a polygon's left-bottom into a spot:
If the polygon is nil, clear the spot; exit.
Put the polygon's box's left-bottom into the spot.

To put a polygon's left-center into a spot:
If the polygon is nil, clear the spot; exit.
Put the polygon's box's left-center into the spot.

To put a polygon's left-top into a spot:
If the polygon is nil, clear the spot; exit.
Put the polygon's box's left-top into the spot.

To put a polygon's right-bottom into a spot:
If the polygon is nil, clear the spot; exit.
Put the polygon's box's right-bottom into the spot.

To put a polygon's right-center into a spot:
If the polygon is nil, clear the spot; exit.
Put the polygon's box's right-center into the spot.

To put a polygon's right-top into a spot:
If the polygon is nil, clear the spot; exit.
Put the polygon's box's right-top into the spot.

To put a polygon's width into a width:
If the polygon is nil, clear the width; exit.
Put the polygon's box's width into the width.

To put a polygon's x-extent into a width:
If the polygon is nil, clear the width; exit.
Put the polygon's box's x-extent into the width.

To put a polygon’s y-extent into a height:
If the polygon is nil, clear the height; exit.
Put the polygon’s box’s y-extent into the height.

To put a rider into another rider:
Copy bytes from the rider’s whereabouts to the other rider’s whereabouts for the rider’s magnitude.

To put the right of a box into a vertical line;
To put the right edge of a box into a vertical line;
To put the right side of a box into a vertical line:
Put the box’s right-top into the vertical line’s start.
Put the box’s right-bottom into the vertical line’s end.

To put a roundy box into another roundy box:
Put the roundy box’s left into the other roundy box’s left.
Put the roundy box’s top into the other roundy box’s top.
Put the roundy box’s right into the other roundy box’s right.
Put the roundy box’s bottom into the other roundy box’s bottom.
Put the roundy box’s radius into the other roundy box’s radius.

To put a row’s working string into a substring:
If the row is nil, clear the substring; exit.
Slap the substring on the row’s string.
Subtract 1 from the substring’s last.

To put a selection into another selection:
Put the selection’s anchor into the other selection’s anchor.
Put the selection’s caret into the other selection’s caret.

To put a sockaddr into another sockaddr:
Copy bytes from the sockaddr’s whereabouts to the other sockaddr’s whereabouts for the sockaddr’s magnitude.

To put a spot and another spot and a radius into a roundy box:
Put the spot into the roundy box’s left-top.
Put the other spot into the roundy box’s right-bottom.
Put the radius into the roundy box’s radius.

To put a spot and another spot into a box:
Put the spot into the box’s left-top.
Put the other spot into the box’s right-bottom.

To put a spot and another spot into an ellipse:
Put the spot into the ellipse’s left-top.
Put the other spot into the ellipse’s right-bottom.

To put a spot and another spot into a line:
Put the spot into the line’s start.
Put the other spot into the line’s end.

To put a spot in the middle of a box;
To center a spot in a box:
Center the spot in the box (horizontally).
Center the spot in the box (vertically).

To put a string into another string:
Put the string's length into a saved length.
Assign a pointer given the saved length.
Copy bytes from the string's first to the pointer for the saved length.
Unassign the other string's first.
Put the pointer into the other string's first.
Put the other string's first plus the saved length minus 1 into the other string's last.

To put a string into a letter:
Put the string's first's target into the letter.

To put a string into a text:
If the text is nil, exit.
Destroy the text's rows.
Reset the origin of the text.
Reset the caret of the text.
Deselect the text.
Privatize the string.
Append the return byte to the string.
Convert the string to the text's rows.
Wrap the text.

To put a string on the windows clipboard:
Call "user32.dll" "OpenClipboard" with the main window.
Call "user32.dll" "EmptyClipboard".
Put the string's length plus 1 into a number.
Call "kernel32.dll" "GlobalAlloc" with 66 [ghnd] and the number returning a handle.
Call "kernel32.dll" "GlobalLock" with the handle returning a pointer.
Copy bytes from the string's first to the pointer for the string's length.
Call "kernel32.dll" "GlobalUnlock" with the handle.
Call "user32.dll" "SetClipboardData" with 1 [cf_text] and the handle.
Call "user32.dll" "CloseClipboard".

To put a string's length into a length:
Intel $8B9D08000000. \ mov ebx,[ebp+8] \ the string
\ load default result
Intel $B900000000. \ mov ecx,0
\ if first is 0, store 0
Intel $833B00. \ cmp [ebx],0
Intel $0F8414000000. \ je store it
\ if last is less than first, store 0
Intel $8B5304. \ mov edx,[ebx+4] \ last pointer
Intel $3B13. \ cmp edx,[ebx]
Intel $0F8C09000000. \ jl store it
\ calc length
Intel $8B8B04000000. \ mov ecx,[ebx+4] \ last pointer
Intel $2B0B. \ sub ecx,[ebx] \ subtract first
Intel $41. \ inc ecx \ add 1
\ STORE IT:
Intel $8B950C000000. \ mov edx,[ebp+12] \ the number
Intel $890A. \ mov [edx],ecx

To put a string's width into a width:
Get the width given the string and the memory canvas and the default font.

To put a substring into another substring:
Copy bytes from the substring's whereabouts to the other substring's whereabouts for the substring's magnitude.

To put the system's date/time into a date/time:
Call "kernel32.dll" "GetLocalTime" with a systemtime's whereabouts.
Put the systemtime's wyear into the date/time's year.
Put the systemtime's wmonth into the date/time's month.
Put the systemtime's wdayofweek into the date/time's week day.
Put the systemtime's wday into the date/time's day.
Put the systemtime's whour into the date/time's hour.
Put the systemtime's wminute into the date/time's minute.
Put the systemtime's wsecond into the date/time's second.
Put the systemtime's wmilliseconds into the date/time's millisecond.

To put the system's last error into a number:
Call "kernel32.dll" "GetLastError" returning the number.

To put the system's last winsock error into a number:
Call "ws2_32.dll" "WSAGetLastError" returning the number.

To put the system's tick count into some ticks: \ wraps every 24.8 days or so
Call "kernel32.dll" "GetTickCount" returning the ticks.
Bitwise and the ticks with the largest number.

To put a terminal in the middle of a box;
To center a terminal in a box:
Privatize the box.
Indent the box 1/4 inch.
Put the box into the terminal's box.

To put a text in the middle of a box;
To center a text in a box:
If the text is nil, exit.
Center the text in the box (horizontally).
Center the text in the box (vertically).

To put a text on a spot;
To center a text on a spot:
If the text is nil, exit.
Get a difference between the spot and the text's box's center.
Round the difference to the nearest multiple of the tpp.
Move the text given the difference.

To put a text's first line into a string:
If the text is nil, clear the string; exit.
Put the text's first row's string into the string.
Remove the last byte from the string.

To put a text's first non-blank line into a string: \ *** new
Clear the string.
If the text is nil, exit.
Loop.
Get a row from the text's rows.
If the row is nil, exit.

```
Put the row's string into the string.
Remove the last byte from the string. \ cr or space (see text rules)
Remove any leading noise from the string.
Remove any trailing noise from the string.
If the string is blank, repeat.

To put a text's globalized origin into a spot:
If the text is nil, clear the spot; exit.
Put the text's origin into the spot.
Globalize the spot given the text's left-top.

To put a text's grid into a grid:
If the text is nil, clear the grid; exit.
Put the text's font's height times 4 into the grid's x.
Put the text's font's height into the grid's y.

To put a text's normalized selection into a selection:
If the text is nil, exit.
Put the text's selection into the selection.
Normalize the selection.

To put a text's row count into a count:
If the text is nil, put 0 into the count; exit.
If the text's rows are empty, put 0 into the count; exit.
Put the text's rows' last's row# into the count.

To put a text's row height into a height:
If the text is nil, put 0 into the height; exit.
Put the text's font's height into the height.

To put a text's rows/box into a count:
If the text is nil, put 0 into the count; exit.
Put the text's box's height divided by the text's row height into the count.

To put a text's selected byte count into a count:
Put 0 into the count.
If the text is nil, exit.
If nothing is selected in the text, exit.
Loop.
Get a row from the text's rows.
If the row is nil, exit.
Slap a substring on any selected bytes in the row of the text.
Add the substring's length to the count.
Repeat.

To put a text's selected row count into a count:
Put 0 into the count.
If the text is nil, exit.
Put the text's normalized selection into a selection.
If the selection's anchor row# is the selection's caret row#, exit.
Put the selection's caret row# into the count.
Subtract the selection's anchor row# from the count.
If the selection's caret column# is not 1, add 1 to the count.

To put a text's status string into a string:
```

```
If the text is nil, clear the string; exit.
Put the text's selected row count into a count.
If the count is not 0, format the count and "line" or "lines" into the string; exit.
Put the text's selected byte count into another count.
If the other count is not 0, format the other count and "byte" or "bytes" into the string; exit.
Convert the text's caret row# to the string.
Append ":" to the string.
Append the text's caret column# to the string.

To put a thing at the end of some things;
To append a thing to some things:
If the thing is nil, exit.
Put the things' last into the thing's previous.
If the things are not empty, put the thing into the things' last's next.
If the things are empty, put the thing into the things' first.
Put the thing into the things' last.

To put some things into some other things:
Put the things' first into the other things' first.
Put the things' last into the other things' last.

To put some things' count into a count:
Put 0 into the count.
Loop.
Get a thing from the things.
If the thing is nil, exit.
Add 1 to the count.
Repeat.

To put a timer into a string;
To convert a timer to a string:
Convert the timer's ticks to the string.

To put a timer's string into a string:
Convert the timer's ticks to the string.

To put a timer's ticks into some ticks:
Put the timer's total ticks into the ticks.
If the timer's count is 0, exit.
Put the system's tick count into some other ticks.
Subtract the timer's start ticks from the other ticks.
Add the other ticks to the ticks.

To put a token on the stack:
Allocate memory for an stack entry.
Put the token into the stack entry's string.
Prepend the stack entry to the stack.

To put the top of a box into a horizontal line;
To put the top edge of a box into a horizontal line;
To put the top side of a box into a horizontal line:
Put the box's left-top into the horizontal line's start.
Put the box's right-top into the horizontal line's end.

To put the top of a box into a line:
```

Make the line with the box’s left-top and the box’s right-top.

To put a wyrd into another wyrd:
Intel $8B8508000000. \ mov eax,[ebp+8] \ the wyrd
Intel $668B00. \ mov ax,[eax]
Intel $8B9D0C000000. \ mov ebx,[ebp+12] \ the other wyrd
Intel $668903. \ mov [ebx],ax

To put a wyrd into a big-endian unsigned wyrd:
Intel $8B8508000000. \ mov eax,[ebp+8] \ the wyrd
Intel $668B00. \ mov ax,word ptr [eax]
Intel $86E0. \ xchg al,ah
Intel $8B9D0C000000. \ mov ebx,[ebp+12] \ the big-endian unsigned wyrd
Intel $668903. \ mov word ptr [ebx],ax

To put a wyrd into a byte:
Intel $8B8508000000. \ mov eax,[ebp+8] \ the wyrd
Intel $668B00. \ mov ax,[eax]
Intel $8B9D0C000000. \ mov ebx,[ebp+12] \ the byte
Intel $8803. \ mov [ebx],al

To put a wyrd into eax:
Intel $8B9D08000000. \ mov ebx,[ebp+8] \ the wyrd
Intel $0FBF03. \ movsx eax,word ptr [ebx]

To put a wyrd into a number:
Intel $8B8508000000. \ mov eax,[ebp+8] \ the wyrd
Intel $0FBF00. \ movsx eax,wyrd ptr [eax]
Intel $8B9D0C000000. \ mov ebx,[ebp+12] \ the number
Intel $8903. \ mov [ebx],eax

To put an x coord and a y coord and another x coord and another y coord into a line:
Put the x into the line’s start’s x.
Put the y into the line’s start’s y.
Put the other x into the line’s end’s x.
Put the other y into the line’s end’s y.

The q-key is a key equal to 81.

A query byte is a query string.

A query string is a string.

The question-mark byte is a byte equal to 63.

To quit;
To tell Windows we’re done;
To relinquish control:
Flush the event queue.
Create an event.
Put "done" into the event’s kind.
Enque the event.

A quora is a thing with a string and a color. \ quora is short for "question or answer"

```
To quote a string: \ inserts leading, trailing and nested double-quotes
Put the double-quote byte into another string.
Slap a substring on the string.
Loop.
If the substring is blank, break.
Append the substring's first's target to the other string.
If the substring's first's target is the double-quote byte, append the double-quote byte to the other string.
Add 1 to the substring's first.
Repeat.
Append the double-quote byte to the other string.
Put the other string into the string.

A quotient is a number.

The r-key is a key equal to 82.

A radius is some twips.

A rainbow color is a color.

To raise a number to another number:
If the other number is 0, put 1 into the number; exit.
If the other number is less than 0, put 0 into the number; exit. \ should be 1/the raised result, but always comes out 0 with numbers
Put 1 into a result number.
Loop.
If a counter is past the other number, break.
Multiply the result by the number.
Repeat.
Put the result into the number.

A random number is a number.

A ratio is a fraction.

A ratio pair is a pair.

To read the bible:
If the bible is not nil, exit.
Extract a directory from the module's path.
Loop.
If the directory is blank, exit.
Put the directory then "bible.txt" into a path.
If the path is in the file system, read the bible given the path; exit.
Extract the directory from the directory.
Repeat.

To read the bible given a path:
If the bible is not nil, exit.
Read the path into a buffer.
If the i/o error is not blank, exit.
Allocate memory for the Bible.
Slap a rider on the buffer.
Loop.
If the rider's source is blank, exit.
```

Allocate memory for a verse. Append the verse to the Bible's verses.
Move the rider (text file rules).
Put the rider's token into the verse's string.
Remove any leading noise from the verse's string.
Remove any trailing noise from the verse's string.
Repeat.

To read a byte from a console:
Read a string from the console.
If the string is blank, put the null byte into the byte; exit.
Put the string's first's target into the byte.

To read a console into a string:
If the console is nil, clear the string; exit.
Flush all events.
Clear the console's reply.
Show the console.
Handle events given the console.
Put the console's reply into the string.
Flush all events.
Refresh the cursor.

To read a file into a buffer:
Clear the i/o error.
Call "kernel32.dll" "GetFileSize" with the file and nil returning a size.
Reassign the buffer's first given the size.
Put the buffer's first plus the size minus 1 into the buffer's last.
Call "kernel32.dll" "ReadFile" with the file and the buffer's first and the size and a number's whereabouts and 0 returning a result number.
If the result number is 0, put "Error reading file." into the i/o error; exit.

To read a flag from a console:
Read a string from the console.
Convert the string to the flag.

To read a fraction from a console:
Read a string from the console.
Convert the string to the fraction.

To read a number from a console:
Read a string from the console.
Convert the string to the number.

To load a path into a buffer;
To read a path into a buffer:
Clear the i/o error.
Privatize the path.
Null terminate the path.
If the path is not in the file system, put "File '" then the path then "' doesn't exist." into the i/o error; exit.
\ set the path to read-write mode.
Call "kernel32.dll" "CreateFileA" with the path's first and -2147483648 [generic_read] and 3 [file_share_read+file_share_write] and 0
And 3 [open_existing] and 0 and 0 returning a handle.
If the handle is -1 [invalid_handle_value], put "Error opening file '" then the path then "'." into the i/o error; exit.

```
Call "kernel32.dll" "GetFileSize" with the handle and nil returning a size.
Reassign the buffer’s first given the size.
Put the buffer’s first plus the size minus 1 into the buffer’s last.
Call "kernel32.dll" "ReadFile" with the handle and the buffer’s first and the size and a number’s
whereabouts and 0 returning the number.
Call "kernel32.dll" "CloseHandle" with the handle.
If the number is not 0, exit.
Put "Error reading file ’" then the path then "’." into the i/o error.

To load a path into a picture:
To read a path into a picture:
Read the path into a buffer.
If the i/o error is not blank, void the picture; exit.
Create the picture given the buffer.

To read a reply from a terminal:
If the terminal is nil, clear the reply; exit.
Flush all events.
Clear the terminal’s reply.
Add a quora to the terminal.
Put "> " into the quora’s string.
Put the terminal’s input color into the quora’s color.
Show the terminal.
Handle events given the terminal.
\  show the terminal.
Put the terminal’s reply into the reply.
Remove any leading noise from the reply.
Remove any trailing noise from the reply.
Flush all events.
\ questionable below
Create an event.
Put "left click" into the event’s kind.
Put the mouse’s spot into the event’s spot.
Enque the event.
Refresh the cursor.

To read a response string from a winhttp request:
If the winhttp request is nil, exit.
Clear the response.
Put 8 kilobytes into a buffer size.
Loop.
Put 0 into a size.
Call "winhttp.dll" "WinHttpQueryDataAvailable"
With the winhttp request’s request and the size’s whereabouts returning a result number.
If the result is 0, put "Unable to query data available." into the i/o error; clear the response; break.
Put the response’s length into a saved length.
Reassign the response’s first given the saved length plus the buffer size.
Put the response’s first plus the saved length into a pointer.
Call "winhttp.dll" "WinHttpReadData" with the winhttp request’s request and the pointer
And the buffer size and a count’s whereabouts returning the result number.
If the result number is 0, put "Error reading data." into the i/o error; clear the response; break.
Put the pointer plus the count minus 1 into the response’s last.
If the count is 0, break.
Repeat.
```

To read stdin into a buffer:
Clear the i/o error.
Clear the buffer.
Find a string given the environment variables and "CONTENT_LENGTH".
If the string is blank, put "Error getting content_length" into the i/o error; exit.
Convert the string into a length.
If the length is 0, exit.
Reassign the buffer's first given the length.
Call "kernel32.dll" "ReadFile" with the stdin handle and the buffer's first and the length and a number's whereabouts and nil.
If the number is not the length, put "Error reading stdin data" into the i/o error; clear the buffer; exit.
Put the buffer's first plus the length minus 1 into the buffer's last.

To read a string from a console:
Read the console into the string.

To read a url into a buffer:
\ prepare
Clear the buffer.
Clear the i/o error.
\ internet open
Call "wininet.dll" "InternetOpenA" with the module's name's first and 0 [internet_open_type_preconfig] and nil and nil returning a internet handle.
If the internet handle is 0, put "Could not connect to the internet." into the i/o error; exit.
\ internet open url
Privatize the url.
Null terminate the url.
Call "wininet.dll" "InternetOpenUrlA" with the internet handle and the url's first and nil and 0 and 0 and 0 returning a url handle.
If the url handle is 0, put "Could not connect to url '" then the url then "'." into the i/o error; call "wininet.dll" "InternetCloseHandle" with the internet handle; exit.
\ read the file
Put 64 kilobytes into a buffer size.
Loop.
Put the buffer's length into a saved length.
Reassign the buffer's first given the saved length plus the buffer size.
Put the buffer's first plus the saved length into a pointer.
Call "wininet.dll" "InternetReadFile" with the url handle and the pointer and the buffer size and a count's whereabouts returning a result number.
If the result number is 0, put "Error reading url '" then the url then "'." into the i/o error; break.
Put the pointer plus the count minus 1 into the buffer's last.
If the count is 0, break.
Repeat.
\ clean up
Call "wininet.dll" "InternetCloseHandle" with the url handle.
Call "wininet.dll" "InternetCloseHandle" with the internet handle.

A really dark color is a color.

A really light color is a color.

A really really dark color is a color.

A really really light color is a color.

To reassign a pointer given a byte count:
If the pointer is nil, assign the pointer given the byte count; exit.
If the byte count is 0, unassign the pointer; exit.
Privatize the byte count.
Round the byte count up to the nearest power of two.
Call "kernel32.dll" "HeapReAlloc" with the heap pointer and 8 [heap_zero_memory] and the pointer and the byte count returning the pointer.

To receive a buffer from a socket:
Clear the i/o error.
Clear the buffer.
Put 8 kilobytes into a buffer size.
Loop.
Put 0 into a size.
Put the buffer's length into a saved length.
Reassign the buffer's first given the saved length plus the buffer size.
Put the buffer's first plus the saved length into a pointer.
Call "ws2_32.dll" "recv" with the socket and the pointer and the buffer size and 0 returning a count.
If the count is not -1 [socket_error], put the pointer plus the count minus 1 into the buffer's last; exit.
If the system's last winsock error is not 10040 [wsamsgsize], put "Error receiving data." into the i/o error; clear the buffer; exit.
Put the pointer plus the count minus 1 into the buffer's last.
Repeat.

To receive the response from a winhttp request:
If the winhttp request is nil, exit.
Call "winhttp.dll" "WinHttpReceiveResponse"
With the winhttp request's request
And 0
Returning a result number.
If the result is 0, put "Could not send request." into the i/o error; exit.

A recipient is a string.

The record-separator byte is a byte equal to 30.

A rectangle is a figure.
A skinny rectangle is a rectangle.
A diamond is a figure.
A desert landscape is a thing.
An octagon is a figure.
An arc is a figure.
A circle is a figure.
A fractal forest is a thing.
A joke is a thing.
A triangle is a figure.
A heptagon is a figure. \ 7 sides
A nonagon is a figure. \ 9 sides
A decagon is a figure. \ 10 sides
A hexagon is a figure.
A koch curve is a figure.
A twelve-sided figure is a figure.
A half circle is a figure.
A half circle flower is a figure.
A quarter circle is a figure.

A spiral is a figure.
A leaf is a figure.
A half leaf is a figure.
A five pointed star is a figure.
A six pointed star is a figure.
A left crescent is a figure.
A right crescent is a figure.
A pentagon is a figure.
A solid is a figure.
A star is a thing.
A cube is a figure.
A yew tree is a figure.
A tree is a figure.

The red color is a color.

The red pen is a pen.

To reduce a fraction:
Get a gcd given the fraction's numerator and the fraction's denominator.
Divide the fraction's numerator by the gcd.
Divide the fraction's denominator by the gcd.

A refer is a thing with a string and a pointer (reference).

To refresh the cursor:
Create an event.
Put "set cursor" into the event's kind.
If the alt key is down, set the event's alt flag.
If the ctrl key is down, set the event's ctrl flag.
If the shift key is down, set the event's shift flag.
Put the mouse's spot into the event's spot.
Enque the event.

To refresh the screen given a box:
Call "gdi32.dll" "BitBlt" with the screen canvas and the box's left and the box's top and the box's width and the box's height
And the current canvas and the box's left and the box's top and 13369376 [srccopy].

The registered byte is a byte equal to 174.

The registered-trade-mark byte is a byte equal to 174.

A remainder is a number.

To remember a text:
If the text is nil, exit.
Destroy the text's redos.
Copy the text into another text.
Scale the other text to 1/1.
Append the other text to the text's undos.
Limit the text's undos to the max text undos.
Set the text's modified flag.

To remember a text with an operation:

If the text is nil, exit.
If the text's last operation is the operation, set the text's modified flag; exit.
Remember the text.
Put the operation into the text's last operation.

To remember where we are:
Save the context.

The remembered pdf path is a path.

To remove any selected bytes in a text:
If the text is nil, exit.
If nothing is selected in the text, exit.
Put the text's selection into a selection.
Normalize the selection.
Get a row given the selection's anchor row# and the text.
Slap a substring on the row's string.
Put the substring's first plus the selection's anchor column# minus 2 into the substring's last.
Get another row given the selection's caret row# and the text.
Slap another substring on the other row's string.
Put the other substring's first plus the selection's caret column# minus 1 into the other substring's first.
Put the substring then the other substring into the row's string.
Remove the rows of the text between the row's next and the other row.
Put the selection's anchor into the text's caret.
Deselect the text.

To remove any trailing backslash from a string:
If the string is blank, exit.
If the string's last's target is not the backslash byte, exit.
Remove the last byte from the string.

To remove any trailing linefeed byte from a string:
If the string is blank, exit.
If the string's last's target is not the linefeed byte, exit.
Remove the last byte from the string.

To remove any trailing return byte from a string:
If the string is blank, exit.
If the string's last's target is not the return byte, exit.
Remove the last byte from the string.

To remove bytes from a string given a substring:
If the string is blank, exit.
If the substring is blank, exit.
Put the string's last minus the substring's last into a length.
Put the substring's last plus 1 into a pointer.
Copy bytes from the pointer to the substring's first for the length.
Put the string's length minus the substring's length into a new length.
Reassign the string's first given the new length.
Put the string's first plus the new length minus 1 into the string's last.

To remove bytes from a text (backspace over a return):
If the text is nil, exit.
If the text's caret row# is 1, exit.
Get a row given the text's caret row# minus 1 and the text.

```
Put the row's string's length and the text's caret row# minus 1 into the text's caret.
Remove any selected bytes in the text.

To remove bytes from a text (backspace with jump):
If the text is nil, exit.
If something is selected in the text, remove any selected bytes in the text; exit.
If the text's caret column# is 1, remove bytes from the text (backspace over a return); exit.
Jump the caret left in the text.
Remove any selected bytes in the text.

To remove bytes from a text (backspace):
If the text is nil, exit.
If something is selected in the text, remove any selected bytes in the text; exit.
If the text's caret column# is 1, remove bytes from the text (backspace over a return); exit.
Move the caret left in the text.
Remove any selected bytes in the text.

To remove bytes from a text (forward delete a return):
If the text is nil, exit.
If the text's caret row# is the text's row count, exit.
Put 1 and the text's caret row# plus 1 into the text's caret.
Remove any selected bytes in the text.

To remove bytes from a text (forward delete with jump):
If the text is nil, exit.
If something is selected in the text, remove any selected bytes in the text; exit.
Get a row given the text's caret row# and the text.
If the text's caret column# is the row's string's length, remove bytes from the text (forward delete a return);
exit.
Jump the caret right in the text.
Remove any selected bytes in the text.

To remove bytes from a text (forward delete):
If the text is nil, exit.
If something is selected in the text, remove any selected bytes in the text; exit.
Get a row given the text's caret row# and the text.
If the text's caret column# is the row's string's length, remove bytes from the text (forward delete a return);
exit.
Move the caret right in the text.
Remove any selected bytes in the text.

To remove every byte in a text:
If the text is nil, exit.
Put "" into the text.

To remove the first byte from a string:
Slap a substring on the first byte of the string.
Remove bytes from the string given the substring.

To remove the last byte from a string:
Slap a substring on the last byte of the string.
Remove bytes from the string given the substring.

To remove the last two bytes from a string:
Remove trailing bytes from the string given 2.
```

To remove leading bytes from a string given a count:
Privatize the count.
If the count is greater than the string's length, clear the string; exit.
Slap a substring on the first byte of the string.
Put the substring's first plus the count minus 1 into the substring's last.
Remove bytes from the string given the substring.

To remove leading noise from a string;
To remove any leading noise from a string:
If the string is blank, exit.
If the string's first's target is not noise, exit.
Remove the first byte from the string.
Repeat.

To remove the rows of a text between a row and another row:
If the text is nil, exit.
If the row is nil, exit.
If the other row is nil, exit.
If the row's row# is greater than the other row's row#, exit.
Privatize the row.
Put the other row's next into a stop row.
Loop.
If the row is the stop row, break.
Put the row's next into a next row.
Remove the row from the text's rows.
Destroy the row.
Put the next row into the row.
Repeat.
Renumber the text's rows.

To remove a thing from some things:
If the thing is nil, exit.
If the thing is the things' first, put the thing's next into the things' first.
If the thing is the things' last, put the thing's previous into the things' last.
If the thing's next is not nil, put the thing's previous into the thing's next's previous.
If the thing's previous is not nil, put the thing's next into the thing's previous' next.
Void the thing's next.
Void the thing's previous.

To remove trailing bytes from a string given a count:
Privatize the count.
If the count is greater than the string's length, clear the string; exit.
Slap a substring on the last byte of the string.
Put the substring's last minus the count plus 1 into the substring's first.
Remove bytes from the string given the substring.

To remove trailing noise from a string;
To remove any trailing noise from a string:
If the string is blank, exit.
If the string's last's target is not noise, exit.
Remove the last byte from the string.
Repeat.

To rename a path to another path in the file system:

```
Privatize the path.
Remove any trailing backslash from the path.
Null terminate the path.
Privatize the other path.
Remove any trailing backslash from the other path.
Null terminate the other path.
Call "kernel32.dll" "MoveFileA" with the path’s first and the other path’s first returning a number.
Clear the i/o error.
If the number is not 0, exit.
Put "Error renaming file ’" then the path then "’." into the i/o error.

To renumber some rows:
Get a row from the rows.
If the row is nil, exit.
Add 1 to a row#.
Put the row# into the row’s row#.
Repeat.

To replace a byte with another byte in a string:
Slap a substring on the string.
Loop.
If the substring is blank, exit.
If the substring’s first’s target is not the byte, add 1 to the substring’s first; repeat.
Put the other byte into the substring’s first’s target.
Add 1 to the substring’s first.
Repeat.

The reply is a reply.

A reply is a string.

To reque an event:
Copy the event into another event.
Enque the other event.

To reset the alphabet:
Put the big-a byte into the next letter.

To reset the caret of a text:
If the text is nil, exit.
Put 1 and 1 into the text’s caret.

To reset the context:
Restore the context.
Save the context.

To reset a count:
Put 0 into the count.

To reset the drawing origin:
Set the drawing origin to the zero spot.

To reset a flag:
Clear the flag.
```

To reset the origin of a text:
If the text is nil, exit.
Put the text's margin into the text's x.
Put 0 into the text's y.

To reset a pointer;
To reset a pointer for the next time around;
To void a pointer:
Intel $8B8508000000. \ mov eax,[ebp+8] \ the pointer
Intel $C70000000000. \ mov [eax],0

To reset the rainbow colors:
Put 0 into the current rainbow color number.

To reset a timer:
Put 0 into the timer's count.
Put 0 into the timer's start ticks.
Put 0 into the timer's total ticks.

To resize a box given a ratio pair;
To resize a box given a fraction pair:
Put the box's x-extent into a width.
Put the box's y-extent into a height.
Scale the width given the fraction pair's fraction.
Scale the height given the fraction pair's other fraction.
Put the box's left plus the width into the box's right.
Put the box's top plus the height into the box's bottom.

To resize a box given a twip pair:
Add the twip pair's x to the box's right.
Add the twip pair's y to the box's bottom.

To resize an ellipse given a ratio pair;
To resize an ellipse given a fraction pair:
Resize the ellipse's box given the fraction pair.

To resize an ellipse given a twip pair:
Resize the ellipse's box given the twip pair.

To resize a line given a ratio pair;
To resize a line given a fraction pair:
Put the line's box into a box.
Subtract the box's left-top from the line's start.
Scale the line's start given the fraction pair.
Add the box's left-top to the line's start.
Subtract the box's left-top from the line's end.
Scale the line's end given the fraction pair.
Add the box's left-top to the line's end.

To resize a line given a twip pair:
Put the line's box into a box.
Put the box into another box.
Resize the other box given the twip pair.
Make a fraction pair given the other box and the box.
Resize the line given the fraction pair.

To resize a picture to a width by a height:
If the picture is nil, exit.
Put the width divided by the tpp into a pixel width.
Put the height divided by the tpp into a pixel height.
Call "gdiplus.dll" "GdipCreateBitmapFromScan0" with the pixel width and the pixel height and 0 and 137224 [pixelformat24bpprgb] and 0 and a gpbitmap's whereabouts.
Call "gdiplus.dll" "GdipGetImageGraphicsContext" with the gpbitmap and a gpgraphic's whereabouts.
Call "gdiplus.dll" "GdipDrawImageRectRectI" with the gpgraphic and the picture's gpbitmap
And 0 and 0 and the pixel width and the pixel height
And 0 and 0 and the picture's gpbitmap's width minus 1 and the picture's gpbitmap's height minus 1
And 2 [unitpixel] and nil and nil and 0.
Call "gdiplus.dll" "GdipDeleteGraphics" with the gpgraphic.
Destroy the picture's gpbitmap.
Put the gpbitmap into the picture's gpbitmap.
Adjust the picture (extract boxes from gpbitmap).
Clear the picture's data.

To resize a polygon given a ratio pair;
To resize a polygon given a fraction pair:
If the polygon is nil, exit.
Put the polygon's box into a box.
Loop.
Get a vertex from the polygon's vertices.
If the vertex is nil, exit.
Subtract the box's left-top from the vertex's spot.
Scale the vertex's spot given the fraction pair.
Add the box's left-top to the vertex's spot.
Repeat.

To resize a polygon given a twip pair:
If the polygon is nil, exit.
Put the polygon's box into a box.
Put the box into another box.
Resize the other box given the twip pair.
Make a fraction pair given the other box and the box.
Resize the polygon given the fraction pair.

To resize a text given a ratio pair;
To resize a text given a fraction pair:
If the text is nil, exit.
Resize the text's box given the fraction pair.
Wrap the text.

To resize a text given a twip pair:
If the text is nil, exit.
Resize the text's box given the twip pair.
Wrap the text.

To restart a timer:
Add 1 to the timer's count.
If the timer's count is not 1, exit.
Put the system's tick count into the timer's start ticks.

To restore a canvas:

Call "gdi32.dll" "RestoreDC" with the canvas and -1. \ need to use -1, windows documentation is wrong

To restore a context:
Get a saved context from the context stack.
If the saved context is nil, exit.
Put the saved context's spot into the context's spot.
Put the saved context's heading into the context's heading.
Put the saved context's letter height into the context's letter height.
Put the saved context's color into the context's color.
Put the saved context's number into the context's number.
Remove the saved context from the context stack.
Destroy the saved context.

To restore a window:
Call "user32.dll" "ShowWindow" with the window and 9 [sw_restore].

The return byte is a byte equal to 13.

To reverse any selected rows of a text:
If the text is nil, exit.
Split the rows of the text into some rows and some selected rows and some other rows.
Reverse the selected rows.
Append the rows to the text's rows.
Append the selected rows to the text's rows.
Append the other rows to the text's rows.
Renumber the text's rows.

To reverse a color:
If the color is the black color, put the white color into the color; exit.
If the color is the white color, put the black color into the color; exit.
Put 1000 minus the color's lightness into the color's lightness.
\Put 1000 minus the color's saturation into the color's saturation.
Add 1800 to the color's hue. Normalize the color's hue.

To reverse a flag:
If the flag is yes, put no into the flag; exit.
Put yes into the flag.

To reverse a number;
To invert a number:
Multiply the number by -1.

To reverse a string: \ could be more efficient
Privatize the string.
Clear the original string.
Loop.
If the string is blank, break.
Get a character from the string (backwards).
Append the character to the original string.
Repeat.

To reverse some things:
Swap the things with some other things.
Loop.
Put the other things' last into a thing.

If the thing is nil, exit.
Move the thing from the other things to the things.
Repeat.

A rgb is a record with
A byte called blue byte,
A byte called green byte,
A byte called red byte.

A rgb pointer is a pointer to a rgb.

A rider has \ fix "bump a rider" if you change me
An original substring,
A source substring and
A token substring.

The right-alligator byte is a byte equal to 62.

The right-alligator-quote byte is a byte equal to 155.

The right-arrow key is a key equal to 39.

The right-brace byte is a byte equal to 125.

The right-bracket byte is a byte equal to 93.

The right-double-alligator-quote byte is a byte equal to 187.

The right-double-quote byte is a byte equal to 148.

The right-paren byte is a byte equal to 41.

The right-single-quote byte is a byte equal to 146.

The right-window key is a key equal to 92.

A rise is a number.
A run is a number.

To rotate a box:
Put the box’s center into a center spot.
Put the box into another box.
Put the center’s y minus the other box’s top plus the center’s x into the box’s right.
Put the other box’s left minus the center’s x plus the center’s y into the box’s top.
Put the center’s y minus the other box’s bottom plus the center’s x into the box’s left.
Put the other box’s right minus the center’s x plus the center’s y into the box’s bottom.

To rotate an ellipse:
Rotate the ellipse’s box.

To rotate a gpimage:
If the gpimage is nil, exit.
Call "gdiplus.dll" "GdipImageRotateFlip" with the gpimage and 1 [rotate90flipnone].

To rotate a gpimage given an angle: \ angle can be 0, 900, 1800, 2700

If the gpimage is nil, exit.
Put 0 [rotatenoneflipnone] into a number.
If the angle is 900, put 1 [rotate90flipnone] into the number.
If the angle is 1800, put 2 [rotate180flipnone] into the number.
If the angle is 2700, put 3 [rotate270flipnone] into the number.
Call "gdiplus.dll" "GdipImageRotateFlip" with the gpimage and the number.

To rotate a line:
Put the line's center into a center spot.
Rotate the line's start around the center.
Rotate the line's end around the center.

To rotate a picture:
If the picture is nil, exit.
Add 900 to the picture's rotate angle.
If the picture's mirror flag is set, add 1800 to the picture's rotate angle.
Normalize the picture's rotate angle.
Rotate the picture's box.
Rotate the picture's uncropped box.
Put the picture's box's center into a center spot.
Put the picture's uncropped box's center into another center spot.
Put the center's y minus the other center's y plus the center's x into a twip pair's x.
Subtract the other center's x from the twip pair's x.
Put the center's y plus the other center's x minus the center's x into the twip pair's y.
Subtract the other center's y from the twip pair's y.
Move the picture's uncropped box given the twip pair.
Rotate the picture's gpbitmap.

To rotate a polygon:
If the polygon is nil, exit.
Put the polygon's center into a center spot.
Loop.
Get a vertex from the polygon's vertices.
If the vertex is nil, exit.
Rotate the vertex's spot around the center.
Repeat.

To rotate a spot around a center spot:
Put the spot into another spot.
Put the center's y minus the other spot's y plus the center's x into the spot's x.
Put the other spot's x minus the center's x plus the center's y into the spot's y.

To rotate a text:
If the text is nil, exit.
Rotate the text's box.
Wrap the text.

To round a number to another number:
Round the number to the nearest multiple of the other number.

To round a number down to the nearest multiple of another number:
Divide the number by the other number.
Multiply the number by the other number.

To round a number to the nearest multiple of another number:

If the other number is 0, exit.
Privatize the other number.
Divide the number by the other number giving a quotient and a remainder.
Divide the other number by 2.
If the remainder is greater than or equal to the other number, round the number up to the nearest multiple of the original other number; exit.
Round the number down to the nearest multiple of the original other number.

To round a number up to the nearest multiple of another number:
Divide the number by the other number giving a quotient and a remainder.
If the remainder is 0, exit.
Add the other number minus the remainder to the number.

To round a number up to the nearest power of two:
Intel $8B8D08000000. \ mov ecx,[ebp+8] \ the number
Intel $8B09. \ mov ecx,[ecx]
Intel $49. \ dec ecx
Intel $0FBDC9. \ bsr ecx,ecx
Intel $41. \ inc ecx
Intel $81F904000000. \ cmp ecx,4
Intel $0F8F05000000. \ jg over the next 1 statement
Intel $B904000000. \ mov ecx,4
Intel $8B8508000000. \ mov eax,[ebp+8] \ the number
Intel $C70001000000. \ mov [eax],1
Intel $D320. \ shl [eax],ecx

To round a pair to another pair:
Round the pair to the nearest multiple of the other pair.

To round a pair to the nearest multiple of another pair:
Round the pair's x to the nearest multiple of the other pair's x.
Round the pair's y to the nearest multiple of the other pair's y.

To round a pair to the nearest multiple of a number:
Round the pair's x to the nearest multiple of the number.
Round the pair's y to the nearest multiple of the number.

To round a pair to a number:
Round the pair to the nearest multiple of the number.

A roundy box is a box with
A left coord, a top coord, a right coord, a bottom coord,
A left-top spot at the left, a right-bottom spot at the right, and
A radius.

A row is a thing with a row# and a string.

A row# is a number.

The s-key is a key equal to 83.

A saturation is a number [0 to 1000].

To save a canvas:
Call "gdi32.dll" "SaveDC" with the canvas.

To save a context:
Allocate memory for a saved context.
Put the context's spot into the saved context's spot.
Put the context's heading into the saved context's heading.
Put the context's letter height into the saved context's letter height.
Put the context's color into the saved context's color.
Put the context's number into the saved context's number.
Prepend the saved context to the context stack.

The saved memory hbitmap is a hbitmap.

The saved tpp is a number.

To say a number:
Put the number into a string.
Say the string.

To say a string;
To speak a string:
If the silent flag is set, exit.
If the talker is nil, exit.
If the string is blank, exit.
Convert the string to a wide string.
Null terminate the wide string.
Call the talker's vtable's speak with the talker and the wide string's first and 17 [svsfdefault+svsflagsasyn+svsfisnotxml] and 0.

To say a string and wait;
To speak a string and wait:
If the silent flag is set, exit.
If the talker is nil, exit.
If the string is blank, exit.
Convert the string to a wide string.
Null terminate the wide string.
Call the talker's vtable's speak with the talker and the wide string's first and 16 [svsfdefault+svsfisnotxml] and 0.

To scale a box given a ratio;
To scale a box given a fraction:
If the fraction is 1/1, exit.
Scale the box's left given the fraction.
Scale the box's top given the fraction.
Scale the box's right given the fraction.
Scale the box's bottom given the fraction.

To scale a box to a percent:
Put the percent / 100 into a fraction.
Scale the box given the fraction.

To scale an ellipse given a ratio;
To scale an ellipse given a fraction:
If the fraction is 1/1, exit.
Scale the ellipse's box given the fraction.

To scale an ellipse to a percent:
Put the percent / 100 into a fraction.
Scale the ellipse given the fraction.

To scale a font given a ratio;
To scale a font given a fraction:
If the fraction is 1/1, exit.
Scale the font's height given the fraction.

To scale a fraction given another fraction;
To multiply a fraction by another fraction:
Multiply the fraction's numerator by the other fraction's numerator.
Multiply the fraction's denominator by the other fraction's denominator.
Reduce the fraction.

To scale a line given a ratio;
To scale a line given a fraction:
If the fraction is 1/1, exit.
Scale the line's start given the fraction.
Scale the line's end given the fraction.

To scale a line to a percent:
Put the percent / 100 into a fraction.
Scale the line given the fraction.

To scale a pair given a ratio;
To scale a pair given a fraction:
If the fraction is 1/1, exit.
Scale the pair's x given the fraction.
Scale the pair's y given the fraction.

To scale a pair given a ratio pair;
To scale a pair given a fraction pair:
Scale the pair's x given the fraction pair's fraction.
Scale the pair's y given the fraction pair's other fraction.

To scale a pair to a percent:
Put the percent / 100 into a fraction.
Scale the pair given the fraction.

To scale a picture given a ratio;
To scale a picture given a fraction:
If the picture is nil, exit.
If the fraction is 1/1, exit.
Scale the picture's box given the fraction.
Scale the picture's uncropped box given the fraction.

To scale a picture to a percent:
If the picture is nil, exit.
Put the percent / 100 into a fraction.
Scale the picture given the fraction.

To scale a polygon given a ratio;
To scale a polygon given a fraction:
If the polygon is nil, exit.

If the fraction is 1/1, exit.
Loop.
Get a vertex from the polygon's vertices.
If the vertex is nil, exit.
Scale the vertex given the fraction.
Repeat.

To scale a polygon to a percent:
If the polygon is nil, exit.
Put the percent / 100 into a fraction.
Scale the polygon given the fraction.

To scale a roundy box given a ratio;
To scale a roundy box given a fraction:
If the fraction is 1/1, exit.
Scale the roundy box as a box given the fraction.
Scale the roundy box's radius given the fraction.

To scale a roundy box to a percent:
Put the percent / 100 into a fraction.
Scale the roundy box given the fraction.

To scale a text to a fraction: \ absolute
If the text is nil, exit.
Put the text's scale into another fraction.
Flip the other fraction.
Multiply the other fraction by the fraction.
Scale the text given the other fraction.

To scale a text given a ratio;
To scale a text given a fraction:
If the text is nil, exit.
If the fraction is 1/1, exit.
Scale the text's box given the fraction.
Scale the text's origin given the fraction.
Scale the text's font given the fraction.
Scale the text's scale given the fraction.

To scale a text to a percent:
If the text is nil, exit.
Put the percent / 100 into a fraction.
Scale the text given the fraction.

To scale a vertex given a ratio;
To scale a vertex given a fraction:
If the vertex is nil, exit.
If the fraction is 1/1, exit.
Scale the vertex's x given the fraction.
Scale the vertex's y given the fraction.

The screen canvas is a canvas.

The screen has a box, a pixel height and a pixel width.

To scroll a console given an event:

```
If the console is nil, exit.
Find a sector given the console's grid and the event's spot.
Loop.
If the mouse's right button is up, exit.
Find another sector given the console's grid and the mouse's spot.
Get a difference between the other sector and the sector.
If the difference is 0, repeat.
Scroll the console's text given the difference.
Show the console.
Add the difference to the sector.
Repeat.

To scroll a text to the bottom:
If the text is nil, exit.
If the text's vertical scroll flag is not set, exit.
Put the text's row count minus 1 into a number.
Put - the number times the text's row height into the text's y.
Limit the origin of the text.

To scroll a text to the caret:
If the text is nil, exit.
If the text's caret's column# is 1, put the text's margin into the text's x.
Get a box for the caret in the text.
Adjust the box given 0 and - the tpp and 0 and the tpp. \ caret boxes don't fill entire row
If the box's top is less than the text's top, put the text's top minus the box's top into a difference's y.
If the box's bottom is greater than the text's bottom, put the text's bottom minus the box's bottom into the
difference's y.
If the box's left is less than the text's left, put the text's left minus the box's left into the difference's x.
If the box's right is greater than the text's right, put the text's right minus the box's right into the
difference's x.
If the difference is 0, exit.
Scroll the text given the difference.

To scroll a text to the caret and center it:
If the text is nil, exit.
Put the text's margin into the text's x.
Get a box for the caret in the text.
If the box is inside the text's box, exit.
Adjust the box given 0 and - the tpp and 0 and the tpp. \ caret boxes don't fill entire row
Put the text's box's y-extent divided by 2 into a height.
Round the height down to the nearest multiple of the text's row height.
Put the text's box's top plus the height into a top coord.
Put the top plus the text's row height into a bottom coord.
If the box's top is less than the top, put the top minus the box's top into a difference's y.
If the box's bottom is greater than the bottom, put the bottom minus the box's bottom into the difference's
y.
If the box's left is less than the text's left, put the text's left minus the box's left into the difference's x.
If the box's right is greater than the text's right, put the text's right minus the box's right into the
difference's x.
If the difference is 0, exit.
Scroll the text given the difference.

To scroll a text down one line:
If the text is nil, exit.
If the text's vertical scroll flag is not set, exit.
```

Put - the text's row height into a difference's y.
Scroll the text given the difference.

To scroll a text down one page:
If the text is nil, exit.
If the text's vertical scroll flag is not set, exit.
Subtract the text's box's y-extent from the text's y.
Add the text's row height to the text's y.
Limit the origin of the text.

To scroll a text given a difference:
If the text is nil, exit.
Privatize the difference.
If the text's horizontal scroll flag is not set, put 0 into the difference's x.
If the text's vertical scroll flag is not set, put 0 into the difference's y.
If the difference is 0, exit.
Move the text's origin given the difference.
Limit the origin of the text.

To scroll a text to the top:
If the text is nil, exit.
If the text's vertical scroll flag is not set, exit.
Put 0 into the text's y.
Limit the origin of the text.

To scroll a text up one line:
If the text is nil, exit.
If the text's vertical scroll flag is not set, exit.
Put the text's row height into a difference's y.
Scroll the text given the difference.

To scroll a text up one page:
If the text is nil, exit.
If the text's vertical scroll flag is not set, exit.
Add the text's box's y-extent to the text's y.
Subtract the text's row height from the text's y.
Limit the origin of the text.

The scrolllock key is a key equal to 145.

A second is 1000 milliseconds.

The sector byte is a byte equal to 167.

A sector is a pair with an x coord and an y coord [indicating the left-top of the sector].

The seed is a number.

To select every byte in a text:
If the text is nil, exit.
Put 1 and 1 into the text's anchor.
Put the text's rows' last's string's length and the text's row count into the text's caret.

To select a row# given a text:
If the text is nil, exit.

Get a row given the row# and the text.
If the row is nil, exit.
Put the row# into the text's anchor row#.
Put 1 into the text's anchor column#.
Put the row# into the text's caret row#.
Put the row's string's length into the text's caret column#.

A selection box is a box.

A selection has
An anchor column#, an anchor row#, an anchor at the anchor column#,
A caret column#, a caret row#, a caret at the caret column#.

The semi-colon byte is a byte equal to 59.

To send a buffer to a socket:
Clear the i/o error.
Put the buffer's first into a pointer.
Put the buffer's length into a length.
Loop.
If the length is 0, break.
Call "ws2_32.dll" "send" with the socket and the pointer and the length and 0 returning a number.
If the number is -1 [socket_error], put "Error sending data." into the i/o error; exit.
Subtract the number from the length.
Add the number to the pointer.
Repeat.

To send a data string to a winhttp request:
If the winhttp request is nil, exit.
Call "winhttp.dll" "WinHttpSendRequest"
With the winhttp request's request
And 0 [winhttp_no_additional_headers]
And 0
And the data's first
And the data's length
And the data's length
And 0
Returning a result number.
If the result is 0, put "Could not send request." into the i/o error; exit.

To send an email:
Clear the i/o error.
\ create socket
Create a socket given the email's smtp server and 25.
If the i/o error is not blank, exit.
\ initial receive here for date/time stuff from server
Receive a response string from the socket.
If the i/o error is not blank, destroy the socket; exit.
If the response starts with "5", put the response into the i/o error; trim the i/o error; destroy the socket; exit.
\ send HELO
Send "HELO " then the module's name then the crlf string to the socket and receive the response string.
If the i/o error is not blank, destroy the socket; exit.
If the response starts with "5", put the response into the i/o error; trim the i/o error; destroy the socket; exit.

\ send MAIL FROM: <xxx>
Send "MAIL FROM: <" then the email's sender then ">" then the crlf string to the socket and receive the response string.
If the i/o error is not blank, destroy the socket; exit.
If the response starts with "5", put the response into the i/o error; trim the i/o error; destroy the socket; exit.
\ send RCPT TO: <xxx>
Send "RCPT TO: <" then the email's recipient then ">" then the crlf string to the socket and receive the response string.
If the i/o error is not blank, destroy the socket; exit.
If the response starts with "5", put the response into the i/o error; trim the i/o error; destroy the socket; exit.
\ send DATA
Send "DATA" then the crlf string to the socket and receive the response string.
If the i/o error is not blank, destroy the socket; exit.
If the response starts with "5", put the response into the i/o error; trim the i/o error; destroy the socket; exit.
\ send From: xxx crlf To: xxx crlf Subject: xxx crlf Reply-To: xxx crlf message crlf . crlf
Clear a temp string.
Append "From: " then the email's sender then the crlf string to the temp string.
Append "To: " then the email's recipient then the crlf string to the temp string.
Append "Subject: " then the email's subject then the crlf string into the temp string.
Append "Reply-To: " then the email's sender then the crlf string into the temp string.
Append the crlf string to the temp string.
Append the email's message to the temp string (handling email transparency).
Append the crlf string then "." then the crlf string to the temp string.
Send the temp string to the socket and receive the response string.
If the i/o error is not blank, destroy the socket; exit.
If the response starts with "5", put the response into the i/o error; trim the i/o error; destroy the socket; exit.
\ send QUIT
Send "QUIT" then the crlf string to the socket.
\ destroy socket
Destroy the socket.

To send a message from a sender to a recipient:
Send the message to the recipient from the sender.

To send a message from a sender to a recipient via a smtp server:
Send the message to the recipient from the sender via the smtp server.

To send a message from a sender to a recipient with a subject:
Send the message to the recipient from the sender with the subject.

To send a message from a sender to a recipient with a subject via a smtp server:
Send the message to the recipient from the sender with the subject via the smtp server.

To send a message to a recipient from a sender:
Put the default smtp server into an email's smtp server.
Put the recipient into the email's recipient.
Put the sender into the email's sender.
Put the message into the email's message.
Send the email.

To send a message to a recipient from a sender via a smtp server:

Put the smtp server into an email's smtp server.
Put the recipient into the email's recipient.
Put the sender into the email's sender.
Put the message into the email's message.
Send the email.

To send a message to a recipient from a sender with a subject:
Put the default smtp server into an email's smtp server.
Put the recipient into the email's recipient.
Put the sender into the email's sender.
Put the subject into the email's subject.
Put the message into the email's message.
Send the email.

To send a message to a recipient from a sender with a subject via a smtp server:
Put the smtp server into an email's smtp server.
Put the recipient into the email's recipient.
Put the sender into the email's sender.
Put the subject into the email's subject.
Put the message into the email's message.
Send the email.

To send a string to a socket and receive a response string:
Clear the response string.
Send the string to the socket.
If the i/o error is not blank, exit.
Receive the response string from the socket.

A sender is a string.

To set the colorref of a canvas given a color:
Convert the color to a colorref.
Call "gdi32.dll" "SetTextColor" with the canvas and the colorref.

To set the drawing origin to a spot:
Call "gdi32.dll" "GetDeviceCaps" with the current canvas and 112 [physicaloffsetx] returning a pair's x.
Call "gdi32.dll" "GetDeviceCaps" with the current canvas and 113 [physicaloffsety] returning the pair's y.
Negate the pair.
If the current canvas is not the printer canvas, clear the pair.
Call "gdi32.dll" "SetViewportOrgEx" with the current canvas and the pair's x and the pair's y and nil.
Privatize the spot.
Call "gdi32.dll" "LPtoDP" with the current canvas and the spot's whereabouts and 1.
Call "gdi32.dll" "SetViewportOrgEx" with the current canvas and the spot's x and the spot's y and nil.

To set a flag:
Put yes into the flag.

To set a path to read-write mode:
Privatize the path.
Null terminate the path.
Call "kernel32.dll" "GetFileAttributesA" with the path's first returning a number.
Bitwise and the number with -2 [everything except file_attribute_readonly].
Call "kernel32.dll" "SetFileAttributesA" with the path's first and the number.

The seven byte is a byte equal to 55.

The seven key is a key equal to 55.

The sharp-s byte is a byte equal to 223.

A sheet is a box.

To shift a byte left some bits:
Intel $8B8D0C000000. \ mov ecx,[ebp+12] \ the bits
Intel $8B09. \ mov ecx,[ecx]
Intel $8B8508000000. \ mov eax,[ebp+8] \ the byte
Intel $D220. \ shl byte pointer [eax],ecx

To shift a byte right some bits:
Intel $8B8D0C000000. \ mov ecx,[ebp+12] \ the bits
Intel $8B09. \ mov ecx,[ecx]
Intel $8B8508000000. \ mov eax,[ebp+8] \ the byte
Intel $D228. \ shr byte pointer [eax],ecx

The shift key is a key equal to 16.

To shift a number left some bits:
Intel $8B8D0C000000. \ mov ecx,[ebp+12] \ the bits
Intel $8B09. \ mov ecx,[ecx]
Intel $8B8508000000. \ mov eax,[ebp+8] \ the number
Intel $D320. \ shl [eax],ecx

To shift a number right some bits:
Intel $8B8D0C000000. \ mov ecx,[ebp+12] \ the bits
Intel $8B09. \ mov ecx,[ecx]
Intel $8B8508000000. \ mov eax,[ebp+8] \ the number
Intel $D328. \ shr [eax],ecx

To shift a wyrd left some bits:
Intel $8B8D0C000000. \ mov ecx,[ebp+12] \ the bits
Intel $8B09. \ mov ecx,[ecx]
Intel $8B8508000000. \ mov eax,[ebp+8] \ the wyrd
Intel $66D320. \ shl word ptr [eax],ecx

To shift a wyrd right some bits:
Intel $8B8D0C000000. \ mov ecx,[ebp+12] \ the bits
Intel $8B09. \ mov ecx,[ecx]
Intel $8B8508000000. \ mov eax,[ebp+8] \ the wyrd
Intel $66D328. \ shr word ptr [eax],ecx

The shift-in byte is a byte equal to 15.

The shift-out byte is a byte equal to 14.

To show a console:
If the console is nil, exit.
Save the current canvas.
Draw the console.
Refresh the screen given the console's box.
Restore the current canvas.

```
To show a cursor:
Call "user32.dll" "SetCursor" with the cursor.
Call "user32.dll" "ShowCursor" with 1 returning a number.
If the number is greater than 0, exit.
Repeat.

To show a terminal:
If the terminal is nil, exit.
Save the current canvas.
Draw the terminal.
Refresh the screen given the terminal's box.
Restore the current canvas.

To shrink a box by some twips;
To indent a box some twips;
To indent a box by some twips;
To indent a box some twips on every side;
To indent a box given some twips:
Add the twips to the box's left.
Add the twips to the box's top.
Subtract the twips from the box's right.
Subtract the twips from the box's bottom.

To shut down:
Destroy the bible.
Destroy the stack.
Destroy the lexicon.
Destroy the console.
Destroy the terminal.
Finalize the context.
Finalize the canvases.
Finalize the mouse.
Finalize the cursors.
Finalize the fonts.
Finalize the window.
Finalize the screen.
Finalize the colors.
Finalize the module.
Finalize the talker.
Finalize gdi+.
Finalize winsock.
Finalize com.

To shut down the cgi:
Finalize the cgi.
Finalize the module.
Finalize winsock.

A side is 1 unit.

The silent flag is a flag.

To simplify a reply:
If the reply is blank, break.
```

Get a byte from the reply.
If the byte is any punctuation mark, repeat.
Append the byte to a string.
Repeat.
Put the string into the reply.

The single-quote byte is a byte equal to 39.

The six byte is a byte equal to 54.

The six key is a key equal to 54.

A size is some twips.

To skip any leading linefeed byte in a substring:
If the substring is blank, exit.
If the substring's first's target is not the linefeed byte, exit.
Add 1 to the substring's first.

To skip any leading noise in a substring:
If the substring is blank, exit.
If the substring's first's target is not noise, exit.
Add 1 to the substring's first.
Repeat.

To skip any non-alphanumeric bytes in a substring:
If the substring is blank, exit.
If the substring's first's target is alphanumeric, exit.
Add 1 to the substring's first.
Repeat.

To skip a line on the terminal:
Write "" on the terminal.

To skip word characters in a substring:
If the substring is blank, exit.
If the substring is on any contraction, add 1 to the substring's first; repeat.
If the substring's first's target is not alphanumeric, exit.
Add 1 to the substring's first.
Repeat.

The sky blue color is a color.

The sky blue pen is a pen.

The sky color is a color.

The sky pen is a pen.

To slap a rider on another rider:
Slap the rider's source on the other rider's source.
Position the rider's token on the rider's source.

To slap a rider on a string:
Slap the rider's original on the string.

Slap the rider's source on the string.
Position the rider's token on the rider's source.

To slap a substring on any selected bytes in a row of a text:
Clear the substring.
If the text is nil, exit.
If the row of the text is not selected, exit.
Slap the substring on the row's string.
Put the text's normalized selection into a selection.
If the row's row# is the selection's caret row#, put the substring's first plus the selection's caret column# minus 2 into the substring's last.
If the row's row# is the selection's anchor row#, put the substring's first plus the selection's anchor column# minus 1 into the substring's first.

To slap a substring on the first byte of a string:
Slap the substring on the string.
If the string is blank, exit.
Put the string's first into the substring's last.

To slap a substring on the last byte of a string:
Slap the substring on the string.
If the string is blank, exit.
Put the string's last into the substring's first.

To slap a substring on a string:
Intel $8B8508000000. \ mov eax,[ebp+8] \ the substring
Intel $8B9D0C000000. \ mov ebx,[ebp+12] \ the string
\ put the string's first into the substring's first
Intel $8B8B00000000. \ mov ecx,[ebx+0] \ the string's first
Intel $898800000000. \ mov [eax+0],ecx \ the substring's first
\ put the string's last into the substring's last
Intel $8B8B04000000. \ mov ecx,[ebx+4] \ the string's last
Intel $898804000000. \ mov [eax+4],ecx \ the substring's last

The slash byte is a byte equal to 47.

The small letter height is a letter height.

The small-bullet byte is a byte equal to 183.

The smallest number is -2147483648.

To smooth a polygon:
If the polygon is nil, exit.
If the polygon's vertices' count is less than 3, exit.
If the polygon is closed, append the polygon's first vertex's next's spot to the polygon; set a flag.
Put the polygon's first vertex into a left vertex.
Loop.
If the left vertex's next is nil, break.
Put the left vertex's next into a right vertex.
Get a center spot given the left vertex's spot and the right vertex's spot.
Insert the center into the polygon after the left vertex.
Put the left vertex's next into a new vertex.
If the left vertex's previous is nil, put the right vertex into the left vertex; repeat.
Get another center spot given the left vertex's previous' spot and the new vertex's spot.

Get a difference between the other center and the left vertex's spot.
Divide the difference by 2.
Add the difference to the left vertex's spot.
Put the right vertex into the left vertex.
Repeat.
If the flag is not set, exit.
Destroy the polygon's first vertex given the polygon.
Destroy the polygon's last vertex given the polygon.

To smooth a polygon some times: \ this use to "times" is a fluke, I think -- see "some times is a number"
Privatize the times.
Loop.
If the times is 0, exit.
Smooth the polygon.
Subtract 1 from the times.
Repeat.

An smtp server is a string.

A sockaddr is a record with
A wyrd called sin_family,
A big-endian unsigned wyrd called sin_port,
A in_addr called sin_addr,
8 bytes called sin_zero.

A sockaddrptr is a pointer to a sockaddr.

A socket is a pointer.

The soft-dash byte is a byte equal to 173.

A solid color is a color.

To sort any selected rows in a text:
If the text is nil, exit.
Split the rows of the text into some rows and some selected rows and some other rows.
Sort the selected rows.
Append the rows to the text's rows.
Append the selected rows to the text's rows.
Append the other rows to the text's rows.
Renumber the text's rows.

To sort some rows:
If the rows' first is the rows' last, exit.
Split the rows into some left rows and some right rows.
Sort the left rows.
Sort the right rows.
Loop.
Put the left rows' first into a left row.
Put the right rows' first into a right row.
If the left row is nil, append the right rows to the rows; exit.
If the right row is nil, append the left rows to the rows; exit.
If the left row's string is greater than the right row's string, move the right row from the right rows to the rows; repeat.
Move the left row from the left rows to the rows.

Repeat.

To space between glyphs:
Turn right. Move 3 squares. Turn left.

The space byte is a byte equal to 32.

The space key is a key equal to 32.

The space string is a string equal to " ".

To split a buffer into some dyads:
Destroy the dyads.
If the buffer is blank, exit.
Slap a rider on the buffer.
Loop.
Move the rider given the ampersand byte.
If the rider's token is blank, exit.
Create a dyad.
Append the dyad to the dyads.
Split the rider's token into a name substring and a query substring given the equal-sign byte.
Put the name substring into the dyad's name.
Convert the query substring as a query string into the dyad's value.
Repeat.

To split a byte into a nibble and another nibble:
Put the byte into the nibble.
Shift the nibble right 4 bits.
Put the byte into the other nibble.
Bitwise and the other nibble with 15.

To split a line into another line and a third line:
Privatize the line.
Put the line's center into a center spot.
Put the line's start and the center into the other line.
Put the center and the line's end into the third line.

To split a number into a wyrd and another wyrd:
Privatize the number.
Shift the number right 16 bits.
Put the number into the wyrd.
Put the original number into the other wyrd.

To split the rows of a text into some rows and some selected rows and some other rows:
If the text is nil, clear the rows; clear the selected rows; clear the other rows; exit.
Loop.
Put the text's rows' first into a row.
If the row is nil, exit.
Remove the row from the text's rows.
If the row of the text is selected, set a flag; append the row to the selected rows; repeat.
If the flag is set, append the row to the other rows; repeat.
Append the row to the rows.
Repeat.

To split a string into a left substring and a right substring given a separator byte:

```
Clear the left.
Clear the right.
If the string is blank, exit.
Put the string's first into a substring's first.
Put the substring's first minus 1 into the substring's last.
Loop.
If the substring's last is greater than the string's last, exit.
Add 1 to the substring's last.
If the substring's last's target is the separator byte, break.
Repeat.
Put the substring's first into the left's first.
Put the substring's last minus 1 into the left's last.
Put the substring's last plus 1 into the right's first.
Put the string's last into the right's last.

To split a string into some string things given a separator byte:
Destroy the string things.
If the string is blank, exit.
Slap a rider on the string.
Loop.
Move the rider given the separator byte.
Add the rider's token to the string things.
If the rider's source is blank, break.
Repeat.
If the string's last's target is not the separator byte, exit.
Add "" to the string things.

To split some things into some left things and some right things:
If the things are empty, clear the left things; clear the right things; exit.
Put the things' count divided by 2 into a count.
Loop.
Get a thing from the things.
If the count is 0, break.
Subtract 1 from the count.
Repeat.
Split the things into the left things and the right things at the thing.

To split some things into some left things and some right things at a thing:
Clear the left things.
Clear the right things.
If the thing is nil, swap the things with the left things; exit.
If the thing's previous is nil, swap the things with the left things; exit.
\ set up the left chain
Put the things' first into the left things' first.
Put the thing's previous into the left things' last.
Void the thing's previous' next.
\ set up the right chain
Put the thing into the right things' first.
Void the thing's previous.
Put the things' last into the right things' last.
\ fix the original chain
Clear the things.

To split a wyrd into a byte and another byte:
Privatize the wyrd.
```

```
Shift the wyrd right 8 bits.
Put the wyrd into the byte.
Put the original wyrd into the other byte.

A spot is a pair with an x coord and a y coord and a left at the x coord and a top at the y coord.

A spot pointer is a pointer to a spot.

A square is 1440 units. \ arbitrary number high enough for precision divides

A square root is a number.

The square size is some twips.

To square up any selection in a text:
If the text is nil, exit.
If nothing is selected in the text, exit.
Normalize the text's selection.
Get a row given the text's caret row# and the text.
Put 1 into the text's anchor column#.
If the text's caret column# is not 1, add 1 to the text's caret row#; put 1 into the text's caret column#.
If the text's caret row# is less than or equal to the text's row count, exit.
Put the text's row count into the text's caret's row#.
Put the row's string's length into the text's caret's column#.

The squirt o' two is a fraction equal to 99/70.

A stack entry is a thing with a string.

The stack is some stack entries.

To start anywhere in a box:
Pick the context's spot in the box.

To start anywhere on a horizontal line:
Pick a spot on the horizontal line.
Put the spot into the context's spot.

To start at the bottom left corner of a box facing east:
Put the box's left-bottom into the context's spot.
Face east.

To start at the bottom left corner of a box facing north:
Put the box's left-bottom into the context's spot.
Face north.

To start at the bottom left corner of a box facing south:
Put the box's left-bottom into the context's spot.
Face south.

To start at the bottom left corner of a box facing west:
Put the box's left-bottom into the context's spot.
Face west.

To start at the bottom of a horizontal line:
```

Put the vertical line's end into the context's spot.

To start at the bottom right corner of a box facing east:
Put the box's right-bottom into the context's spot.
Face east.

To start at the bottom right corner of a box facing north:
Put the box's right-bottom into the context's spot.
Face north.

To start at the bottom right corner of a box facing south:
Put the box's right-bottom into the context's spot.
Face south.

To start at the bottom right corner of a box facing west:
Put the box's right-bottom into the context's spot.
Face west.

To start at the left of a horizontal line:
Put the horizontal line's start into the context's spot.

To start at the middle of the bottom of a box;
To start in the middle of the bottom of a box;
To start at the center of the bottom of a box;
To start in the center of the bottom of a box:
Put the box's center's x into the context's spot's x.
Put the box's bottom into the context's spot's y.

To start at the middle of the bottom of a box facing east;
To start in the middle of the bottom of a box facing east;
To start at the center of the bottom of a box facing east;
To start in the center of the bottom of a box facing east:
Put the box's center's x into the context's spot's x.
Put the box's bottom into the context's spot's y.
Face east.

To start at the middle of the bottom of a box facing north;
To start in the middle of the bottom of a box facing north;
To start at the center of the bottom of a box facing north;
To start in the center of the bottom of a box facing north:
Put the box's center's x into the context's spot's x.
Put the box's bottom into the context's spot's y.
Face north.

To start at the middle of the bottom of a box facing south;
To start in the middle of the bottom of a box facing south;
To start at the center of the bottom of a box facing south;
To start in the center of the bottom of a box facing south:
Put the box's center's x into the context's spot's x.
Put the box's bottom into the context's spot's y.
Face south.

To start at the middle of the bottom of a box facing west;
To start in the middle of the bottom of a box facing west;
To start at the center of the bottom of a box facing west;

To start in the center of the bottom of a box facing west:
Put the box's center's x into the context's spot's x.
Put the box's bottom into the context's spot's y.
Face west.

To start at the middle of the left of a box facing east;
To start in the middle of the left of a box facing east;
To start at the center of the left of a box facing east;
To start in the center of the left of a box facing east:
Put the box's left into the context's spot's x.
Put the box's center's y into the context's spot's y.
Face east.

To start at the middle of the left of a box facing north;
To start in the middle of the left of a box facing north;
To start at the center of the left of a box facing north;
To start in the center of the left of a box facing north:
Put the box's left into the context's spot's x.
Put the box's center's y into the context's spot's y.
Face north.

To start at the middle of the left of a box facing south;
To start in the middle of the left of a box facing south;
To start at the center of the left of a box facing south;
To start in the center of the left of a box facing south:
Put the box's left into the context's spot's x.
Put the box's center's y into the context's spot's y.
Face south.

To start at the middle of the left of a box facing west;
To start in the middle of the left of a box facing west;
To start at the center of the left of a box facing west;
To start in the center of the left of a box facing west:
Put the box's left into the context's spot's x.
Put the box's center's y into the context's spot's y.
Face west.

To start at the middle of the right of a box facing east;
To start in the middle of the right of a box facing east;
To start at the center of the right of a box facing east;
To start in the center of the right of a box facing east:
Put the box's right into the context's spot's x.
Put the box's center's y into the context's spot's y.
Face east.

To start at the middle of the right of a box facing north;
To start in the middle of the right of a box facing north;
To start at the center of the right of a box facing north;
To start in the center of the right of a box facing north:
Put the box's right into the context's spot's x.
Put the box's center's y into the context's spot's y.
Face north.

To start at the middle of the right of a box facing south;
To start in the middle of the right of a box facing south;

To start at the center of the right of a box facing south;
To start in the center of the right of a box facing south:
Put the box's right into the context's spot's x.
Put the box's center's y into the context's spot's y.
Face south.

To start at the middle of the right of a box facing west;
To start in the middle of the right of a box facing west;
To start at the center of the right of a box facing west;
To start in the center of the right of a box facing west:
Put the box's right into the context's spot's x.
Put the box's center's y into the context's spot's y.
Face west.

To start at the middle of the top of a box;
To start in the middle of the top of a box;
To start at the center of the top of a box;
To start in the center of the top of a box:
Put the box's center's x into the context's spot's x.
Put the box's top into the context's spot's y.

To start at the middle of the top of a box facing east;
To start in the middle of the top of a box facing east;
To start at the center of the top of a box facing east;
To start in the center of the top of a box facing east:
Put the box's center's x into the context's spot's x.
Put the box's top into the context's spot's y.
Face east.

To start at the middle of the top of a box facing north;
To start in the middle of the top of a box facing north;
To start at the center of the top of a box facing north;
To start in the center of the top of a box facing north:
Put the box's center's x into the context's spot's x.
Put the box's top into the context's spot's y.
Face north.

To start at the middle of the top of a box facing south;
To start in the middle of the top of a box facing south;
To start at the center of the top of a box facing south;
To start in the center of the top of a box facing south:
Put the box's center's x into the context's spot's x.
Put the box's top into the context's spot's y.
Face south.

To start at the middle of the top of a box facing west;
To start in the middle of the top of a box facing west;
To start at the center of the top of a box facing west;
To start in the center of the top of a box facing west:
Put the box's center's x into the context's spot's x.
Put the box's top into the context's spot's y.
Face west.

To start at the right of a horizontal line:
Put the horizontal line's end into the context's spot.

To start at a spot:
Put the spot into the context's spot.

To start at a spot facing east:
Put the spot into the context's spot.
Face east.

To start at a spot facing north:
Put the spot into the context's spot.
Face north.

To start at a spot facing south:
Put the spot into the context's spot.
Face south.

To start at a spot facing west:
Put the spot into the context's spot.
Face west.

To start at the top left corner of a box facing east:
Put the box's left-top into the context's spot.
Face east.

To start at the top left corner of a box facing north:
Put the box's left-top into the context's spot.
Face north.

To start at the top left corner of a box facing south:
Put the box's left-top into the context's spot.
Face south.

To start at the top left corner of a box facing west:
Put the box's left-top into the context's spot.
Face west.

To start at the top of a vertical line:
Put the vertical line's start into the context's spot.

To start at the top right corner of a box facing east:
Put the box's right-top into the context's spot.
Face east.

To start at the top right corner of a box facing north:
Put the box's right-top into the context's spot.
Face north.

To start at the top right corner of a box facing south:
Put the box's right-top into the context's spot.
Face south.

To start at the top right corner of a box facing west:
Put the box's right-top into the context's spot.
Face west.

To start in the middle of a box facing east;
To start at the middle of a box facing east;
To move to the middle of a box facing east;
To move to the middle of a box and face east;
To start in the center of a box facing east;
To start at the center of a box facing east;
To move to the center of a box facing east;
To move to the center of a box and face east:
Put the box's center into the context's spot.
Face east.

To start in the middle of a box facing north;
To start at the middle of a box facing north;
To move to the middle of a box facing north;
To move to the middle of a box and face north;
To start in the center of a box facing north;
To start at the center of a box facing north;
To move to the center of a box facing north;
To move to the center of a box and face north:
Put the box's center into the context's spot.
Face north.

To start in the middle of a box facing north minus some points;
To start in the center of a box facing north minus some points:
Put the box's center into the context's spot.
Face north.
Turn left the points.

To start in the middle of a box facing south;
To start at the middle of a box facing south;
To move to the middle of a box facing south:
To move to the middle of a box and face south;
To start in the center of a box facing south;
To start at the center of a box facing south;
To move to the center of a box facing south:
To move to the center of a box and face south:
Put the box's center into the context's spot.
Face south.

To start in the middle of a box facing west;
To start at the middle of a box facing west;
To move to the middle of a box facing west;
To move to the middle of a box and face west;
To start in the center of a box facing west;
To start at the center of a box facing west;
To move to the center of a box facing west;
To move to the center of a box and face west:
Put the box's center into the context's spot.
Face west.

To start a process given a path: \ must be called with a global variable
Clear the i/o error.
If the process is not 0, put "I'm sorry, but that process is already running." into the i/o error; exit.
Put a startupinfo's magnitude into the startupinfo's cb.
Extract a directory from the path. null terminate the directory.

Privatize the path. null terminate the path.
Call "kernel32.dll" "CreateProcessA" with the path's first and 0 and 0 and 0 and 0 and 67108904 [create_default_error_mode + normal_priority_class + detached_process] and 0
And the directory's first and the startupinfo's whereabouts and a processinfo's whereabouts returning a number.
If the number is 0, put "I'm unable to run the program." into the i/o error; exit.
Put the processinfo's hprocess into the process.
Call "kernel32.dll" "CloseHandle" with the processinfo's hthread.
Point a pointer to routine wait for a process pointer.
Call "kernel32.dll" "CreateThread" with 0 and 0 and the pointer and the process's whereabouts and 0 and another number's whereabouts returning a handle.
Call "kernel32.dll" "CloseHandle" with the handle. \ does not end the thread, just dumps the handle

To start a timer:
Reset the timer.
Restart the timer.

To start some twips above the middle of the bottom of a box: \ incomplete set of these
Put the box's center's x into the context's spot's x.
Put the box's bottom minus the twips into the context's spot's y.

To start some twips down from a spot;
To start some twips below a spot:
Put the spot's x into the context's x.
Put the spot's y plus the twips into the context's y.

To start some twips from the center of a box:
Put the box's center into the context's spot.
Move the twips.

To start some twips to the left and some other twips down from a spot;
To start some twips left and some other twips down from a spot:
Put the spot's x minus the twips into the context's x.
Put the spot's y plus the other twips into the context's y.

To start some twips left and some other twips up from a spot:
Put the spot's x minus the twips into the context's x.
Put the spot's y minus the other twips into the context's y.

To start some twips to the left and some other twips up from a spot;
To start some twips to the left of a spot;
To start some twips left of a spot:
Put the spot's x minus the twips into the context's x.

To start some twips to the right and some other twips down from a spot;
To start some twips right and some other twips down from a spot:
Put the spot's x plus the twips into the context's x.
Put the spot's y plus the other twips into the context's y.

To start some twips to the right and some other twips up from a spot;
To start some twips right and some other twips up from a spot:
Put the spot's x plus the twips into the context's x.
Put the spot's y minus the other twips into the context's y.

To start some twips up from a coord:

Put the coord minus the twips into the context's y.

To start some twips up from a spot;
To start some twips above a spot:
Put the spot's y into the context's y.
Subtract the twips from the context's y.

To start up:
Initialize com.
Initialize winsock.
Initialize gdi+.
Initialize the talker.
Initialize the module.
Initialize the colors.
Initialize the screen.
Initialize the window.
Initialize the fonts.
Initialize the cursors.
Initialize the mouse.
Initialize the canvases.
Initialize the context.
Initalize the terminal.
Create the console.

To start up the cgi:
Initialize winsock.
Initialize the module.
Initialize the cgi.

To start with a color:
Put the color into the context's color.

To start with nothing in a pointer:
Void the pointer.

The start-of-heading byte is a byte equal to 1.

The start-of-text byte is a byte equal to 2.

A startupinfo is a record with
A number called cb,
A pointer called lpreserved,
A pointer called lpdesktop,
A pointer called lptitle,
A number called dwx,
A number called dwy,
A number called dwxsize,
A number called dwysize,
A number called dwxcountchars,
A number called cwycountchars,
A number called dwfillattribute,
A number called dwflags,
A wyrd called wshowwindow,
A wyrd called cbreserved2,
A pointer called lpreserved2,

A handle called hstdinput,
A handle called hstdoutput,
A handle called hstderror.

The stdin handle is a handle.

The stdout handle is a handle.

To stop a process:
If the process is 0, exit.
Call "kernel32.dll" "TerminateProcess" with the process and 0.
Put 0 into the process.

To stop a timer:
If the timer’s count is 0, exit.
Subtract 1 from the timer’s count.
If the timer’s count is not 0, exit.
Put the system’s tick count into some ticks.
Subtract the timer’s start ticks from the ticks.
Add the ticks to the timer’s total ticks.

A string has a first byte pointer and a last byte pointer.

A string thing is a thing with a string.

A string# is a number.

To stroke the accent glyph:
Save the context.
Move 4 squares.
Turn right.
Turn right 7/96 of the way.
Stroke 9/4 square.
Restore the context.

To stroke the asterisk glyph:
Save the context.
Move 2 squares.
Turn right.
Stroke 2 squares.
Turn left.
Move 1 square.
Turn left.
Move 1 square.
Turn left.
Stroke 2 squares.
Reset the context.
Move 1 square.
Turn right 1/8.
Stroke 2 squares slantways.
Reset the context.
Move 3 squares.
Turn right 3/8.
Stroke 2 squares slantways.
Restore the context.

To stroke the at-sign glyph:
Save the context.
Turn right.
Move 2 squares.
Turn around.
Stroke 1 square.
Turn right 1/8.
Stroke 1 square slantways.
Turn right 1/8.
Stroke 2 squares.
Turn right 1/8.
Stroke 1 square slantways.
Turn right.
Stroke 1 square slantways.
Turn right 1/8.
Stroke 1 square.
Turn right 3/8.
Stroke 1/2 square slantways.
Turn left.
Stroke 1/2 square slantways.
Turn left.
Stroke 1/2 square slantways.
Turn left.
Stroke 1/2 square slantways.
Restore the context.

To stroke the backslash glyph:
Save the context.
Turn right.
Move 2 squares.
Turn left.
Turn left 7/96 of the way.
Stroke 9/2 square.
Restore the context.

To stroke the big-a glyph:
Save the context.
Stroke 3 squares.
Turn right 1/8.
Stroke 1 square slantways.
Turn right.
Stroke 1 square slantways.
Turn right 1/8.
Stroke 3 squares.
Turn around.
Move 2 squares.
Turn left.
Stroke 1 square.
Restore the context.

To stroke the big-b glyph:
Save the context.
Stroke 4 squares.
Turn right.

```
Stroke 1 square.
Turn right 1/8.
Stroke 1 square slantways.
Turn right.
Stroke 1 square slantways.
Turn left.
Stroke 1 square slantways.
Turn right.
Stroke 1 square slantways.
Restore the context.

To stroke the big-c glyph:
Save the context.
Move 2 squares.
Stroke 1 square.
Turn right 1/8.
Stroke 1 square slantways.
Turn right.
Stroke 1 square slantways.
Reset the context.
Move 2 squares.
Turn around.
Stroke 1 square.
Turn left 1/8.
Stroke 1 square slantways.
Turn left.
Stroke 1 square slantways.
Restore the context.

To stroke the big-d glyph:
Save the context.
Stroke 4 squares.
Turn right.
Stroke 1 square.
Turn right 1/8.
Stroke 1 square slantways.
Turn right 1/8.
Stroke 2 squares.
Turn right 1/8.
Stroke 1 square slantways.
Turn right 1/8.
Stroke 1 square.
Restore the context.

To stroke the big-e glyph:
Save the context.
Stroke 4 squares.
Turn right.
Stroke 2 squares.
Reset the context.
Move 2 squares.
Turn right.
Stroke 1 square.
Reset the context.
Turn right.
```

Stroke 2 squares.
Restore the context.

To stroke the big-f glyph:
Save the context.
Stroke 4 squares.
Turn right.
Stroke 2 squares.
Reset the context.
Move 2 squares.
Turn right.
Stroke 1 square.
Restore the context.

To stroke the big-g glyph:
Save the context.
Move 2 squares.
Stroke 1 square.
Turn right 1/8.
Stroke 1 square slantways.
Turn right.
Stroke 1 square slantways.
Reset the context.
Move 2 squares.
Turn around.
Stroke 1 square.
Turn left 1/8.
Stroke 1 square slantways.
Turn left.
Stroke 1 square slantways.
Turn left 1/8.
Stroke 1 square.
Turn left.
Stroke 1 square.
Restore the context.

To stroke the big-h glyph:
Save the context.
Stroke 4 squares.
Reset the context.
Move 2 squares.
Turn right.
Stroke 2 squares.
Reset the context.
Turn right.
Move 2 squares.
Turn left.
Stroke 4 squares.
Restore the context.

To stroke the big-i glyph:
Save the context.
Turn right.
Move 1 square.
Turn left.

Stroke 4 squares.
Reset the context.
Turn right.
Stroke 2 squares.
Reset the context.
Move 4 squares.
Turn right.
Stroke 2 squares.
Restore the context.

To stroke the big-j glyph:
Save the context.
Move 2 squares.
Turn around.
Stroke 1 square.
Turn left 1/8.
Stroke 1 square slantways.
Turn left.
Stroke 1 square slantways.
Turn left 1/8.
Stroke 3 squares.
Restore the context.

To stroke the big-k glyph:
Save the context.
Stroke 4 squares.
Turn around.
Move 2 squares.
Turn left.
Stroke 1 square.
Save the context.
Turn left 1/8.
Stroke 1 square slantways.
Turn left 1/8.
Stroke 1 square.
Restore the context.
Turn right 1/8.
Stroke 1 square slantways.
Turn right 1/8.
Stroke 1 square.
Restore the context.

To stroke the big-l glyph:
Save the context.
Move 4 squares.
Turn around.
Stroke 4 squares.
Turn left.
Stroke 2 squares.
Restore the context.

To stroke the big-m glyph:
Save the context.
Stroke 4 squares.
Turn right 41/96.

```
Stroke 9/4 square.
Turn right 62/96.
Stroke 9/4 square.
Turn right 41/96.
Stroke 4 squares.
Restore the context.

To stroke the big-n glyph:
Save the context.
Stroke 4 squares.
Turn right 41/96.
Stroke 9/2 square.
Turn left 41/96.
Stroke 4 squares.
Restore the context.

To stroke the big-o glyph:
Save the context.
Move 1 square.
Stroke 2 squares.
Turn right 1/8.
Stroke 1 square slantways.
Turn right 1/4.
Stroke 1 square slantways.
Turn right 1/8.
Stroke 2 squares.
Turn right 1/8.
Stroke 1 square slantways.
Turn right 1/4.
Stroke 1 square slantways.
Restore the context.

To stroke the big-p glyph:
Save the context.
Stroke 4 squares.
Turn right.
Stroke 1 square.
Turn right 1/8.
Stroke 1 square slantways.
Turn right.
Stroke 1 square slantways.
Restore the context.

To stroke the big-q glyph:
Save the context.
Move 1 square.
Stroke 2 squares.
Turn right 1/8.
Stroke 1 square slantways.
Turn right 1/4.
Stroke 1 square slantways.
Turn right 1/8.
Stroke 2 squares.
Turn right 1/8.
Stroke 1 square slantways.
```

Turn right 1/4.
Stroke 1 square slantways.
Reset the context.
Move 1 square.
Turn right.
Move 1 square.
Turn right 1/8 of the way.
Stroke 1-1/2 square.
Restore the context.

To stroke the big-r glyph:
Save the context.
Stroke 4 squares.
Turn right.
Stroke 1 square.
Turn right 1/8.
Stroke 1 square slantways.
Turn right.
Stroke 1 square slantways.
Turn left 1/8.
Turn left 7/96.
Stroke 7/2 square.
Restore the context.

To stroke the big-s glyph:
Save the context.
Move 1 square.
Turn around.
Turn left 1/8.
Stroke 1 square slantways.
Turn left.
Stroke 1 square slantways.
Turn left.
Stroke 2 squares slantways.
Turn right.
Stroke 1 square slantways.
Turn right.
Stroke 1 square slantways.
Restore the context.

To stroke the big-t glyph:
Save the context.
Turn right.
Move 1 square.
Turn left.
Stroke 4 squares.
Turn left.
Move 1 square.
Turn around.
Stroke 2 squares.
Restore the context.

To stroke the big-u glyph:
Save the context.
Move 4 squares.

Turn around.
Stroke 4 squares.
Turn left.
Stroke 2 squares.
Turn left.
Stroke 4 squares.
Restore the context.

To stroke the big-v glyph:
Save the context.
Move 4 squares.
Turn around.
Stroke 3 squares.
Turn left 1/8.
Stroke 1 square slantways.
Turn left.
Stroke 1 square slantways.
Turn left 1/8.
Stroke 3 squares.
Restore the context.

To stroke the big-w glyph:
Save the context.
Move 4 squares.
Turn around.
Stroke 4 squares.
Turn left 41/96.
Stroke 9/4 square.
Turn left 61/96.
Stroke 9/4 square.
Turn left 41/96.
Stroke 4 squares.
Restore the context.

To stroke the big-x glyph:
Save the context.
Stroke 1 square.
Turn right 1/8.
Stroke 2 squares slantways.
Turn left 1/8.
Stroke 1 square.
Reset the context.
Turn right.
Move 2 squares.
Turn left.
Stroke 1 square.
Turn left 1/8.
Stroke 2 squares slantways.
Turn right 1/8.
Stroke 1 square.
Restore the context.

To stroke the big-y glyph:
Save the context.
Move 4 squares.

```
Turn around.
Stroke 1 square.
Turn left 1/8.
Stroke 1 square slantways.
Turn left.
Stroke 1 square slantways.
Turn left 1/8.
Stroke 1 square.
Reset the context.
Turn right.
Move 1 square.
Turn left.
Stroke 2 squares.
Restore the context.

To stroke the big-z glyph:
Save the context.
Move 4 squares.
Turn right.
Stroke 2 squares.
Turn right.
Stroke 1 square.
Turn right 1/8.
Stroke 2 squares slantways.
Turn left 1/8.
Stroke 1 square.
Turn left.
Stroke 2 squares.
Restore the context.

To stroke a box with a color:
Put the color into the context's color.
Put the box's left-bottom into the context's spot.
Face north.
Stroke the box's height.
Turn right.
Stroke the box's width.
Turn right.
Stroke the box's height.
Turn right.
Stroke the box's width.

To stroke a byte:
Put the context's letter height divided by 4 into the square size. \ ***
If the byte is the accent byte, stroke the accent glyph.
If the byte is the asterisk byte, stroke the asterisk glyph.
If the byte is the at-sign byte, stroke the at-sign glyph.
If the byte is the backslash byte, stroke the backslash glyph.
If the byte is the big-a byte, stroke the big-a glyph.
If the byte is the big-b byte, stroke the big-b glyph.
If the byte is the big-c byte, stroke the big-c glyph.
If the byte is the big-d byte, stroke the big-d glyph.
If the byte is the big-e byte, stroke the big-e glyph.
If the byte is the big-f byte, stroke the big-f glyph.
If the byte is the big-g byte, stroke the big-g glyph.
```

If the byte is the big-h byte, stroke the big-h glyph.
If the byte is the big-i byte, stroke the big-i glyph.
If the byte is the big-j byte, stroke the big-j glyph.
If the byte is the big-k byte, stroke the big-k glyph.
If the byte is the big-l byte, stroke the big-l glyph.
If the byte is the big-m byte, stroke the big-m glyph.
If the byte is the big-n byte, stroke the big-n glyph.
If the byte is the big-o byte, stroke the big-o glyph.
If the byte is the big-p byte, stroke the big-p glyph.
If the byte is the big-q byte, stroke the big-q glyph.
If the byte is the big-r byte, stroke the big-r glyph.
If the byte is the big-s byte, stroke the big-s glyph.
If the byte is the big-t byte, stroke the big-t glyph.
If the byte is the big-u byte, stroke the big-u glyph.
If the byte is the big-v byte, stroke the big-v glyph.
If the byte is the big-w byte, stroke the big-w glyph.
If the byte is the big-x byte, stroke the big-x glyph.
If the byte is the big-y byte, stroke the big-y glyph.
If the byte is the big-z byte, stroke the big-z glyph.
If the byte is the caret byte, stroke the caret glyph.
If the byte is the colon byte, stroke the colon glyph.
If the byte is the comma byte, stroke the comma glyph.
If the byte is the dollar-sign byte, stroke the dollar-sign glyph.
If the byte is the double-quote byte, stroke the double-quote glyph.
If the byte is the eight byte, stroke the eight glyph.
If the byte is the equal-sign byte, stroke the equal-sign glyph.
If the byte is the exclamation-mark byte, stroke the exclamation-mark glyph.
If the byte is the five byte, stroke the five glyph.
If the byte is the four byte, stroke the four glyph.
If the byte is the left-alligator byte, stroke the left-alligator glyph.
If the byte is the left-brace byte, stroke the left-brace glyph.
If the byte is the left-bracket byte, stroke the left-bracket glyph.
If the byte is the left-paren byte, stroke the left-paren glyph.
If the byte is the little-a byte, stroke the little-a glyph.
If the byte is the little-b byte, stroke the little-b glyph.
If the byte is the little-c byte, stroke the little-c glyph.
If the byte is the little-d byte, stroke the little-d glyph.
If the byte is the little-e byte, stroke the little-e glyph.
If the byte is the little-f byte, stroke the little-f glyph.
If the byte is the little-g byte, stroke the little-g glyph.
If the byte is the little-h byte, stroke the little-h glyph.
If the byte is the little-i byte, stroke the little-i glyph.
If the byte is the little-j byte, stroke the little-j glyph.
If the byte is the little-k byte, stroke the little-k glyph.
If the byte is the little-l byte, stroke the little-l glyph.
If the byte is the little-m byte, stroke the little-m glyph.
If the byte is the little-n byte, stroke the little-n glyph.
If the byte is the little-o byte, stroke the little-o glyph.
If the byte is the little-p byte, stroke the little-p glyph.
If the byte is the little-q byte, stroke the little-q glyph.
If the byte is the little-r byte, stroke the little-r glyph.
If the byte is the little-s byte, stroke the little-s glyph.
If the byte is the little-t byte, stroke the little-t glyph.
If the byte is the little-u byte, stroke the little-u glyph.
If the byte is the little-v byte, stroke the little-v glyph.

If the byte is the little-w byte, stroke the little-w glyph.
If the byte is the little-x byte, stroke the little-x glyph.
If the byte is the little-y byte, stroke the little-y glyph.
If the byte is the little-z byte, stroke the little-z glyph.
If the byte is the minus-sign byte, stroke the minus-sign glyph.
If the byte is the nine byte, stroke the nine glyph.
If the byte is the number-sign byte, stroke the number-sign glyph.
If the byte is the one byte, stroke the one glyph.
If the byte is the percent-sign byte, stroke the percent-sign glyph.
If the byte is the period byte, stroke the period glyph.
If the byte is the plus-sign byte, stroke the plus-sign glyph.
If the byte is the question-mark byte, stroke the question-mark glyph.
If the byte is the right-alligator byte, stroke the right-alligator glyph.
If the byte is the right-brace byte, stroke the right-brace glyph.
If the byte is the right-bracket byte, stroke the right-bracket glyph.
If the byte is the right-paren byte, stroke the right-paren glyph.
If the byte is the semi-colon byte, stroke the semi-colon glyph.
If the byte is the seven byte, stroke the seven glyph.
If the byte is the single-quote byte, stroke the single-quote glyph.
If the byte is the six byte, stroke the six glyph.
If the byte is the slash byte, stroke the slash glyph.
If the byte is the three byte, stroke the three glyph.
If the byte is the tilde byte, stroke the tilde glyph.
If the byte is the two byte, stroke the two glyph.
If the byte is the underscore byte, stroke the underscore glyph.
If the byte is the vertical-bar byte, stroke the vertical-bar glyph.
If the byte is the zero byte, stroke the zero glyph.
\ Refresh the screen. \ *** questionable doesn’t work screws up console.

To stroke the caret glyph:
Save the context.
Move 3 squares.
Turn right 1/8.
Stroke 1 square slantways.
Turn right.
Stroke 1 square slantways.
Restore the context.

To stroke the colon glyph:
Save the context.
Turn right.
Move 1 square.
Turn left.
Stroke 1/2 square.
Move 1/2 square.
Move 1 square.
Stroke 1/2 square.
Restore the context.

To stroke the comma glyph:
Save the context.
Turn around.
Move 1 square.
Turn left 3/8.
Stroke 1 square slantways.

Turn left 1/8.
Stroke 1/2 square.
Restore the context.

To stroke the dollar-sign glyph:
Save the context.
Move 1 square.
Turn around.
Turn left 1/8.
Stroke 1 square slantways.
Turn left.
Stroke 1 square slantways.
Turn left.
Stroke 2 squares slantways.
Turn right.
Stroke 1 square slantways.
Turn right.
Stroke 1 square slantways.
Reset the context.
Turn right.
Move 1 square.
Turn right.
Move 1/2 square.
Turn around.
Stroke 5 squares.
Restore the context.

To stroke the double-quote glyph:
Save the context.
Turn right.
Move 1/2 square.
Turn left.
Move 2-1/2 squares.
Stroke 1-1/2 squares.
Reset the context.
Turn right.
Move 1-1/2 squares.
Turn left.
Move 2-1/2 squares.
Stroke 1-1/2 squares.
Restore the context.

To stroke the eight glyph:
Save the context.
Turn right.
Move 1 square.
Turn left 3/8.
Stroke 1 square slantways.
Turn right.
Stroke 2 squares slantways.
Turn left.
Stroke 1 square slantways.
Turn left.
Stroke 1 square slantways.
Turn left.

Stroke 2 squares slantways.
Turn right.
Stroke 1 square slantways.
Restore the context.

To stroke the equal-sign glyph:
Save the context.
Move 1-1/2 squares.
Turn right.
Stroke 2 squares.
Turn left.
Move 1 square.
Turn left.
Stroke 2 squares.
Restore the context.

To stroke the exclamation-mark glyph:
Save the context.
Move 4 squares.
Turn right.
Move 1 square.
Turn right.
Stroke 3 squares.
Move 1 square.
Stroke 1/2 square.
Restore the context.

To stroke the five glyph:
Save the context.
Turn around.
Move 1 square.
Turn left 3/8.
Stroke 2 squares slantways.
Turn left.
Stroke 1 square slantways.
Turn left 1/8.
Stroke 1 square.
Turn right.
Stroke 2 squares.
Turn right.
Stroke 2 squares.
Restore the context.

To stroke the four glyph:
Save the context.
Turn right.
Move 2 squares.
Turn left.
Stroke 4 squares.
Turn left 3/8.
Stroke 2 squares slantways.
Turn left 3/8.
Stroke 1 square.
Restore the context.

To stroke the left-alligator glyph:
Save the context.
Turn right.
Move 2 squares.
Turn left.
Move 1 square.
Turn left 17/96 of the way.
Stroke 9/4 square.
Turn right 34/96 of the way.
Stroke 9/4 square.
Restore the context.

To stroke the left-brace glyph:
Save the context.
Turn right.
Move 2 squares.
Turn around.
Stroke 1 square.
Turn right.
Stroke 1 square.
Turn left 1/8.
Stroke 1 square slantways.
Turn right.
Stroke 1 square slantways.
Turn left 1/8.
Stroke 1 square.
Turn right.
Stroke 1 square.
Restore the context.

To stroke the left-bracket glyph:
Save the context.
Turn right.
Move 2 squares.
Turn around.
Stroke 1 square.
Turn right.
Stroke 4 squares.
Turn right.
Stroke 1 square.
Restore the context.

To stroke the left-paren glyph:
Save the context.
Turn right.
Move 1-1/2 squares.
Turn left 3/8.
Stroke 1 square slantways.
Turn right 1/8.
Stroke 2 squares.
Turn right 1/8.
Stroke 1 square slantways.
Restore the context.

To stroke a line as high as a box;

To stroke a line as tall as a box;
To draw a line as high as a box;
To draw a line as tall as a box:
Stroke the box's height.

To stroke a line as wide as a box;
To draw a line as wide as a box:
Stroke the box's width.

To stroke the little-a glyph:
Save the context.
Turn right.
Move 2 squares.
Turn left.
Stroke 3 squares.
Turn around.
Move 1 square.
Turn right 3/8.
Stroke 1 square slantways.
Turn left.
Stroke 1 square slantways.
Turn left 1/8.
Stroke 1 square.
Turn left 1/8.
Stroke 1 square slantways.
Turn left.
Stroke 1 square slantways.
Restore the context.

To stroke the little-b glyph:
Save the context.
Stroke 4 squares.
Turn around.
Move 2 squares.
Turn left 3/8.
Stroke 1 square slantways.
Turn right.
Stroke 1 square slantways.
Turn right 1/8.
Stroke 1 square.
Turn right 1/8.
Stroke 1 square slantways.
Turn right.
Stroke 1 square slantways.
Restore the context.

To stroke the little-c glyph:
Save the context.
Turn right.
Move 2 squares.
Turn left.
Move 1 square.
Turn left 3/8.
Stroke 1 square slantways.
Turn right.

Stroke 1 square slantways.
Turn right 1/8.
Stroke 1 square.
Turn right 1/8.
Stroke 1 square slantways.
Turn right.
Stroke 1 square slantways.
Restore the context.

To stroke the little-d glyph:
Save the context.
Turn right.
Move 2 squares.
Turn left.
Stroke 4 squares.
Turn around.
Move 2 squares.
Turn right 3/8.
Stroke 1 square slantways.
Turn left.
Stroke 1 square slantways.
Turn left 1/8.
Stroke 1 square.
Turn left 1/8.
Stroke 1 square slantways.
Turn left.
Stroke 1 square slantways.
Restore the context.

To stroke the little-e glyph:
Save the context.
Turn right.
Move 2 squares.
Turn around.
Stroke 1 square.
Turn right 1/8.
Stroke 1 square slantways.
Turn right 1/8.
Stroke 1 square.
Turn right 1/8.
Stroke 1 square slantways.
Turn right.
Stroke 1 square slantways.
Turn right 1/8.
Stroke 1/2 square.
Turn right.
Stroke 2 squares.
Restore the context.

To stroke the little-f glyph:
Save the context.
Turn right.
Move 1 square.
Turn left.
Save the context.

Turn around.
Turn right 1/8.
Stroke 1 square slantways.
Restore the context.
Stroke 4 squares.
Turn right 1/8.
Stroke 1 square slantways.
Reset the context.
Move 3 squares.
Turn right.
Stroke 2 squares.
Restore the context.

To stroke the little-g glyph:
Save the context.
Turn right.
Move 2 squares.
Turn left.
Move 1 square.
Turn left 3/8.
Stroke 1 square slantways.
Turn right.
Stroke 1 square slantways.
Turn right 1/8.
Stroke 1 square.
Turn right 1/8.
Stroke 1 square slantways.
Turn right.
Stroke 1 square slantways.
Reset the context.
Move 3 squares.
Turn right.
Move 2 squares.
Turn right.
Stroke 3 squares.
Turn right 1/8.
Stroke 1-1/2 square slantways.
Restore the context.

To stroke the little-h glyph:
Save the context.
Stroke 4 squares.
Turn around.
Move 2 squares.
Turn left 3/8.
Stroke 1 square slantways.
Turn right.
Stroke 1 square slantways.
Turn right 1/8.
Stroke 2 squares.
Restore the context.

To stroke the little-i glyph:
Save the context.
Turn right.

Stroke 2 squares.
Turn around.
Move 1 square.
Turn right.
Stroke 3 squares.
Turn left.
Stroke 1 square.
Turn around.
Move 1 square.
Turn left.
Move 1/2 square.
Stroke 1/2 square.
Restore the context.

To stroke the little-j glyph:
Save the context.
Move 1 square.
Turn around.
Stroke 1 square.
Turn left 1/8.
Stroke 1 square slantways.
Turn left.
Stroke 1 square slantways.
Turn left 1/8.
Stroke 3 squares.
Move 1/2 square.
Stroke 1/2 square.
Restore the context.

To stroke the little-k glyph:
Save the context.
Stroke 4 squares.
Turn around.
Move 2 squares.
Turn left 1/8.
Stroke 2 squares slantways.
Reset the context.
Move 2 squares.
Turn right 17/96.
Stroke 9/4 square.
Restore the context.

To stroke the little-l glyph:
Save the context.
Turn right.
Stroke 2 squares.
Turn around.
Move 1 square.
Turn right.
Stroke 4 squares.
Turn left.
Stroke 1 square.
Restore the context.

To stroke the little-m glyph:

Save the context.
Stroke 3 squares.
Turn around.
Move 1 square.
Turn left 3/8.
Stroke 1 square slantways.
Turn right 3/8.
Stroke 2 squares.
Turn around.
Move 1 square.
Turn right 1/8.
Stroke 1 square slantways.
Turn right 3/8.
Stroke 3 squares.
Restore the context.

To stroke the little-n glyph:
Save the context.
Stroke 3 squares.
Turn around.
Move 1 square.
Turn left 3/8.
Stroke 1 square slantways.
Turn right.
Stroke 1 square slantways.
Turn right 1/8.
Stroke 2 squares.
Restore the context.

To stroke the little-o glyph:
Save the context.
Turn right.
Move 2 squares.
Turn left.
Move 1 square.
Turn left 3/8.
Stroke 1 square slantways.
Turn right.
Stroke 1 square slantways.
Turn right 1/8.
Stroke 1 square.
Turn right 1/8.
Stroke 1 square slantways.
Turn right.
Stroke 1 square slantways.
Turn right 1/8.
Stroke 1 square.
Restore the context.

To stroke the little-p glyph:
Save the context.
Move 3 squares.
Turn around.
Stroke 4 squares.
Turn around.

Move 3 squares.
Turn right 1/8.
Stroke 1 square slantways.
Turn right.
Stroke 1 square slantways.
Turn right 1/8.
Stroke 1 square.
Turn right 1/8.
Stroke 1 square slantways.
Turn right.
Stroke 1 square slantways.
Restore the context.

To stroke the little-q glyph:
Save the context.
Turn right.
Move 2 squares.
Turn left.
Move 1 square.
Turn left 3/8.
Stroke 1 square slantways.
Turn right.
Stroke 1 square slantways.
Turn right 1/8.
Stroke 1 square.
Turn right 1/8.
Stroke 1 square slantways.
Turn right.
Stroke 1 square slantways.
Turn left 3/8.
Move 1 square.
Turn around.
Stroke 4 squares.
Restore the context.

To stroke the little-r glyph:
Save the context.
Stroke 3 squares.
Turn around.
Move 1 square.
Turn left 3/8.
Stroke 1 square slantways.
Turn right.
Stroke 1 square slantways.
Restore the context.

To stroke the little-s glyph:
Save the context.
Move 1 square.
Turn right 3/8.
Stroke 1 square slantways.
Turn left.
Stroke 1 square slantways.
Reset the context.
Move 2 squares.

Turn right 1/8.
Stroke 1 square slantways.
Turn right.
Stroke 1 square slantways.
Reset the context.
Move 2 squares.
Turn right 31/96.
Stroke 9/4 squares.
Restore the context.

To stroke the little-t glyph:
Save the context.
Turn right.
Move 1 square.
Turn right 1/8.
Stroke 1 square slantways.
Reset the context.
Turn right.
Move 1 square.
Turn left.
Stroke 4 squares.
Turn around.
Move 1 square.
Turn right.
Move 1 square.
Turn around.
Stroke 2 squares.
Restore the context.

To stroke the little-u glyph:
Save the context.
Move 3 squares.
Turn around.
Stroke 2 squares.
Turn left 1/8.
Stroke 1 square slantways.
Turn left.
Stroke 1 square slantways.
Turn left 1/8.
Move 2 squares.
Turn around.
Stroke 3 squares.
Restore the context.

To stroke the little-v glyph:
Save the context.
Move 3 squares.
Turn around.
Stroke 2 squares.
Turn left 1/8.
Stroke 1 square slantways.
Turn left.
Stroke 1 square slantways.
Turn left 1/8.
Stroke 2 squares.

Restore the context.

To stroke the little-w glyph:
Save the context.
Move 3 squares.
Turn around.
Stroke 3 squares.
Turn around.
Turn right 7/96.
Stroke 9/4 squares.
Turn right 34/96.
Stroke 9/4 squares.
Turn left 41/96.
Stroke 3 squares.
Restore the context.

To stroke the little-x glyph:
Save the context.
Stroke 1 square.
Turn right 17/96.
Stroke 9/4 square.
Turn left 17/96.
Stroke 1 square.
Reset the context.
Turn right.
Move 2 squares.
Turn left.
Stroke 1 square.
Turn left 17/96.
Stroke 9/4 square.
Turn right 17/96.
Stroke 1 square.
Restore the context.

To stroke the little-y glyph:
Save the context.
Move 3 squares.
Turn around.
Stroke 2 squares.
Turn left 1/8.
Stroke 1 square slantways.
Turn left.
Stroke 1 square slantways.
Turn left 1/8.
Move 2 squares.
Turn around.
Stroke 3 squares.
Turn right 1/8.
Stroke 1-1/2 squares slantways.
Restore the context.

To stroke the little-z glyph:
Save the context.
Move 3 squares.
Turn right.

Stroke 2 squares.
Turn right.
Stroke 1 square.
Turn right 17/96.
Stroke 9/4 square.
Turn left 17/96.
Stroke 1 square.
Turn left.
Stroke 2 squares.
Restore the context.

To stroke the minus-sign glyph:
Save the context.
Move 2 squares.
Turn right.
Stroke 2 squares.
Restore the context.

To stroke the nine glyph:
Save the context.
Move 1 square.
Turn right 3/8.
Stroke 1 square slantways.
Turn left.
Stroke 1 square slantways.
Turn left 1/8.
Stroke 2 squares.
Turn left 1/8.
Stroke 1 square slantways.
Turn left.
Stroke 1 square slantways.
Turn left.
Stroke 1 square slantways.
Restore the context.

To stroke the number-sign glyph:
Save the context.
Move 3 squares.
Turn right.
Stroke 2 squares.
Reset the context.
Move 1 square.
Turn right.
Stroke 2 squares.
Reset the context.
Turn right.
Move 1/2 square.
Turn left.
Stroke 4 squares.
Reset the context.
Turn right.
Move 1-1/2 squares.
Turn left.
Stroke 4 squares.
Restore the context.

To stroke the one glyph:
Save the context.
Turn right.
Move 1 square.
Turn left.
Stroke 4 squares.
Reset the context.
Turn right.
Stroke 2 squares.
Reset the context.
Move 3 squares.
Turn right 1/8.
Stroke 1 square slantways.
Restore the context.

To stroke the percent-sign glyph:
Save the context.
Turn right 7/96.
Stroke 9/2 squares.
Reset the context.
Move 2-1/2 squares.
Stroke 1/2 square.
Reset the context.
Turn right.
Move 2 squares.
Turn left.
Move 1 square.
Stroke 1/2 square.
Restore the context.

To stroke the period glyph:
Save the context.
Turn right.
Move 1 square.
Turn left.
Stroke 1/2 square.
Restore the context.

To stroke the plus-sign glyph:
Save the context.
Move 2 squares.
Turn right.
Stroke 2 squares.
Turn left.
Move 1 square.
Turn left.
Move 1 square.
Turn left.
Stroke 2 squares.
Restore the context.

To stroke the question-mark glyph:
Save the context.
Move 3 squares.

Turn right 1/8.
Stroke 1 square slantways.
Turn right.
Stroke 1 square slantways.
Turn right.
Stroke 1 square slantways.
Turn left 1/8.
Stroke 1 square.
Move 1 square.
Stroke 1/2 square.
Restore the context.

To stroke the right-alligator glyph:
Save the context.
Move 1 square.
Turn right 17/96 of the way.
Stroke 9/4 square.
Turn left 34/96 of the way.
Stroke 9/4 square.
Restore the context.

To stroke the right-brace glyph:
Save the context.
Turn right.
Stroke 1 square.
Turn left.
Stroke 1 square.
Turn right 1/8.
Stroke 1 square slantways.
Turn left.
Stroke 1 square slantways.
Turn right 1/8.
Stroke 1 square.
Turn left.
Stroke 1 square.
Restore the context.

To stroke the right-bracket glyph:
Save the context.
Turn right.
Stroke 1 square.
Turn left.
Stroke 4 squares.
Turn left.
Stroke 1 square.
Restore the context.

To stroke the right-paren glyph:
Save the context.
Turn right.
Move 1/2 square.
Turn left.
Turn right 1/8.
Stroke 1 square slantways.
Turn left 1/8.

Stroke 2 squares.
Turn left 1/8.
Stroke 1 square slantways.
Restore the context.

To stroke the semi-colon glyph:
Save the context.
Turn around.
Move 1 square.
Turn left 3/8.
Stroke 1 square slantways.
Turn left 1/8.
Stroke 1/2 square.
Move 1/2 square.
Move 1 square.
Stroke 1/2 square.
Restore the context.

To stroke the seven glyph:
Save the context.
Move 3 squares.
Stroke 1 square.
Turn right.
Stroke 2 squares.
Turn right.
Stroke 1 square.
Turn right 1/8.
Stroke 1 square slantways.
Turn left 1/8.
Stroke 2 squares.
Restore the context.

To stroke the single-quote glyph:
Save the context.
Turn right.
Move 1 square.
Turn left.
Move 2-1/2 squares.
Stroke 1-1/2 squares.
Restore the context.

To stroke the six glyph:
Save the context.
Turn right.
Move 2 squares.
Turn left.
Move 3 squares.
Turn left 1/8.
Stroke 1 square slantways.
Turn left.
Stroke 1 square slantways.
Turn left 1/8.
Stroke 2 squares.
Turn left 1/8.
Stroke 1 square slantways.

Turn left.
Stroke 1 square slantways.
Turn left.
Stroke 1 square slantways.
Restore the context.

To stroke the slash glyph:
Save the context.
Turn right 7/96 of the way.
Stroke 9/2 square.
Restore the context.

To stroke some squares:
Stroke the square size times the squares divided by 1 square. \ squares are scaled up for precision hence the division at the end

To stroke some squares diagonally;
To stroke some squares slantways:
Stroke the square size times the squares times the squirt o' two divided by 1 square. \ squares are scaled up for precision hence the division at the end

To stroke the three glyph:
Save the context.
Move 3 squares.
Turn right 1/8.
Stroke 1 square slantways.
Turn right.
Stroke 1 square slantways.
Turn right.
Stroke 1 square slantways.
Turn left.
Stroke 1 square slantways.
Turn right.
Stroke 1 square slantways.
Turn right.
Stroke 1 square slantways.
Restore the context.

To stroke the tilde glyph:
Save the context.
Move 2 squares.
Turn right 7/96 of the way.
Stroke 9/8 square.
Turn right.
Stroke 9/6 square.
Turn left.
Stroke 9/8 square.
Restore the context.

To stroke some twips:
Draw a line the twips long.

To stroke the two glyph:
Save the context.
Move 3 squares.

Turn right 1/8.
Stroke 1 square slantways.
Turn right.
Stroke 1 square slantways.
Turn right.
Stroke 2 squares slantways.
Turn left 1/8.
Stroke 1 square.
Turn left.
Stroke 2 squares.
Restore the context.

To stroke the underscore glyph:
Save the context.
Turn around.
Move 1 square.
Turn left.
Stroke 2 squares.
Restore the context.

To stroke the vertical-bar glyph:
Save the context.
Turn right.
Move 1 square.
Turn right.
Move 1 square.
Turn around.
Stroke 5 squares.
Restore the context.

To stroke the zero glyph:
Save the context.
Move 1 square.
Stroke 2 squares.
Turn right 1/8.
Stroke 1 square slantways.
Turn right 1/4.
Stroke 1 square slantways.
Turn right 1/8.
Stroke 2 squares.
Turn right 1/8.
Stroke 1 square slantways.
Turn right 1/4.
Stroke 1 square slantways.
Turn right 1/8.
Move 1 square.
Turn right.
Move 1 square.
Stroke 1 pixel.
Restore the context.

A subject is a string.

The substitute byte is a byte equal to 26.

A substring is a string.

To subtract a byte from another byte:
Intel $8B8508000000. \ mov eax,[ebp+8] \ the byte
Intel $0FB600. \ movzx eax,[eax]
Intel $8B9D0C000000. \ mov ebx,[ebp+12] \ the other byte
Intel $2803. \ sub [ebx],al

To subtract a byte from a number:
Intel $8B8508000000. \ mov eax,[ebp+8] \ the byte
Intel $0FB600. \ movzx eax,[eax]
Intel $8B9D0C000000. \ mov ebx,[ebp+12] \ the number
Intel $2903. \ sub [ebx],eax

To subtract a fraction from another fraction:
Privatize the fraction.
Normalize the fraction and the other fraction.
Subtract the fraction's numerator from the other fraction's numerator.
Reduce the other fraction.

To subtract a number and another number from a pair:
Subtract the number from the pair's x.
Subtract the other number from the pair's y.

To subtract a number from a byte:
Intel $8B8508000000. \ mov eax,[ebp+8] \ the number
Intel $8B00. \ mov eax,[eax]
Intel $8B9D0C000000. \ mov ebx,[ebp+12] \ the byte
Intel $0FB60B. \ movzx ecx,[ebx]
Intel $2BC8. \ sub ecx,eax
Intel $880B. \ mov [ebx],cl

To subtract a number from a fraction:
Subtract the number / 1 from the fraction.

To subtract a number from a pair:
Subtract the number from the pair's x.
Subtract the number from the pair's y.

To subtract a pair from another pair:
Subtract the pair's x from the other pair's x.
Subtract the pair's y from the other pair's y.

To subtract a pointer from another pointer;
To subtract a number from a pointer;
To subtract a number from another number:
Intel $8B8508000000. \ mov eax,[ebp+8] \ the number
Intel $8B00. \ mov eax,[eax]
Intel $8B9D0C000000. \ mov ebx,[ebp+12] \ the other number
Intel $2903. \ sub [ebx],eax

The superscript-one byte is a byte equal to 185.

The superscript-three byte is a byte equal to 179.

The superscript-two byte is a byte equal to 178.

To swap a color with another color:
Swap the color's hue with the other color's hue.
Swap the color's saturation with the other color's saturation.
Swap the color's lightness with the other color's lightness.

To swap a pair with another pair:
Swap the pair's x with the other pair's x.
Swap the pair's y with the other pair's y.

To swap a pointer with another pointer;
To swap a number with another number:
Put the number into a third number.
Put the other number into the number.
Put the third number into the other number.

To swap some things with some other things:
Swap the things' first with the other things' first.
Swap the things' last with the other things' last.

The synchronous-idle byte is a byte equal to 22.

A systemtime is a record with
A wyrd called wyear,
A wyrd called wmonth,
A wyrd called wdayofweek,
A wyrd called wday,
A wyrd called whour,
A wyrd called wminute,
A wyrd called wsecond,
A wyrd called wmilliseconds.

The t-key is a key equal to 84.

The tab byte is a byte equal to 9.

The tab key is a key equal to 9.

To take off all the masking tape: unmask everything.

To take off any masking tape: unmask everything.

A talker is a pointer to a talker object.

The talker is a talker.

A talker object is a record with a talker vtable called vtable.

A talker vtable is a pointer to a talker vtable record.

A talker vtable record is a record with
\ iunknown
A pointer called queryinterface,
A pointer called addref,

A pointer called release, \ in this:pspvoice out number
\ italker
A pointer called setnotifysink,
A pointer called setnotifywindowmessage,
A pointer called setnotifycallbackfunction,
A pointer called setnotifycallbackinterface,
A pointer called setnotifywin32event,
A pointer called waitfornotifyevent,
A pointer called getnotifyeventhandle,
A pointer called setinterest,
A pointer called getevents,
A pointer called getinfo,
A pointer called setoutput,
A pointer called getoutputobjecttoken,
A pointer called getoutputstream,
A pointer called pause,
A pointer called resume,
A pointer called setvoice,
A pointer called getvoice,
A pointer called speak, \ in this:pspvoice; pwcs:pwchar; dwflags:number; pulstreamnumber:pnumber out number
A pointer called speakstream,
A pointer called getstatus,
A pointer called skip,
A pointer called setpriority,
A pointer called getpriority,
A pointer called setalertboundary,
A pointer called getalertboundary,
A pointer called setrate,
A pointer called getrate,
A pointer called setvolume,
A pointer called getvolume,
A pointer called waituntildone,
A pointer called setsyncspeaktimeout,
A pointer called getsyncspeaktimeout,
A pointer called speakcompleteevent,
A pointer called isuisupported,
A pointer called displayui.

The tan color is a color.

The tan pen is a pen.

The teal color is a color.

The teal pen is a pen.

The temp path is a path.

The terminal is a terminal.

A terminal is a thing with a box, some quoras, an output color, an input color, and a reply string.

The text cutoff is a number equal to 500.

A text is a thing with
A box,
An origin,
A pen color,
A font,
An alignment,
Some rows,
A margin,
A scale fraction,
A wrap flag,
A horizontal scroll flag,
A vertical scroll flag,
A selection,
A modified flag,
A last operation,
Some texts called undos,
Some texts called redos.

A textmetric is a record with
A number called tmheight,
A number called tmascent,
A number called tmdescent,
A number called tminternalleading,
A number called tmexternalleading,
A number called tmavecharwidth,
A number called tmmaxcharwidth,
A number called tmweight,
A number called tmoverhang,
A number called tmdigitizedaspectx,
A number called tmdigitizedaspecty,
A byte called tmfirstchar,
A byte called tmlastchar,
A byte called tmdefaultchar,
A byte called tmbreakchar,
A byte called tmitalic,
A byte called tmunderlined,
A byte called tmstruckout,
A byte called tmpitchandfamily,
A byte called tmcharset.

A thing is a pointer to a thing record.

A thing record has a next thing and a previous thing.

Some things has a first thing and a last thing.

A thousand is 10 hundreds.

The three byte is a byte equal to 51.

The three key is a key equal to 51.

The three-quarter byte is a byte equal to 190.

A tick is a number.

The tilde byte is a byte equal to 126.

A timer has a count, some start ticks and some total ticks.

Some times is a number. \ this is a fluke, I think -- see "smooth a polygon some times"

A token is a string.

A top is some twips.

The tpi is some twips equal to 1440.

The tpp is some twips.

The trade-mark byte is a byte equal to 153.

To trim a string:
Remove any leading noise from the string.
Remove any trailing noise from the string.

To turn around:
Turn right 1/2.

To turn a fraction equal to a number over another number:
Put the number into the fraction's top.
Put the other number into the fraction's bottom.
Turn the fraction.

To turn a fraction of the way;
To turn a fraction of the way around;
To turn a fraction:
If the fraction is 1/1, exit.
Put 3840 times the fraction plus the context's heading into the context's heading.
Normalize the context's heading.

To turn left:
Turn -1/4.

To turn left a fraction equal to a number over another number:
Put the number into the fraction's top.
Put the other number into the fraction's bottom.
Turn left the fraction.

To turn left a fraction of the way;
To turn left a fraction of the way around;
To turn left a fraction:
Privatize the fraction.
Negate the fraction.
Turn the fraction.

To turn left some points:
Put the points and 3840 into a fraction.
Turn left the fraction.

To turn right:
Turn 1/4.

To turn right some degrees:
Put the degrees times 10 and 3600 into a fraction.
Turn right the fraction.

To turn right a fraction equal to a number over another number:
Put the number into the fraction's top.
Put the other number into the fraction's bottom.
Turn right the fraction.

To turn right a fraction of the way;
To turn right a fraction of the way around;
To turn right a fraction:
Turn the fraction.

To turn right a fraction of the way some percent of the time;
To turn right a fraction about some percent of the time;
To turn right a fraction of the way about some percent of the time;
To turn right a fraction some percent of the time:
Pick a number between 1 and 100.
If the number is greater than the percent, exit.
Turn right the fraction.

To turn right some points:
Put the points and 3840 into a fraction.
Turn right the fraction.

A twip is a number.

The two byte is a byte equal to 50.

The two key is a key equal to 50.

The u-key is a key equal to 85.

To unassign a pointer:
If the pointer is nil, exit.
Call "kernel32.dll" "HeapFree" with the heap pointer and 0 [no options] and the pointer returning a number.
If the number is 0, exit.
Void the pointer.
Subtract 1 from the heap count.

The underscore byte is a byte equal to 95.

A unit is a number.

The unit-separator byte is a byte equal to 31.

To unlock a gpbitmap given a bitmapdata:
Call "gdiplus.dll" "GdipBitmapUnlockBits" with the gpbitmap and the bitmapdata's whereabouts.

To unmask everything:

Call "gdi32.dll" "SelectClipRgn" with the current canvas and 0.

To unmask inside a box:
Create an hrgn given the box.
Unmask inside the hrgn.
Destroy the hrgn.

To unmask inside an ellipse:
Create an hrgn given the ellipse.
Unmask inside the hrgn.
Destroy the hrgn.

To unmask inside an hrgn:
Call "gdi32.dll" "ExtSelectClipRgn" with the current canvas and the hrgn and 2 [rgn_or].

To unmask inside a polygon:
Create an hrgn given the polygon.
Unmask inside the hrgn.
Destroy the hrgn.

To unmask inside a roundy box:
Create an hrgn given the roundy box.
Unmask inside the hrgn.
Destroy the hrgn.

To unmask outside a box:
Create an hrgn given the box.
Unmask outside the hrgn.
Destroy the hrgn.

To unmask outside an ellipse:
Create an hrgn given the ellipse.
Unmask outside the hrgn.
Destroy the hrgn.

To unmask outside an hrgn:
Create an old hrgn given the zero box.
Call "gdi32.dll" "GetClipRgn" with the current canvas and the old hrgn returning a number.
If the number is not 1, clear the old hrgn.
Call "gdi32.dll" "SelectClipRgn" with the current canvas and 0.
Call "gdi32.dll" "ExtSelectClipRgn" with the current canvas and the hrgn and 4 [rgn_diff].
Call "gdi32.dll" "ExtSelectClipRgn" with the current canvas and the old hrgn and 2 [rgn_or].
Destroy the old hrgn.

To unmask outside a polygon:
Create an hrgn given the polygon.
Unmask outside the hrgn.
Destroy the hrgn.

To unmask outside a roundy box:
Create an hrgn given the roundy box.
Unmask outside the hrgn.
Destroy the hrgn.

To unquote a string:

Slap a substring on the string.
If the substring is blank, break.
If the substring's first's target is not the double-quote byte, exit.
Add 1 to the substring's first.
Loop.
If the substring is blank, break.
If the substring's first is the substring's last, break.
Append the substring's first's target to another string.
If the substring's first's target is the double-quote byte, add 1 to the substring's first.
Add 1 to the substring's first.
Repeat.
Put the other string into the string.

The up-arrow key is a key equal to 38.

To update the screen;
To show it;
To show it all;
To show reveal the canvas;
To refresh the screen:
Refresh the screen given the screen's box.

To uppercase any selected bytes in a text:
If the text is nil, exit.
Loop.
Get a row from the text's rows.
If the row is nil, exit.
If the row of the text is not selected, repeat.
Slap a substring on any selected bytes in the row of the text.
Uppercase the substring.
Repeat.

To uppercase a byte:
Intel $8B8508000000. \ mov eax,[ebp+8] \ the byte
Intel $803861. \ cmp byte ptr [eax],'a'
Intel $0F820C000000. \ jb END
Intel $80387A. \ cmp byte ptr [eax],'z'
Intel $0F8703000000. \ ja END
Intel $802820. \ sub byte ptr [eax],$20
\ END

To uppercase the character under a finger and put it into a string:
If the finger is nil, exit.
Put the finger's target into the string.
Uppercase the string.

To uppercase a string:
Slap a substring on the string.
Loop.
If the substring is blank, exit.
Uppercase the substring's first's target.
Add 1 to the substring's first.
Repeat.

To uppercase a text:

If the text is nil, exit.
Loop.
Get a row from the text's rows.
If the row is nil, break.
Uppercase the row's string.
Repeat.
Wrap the text.

The upperscore byte is a byte equal to 175.

A url is a string.

A url record has
A scheme string,
A host name string,
A path string,
An extra string,
A port number.

A urlcomponents is a record with
A number called dwstructsize,
A pchar called lpszscheme,
A number called dwschemelength,
A number called nscheme,
A pchar called lpszhostname,
A number called dwhostnamelength,
A number called nport, \ this is typed as a wyrd in windows documentation, but dosen't work
A pchar called lpszusername,
A number called dwusernamelength,
A pchar called lpszpassword,
A number called dwpasswordlength,
A pchar called lpszurlpath,
A number called dwurlpathlength,
A pchar called lpszextrainfo,
A number called dwextrainfolength.

To use a color:
Put the color into the context's color.

To use the fat pen:
Put 3 into the pen size.

To use a letter height:
Put the letter height into the context's letter height.

To use a letter height of some twips:
Put the twips into the letter height.
Put the twips into the context's letter height.

To use a pen:
Put the pen into the context's pen.

To use the skinny pen:
Put 1 into the pen size.

To use small pointy letters: \ as opposed to "roundy letters" not yet implemented
Use small letters.

A uuid is a record with
A number called d1,
A wyrd called d2,
A wyrd called d3,
8 bytes called d4.

The v-key is a key equal to 86.

A vertex array is a pointer to a vertex array record.

A vertex array record has a count and a spot pointer.

A vertex is a thing with an x coord, a y coord, a spot at the x.

The vertical-bar byte is a byte equal to 124.

The vertical-tab byte is a byte equal to 11.

A very dark color is a color.

A very light color is a color.

A very very dark color is a color.

A very very light color is a color.

The violet color is a color.

The violet pen is a pen.

The w-key is a key equal to 87.

A w-param is a number.

To wait for an event;
To deque an event:
Yield to windows.
Put the event queue’s first into the event.
If the event is nil, repeat.
Remove the event from the event queue.
If the event’s kind is "done", destroy the event; exit.
Destroy the current event.
Put the event into the current event.

To wait for some grains of sand;
To wait for some grains of sand to fall;
To wait for some grains of sand to fall in the hourglass;
To wait some milliseconds;
To wait for some milliseconds:
If the milliseconds are less than or equal to 0, exit.
Call "kernel32.dll" "Sleep" with the milliseconds.

To wait for a key to come back up:
If the key is not up, repeat.

To wait for a key to come up:
If the key is not up, repeat.

To wait for a key to go down:
If the key is not down, repeat.

To wait on that there key with the ESC on it:
Wait for the escape key.

To wait until speaking is done:
If the talker is nil, exit.
Call the talker's vtable's waituntildone with the talker and -1.

To wait until we hit a key;
To wait for a key:
Wait for the key to go down.
Wait for the key to come up.
Flush all events.

A wave file is a path.

A wave is a hex string.

The white color is a color.

The white pen is a pen.

A wide string is a string.

A width is some twips.

A win32finddata is a record with
A number called dwfileattributes,
A filetime called ftcreationtime,
A filetime called ftlastaccesstime,
A filetime called ftlastwritetime,
A number called nfilesizehigh,
A number called nfilesizelow,
A number called dwreserved0,
A number called dwreserved1,
260 bytes called cfilename,
14 bytes called calternatefilename.

A window class is a record with
A number called cbsize,
A number called style,
A pointer called lpfnwndproc,
A number called cbclsextra,
A number called cbwndextra,
A handle called hinstance,
A hicon called hicon,
A cursor called hcursor,

An hbrush called hbrbackground,
A pointer called lpszmenuname,
A pointer called lpszclassname,
A hicon called hiconsm.

The window class is a window class.

A window is a handle.

A winhttp request is a thing with
A session handle,
A connection handle,
A request handle.

A word is a substring.

To wrap a text:
If the text is nil, exit.
If the text's wrap flag is not set, exit.
Convert the text's anchor to an absolute position given the text.
Convert the text's caret to another absolute position given the text.
Put the text's scale into a fraction.
Scale the text to 1/1.
Extract a string from the text.
Append the return byte to the string.
Destroy the text's rows.
Slap a rider on the string.
Create the hfont of the memory canvas given the text's font.
Loop.
Move the rider given the text's box (word wrapping rules).
If the rider's token is blank, break.
Create a row given the rider's token.
Append the row to the text's rows.
Repeat.
Destroy the hfont of the memory canvas.
Renumber the text's rows.
Scale the text to the fraction.
Convert the absolute position to the text's anchor given the text.
Convert the other absolute position to the text's caret given the text.
Limit the origin of the text.

To store a buffer in a file;
To write a buffer to a file:
Clear the i/o error.
Call "kernel32.dll" "SetFilePointer" with the file and 0 and 0 and 0 [file_begin] returning a result number.
If the result number is -1, put "Error positioning file pointer." into the i/o error; exit.
Call "kernel32.dll" "WriteFile" with the file and the buffer's first and the buffer's length and a number's whereabouts and 0 returning the result number.
If the result number is 0, put "Error writing file." into the i/o error; exit.

To store a buffer in a path;
To write a buffer to a path:
Clear the i/o error.
Extract a directory from the path.
If the directory is not in the file system, put "Directory '" then the directory then "' doesn't exist." into the i/o

error; exit.
Set the path to read-write mode.
Privatize the path.
Null terminate the path.
Call "kernel32.dll" "CreateFileA" with the path's first and 1073741824 [generic_write]
And 0 and 0 and 2 [create_always] and -2147483520 [file_flag_write_through or file_attribute_normal]
and 0 returning a handle.
If the handle is -1 [invalid_handle_value], put "Error opening file '" then the path then "'." into the i/o error; exit.
Call "kernel32.dll" "WriteFile" with the handle and the buffer's first and the buffer's length and a number's whereabouts and 0 returning the number.
Call "kernel32.dll" "CloseHandle" with the handle.
If the number is not 0, exit.
Put "Error writing file '" then the path then "'." into the i/o error.

To write a byte:
Put the byte into a string.
Write the string.

To write a byte to stdout:
Call "kernel32.dll" "WriteFile" with the stdout handle and the byte's whereabouts and 1 and a number's whereabouts and nil.

To write a byte without advancing:
Put the byte into a string.
Write the string without advancing.

To write a flag:
Convert the flag to a string.
Write the string.

To write a flag without advancing:
Convert the flag to a string.
Write the string without advancing.

To write a fraction:
Convert the fraction to a string.
Write the string.

To write a fraction without advancing:
Convert the fraction to a string.
Write the string without advancing.

To write a number:
Convert the number to a string.
Write the string.

To write a number without advancing:
Convert the number to a string.
Write the string without advancing.

To write some quoras in a box:
\Draw the box with the red color and the clear color. \ temp ***
Put the box into a quora box.
Put the quora box's top plus 1/4 inch into the quora box's bottom.

```
Loop.
Get a quora from the quoras.
If the quora is nil, break.
\draw really fast. ***
\ Draw the quora box with the yellow color. \ temp ***
Write the quora's string in the quora box with the quora's color.
Move the quora box down 1/4 inch.
Repeat.

To write a string;
To stroke a string:
Privatize the string.
Loop.
If the string is blank, exit.
Get a byte from the string.
Stroke the byte.
If the string is not blank, space between glyphs.
Repeat.

To write a string around a center spot at a radius;
To write a string given a center spot and a radius;
To stroke a string around a center spot at a radius;
To stroke a string given a center spot and a radius:
Privatize the string.
Put 1 and the string's length into a fraction.
Loop.
If the string is blank, exit.
Get a byte from the string.
Start at the center spot.
Move the radius.
Stroke the byte.
Turn the fraction.
Repeat.

To write a string at a spot with a color;
To stroke a string at a spot with a color:
Start at the spot.
Put the color into the context's color.
Stroke the string.

To write a string to a console;
To write a string on a console:
If the console is nil, exit.
Insert the string into the console's text.
Insert the return byte into the console's text.
Wrap the console's text.
Scroll the console's text to the caret.
Show the console.

To write a string to a console without advancing;
To write a string on a console without advancing:
If the console is nil, exit.
Insert the string into the console's text.
Wrap the console's text.
Scroll the console's text to the caret.
```

Show the console.

To write a string in a box;
To stroke a string in a box:
Stroke the string in the box with the context's color.

To write a string in a box with a color;
To stroke a string in a box with a color:
Put the color into the context's color.
Put the box's left-bottom into the context's spot.
Put the box's height divided by 2 into the context's letter height.
\Put the box's height into the context's letter height.
Face north.
Move the box's height divided by 4. \ was 4 and still is now! ***
Stroke the string.

To write a string in the middle of a box;
To stroke a string in the middle of a box:
Put the context's letter height divided by 4 into a square size. \ was 4 ***
\ glyphs are two squares wide plus one square of intercharacter spacing. no spacing at the end.
Put the string's length times the square size times 3 minus the square size into a width.
Divide the width by 2.
Start in the middle of the box.
Move down the context's letter height divided by 2. \ was 2 ***
Move left the width.
Face north.
Stroke the string.

To write a string in the middle of the screen:
Stroke the string in the middle of the screen's box.
Refresh the screen.

To write a string on a terminal:
If the terminal is nil, exit.
Add a quora to the terminal.
Put the string into the quora's string.
Put the terminal's output color into the quora's color.
Show the terminal.

To write a string a radius away from a center spot;
To write a string a radius around a center spot;
To write a string about a radius from a center spot;
To write a string a radius from a center spot;
To stroke a string a radius away from a center spot;
To stroke a string a radius around a center spot;
To stroke a string about a radius from a center spot;
To stroke a string a radius from a center spot:
Stroke the string given the center spot and the radius.

To write a string a radius away from the middle of a box;
To write a string a radius around the middle of a box;
To write a string about a radius from the middle of a box;
To write a string a radius from the middle of a box;
To stroke a string a radius away from the middle of a box;
To stroke a string a radius around the middle of a box;

To stroke a string about a radius from the middle of a box;
To stroke a string a radius from the middle of a box:
Stroke the string given the box's center and the radius.

To write a string to stdout:
Call "kernel32.dll" "WriteFile" with the stdout handle and the string's first and the string's length and a number's whereabouts and nil.

To write a string while turning a fraction of the way;
To write a string while turning a fraction of the way around;
To write a string while turning a fraction;
To stroke a string while turning a fraction of the way;
To stroke a string while turning a fraction of the way around;
To stroke a string while turning a fraction:
Privatize the string.
Loop.
If the string is blank, exit.
Get a byte from the string.
Stroke the byte.
Turn the fraction.
If the string is not blank, space between glyphs.
Repeat.

To write a string with a color;
To stroke a string with a color:
Put the color into the context's color.
Stroke the string.

To write a string with a color at the bottom of a box;
To stroke a string with a color at the bottom of a box:
Put the context's letter height divided by 4 into a square size. \ ***
\ glyphs are two squares wide plus one square of intercharacter spacing. no spacing at the end.
Put the string's length times the square size times 3 minus the square size into a width.
Divide the width by 2.
Start in the middle of the bottom of the box.
Move up the context's letter height times 2. \ was without the times 2 ***
Move left the width.
Face north.
Stroke the string with the color.

To write a string with a color at the top of a box;
To stroke a string with a color at the top of a box:
Put the context's letter height divided by 4 into a square size. \ ***
\ glyphs are two squares wide plus one square of intercharacter spacing. no spacing at the end.
Put the string's length times the square size times 3 minus the square size into a width.
Divide the width by 2.
Start in the middle of the top of the box.
Move down the context's letter height times 4. \ was times 2 ***
Move left the width.
Face north.
Stroke the string with the color.

To write a string with a color in the middle of a box;
To stroke a string with a color in the middle of a box:
Put the context's letter height divided by 4 into a square size. \ was 4 ***

\ glyphs are two squares wide plus one square of intercharacter spacing. no spacing at the end.
Put the string's length times the square size times 3 minus the square size into a width.
Divide the width by 2.
Start in the middle of the box.
Move down the context's letter height divided by 2. \ was 2 ***
Move left the width.
Face north.
Stroke the string with the color.

To write a string with a color a radius away from a center spot;
To write a string with a color a radius around a center spot;
To write a string with a color about a radius from a center spot;
To write a string with a color a radius from a center spot;
To stroke a string with a color a radius away from a center spot;
To stroke a string with a color a radius around a center spot;
To stroke a string with a color about a radius from a center spot;
To stroke a string with a color a radius from a center spot:
Put the color into the context's color.
Stroke the string given the center spot and the radius.

To write a string with a color some twips down from the top of a box;
To write a string with a color some twips down from the top center of a box;
To stroke a string with a color some twips down from the top of a box;
To stroke a string with a color some twips down from the top center of a box:
Put the context's letter height divided by 4 into a square size. \ was 4 ***
\ glyphs are two squares wide plus one square of intercharacter spacing. no spacing at the end.
Put the string's length times the square size times 3 minus the square size into a width.
Divide the width by 2.
Start in the middle of the top of the box.
Move down the twips.
Move left the width.
Face north.
Stroke the string with the color.

To write a string with a font and a size and a color and a spot:
Put the size into the font's height.
Put the string's width into a width.
Put the spot and the spot into a box.
Subtract the width divided by 2 from the box's left.
Add the width divided by 2 to the box's right.
Subtract the size divided by 2 from the box's top.
Add the size divided by 2 to the box's bottom.
Draw the string in the box with the color and the font and "center".
Refresh the screen.

To write with large letters;
To use large letters:
Put the large letter height into the context's letter height.

To write with medium letters;
To use medium size letters;
To use medium-size letters;
To use medium sized letters;
To use medium-sized letters;
To use medium letters:

Put the medium letter height into the context’s letter height.

To write with small letters;
To use small letters:
Put the small letter height into the context’s letter height.

A wsadata is a record with
A wyrd called wversion,
A wyrd called whighversion,
257 bytes called szdescription,
127 bytes called szsystemstatus,
A wyrd [unsigned] called imaxsockets,
A wyrd [unsigned] called imaxudpdg,
A pointer called lpvendorinfo.

A wyrd has a low byte and a high byte.

The x-key is a key equal to 88.

An xor-mask is a mask.

The y-key is a key equal to 89.

The yellow color is a color.

The yellow pen is a pen.

The yen-sign byte is a byte equal to 165.

To yield to windows:
If the event queue is not empty, exit.
Call "user32.dll" "GetMessageA" with an msg’s whereabouts and 0 and 0 and 0 returning a number.
If the number is 0, exit.
Call "user32.dll" "TranslateMessage" with the msg’s whereabouts.
Call "user32.dll" "DispatchMessageA" with the msg’s whereabouts.

The z-key is a key equal to 90.

To zero a box: \ was clear a box, got confounded with "clear a box" (which should draw the box all black as does "clear the screen")
Put 0 into the box’s left.
Put 0 into the box’s top.
Put 0 into the box’s right.
Put 0 into the box’s bottom.

The zero box is a box.

The zero byte is a byte equal to 48.

To zero fill a number given a count and append it to a string:
Convert the number to another string.
Zero fill the other string given the count.
Append the other string to the string.

To zero fill a string given a count:

If the string’s length is greater than or equal to the count, exit.
Prepend the zero byte to the string.
Repeat.

The zero key is a key equal to 48.

The zero line is a line. \ tracer

The zero spot is a spot.